

# Kolibri: A Sovereign European Model on the Pareto Frontier

Aleph Alpha

## Abstract

This report introduces Kolibri, an English–German Mixture-of-Experts transformer with 78.1B total parameters and 3.46B active parameters per token, released as open weights under the Apache 2.0 licence. We built Kolibri for sovereign and specialised deployment, with a particular focus on German and on regulated domains such as public administration, industry, and aerospace. To support these settings, we control the full model-development process, including data, architecture, training infrastructure, post-training, and evaluation.

We train Kolibri on 24T tokens across pre-training, mid-training, and long-context extension, with German accounting for more than 20 % of the data mix. To reach this share, we complement German datasets with more than 2T German tokens that we curate from the web with our own pipeline and generate synthetically. Post-training combines supervised fine-tuning on curated German and English data with reinforcement learning on more than 1.2M internally curated tasks, which cover reasoning, tool use, instruction following, code, and retrieval. The model activates 4.4 % of its parameters per token, and among the open models evaluated in this report, it lies on the Pareto frontier of quality and serving cost in English and German.

Kolibri builds on Kolibri Origin, an earlier iteration that validated our full pipeline at scale. Both models come from our Model Factory, which connects data, experimentation, training, evaluation, and deployment through version-controlled workflows<sup>1</sup>.

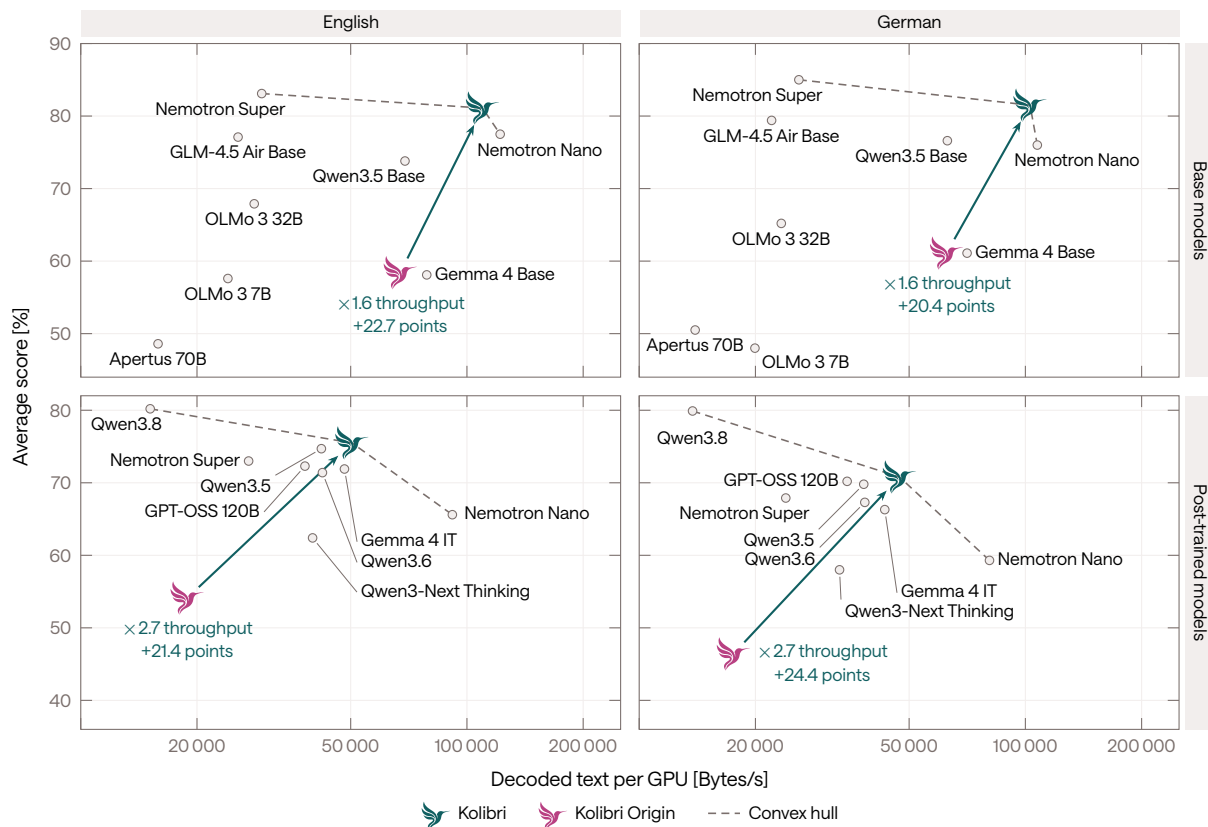


Figure 1. Average English and German benchmark scores of base (top) and post-trained (bottom) models against decoded bytes of text per second and GPU, where higher and further right is better. Every model runs in its fastest configuration on eight B200 GPUs at a sequence length of 4k (base) or 16k (post-trained). Scores taken from Tables 17, 28 and 29; eval description in Section 3.3.1; plot description in Appendix A.

<sup>1</sup><https://huggingface.co/Aleph-Alpha/kolibri-1>

# Contents

|          |                                                                  |            |
|----------|------------------------------------------------------------------|------------|
| <b>1</b> | <b>Introduction</b>                                              | <b>4</b>   |
| <b>2</b> | <b>Pre-Training</b>                                              | <b>6</b>   |
| 2.1      | <b>Model Architecture</b>                                        | <b>7</b>   |
| 2.1.1    | Ablation Methodology                                             | 9          |
| 2.1.2    | Tokeniser                                                        | 9          |
| 2.1.3    | Hybrid Sliding-Window Attention                                  | 12         |
| 2.1.4    | Scaling Sparsity                                                 | 14         |
| 2.1.5    | MoE Load Balancing                                               | 16         |
| 2.1.6    | Infrastructure                                                   | 18         |
| 2.2      | <b>Optimisation</b>                                              | <b>18</b>  |
| 2.2.1    | Hyperparameter Transfer                                          | 19         |
| 2.3      | <b>Data and Training Stages</b>                                  | <b>24</b>  |
| 2.3.1    | Overview                                                         | 24         |
| 2.3.2    | Stage 1: Pre-training Data                                       | 24         |
| 2.3.3    | Stage 2: Mid-Training                                            | 32         |
| 2.3.4    | Stage 3: Long Context Extension                                  | 38         |
| 2.4      | <b>Base Model Evaluation</b>                                     | <b>44</b>  |
| 2.4.1    | Evaluation Suite                                                 | 44         |
| 2.4.2    | Baseline Models                                                  | 45         |
| 2.4.3    | Results                                                          | 45         |
| <b>3</b> | <b>Post-Training</b>                                             | <b>49</b>  |
| 3.1      | <b>Supervised Fine-Tuning (SFT)</b>                              | <b>52</b>  |
| 3.1.1    | Datasets                                                         | 53         |
| 3.1.2    | Data Pool Curation                                               | 59         |
| 3.1.3    | Data Mixing with MergeMix                                        | 66         |
| 3.1.4    | Kolibri SFT Training                                             | 68         |
| 3.2      | <b>Reinforcement Learning (RL)</b>                               | <b>71</b>  |
| 3.2.1    | Training Recipe                                                  | 71         |
| 3.2.2    | Environment Mix                                                  | 76         |
| 3.2.3    | Infrastructure                                                   | 82         |
| 3.2.4    | Kolibri RL Training                                              | 86         |
| 3.2.5    | Lessons Learned and Hills Climbed                                | 89         |
| 3.3      | <b>Released Model Evaluation</b>                                 | <b>96</b>  |
| 3.3.1    | Evaluation Suite                                                 | 96         |
| 3.3.2    | Baseline Models                                                  | 98         |
| 3.3.3    | Results and Discussion                                           | 98         |
| <b>4</b> | <b>Model Factory</b>                                             | <b>104</b> |
| 4.1      | <b>Savanna: Model Training as Code</b>                           | <b>105</b> |
| 4.1.1    | Core Abstractions                                                | 105        |
| 4.1.2    | Continuous Integration                                           | 106        |
| 4.1.3    | Infrastructure                                                   | 106        |
| 4.1.4    | Agents in the Pipeline                                           | 107        |
| 4.2      | <b>Savanna in Kolibri Training</b>                               | <b>107</b> |
| <b>5</b> | <b>Summary</b>                                                   | <b>109</b> |
|          | <b>Legal Due Diligence Example: Deep Dive Into Copyright Law</b> | <b>111</b> |
| <b>6</b> | <b>Appendix</b>                                                  | <b>130</b> |
| A        | <b>Quality-Cost Analysis</b>                                     | <b>130</b> |
| B        | <b>Model Architecture</b>                                        | <b>132</b> |
| B.1      | Tokeniser                                                        | 132        |
| B.2      | QK Normalisation                                                 | 134        |

|          |                                                                |            |
|----------|----------------------------------------------------------------|------------|
| B.3      | Model-Shape Evaluation Suite . . . . .                         | 135        |
| B.4      | Sparsity Sweep . . . . .                                       | 135        |
| B.5      | Model Shape at 80B Total Parameters . . . . .                  | 136        |
| B.6      | Leading Dense Layers and Routing in the First Layers . . . . . | 137        |
| B.7      | Pre-Training Infrastructure . . . . .                          | 140        |
| <b>C</b> | <b>Optimisation</b>                                            | <b>143</b> |
| C.1      | Training Batch Construction . . . . .                          | 143        |
| <b>D</b> | <b>Data and Training Stages</b>                                | <b>146</b> |
| D.1      | Common Crawl Internal Pipeline . . . . .                       | 146        |
| D.2      | Mid-training Details . . . . .                                 | 149        |
| D.3      | Full Long-Context Results of the Mix Ablation . . . . .        | 161        |
| <b>E</b> | <b>Base Model Evaluation</b>                                   | <b>163</b> |
| E.1      | Baseline Models . . . . .                                      | 163        |
| <b>F</b> | <b>“I Don’t Know” Evaluations</b>                              | <b>165</b> |
| <b>G</b> | <b>Reasoning Mode at Long Context</b>                          | <b>167</b> |
| <b>H</b> | <b>Supervised Fine-Tuning (SFT)</b>                            | <b>168</b> |
| H.1      | Long-Context SFT Data Ablation . . . . .                       | 168        |
| H.2      | Impact of German Reasoning Data . . . . .                      | 168        |
| H.3      | SFT MergeMix Sweep . . . . .                                   | 169        |
| H.4      | SFT Ablation Recipes . . . . .                                 | 170        |
| H.5      | KulturBank . . . . .                                           | 171        |
| <b>I</b> | <b>Reinforcement Learning</b>                                  | <b>172</b> |
| I.1      | Environments . . . . .                                         | 172        |
| I.2      | Harness Tools . . . . .                                        | 178        |
| I.3      | Per-Token Logit-Gradient Norms . . . . .                       | 178        |
| I.4      | Training Recipe Details . . . . .                              | 180        |
| I.5      | RL Hyperparameters . . . . .                                   | 181        |
| I.6      | Further OPSD Analysis . . . . .                                | 184        |

## Part 1

# Introduction

Organisations in public administration, industry, and aerospace process sensitive data under regulation. For these domains, a language model has to run on infrastructure that the organisation controls, rest on training in line with European values, and serve German as well as English. In addition, the serving cost of the model limits the scale at which an organisation can deploy it.

Kolibri, which we built for these requirements, is an English–German MoE transformer with 78.1B total parameters, of which 3.46B, or 4.4 %, are active per token. The Apache 2.0 licence of the open weights permits the deploying organisation to operate the model on its own infrastructure. Among the open models we evaluate, Kolibri lies on the Pareto frontier of quality and serving cost in English and German, both as a base and as a post-trained model (Figure 1). Figure 2 shows that Kolibri scores higher than three open models on most benchmarks across Agentic, Industry, Math, Code and General Knowledge. The model processes contexts of up to 1M tokens, and users set a reasoning effort per request to trade cost and latency against answer quality.

We designed the development pipeline of Kolibri with applicable regulation, such as the EU AI Act, its General-Purpose AI Code of Practice and the General Data Protection Regulation in mind from the ground up. Our data pipeline filters against illegal, harmful, and pirated content and redacts personal data from data sources. In addition, we align the answers of the model with the values of the democratic consensus in Europe and train the model to abstain when the provided context does not support an answer.

Teams in Germany developed Kolibri end to end and train it on infrastructure in Germany and Finland. Training spans 24T tokens across pre-training, mid-training, and long-context extension, with a German share of more than 20 % and sequences of up to 256k tokens in the final stage. To reach this share, we complement German datasets with more than 2T German tokens that we curate from the web with our own pipeline and generate synthetically. Post-training combines SFT on curated German and English data with RL on more than 1.2M internally curated tasks, which cover reasoning, tool use, instruction following, code, and retrieval. The predecessor of Kolibri, Kolibri Origin, is an internal model that we used to validate the full development pipeline at scale. From Kolibri Origin to Kolibri, we scaled the data, the model size, the context length, and post-training, and we refined the architecture and the training recipe (Table 1).

The entire pipeline, from data curation through pre- and post-training to evaluation, runs in our Model Factory, which comprises the infrastructure, tooling, and development processes that connect data, experimentation, training, evaluation, and deployment. The Model Factory expresses these workflows as

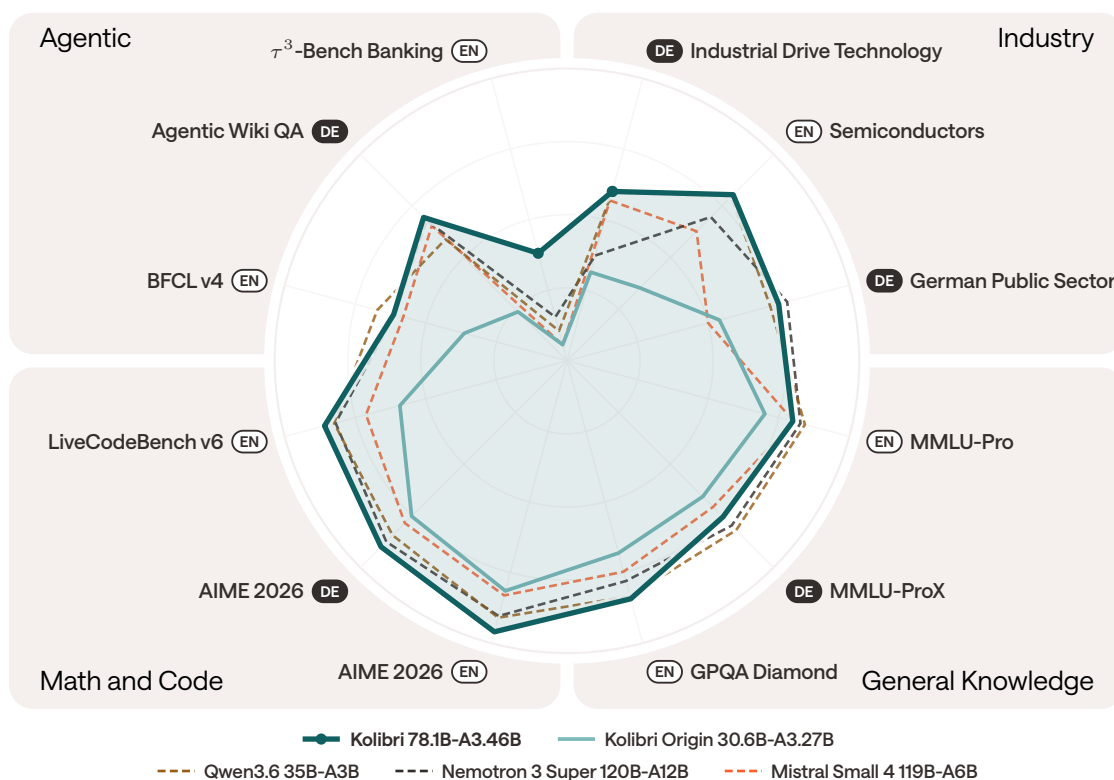


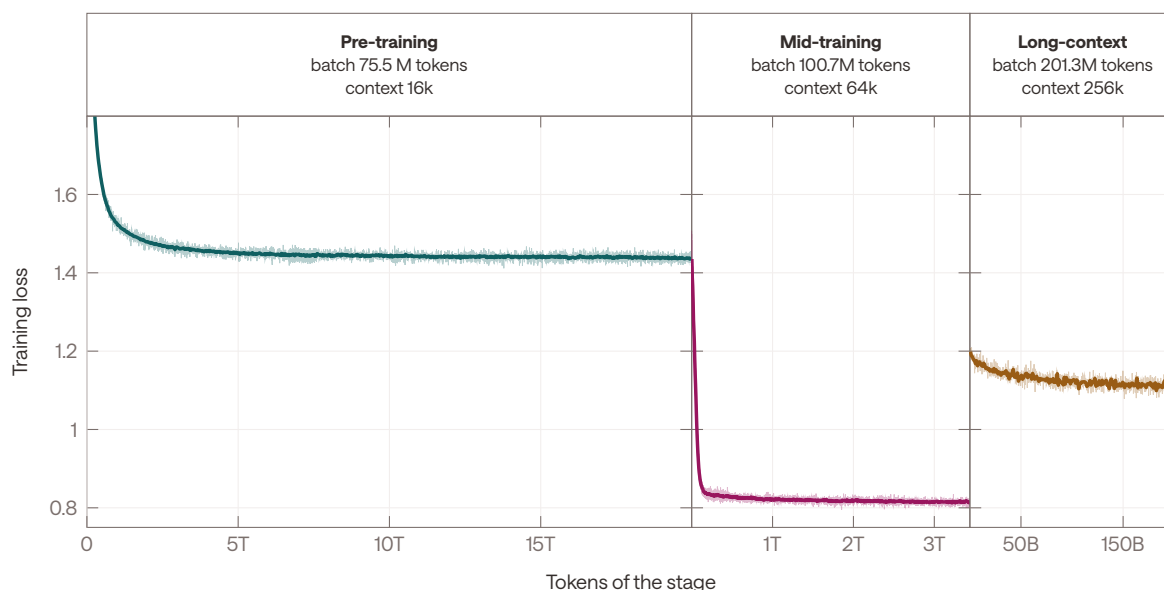
Figure 2. Post-training benchmark scores of Kolibri, Kolibri Origin and three open models in four domains, from Tables 28 and 29. Badges mark the English (EN) and German (DE) version of a benchmark; the scale runs from 0 % at the centre to 100 % at the outer ring.

| Property                 | Kolibri Origin | Kolibri   |
|--------------------------|----------------|-----------|
| Size                     | 30.6A3B        | 78.1A3B   |
| Pre-training tokens      | 8T             | 24T       |
| Supported context length | 64k tokens     | 1M tokens |

Table 1. Main differences between Kolibri Origin, an internal model that is not released, and Kolibri. Active parameters exclude the embeddings. Table 2 lists the full architecture of both models.

version-controlled code, which makes every experiment reproducible, composable, and traceable. A new model generation, an additional language, or a further specialisation thus becomes a change to this code.

The remainder of this report follows the development of Kolibri. Part 2 describes the architecture, the optimisation, the data, and the stages of pre-training, mid-training, and long-context extension, together with the experiments behind the design choices, and evaluates the base model. Post-training follows in Part 3, which covers SFT and RL and evaluates the post-trained model. Part 4 describes the Model Factory, and Part 5 summarises the results. The appendices in Part 6 give the details of the individual stages and the quality-cost analysis.



**Figure 3.** Training loss of the three stages of Kolibri, each against the tokens of that stage and on its own token axis. Mid-training starts from the pre-training checkpoint at step 264 750 and long-context extension from the mid-training checkpoint at step 34 200. Thin lines are the per-step loss at a sample of the steps, thick lines its mean over bins of 20B, 5B and 1B tokens. Each stage trains on its own mix, so the loss levels of different stages are not comparable.

## Part 2

# Pre-Training

This part covers the three stages that produce the Kolibri base model: pre-training, mid-training and long-context extension.

After pre-training for 20T tokens at a 16k context length on data from nine domains (Section 2.3), mid-training continues for 3.4T tokens at a context length of 64k tokens, on a mix drawn mostly from curated datasets rather than from web text (Section 2.3.3). The long-context extension then adds 200B training tokens at 256k token context length (Section 2.3.4). Figure 3 shows the training loss throughout pre-training.

Every stage was scaled with respect to Kolibri Origin, which pre-trained on 7.5T tokens at a context of 4k tokens, mid-trained on 122B tokens at 8k and extended the context on 96B tokens at 64k. Kolibri trains on about three times as many tokens in total, at four times the maximum context. Mid-training grows the most, by a factor of about 28.

We start by describing the model architecture and how we run the pre-training in Sections 2.1 and 2.1.6, followed by our specific optimisation settings in Section 2.2. Section 2.3 describes the pre-training data: the Common Crawl pipeline, the mixing of the data sources, PII redaction and decontamination. Sections 2.3.3

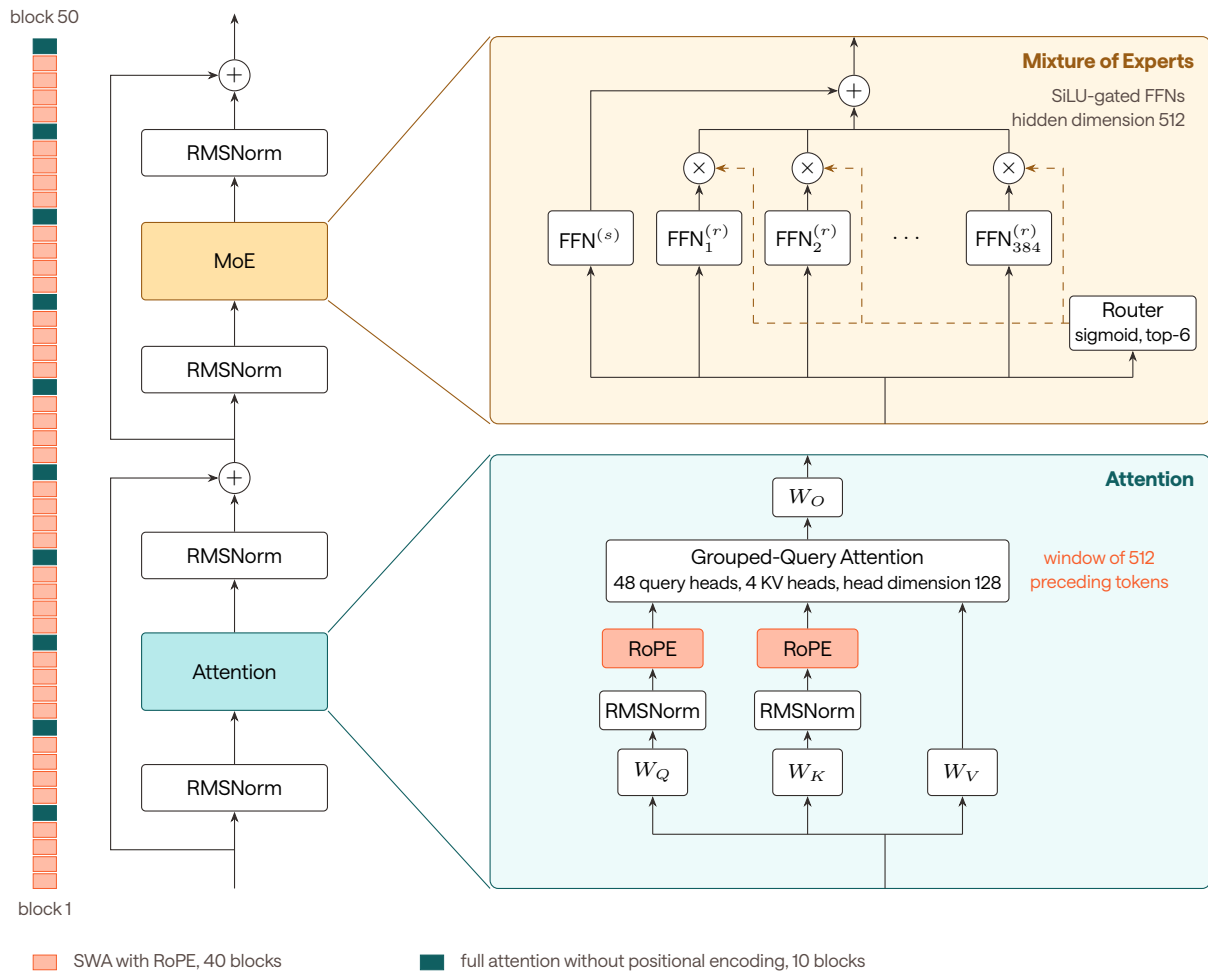


Figure 4. Architecture of one Kolibri block, which all 50 blocks share. The attention and MoE sublayers each read from the residual stream through an RMSNorm and write back through a second one. Orange marks what only the 40 SWA blocks have: RoPE on queries and keys, and a mask over the 512 preceding tokens and the query token. The strip on the left shows which blocks use full attention. The router scores all 384 routed experts, sends the token to the 6 with the highest scores and weights their outputs by those sigmoid scores  $g_i$  without renormalisation.

and 2.3.4 describe the data of the two later stages and the ablations that selected their mixes. Section 2.4 defines the evaluation suite that the ablations of the following sections report scores on and describes the benchmark scores of the resulting base model.

## 2.1 Model Architecture

Kolibri is a decoder-only causal Transformer (Vaswani et al. 2017) with hybrid attention, which interleaves SWA and full-attention blocks, and sparse MoE layers of FFNs (Shazeer et al. 2017). It has 78.1B total parameters and 3.46B active parameters per token. Therefore, each token uses 4.4 % of the parameters, keeping the computation per token small at a large total capacity (Section 2.1.4). Figure 4 shows one block and Table 2 lists its dimensions. Each block uses sandwich normalisation (M. Ding et al. 2021; V. Singh et al.

| Property                       | Kolibri               | Kolibri Origin        |
|--------------------------------|-----------------------|-----------------------|
| Transformer blocks             | 50                    | 50                    |
| Leading dense layers           | 0                     | 2                     |
| Full attention / SWA blocks    | 10 / 40               | 50 / 0                |
| SWA window size                | 512 preceding tokens  | —                     |
| Model width                    | 2560                  | 2048                  |
| Vocabulary                     | 128 000               | 96 000                |
| Attention heads / KV heads     | 48 / 4                | 32 / 4                |
| Attention head dimension       | 128                   | 128                   |
| Dense FFN hidden dimension     | —                     | 6144                  |
| Routed experts per MoE layer   | 384                   | 128                   |
| Experts per token              | 6 routed and 1 shared | 8 routed and 1 shared |
| Routed-expert hidden dimension | 512                   | 768                   |
| Total parameters               | 78.1B                 | 30.6B                 |
| Active parameters per token    | 3.46B                 | 3.27B                 |
| Trained maximum context        | 256k tokens           | 64k tokens            |

Table 2. Architecture of Kolibri and, for comparison, Kolibri Origin. Active parameters exclude the embeddings.

2026), applying RMS normalisation (B. Zhang and Sennrich 2019) before and after both the attention and MoE layers.

The tokeniser vocabulary of 128 000 tokens is trained with UniBPE (Section 2.1.2), which builds merges bottom-up like BPE but selects each merge by Unigram loss. Compared with the GPT-5 tokeniser, it places more token boundaries on morpheme boundaries, in German 65 % against 47 %, and needs 11.2 % fewer tokens on German web text.

Attention uses GQA (Ainslie et al. 2023) with 48 query heads, 4 KV heads and head dimension 128. The blocks combine SWA and full attention at a 4:1 ratio: every fifth block uses full attention, and the other blocks use SWA (Beltagy, Peters, and Cohan 2020) over the 512 preceding tokens. Positional information is injected only in the SWA blocks using RoPE (Jianlin Su et al. 2024) with  $\theta = 10\,000$ . The full-attention blocks use no positional encoding. Queries and keys are normalised per head with RMS normalisation (Henry et al. 2020) before RoPE. Section 2.1.3 compares the hybrid attention pattern with full attention, selects the window size and gives the reasons for the positional encoding and QK normalisation.

Every MoE layer routes each token to 6 of 384 routed experts and also evaluates one shared expert (Dai et al. 2024). The experts are SiLU-gated FFNs (Shazeer 2020). The total parameter count follows from the sparsity sweep of Section 2.1.4. The routing scores are sigmoids of the router logits (DeepSeek-AI 2024b), but are not renormalised after top- $k$  selection. Expert loads are balanced with Exact Quantile Balancing (EQB) and Load-Error Injection (LEI) (Neitemeier et al. 2026) (Section 2.1.5).

Kolibri Origin has a simpler architecture (Table 2), in which every block uses full attention with RoPE, the first two blocks are dense, and each MoE layer routes a token to 8 of 128 experts. It is less sparse, with 10.7 % of its parameters active per token against 4.4 % for Kolibri. Its expert loads are balanced with the combination of Soft-Clamped Momentum Expert Bias Updates (SMEBU) (V. Singh et al. 2026) expert-bias updates and a GShard-style auxiliary loss (Lepikhin et al. 2021).

## 2.1.1 Ablation Methodology

We compare recipe choices through controlled proxy-scale ablations before validating at larger scale. Each study specifies its resource-matching criteria, training budget and evaluation stage, with checkpoints compared at a common training horizon.

**Proxy Models** We run ablations on two smaller *proxy models*: versions of Kolibri which share the main architectural choices but fewer parameters (Table 3). Both proxy models differ in width and have fewer routed experts per layer at a lower dimension compared to the target Kolibri. The proxy at scale M has 0.6B active out of 7.8B total parameters trained on 97.6B tokens. The proxy at scale L has 2.1B active out of 30.6B total parameters trained on 327.6B tokens.

| Scale                          | M              | L               | Target (Kolibri) |
|--------------------------------|----------------|-----------------|------------------|
| Transformer blocks             | 50             | 50              | 50               |
| Model width                    | 1024           | 2048            | 2560             |
| Attention heads / KV heads     | 16 / 2         | 32 / 4          | 48 / 4           |
| Routed experts per layer       | 256            | 256             | 384              |
| Routed experts per token       | 6              | 6               | 6                |
| Routed-expert hidden dimension | 192            | 384             | 512              |
| Total parameters               | 7.8B           | 30.6B           | 78.1B            |
| Active parameters per token    | 0.6B           | 2.1B            | 3.46B            |
| Pre-training tokens (steps)    | 97.6B (10 342) | 327.6B (13 016) | 20T              |
| Long-context tokens (steps)    | 5B (299)       | 15B (448)       | 200B             |

Table 3. Scale M and L proxy models, used for ablation studies next to the target configuration of Kolibri (Table 2). Tokens are the pre-training horizon of each scale, in batches of 576 (M) and 1536 (L) sequences of 16k tokens.

**Evaluation** We evaluate every candidate change along three axes: modelling quality, stability in the attention and MoE layers, and efficiency, both in training and decode during inference. To assess modelling quality, we use validation loss and a suite of benchmarks, which are described in Section 2.4.1. Because we broadened the evaluation suite as the model was developed, aggregates are not directly comparable across studies. To ablate the impact on long-context performance, some studies applied a shortened long-context phase directly after pre-training.

**Learning-Rate Selection** Architectural changes can shift the optimal learning rate, so using only the scaling-law prediction can confound shape quality with a suboptimal learning rate. For model-shape sweeps, we tune learning rates at the smallest ablation scale using  $\eta \in \{\eta_0/\sqrt{2}, \eta_0, \sqrt{2}\eta_0\}$ , where  $\eta_0$  is the predicted learning rate.

## 2.1.2 Tokeniser

We designed the Kolibri tokeniser to capture the specificity of the German language, without compromising on English efficiency and downstream model performance.

The model uses a vocabulary of 128 000 tokens, up from 96 000 tokens used by Kolibri Origin. We train both Kolibri tokenisers with *UniBPE*, a new hybrid tokeniser training method: it keeps the greedy bottom-up approach of BPE (Gage 1994; Sennrich, Haddow, and Birch 2016) and scores merges with the Unigram loss objective (Kudo 2018).

**UniBPE Merge Score** A tokeniser state assigns every token  $t$  a count  $c_t$  over the training corpus, with  $N = \sum_t c_t$  tokens in total. The Unigram model puts probability  $c_t/N$  on  $t$ , and the negative log-likelihood of the corpus under it is

$$\mathcal{L} = - \sum_t c_t \log \frac{c_t}{N} = h(N) - \sum_t h(c_t), \quad h(x) = x \log x, \quad h(0) = 0. \quad (1)$$

This is the quantity Unigram tokenisers minimise top-down by pruning tokens. UniBPE minimises it bottom-up: at each step it applies the merge that lowers  $\mathcal{L}$  the most, and otherwise runs exactly like BPE, producing an ordinary vocabulary and merge list.

Merging the pair  $(a, b)$  into a new token  $ab$ , where the pair occurs  $c_{ab}$  times, moves  $c_{ab}$  counts out of  $a$  and out of  $b$ , creates a token with count  $c_{ab}$ , and shortens the corpus by  $c_{ab}$  tokens. Every other count is untouched, so for  $a \neq b$  the change of the loss

$$\Delta\mathcal{L} = \underbrace{h(c_a) - h(c_a - c_{ab})}_{a \text{ loses } c_{ab}} + \underbrace{h(c_b) - h(c_b - c_{ab})}_{b \text{ loses } c_{ab}} - \underbrace{h(c_{ab})}_{\text{new } ab \text{ token}} + \underbrace{h(N - c_{ab}) - h(N)}_{\text{shorter corpus}} \quad (2)$$

is closed-form. BPE ranks candidates by  $c_{ab}$  alone; UniBPE ranks them by  $\Delta\mathcal{L}$ , the most negative first. Both only need the pair count and the two parent counts, so no new information is required over BPE.

**Training** We train the tokeniser on a shuffled 100 GB slice of the pre-training dataset (Section 2.3) without Unicode normalisation. For pre-tokenisation we adapt the GPT4 pre-tokeniser (OpenAI 2023), changing the regex to split every digit into its own token. The tokeniser has byte fallback, starting with 256 byte tokens. The next  $120\,000 - 256 = 119\,744$  merges use the UniBPE algorithm, standard BPE performs the next 7900 merges, and we then add 100 reserved and special tokens, for a total vocabulary of 128 000 tokens. The second stage of standard BPE has two purposes. First, some useful merges are difficult under UniBPE, such as `. . .`, preventing `. . .` from being a single token. Second, Kolibri Origin used a pure UniBPE algorithm, leading to degraded compression. This hybrid approach matches the compression of standard BPE in our experiments.

**Compression** On German text, the Kolibri tokeniser reaches 4.90 bytes per token, the highest compression across twelve evaluated tokenisers in Figure 5, with the best of the nine other tokenisers being 4.35 for GPT-5, and is on par with BPE trained on the same pre-training corpus. On English, Kolibri reaches 4.58 bytes per token, against 4.67 for GPT-5, the best tokeniser on English. This lowers the cost of training on and serving German text, and leads to shorter context lengths in long-context tasks while staying competitive in English.

**Morphology** UniBPE consistently improves on morphological alignment compared to plain BPE. In the first 60 merges each performs on the pre-training dataset, UniBPE creates the units of three or

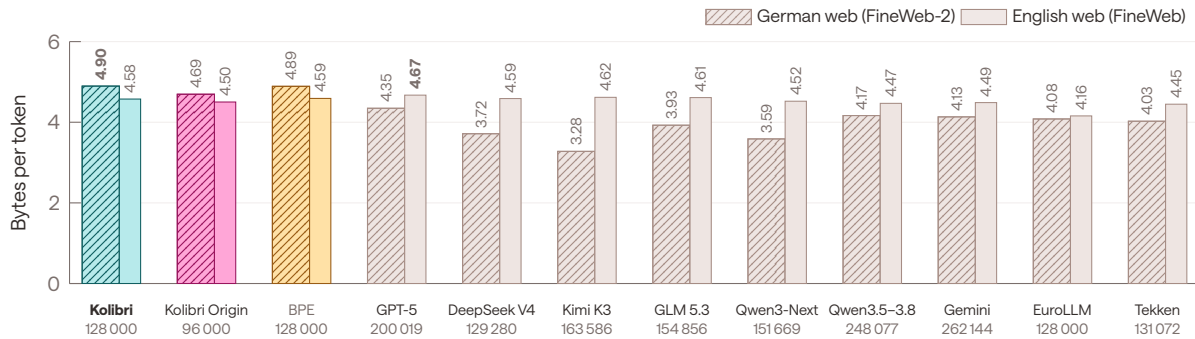


Figure 5. Compression of the Kolibri tokenisers and other tokenisers on German text from FineWeb-2 (Penedo, Kydlíček, Sabolčec, et al. 2025) and English text from FineWeb (Penedo, Kydlíček, Ben Allal, et al. 2024). The compression is given in UTF-8 bytes per token (higher is better); the wide hatched bars are German, the narrow ones English. The number under each name denotes the vocabulary size. BPE is plain Byte Pair Encoding trained on the same corpus with the same vocabulary size as Kolibri. *Tekken* is Mistral’s tokeniser, also used by Apertus and Nemotron, and *Gemini* the tokeniser shared by Gemma 4 and Gemini (Gemma Team 2026).

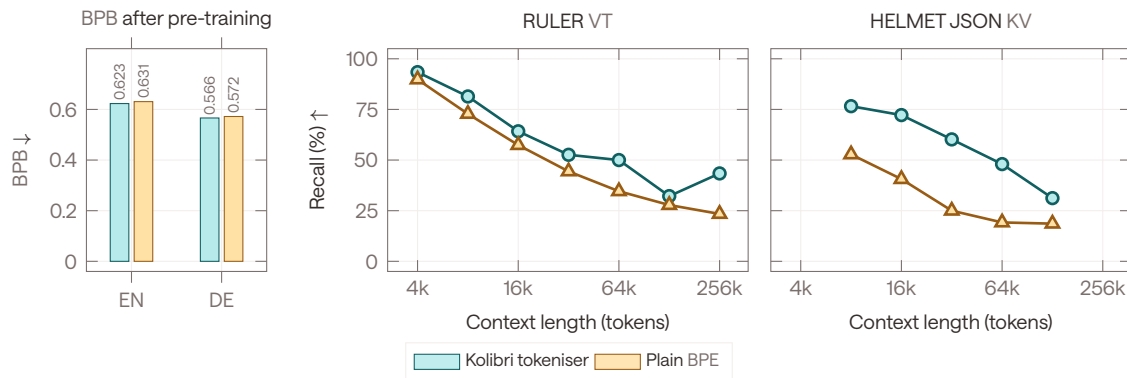


Figure 6. Proxy at scale M trained with the Kolibri tokeniser and with plain BPE of the same vocabulary size. Left: BPB averaged over the English and German pre-training benchmarks after pre-training. Middle and right: recall by context length after the long-context stage, the recall of assigned variables for RULER VT and the ROUGE-L recall of the retrieved value for HELMET JSON KV.

more letters `ing`, `the`, `the`, `and`, `ion`, `tion`, `for`, `and`, `ation`, `pro`, `that`, `con`, `ent`, `ment`, `with`, `ther` and the German `und`, `ung`, `ver`, and the German characters `ü`, `ä`, `ö`, `ß`. BPE has no German units or characters, and only five units of three or more letters, `the`, `ing`, `ion`, `ent` and `and` by merge 60.

We quantify this morphological alignment in Appendix B.1.1. Kolibri creates splits at morpheme boundaries with a precision of 65 % in German and 50 % in English, compared to 49 % and 42 % respectively for the best of the external tokenisers.

**Downstream Model Performance** We ran a Kolibri tokeniser vs BPE ablation at scale M (see Table 3). For both training runs, Figure 6 compares the Bits per Byte (BPB) after pre-training and the recall on two retrieval tasks after the long-context stage. Both runs reach the same BPB in English and German, so the two tokenisers are matched in language-modelling quality, while the Kolibri tokeniser improves recall on long-context Variable Tracking (VT) and KV retrieval at every length.

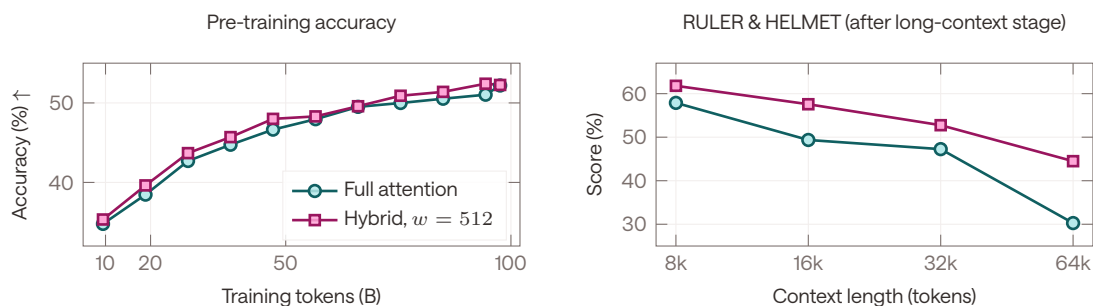


Figure 7. Average accuracy along the pre-training stage (left) and long-context score after a 5B long-context stage (right). We evaluate a model with full attention in every block and a hybrid model with  $w = 512$ , both FLOP-matched at proxy scale M (Table 3). Full attention uses 14 heads per block and pre-trains on sequences of 4k tokens, the hybrid on sequences of 16k tokens. During the long-context stage, the full-attention model is trained on sequences of 64k tokens with YaRN (Peng et al. 2024), while the hybrid model is trained over sequences of 256k tokens (without any RoPE extension). Left: log-likelihood accuracy averaged over ARC, HellaSwag and MMLU in English and German at raw pre-training checkpoints. Right: Mean average of RULER score (NIAH, VT, WE and QA) and the HELMET score (JSON-KV and RAG-QA), up to the 64k tokens full attention trained on.

## 2.1.3 Hybrid Sliding-Window Attention

One target of Kolibri is to improve long-context inference efficiency compared to Kolibri Origin which uses full attention in every block: each token attends to all preceding tokens, so the compute grows quadratically with the sequence length in all layers. During inference, the KV cache grows linearly with the sequence length, leading to increased memory usage and computational cost. A mitigation is to use a hybrid of full attention and SWA (Beltagy, Peters, and Cohan 2020) layers, which restrict attention to a fixed-size window of preceding tokens, reducing both memory usage and computational cost. We integrate SWA into Kolibri, increasing long-context inference efficiency and benchmark performance.

**Hybrid Attention** Kolibri uses full attention in every fifth block (10 of 50) and SWA in the other 40. First experiments at proxy scale M (Table 3) showed improvements in long-context tasks without a loss on pre-training benchmarks (Figure 7). At matched training compute, the hybrid model supports sequences four times as long as the full-attention model. Moreover, Figure 9 (right) shows that the hybrid model is generally more efficient during inference.

**Positional Encoding** The SWA blocks apply RoPE (Jianlin Su et al. 2024) with base  $\theta = 10\,000$  to queries and keys; the full-attention blocks use No Positional Encoding (NoPE). In early experiments, removing RoPE from the full-attention blocks improved long-context performance, in line with B. Yang et al. (2025).

**QK Normalisation** Every block normalises queries and keys per head with RMS normalisation before RoPE (Henry et al. 2020; B. Zhang and Sennrich 2019) which bounds the attention logits and keeps training stable at higher learning rates. We refer to Appendix B.2 for an analysis of QK normalisation effects. During RL, assuming sufficiently small gain parameters, the QK normalisation also bounds every query and key element below the FP8 range, so the FP8 KV cache cannot overflow (Appendix I.4.1).

| Window size $w$ | SWA Q/KV heads | Experts | Expert hidden | TFLOPs per seq. | KV MiB per seq. | Total params | Active params | Accuracy (%) $\uparrow$ | BPB $\downarrow$ |
|-----------------|----------------|---------|---------------|-----------------|-----------------|--------------|---------------|-------------------------|------------------|
| 128             | 32/32          | 144     | 320           | 61.3            | 400             | 7.8B         | 1.2B          | 35.2                    | 0.641            |
| 256             | 32/32          | 144     | 320           | 62.7            | 480             | 7.8B         | 1.2B          | <b>35.8</b>             | <b>0.637</b>     |
| 512             | 32/16          | 144     | 320           | 60.0            | 480             | 7.6B         | 1.0B          | 35.5                    | 0.647            |
| 1024            | 24/8           | 96      | 512           | 63.7            | 480             | 7.9B         | 1.0B          | 35.3                    | 0.643            |
| 2048            | 16/4           | 96      | 512           | 62.3            | 480             | 7.8B         | 0.9B          | 34.9                    | 0.650            |
| 4096            | 16/2           | 256     | 192           | 61.8            | 480             | 7.7B         | 0.6B          | 35.2                    | 0.642            |

Table 4. Resource-matched window size sweep at scale M with adapted SWA heads, routed experts and expert width that match the forward FLOPs and KV cache per sequence of 16k tokens and the total parameters across windows. The configuration at window size 128 uses a smaller KV cache because an exact match would require an impractically large number of KV heads. *Accuracy* denotes the average of 15 accuracy metrics, one per benchmark, at the last pre-training checkpoint (higher is better). *BPB* denotes the average of 13 BPB metrics at the last pre-training checkpoint (lower is better).

### 2.1.3.1 Window Size

We select the window size  $w^* \in \{128, 256, \dots, 4096\}$  by running ablations at scale M and validating the best two variants at scale L (Table 3).

After the pre-training stage, we add a long-context stage of 5B tokens on sequences of 256k tokens. We evaluate the pre-training benchmarks at the last pre-training checkpoint and the long-context benchmarks after the long-context stage. Across windows, we approximately match total FLOPs, KV-cache size and parameter count (Table 4).

**Ablations at Scale M** We select the window for long-context score and inference efficiency, and also compare the scores on the pre-training benchmarks.

Table 4 also reports the average accuracy and BPB of our pre-training benchmarks at the last pre-training checkpoint. While the scores are generally similar across window sizes, the configuration with window size 256 achieves the highest scores.

Figure 8 shows the pooled long-context score for each window size configuration depending on the context length. At scale M,  $w = 128$  yields the highest pooled score and leads from 8k to 64k tokens, but degrades at context length 128k.  $w = 512$  follows closely and degrades the least with context length.

**Validation at Scale L** Based on the ablations at scale M, we validate  $w = 128$  and  $w = 512$  at scale L (Table 3). Figure 8 also reports the pooled long-context score for these models. Their average accuracies on the pre-training benchmarks are 48.0 % for  $w = 128$  and 47.8 % for  $w = 512$ , while their pooled long-context scores are 76.2 % and 77.1 %. Through 128k tokens, the two windows are generally close, with the largest absolute difference being 3.1 percentage points. At context length 256k, however,  $w = 512$  scores 6.7 percentage points higher than  $w = 128$ .

We therefore use a sliding window of  $w = 512$  in Kolibri, at the cost of 0.2 pp of accuracy on the pre-training benchmarks.

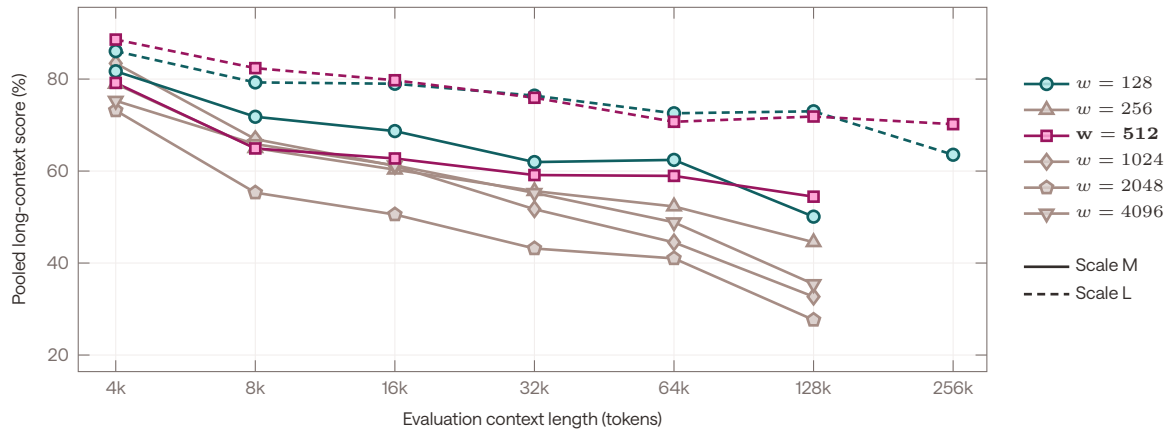


Figure 8. Pooled long-context scores by window size  $w$  after the long-context stage, in the resource-matched sweep at scale M (solid) and in the validation at scale L (dashed). Each point denotes the average of RULER NIAH, VT, WE and QA, and the HELMET JSON-KV and RAG-QA F1-scores evaluated at that length. At the longest evaluated context, window size 512 leads both sweeps. Axis labels use  $k = 1024$  tokens.

## 2.1.4 Scaling Sparsity

Sparse MoE architectures are able to increase model capacity at matched compute by routing each token through a subset of experts. We measure their sparsity as the share of the parameters that is active for a token. At fixed expert width and routing top- $k$ , adding experts increases the total parameter count while leaving the number of FLOPs per token approximately unchanged. A larger total parameter count increases parameter storage, communication volume and optimiser states, and makes expert balance harder to maintain. Downstream performance, however, improves substantially with total capacity (A. Clark et al. 2022; Frantar et al. 2024; Ludziejewski et al. 2024; Abnar et al. 2025; Tian, K. Chen, et al. 2025; Kimi Team 2025), a trend we confirm in our own ablations (Figure 9). We compare configurations of different sparsity by their quality, inference efficiency and deployment cost.

**Quality** We vary sparsity by increasing the number of routed experts from 128 to 256 and 384 at 3.4B active parameters, which gives 42.1B, 82.3B and 122.6B total parameters.

Moving from 42.1B to 82.3B improves loss and aggregate eval accuracy (Figure 9). The 122.6B configuration improves loss and aggregate eval accuracy only slightly further. The load balancing of Section 2.1.5 keeps the expert loads well balanced in all three configurations, including the 122.6B one. A preliminary sweep at 0.9B active parameters shows the same ordering in loss and accuracy. Appendix B.4 shows the loss and accuracy of both sweeps and the deployment footprint of each configuration.

**Inference Efficiency** We focus on the inference regime that is most interesting for the RL phase, where most of the runtime is spent decoding rollouts. We therefore compare the configurations by their decode throughput: the tokens generated per second, summed over all concurrent requests. Since the decode phase is mainly memory-bound, increasing total parameters can increase inference cost through greater parameter storage and weight movement, as well as additional communication overhead. To capture these effects, we measure this throughput for the three configurations with 3.4B active parameters on 8 H100

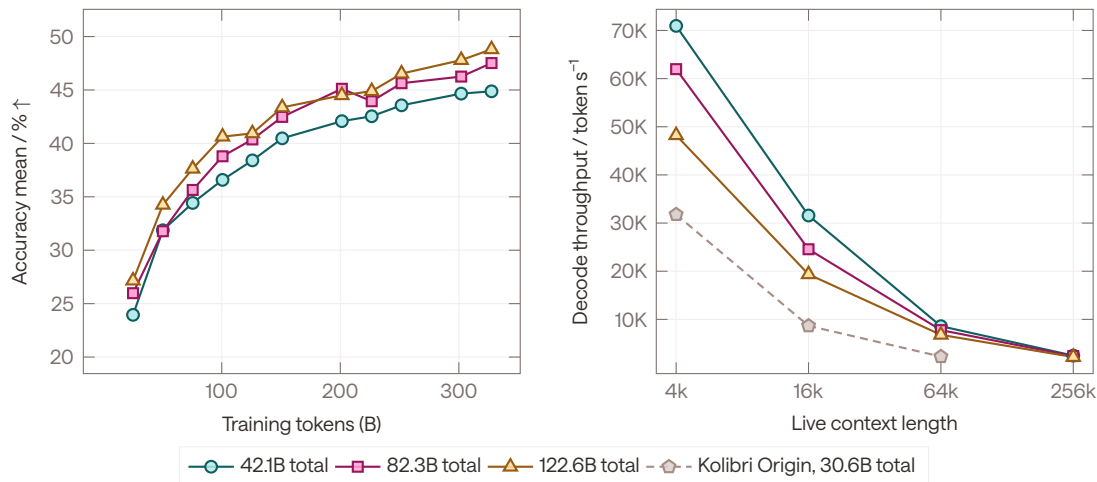


Figure 9. Accuracy mean during pre-training (left) and maximum pure-decode throughput by live context length (right) of three configurations with 3.4B active parameters and different total parameters. Accuracy averages the 17 benchmarks of Appendix B.3. Throughput is measured on eight H100 SXM5 GPUs in BF16, with the best tested parallel configuration for each model and context length. Kolibri Origin is shown for reference over its supported context range.

SXM5 GPUs in BF16, using for each model and context length the best tensor-, data- and expert-parallel configuration identified through a combinatorial search (Figure 9).

The main sources of memory traffic are the active model weights and the KV cache. Weight reads increase with the total parameter count, although they can be amortised across tokens routed to the same experts within a large batch. KV-cache reads, by contrast, remain specific to each request and grow with its live context. At 4K context length, where weight reads account for a larger share of memory traffic, the 82.3B and 122.6B configurations are 13 % and 32 % slower than the 42.1B configuration. At 256k, the gaps narrow to 4 % and 12 % as KV-cache reads account for a larger share of total memory traffic. Most of this long-context traffic comes from the 10 full-attention layers, while the 40 sliding-window layers read a fixed window and contribute negligibly.

**Deployment Footprint** In resource-constrained deployments, total parameter count directly reduces the memory available for activations and KV cache, limiting request concurrency at long context lengths (Table 31). For example, on two H100s, the 122.6B configuration fits only 3 concurrent 256k-token requests, compared with 18 for 82.3B.

The 82.3B configuration improves all reported quality metrics over 42.1B, while increasing capacity to 122.6B yields only a small further improvement, reduces decode throughput and fits far fewer long requests. We therefore target 80B total parameters at 3.4B active parameters. With the total size fixed, further ablations set the number and width of the experts and the other shape parameters. Appendix B.5.1 compares three expert configurations at this size. Informed by a preliminary sweep at 0.9B active parameters, we choose to widen the residual stream (Appendix B.5.2).

## 2.1.5 MoE Load Balancing

An MoE layer combines a shared expert, evaluated for every token, with  $E$  routed experts, of which only  $K$  are selected per token. For  $T$  tokens and  $E$  routed experts, each MoE layer computes router logits  $Z \in \mathbb{R}^{T \times E}$ . Expert biases  $\beta \in \mathbb{R}^E$  are added to the router logits  $Z$  to produce the biased logits  $B$  used for balanced expert selection:

$$B = Z + \mathbf{1}_T \beta^\top, \quad \mathcal{I}_t = \text{Top-K}(B_{t,:}, K), \quad S_{t,e} = \text{sigmoid}(Z_{t,e}). \quad (3)$$

Here  $B_{t,:}$  denotes the row of  $B$  for token  $t$ , and  $S$  denotes the routing scores that weight the outputs of the selected experts;  $\mathbf{1}_T$  is the all-ones column vector, and  $\text{Top-K}(x, K)$  returns the indices of the  $K$  largest entries of  $x$ . For an input representation  $x_t$ , the layer output is

$$y_t = f_{\text{shared}}(x_t) + \sum_{e \in \mathcal{I}_t} S_{t,e} f_e(x_t), \quad (4)$$

where  $f_{\text{shared}}$  and  $f_e$  are the shared and routed expert networks. The selected scores are not renormalised, and the biases affect selection only; the mixture weights use the unbiased sigmoid scores.

Load balancing ensures sufficient learning signal for every expert and efficient expert-parallel execution. The former requires balance across the global optimiser-step batch; the latter requires balance within each local microbatch. For either batch  $\mathcal{B}$ , we measure the maximum relative excess load by MaxVio (DeepSeek-AI 2024b):

$$\text{MaxVio}(\mathcal{B}) = \max_e \left[ \frac{c_e(\mathcal{B})}{K|\mathcal{B}|/E} - 1 \right], \quad (5)$$

where  $c_e(\mathcal{B})$  counts the tokens assigned to expert  $e$ .

Balancing methods include auxiliary losses (Lepikhin et al. 2021) and expert-bias updates, either with a fixed step (DeepSeek-AI 2024b) or based on quantiles (Kimi Team 2026b).

Kolibri combines Exact Quantile Balancing (EQB) for global balance with Load-Error Injection (LEI) for local balance (Neitemeier et al. 2026). EQB maintains stable global load balance without a tunable balancing hyperparameter. The LEI coefficient therefore primarily controls local balance and training efficiency, giving the recipe fewer and less sensitive hyperparameter choices than the recipe of Kolibri Origin, which combines SMEBU bias updates (V. Singh et al. 2026) with a GShard-style auxiliary loss (Lepikhin et al. 2021). Over the Kolibri pre-training run, the global MaxVio averaged over layers stays at 0.31. Only the first two layers, whose routing the bias overrides (Section 2.1.5.3), reach 2.39 on average.

### 2.1.5.1 Exact Quantile Balancing (EQB)

Quantile Balancing (QB) (Kimi Team 2026b) computes token biases  $\alpha \in \mathbb{R}^T$  and expert biases  $\beta$  through alternating quantile updates:

$$\alpha_t \leftarrow -Q_K(Z_{t,:} + \beta^\top), \quad \beta_e \leftarrow -Q_r(Z_{:,e} + \alpha), \quad r = \frac{TK}{E}, \quad (6)$$

where  $Q_m$  denotes the  $(m + 1)$ th largest entry. Subtracting this cutoff leaves  $m$  positive entries, so the token bias update targets  $K$  assignments per token and the expert bias update  $r$  assignments per expert.

Training performs one token-bias update followed by one expert-bias update per optimiser step. The resulting expert biases  $\beta$  are centred by subtracting their mean and we use the top- $K$  of the biased logits for routing. Using top- $K$  per token approximates the token biases computed by the first step of the QB iteration during training and inference.

The global batch is partitioned across data-parallel ranks and gradient-accumulation microbatches. Each rank holds the router scores  $Z_{t,:}$  for all experts for its local tokens, together with the shared expert biases  $\beta$ . The token bias  $\alpha_t$  can therefore be computed locally from  $Z_{t,:} + \beta^\top$ . Updating an expert bias  $\beta_e$ , however, requires a quantile of  $Z_{:,e} + \alpha$  across all tokens in the global batch. These values are distributed across ranks and microbatches, so their statistics must be combined.

Marin (Dial 2026) averages quantiles computed separately on each rank, which generally differs from the quantile of the combined batch. Kimi (Kimi Team 2026b) accumulates a 1000-bin histogram per expert over local microbatches and sums the counts across ranks with an all-reduce. The resulting histogram represents the global batch, but its quantile estimate is limited by the bin width.

EQB (Neitemeier et al. 2026) computes this global quantile exactly for bfloat16 adjusted logits using two-pass radix selection (Yifei Li et al. 2024). Each rank encodes its local values as order-preserving 16 bit keys. In the first pass, we build 256-bin histograms over the high byte and sum the counts across microbatches to locate the bin containing the global quantile. In the second pass, we repeat this for the histogram of the low byte of values in that bin to recover the exact quantile. Each layer requires two all-reduces of  $256E$  int32 counts per optimiser step, independent of  $T$ .

### 2.1.5.2 Load-Error Injection (LEI)

A shared global bias cannot balance all microbatches individually. A GShard-style auxiliary loss is a common choice for local balancing, originally formulated for normalised softmax probabilities (Lepikhin et al. 2021). With sigmoid scores, models such as DeepSeek-V3 and Arcee Trinity add a separate normalisation for this loss (DeepSeek-AI 2024b; V. Singh et al. 2026). We introduce LEI to balance each microbatch, which directly injects each expert’s deviation from its target into the score gradients. Neitemeier et al. (2026, App. D) derive the LEI update as the straight-through estimate for unnormalised sigmoid scores, while the GShard loss is the corresponding estimate for normalised softmax probabilities. For a local microbatch  $\mathcal{B}_1$ , with  $c_e(\mathcal{B}_1)$  assignments to expert  $e$ , the relative load and relative load error are

$$f_e = \frac{c_e(\mathcal{B}_1)}{K|\mathcal{B}_1|/E}, \quad \rho_e = f_e - 1. \quad (7)$$

We inject a rescaled version of the load-error vector into the sigmoid-score gradient of every token in the microbatch:

$$G_{t,e} \leftarrow G_{t,e} + \lambda \frac{\tanh \xi}{\xi} \rho_e, \quad \xi = \frac{\|\rho\|_\infty}{\tau}, \quad t \in \mathcal{B}_1. \quad (8)$$

Here  $G_{t,e}$  is the incoming gradient with respect to  $S_{t,e}$ ,  $\lambda > 0$  controls the correction strength, and  $\tau > 0$  bounds the rescaled load error, which we found to help training stability. At  $\xi = 0$ , the scaling factor is 1 by continuity. Kolibri uses  $\lambda = 1 \times 10^{-5}$  and  $\tau = 1$ .

| Dense blocks | Loss ↓       | BPB ↓        |              |              | Accuracy (%) ↑ |             |             | Dead (%) ↓  |
|--------------|--------------|--------------|--------------|--------------|----------------|-------------|-------------|-------------|
|              |              | Mean         | EN           | DE           | Mean           | EN          | DE          |             |
| None         | <b>1.847</b> | <b>0.486</b> | <b>0.491</b> | <b>0.466</b> | <b>38.0</b>    | <b>37.3</b> | <b>40.8</b> | <b>0.46</b> |
| Two          | 1.850        | 0.491        | 0.496        | 0.470        | 36.9           | 36.3        | 39.5        | 0.52        |

Table 5. Leading dense blocks in the Kolibri architecture, at the last checkpoint. Both models train for 327.6B tokens. Loss is the training loss averaged over the last 2B tokens, and Dead the effective dead-expert fraction (Equation (20)) of the most affected layer at the last step; the best value of each column is bold.

### 2.1.5.3 Leading Dense Layers

Unlike Kolibri Origin, which has dense FFNs in its first two blocks, Kolibri uses MoE layers in all 50 blocks. This is motivated by an ablation of the Kolibri architecture over the 327.6B-token budget of scale L that found slightly lower loss and BPB and higher accuracy when replacing leading dense blocks with MoE layers (Table 5). Diagnostics of the full Kolibri run, however, show that while the expert loads are balanced, its first two MoE layers make almost no use of their routed experts. Appendix B.6 gives the routing diagnostics of the full run and an analysis of the first two MoE layers.

## 2.1.6 Infrastructure

Pre-training ran on 768 NVIDIA B200 GPUs, across 96 nodes with 8 GPUs per node. The cluster has an NVIDIA SuperPOD™ topology. Within each node, the GPUs share an NVLink™ Generation 5 interconnect with  $1.8 \text{ TB s}^{-1}$  bidirectional bitrate. The nodes are connected via InfiniBand by 8 NVIDIA ConnectX-7® network adapters per node, so each GPU can sustain a cross-node bidirectional bitrate of  $100 \text{ GB s}^{-1}$ .

Our parallelism setup for pre-training is EP 8, FSDP 16, DP 6. With this, we make use of the hierarchical interconnect in pre-training: Expert parallel across the 8 GPUs within each node and (fully-sharded) data parallel across nodes. Thus, dense parameters are sharded across  $\text{EP} \times \text{FSDP} = 128$  GPUs (one full replica), while sparse expert parameters are sharded across  $\text{FSDP} = 16$  GPUs—the same local GPU on each node in one full replica. Mid-training and long-context extension use FSDP 128, DP 6.

We leverage several existing libraries, namely PyTorch 2.14 (Ansel et al. 2024), DeepEP v2 (C. Zhao et al. 2025) for expert parallelism, and FlashAttention 4 (Zadouri et al. 2026) for both SWA and full attention computations. Ablations without EP use SonicMoE (W. Guo et al. 2025) as an efficient MoE implementation.

For further details, see Appendix B.7, including Appendix B.7.1 on straggler detection.

m

## 2.2 Optimisation

Kolibri is trained with a Warmup-Stable-Merge (WSM) recipe (Tian, J. Wang, et al. 2025). The learning rate warms up linearly over the first 100B tokens and then stays constant through pre-training, mid-training

and long-context training. In our experiments, a constant learning rate with checkpoint merging improves post-trained performance over a learning-rate cooldown (Maskey et al. 2026). Each stage (except pre-training) continues with the optimiser state after finishing the previous one and therefore needs no further warmup. We then determine the base model weights as the average of the last 20 checkpoints of the long-context stage (Yunshui Li et al. 2025), which are 10B tokens apart. Over the three stages the context length grows from 16k to 256k tokens and the token batch from 75.5M to 201M tokens per step (Figure 3). The architecture improvements, particularly the hybrid of sliding-window and full attention, enable longer training sequences than in Kolibri Origin, which pre-trains at 4k tokens and reaches 64k tokens only in its long-context stage (Sections 2.1 and 2.1.3.1). Sequences are assembled online from a Random Sequential Document Buffer (RSDB) (V. Singh et al. 2026), which spreads the chunks of a long document over many optimiser steps, by bin packing these chunks without padding tokens or unnecessary document cuts (H. Ding et al. 2024) (Appendices C.1, C.1.1 and C.1.2).

We optimise attention and expert weights with Muon (Jordan et al. 2024), embedding weights and norm scales with Adam (Kingma and Ba 2015), and LM head and router weights with AdamW (Loshchilov and Hutter 2019) (Table 6). In contrast to the commonly used convention of matching Muon’s update RMS to that of AdamW (Jingyuan Liu et al. 2025; Kimi Team 2025), we adopt the spectral convention (Bernstein 2025), which keeps the Muon learning rate independent of model width (Section 2.2.1). We use decoupled weight decay for all optimiser groups except the embedding and norm scales. Adam and AdamW use  $\epsilon = 1 \times 10^{-15}$ , small enough that the updates of parameters with small gradients, such as the LM head and the routers, stay independent of the gradient scale, as Muon updates are by construction.

Table 6 lists the optimiser, peak learning rate, weight decay and initialisation of every parameter group; load balancing is described in Section 2.1.5. Our hyperparameter optimisation and transfer scheme is described in Section 2.2.1.

| Optimiser      | Peak learning rate   | Weight decay | Parameters                                                             | Initialisation                       |
|----------------|----------------------|--------------|------------------------------------------------------------------------|--------------------------------------|
| Muon, spectral | $1 \times 10^{-3}$   | $2^{-12}$    | Q, K, V; expert up and gate<br>Attention output; expert down           | std $1/\sqrt{d_{\text{in}}}$<br>zero |
| AdamW          | $2.5 \times 10^{-5}$ | $2^{-12}$    | Router gates<br>LM head                                                | std $1/d_{\text{in}}$<br>zero        |
| Adam           | $2 \times 10^{-3}$   | none         | Token embedding<br>Norm scales<br>Post-sublayer (sandwich) norm scales | std 1<br>1<br>$1/\sqrt{L}$           |

Table 6. Optimiser and initialisation of Kolibri by parameter group. Muon uses momentum  $\mu = 0.95$  with Nesterov and spectral scaling, which multiplies the orthogonalised update by  $\sqrt{d_{\text{out}}/d_{\text{in}}}$ ; Adam and AdamW use  $\beta_1 = 0.9$ ,  $\beta_2 = 0.95$  and  $\epsilon = 1 \times 10^{-15}$ . Decoupled weight decay multiplies the decayed weights by  $1 - 2^{-12}$  in every step, independent of the learning rate. Normal initialisations are truncated at three standard deviations and rescaled to the stated standard deviation;  $d_{\text{in}}$  is the fan-in of a layer,  $L = 50$  the number of layers, and expert biases start at zero.

## 2.2.1 Hyperparameter Transfer

The optimal hyperparameters depend on model size and token horizon (G. Yang, E. J. Hu, et al. 2021; McCandlish et al. 2018; Bjorck et al. 2024; Porian et al. 2024; Bergsma et al. 2025), but a sweep at the

scale of the target run is infeasible. Proxy runs on smaller models and shorter horizons therefore guide the choice of hyperparameters for the full-scale run. A common approach fits scaling laws that extrapolate along both dimensions jointly (DeepSeek-AI 2024a; H. Li et al. 2025; Porian et al. 2024; Poolside 2026), which requires sweeps at many points of a 2D grid over model size and horizon. Instead, we factorise the transfer across model size and token horizon, and vary model size only through the width, at the 50 layers of the target model (Section 2.2.1.2). For the width transfer we use Maximal Update Parametrisation ( $\mu P$ ) (G. Yang, E. J. Hu, et al. 2021; G. Yang, Simon, and Bernstein 2024), which makes the optimal base learning rate of every parameter group approximately width-independent, so that hyperparameters found on narrower models carry over to the full-scale model (Dey et al. 2025; Meituan LongCat Team 2025). The dependence on the token horizon can then be studied at a single narrower width. Instead of scaling laws for the optimal learning rate and batch (DeepSeek-AI 2024a; H. Li et al. 2025; Bjorck et al. 2024; Bergsma et al. 2025), we use loss extrapolation (Domhan, Springenberg, and Hutter 2015) to predict the learning-rate landscape at the target horizon and pick the best cell of a grid of Muon learning rates and batch sizes on a narrower proxy (Section 2.2.1.3).

The remainder of this section fixes the learning-rate ratios between the parameter groups (Section 2.2.1.1), introduces the width transfer with  $\mu P$  (Section 2.2.1.2) and the horizon transfer by loss extrapolation (Section 2.2.1.3) for learning rate, batch size and weight decay.

### 2.2.1.1 Learning-Rate Ratios

We find that different parameter groups require different learning-rate scales to achieve optimal performance. To avoid an explosion of the grid in later sweeps we optimise the ratios between the learning rates of the different embedding, norm and head parameter groups as a factor of the Muon learning rate in a preliminary sweep. In internal sweeps, we find these ratios to be relatively stable across widths and token horizons, and therefore assume that they can be held fixed when transferring hyperparameters. The router learning rate is set equal to the head learning rate, based on internal ablations. The resulting production learning rates are listed in Table 6.

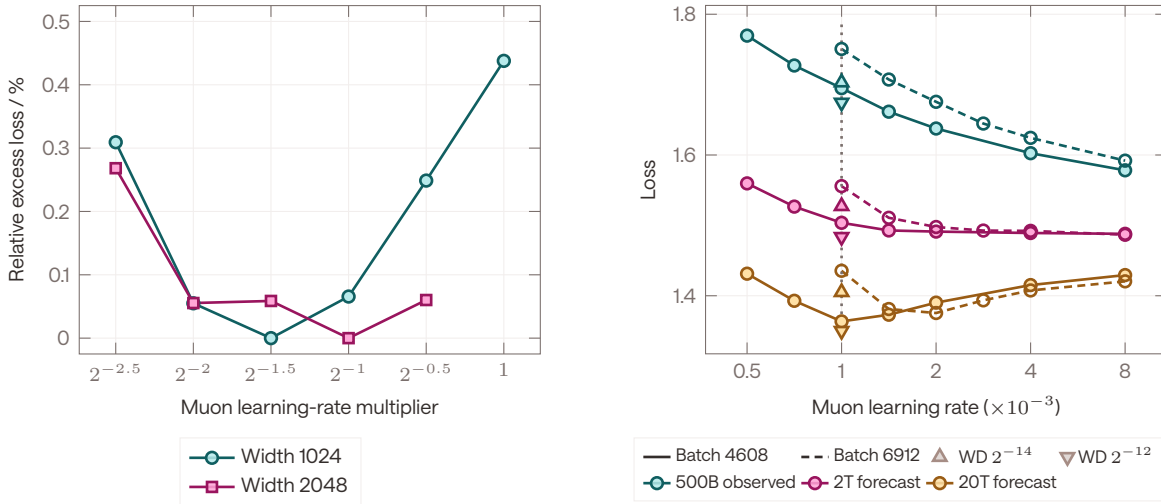
### 2.2.1.2 Transfer Across Width

While there is work on joint width and depth transfer with  $\mu P$  (Bordelon et al. 2023; G. Yang, D. Yu, et al. 2023; Dey et al. 2025), it is less established and introduces additional complexity, and we therefore restrict ourselves to width transfer only. We keep the number of experts and top- $k$  fixed and scale the size of every expert, so that the routing and load-balancing dynamics of the proxy match the production model. Attention heads keep their dimension of 128 and we only scale the number of heads, as in  $\mu P$  (G. Yang, E. J. Hu, et al. 2021). We assume the optimal batch size to be only dependent on token horizon (H. Li et al. 2025).

As described in Section 2.2, Muon runs in the spectral convention: after the Newton–Schulz orthogonalisation the update is scaled by  $\sqrt{d_{\text{out}}/d_{\text{in}}}$ , so that every layer receives an update of unit RMS operator norm times the learning rate regardless of its shape (Bernstein 2025). This ensures that the optimisation dynamics satisfy the spectral condition (G. Yang, Simon, and Bernstein 2024), and makes the Muon learn-

ing rate width-independent. The learning rates of embedding and norm scales are width-independent as well, whereas those of the LM head and router scale with  $1/d$ .  $\mu P$  also shrinks the initialisation of the output layers with width. We initialise the LM head at the infinite-width limit of this scaling, zero, which removes the width dependence of the initial outputs and brings the optimal hyperparameters of narrow and wide models closer together (G. Yang, E. J. Hu, et al. 2021, App. D.2). The attention output and expert down projections also start at zero, so that every block starts as the identity map (Hongyi Zhang, Dauphin, and T. Ma 2019); in our ablations this converged faster and still transfers across width (Table 6). The router is initialised with standard deviation  $1/d_{\text{in}}$  (Vankadara et al. 2026).

To validate the transfer we swept the base learning rate on the proxies M and L (Table 3), where L is M scaled to twice the width, both trained for 97.6B tokens, the horizon of M, with a batch of 576 sequences of 16k tokens and with learning rates scaled according to the rules described above. They are not scaled down from the production model, since the width sweep is an earlier ablation. Results are visualised in Figure 10a, with the loss averaged over the trailing 3B tokens of each run. We see a shift of the optimal learning rate by  $\sqrt{2}$ , which we deem acceptable. The loss differences within a  $\sqrt{2}$  learning rate factor of the optimum are less than 0.1%.



(a) Across width: Relative excess training loss at 97.6B tokens above that of the best learning rate of each width, for proxies of width 1024 and 2048. The optima differ by one grid step, a factor of  $\sqrt{2}$ , and the wider model is flatter around its optimum.

(b) Across the training horizon: observed loss of the width-scaled proxy (Table 7) at 500B tokens and forecasts at 2T and 20T tokens, for batches of 4608 (solid) and 6912 (dashed) sequences. With increased target horizon, the optimum moves from the largest learning rate to the production value of  $1 \times 10^{-3}$  (dotted) at 20T tokens; the larger batch is best at  $2 \times 10^{-3}$ , 0.9% behind. Triangles indicate halved  $2^{-14}$  (up), and doubled weight decay,  $2^{-12}$  (down), relative to the  $2^{-13}$  baseline used for the remaining sweep points.

Figure 10. Learning-rate transfer across width and training horizon.

### 2.2.1.3 Transfer Across Token Horizon

Transferring the optimal learning rate and batch size to a target token horizon is often done via scaling laws (DeepSeek-AI 2024a; H. Li et al. 2025; Bjorck et al. 2024; Porian et al. 2024). These necessitate a large grid of values to have enough points for a reliable power-law fit, making them expensive. Additionally, they

complicate transferring other hyperparameters such as weight decay from short to long horizons, as this would combinatorially explode the grid.

For Kolibri Origin, we fit power laws to the optimal learning rate and batch size at horizons from 100B to 430B tokens and extrapolate them to its pre-training budget of 7.5T tokens. Extrapolating those laws to the Kolibri setting gives unrealistic hyperparameter predictions, motivating a method that uses the observed loss trajectories of the candidate configurations.

We employ loss extrapolation (Domhan, Springenberg, and Hutter 2015) to predict the hyperparameters with the lowest loss at longer token horizons based on the observed training dynamics at shorter horizons. This allows us to use smaller grids for the search and employ coordinate descent to tune further hyperparameters iteratively. Since the sweep uses the batch size, warmup, learning-rate schedule and data of the production run on the width-scaled proxy (Table 7), it doubles as a test of the production configuration, and its runs are reused for the weight-decay sweep described below.

We use a power law with floor (Hoffmann et al. 2022) in the cumulative-learning-rate coordinate (Tissue, V. Wang, and L. Wang 2025),

$$L(D) = E + A(D - W/2)^{-\alpha}, \quad (9)$$

where  $D$  is the number of tokens,  $W$  the length of the linear warmup in tokens,  $E$  the irreducible loss, and  $A$  and  $\alpha$  the scale and exponent of the reducible part. After a linear warmup of  $W$  tokens at a constant learning rate, the cumulative learning rate is proportional to  $D - W/2$ . We validated the method on internal loss curves, not reported here, and find an average prediction error at the target token horizon below 1%. All tested methods are fairly sensitive to the fit start point. For the method used, we discard the first 20 % of the post-warmup prefix before fitting. We find that the optimal grid point is robust to the specific form of the power law used for extrapolation: we cross-checked with a shifted power law with floor (Domhan, Springenberg, and Hutter 2015) and a one-break broken power law (Caballero et al. 2023) and obtained the same ranking.

**Learning Rate and Batch Size** We sweep a grid of Muon learning rates from  $5 \times 10^{-4}$  to  $8 \times 10^{-3}$  and two batch sizes on the width-scaled proxy model (Table 7), which halves the width of the production model with the same rules as in Section 2.2.1.2, for 500B tokens. The observed loss is averaged over the trailing 2B tokens of each run and, at 500B tokens, is still lowest at the largest learning rate for both batches. The prediction at 20T tokens is visualised in Figure 10b, with an optimal Muon learning rate of  $1 \times 10^{-3}$  and a batch size of 4608 sequences of 16k tokens each. The selected batch is the smaller of the two, so while a further reduction in batch size might be beneficial, we prioritise maintaining training efficiency.

|         | Width | Heads / KV heads | Expert hidden | Experts, top- $k$ | Layers |
|---------|-------|------------------|---------------|-------------------|--------|
| Kolibri | 2560  | 48 / 4           | 512           | 384, 6            | 50     |
| Proxy   | 1280  | 24 / 2           | 256           | 384, 6            | 50     |

Table 7. Production model and the width-scaled proxy used for the loss-extrapolation sweeps over learning rate, batch and weight decay. Layers, experts, head dimension and the attention pattern are shared.

**Weight Decay** We use the same loss extrapolation to set the weight decay in a one-dimensional sweep. Starting from the selected learning rate and batch size, we vary the weight decay and compare the predicted loss at 20T tokens (triangles in Figure 10b). The baseline cell was trained with a weight decay of  $2^{-13}$ . Doubling the weight decay to  $2^{-12}$  lowers both the observed loss and the forecasts at every horizon, whereas halving it raises them. All estimators agree on this order, and we use  $2^{-12}$  for Kolibri. Larger values were not tested.

**Limitations** This optimisation is done purely with respect to the training loss under a warmup-stable learning-rate schedule. While evidence shows that a WSM schedule can lead to better post-trained performance (Maskey et al. 2026), merging and post-trained performance are not considered here. Additionally, uncertainties are not modelled and some of the differences between grid points may be within the noise level of the measurements, although all three forms of the extrapolation law select the same cell (Section 2.2.1.3). We leave modelling merged-checkpoint and post-trained performance, as well as uncertainty of the forecasts, to future iterations of the recipe.

## 2.3 Data and Training Stages

### 2.3.1 Overview

Kolibri Base is trained in three stages: pre-training, mid-training, and long-context extension. Pre-training consumes 20T tokens from a broad English-German data pool and establishes the model's general capabilities. Mid-training shifts the data distribution towards higher-quality reasoning, mathematics, code, and instruction-following data, training on 3.4T tokens. Finally, long-context extension trains on 200B tokens and extends the context length to 256k tokens.

Across these stages, data selection progressively becomes more selective and task-oriented.

Pre-training emphasises breadth and coverage, mid-training concentrates on capabilities that benefit from higher-quality and more structured data, and long-context extension adapts these capabilities to substantially longer sequences.

In addition to filtering illegal, harmful, toxic and pirated content, we redact PII from all training sources. Each category is detected by a regular expression, checked by a validator that rejects false positives, and replaced by a special token. A document whose redacted bytes exceed 15 % of its length is dropped. In one illustrative run on a mid-training data sample, 5.3 % of scanned documents received at least one redaction, with PDF data accounting for 92 % of all redactions. We report these numbers only as an example of the redaction behaviour; the sample is not representative of the overall training corpus.

The following sections describe the data construction, mixing, and evaluation choices for each stage.

### 2.3.2 Stage 1: Pre-training Data

The pre-training dataset pool consists of 78 English- and German-language datasets with approximately 29.3T materialised tokens, from which the final training mix consumes 20T. The pool spans web text, code, instruction data, scientific and technical text, and legal and public-sector data.

We group datasets into nine domains according to language, subject, and degree of structure. Datasets spanning several domains are assigned to the domain containing the majority of their tokens. Table 8 summarises the resulting pool, and Figure 11 shows each domain's share of its tokens and datasets.

Web data makes up the largest part of the pool. Its largest individual source comes from our internally developed CC pipeline, which processes and filters Common Crawl snapshots to produce more than 308B quality-scored documents for downstream data mixing.

German accounts for 21.3 % of the final pre-training tokens and is treated separately in the data construction process because its availability and filtering requirements differ from English.

Approximately 24 % (4.8T tokens) of our pre-training data is synthetic in origin (including rephrases and translations). We produced synthetic data in English, using our deduplicated English CC data (see Sec-

| Domain                            | Short     | Datasets  | Tokens (B)    |
|-----------------------------------|-----------|-----------|---------------|
| English web and documents         | EN web    | 23        | 12,248        |
| German web and documents          | DE web    | 16        | 6,712         |
| English instruction and reasoning | EN instr. | 7         | 3,548         |
| German instruction and reasoning  | DE instr. | 3         | 139           |
| Code                              | Code      | 3         | 4,323         |
| Agentic code and tool use         | Agentic   | 1         | 1             |
| STEM (documents)                  | STEM docs | 7         | 1,462         |
| STEM (QA-style)                   | STEM QA   | 9         | 789           |
| Specialized domains               | Spec.     | 9         | 80            |
| <b>Total</b>                      |           | <b>78</b> | <b>29,301</b> |

Table 8. The pre-training pool by domain, in billions of materialised tokens.

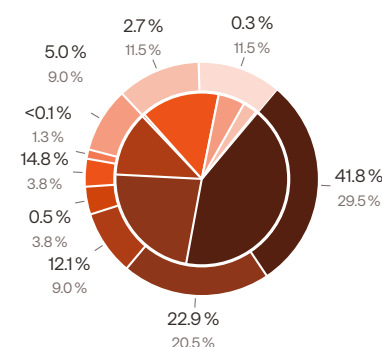


Figure 11. Share of the pre-training pool by domain, in materialised tokens (inner disk) and in datasets (outer ring), colours as in the adjacent Table 8. Each label gives the domain's share of the tokens and, below it, its share of the datasets.

tion 2.3.2.1) as seeds for rephrasing with Gemma-4-26B-A4B. We followed a Nemotron-CC-style recipe (D. Su et al. 2025), using five prompt templates applied to each seed document: diverse QA pairs, distillation, knowledge list, knowledge extraction, and encyclopedia-style rewriting. The outputs were post-processed and filtered to remove LLM-generation artefacts and erroneous generations. In Section 2.3.2.2 we describe our work on German synthetic data.

The following subsections describe key decisions around the Common Crawl (Common Crawl Foundation 2026) pipeline, the construction of the German data pool, quality annotation, and the final data-source mixing procedure in more detail.

### 2.3.2.1 Common Crawl Pipeline

The CC pipeline is designed to process Common Crawl snapshots to curate high quality pre-training data at scale. Text is extracted from the raw archives with Resiliparse (Bevendorff et al. 2018), yielding more than 308B documents.

Language-agnostic heuristic filters and language identification route each document to an English or a German branch, which apply separate thresholds. Exact deduplication removes byte-identical documents across all snapshots, which yields 21.16B distinct documents. Fuzzy deduplication, which compares MinHash signatures of documents bucketed by locality-sensitive hashing, removes a further 25.9 % as near-duplicates (Appendix D.1.3).

Substring deduplication cuts repeated blocks of at least 500 bytes, such as navigation menus and page templates, out of the remaining documents, removing 21.3 % of text bytes (Appendix D.1.4). Appendix D.1 describes the stages in more detail.

Figure 12 shows how many documents each stage of the pipeline had as an input and removed, from the extracted documents entering the heuristic filters to substring deduplication.

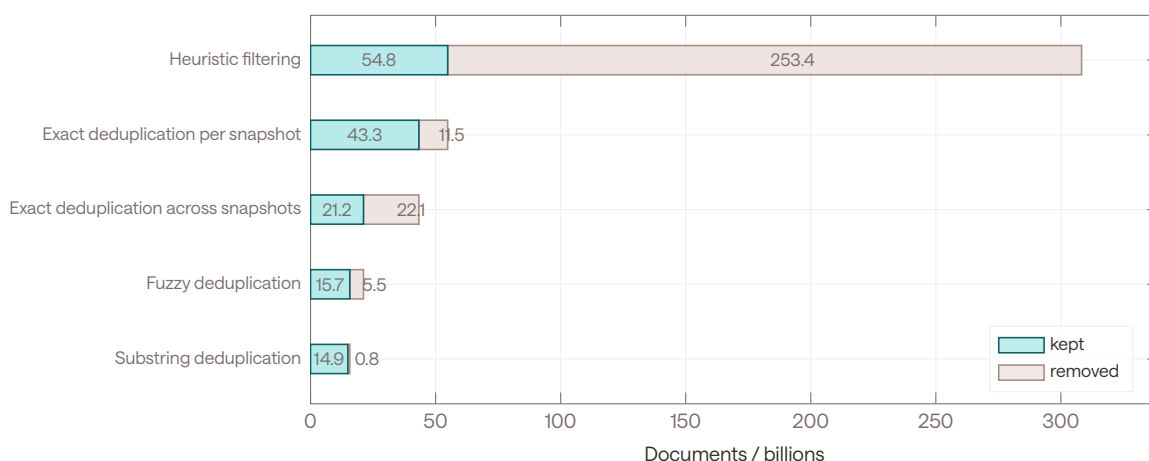


Figure 12. Documents entering each stage of the CC pipeline, split into the documents the stage kept and removed. Each stage receives what the previous one kept. Substring deduplication removes text within documents and drops a document only when nothing of it remains, so its effect on bytes is larger than on documents.

| Dataset                 | Size (B tokens) | Latest source data |
|-------------------------|-----------------|--------------------|
| Klettermix              | 500             | 2024               |
| HPLT 3.0 (German slice) | 609             | 2025               |
| FinePDFs (German slice) | 177             | 2025               |
| German Commons          | 154             | 2025               |
| MT-Nemotron-CC          | 117             | 2024               |
| FineWeb2 (German slice) | 378             | 2024               |
| Our organic web data    | 1300            | 2026               |
| Rephrased German data   | 1100            | 2025               |

Table 9. Sizes of selected German pre-training data sources in billions of tokens to illustrate German dataset scale. Note that this table does not describe our training set.

### 2.3.2.2 German Pre-Training Data

At our 20 T token budget, our ablations show strong German benchmark performance with around 20 % German data in the training mix. This leaves us with a lower bound of 4 T German tokens needed for training. Note that this refers to training tokens, not unique tokens. Since datasets are commonly upsampled during training, the underlying data pool can be smaller than 4 T, though not arbitrarily so, because repeating data can have diminishing returns after roughly four epochs (Muennighoff, Rush, et al. 2023).

To put the scale of this requirement into context, we first look at the amount of German pre-training data available in existing open datasets. Table 9 summarises several major open German datasets. These datasets are shown only as a reference point for the scale of publicly available German data and does not describe our training set. For further details on training data, refer to the Public Summary of Training Content.

The open datasets in the upper part of Table 9 add up to around 1.94T tokens. This token count does not account for filtering, deduplication, or overlap between datasets, so the amount of usable unique data would in practice be smaller. This comparison illustrates the challenge of sourcing German data at

the scale implied by our 4T-token training target. In the following, we describe the three approaches we considered to construct our German data pool.

First, we process raw Common Crawl dumps with a pipeline that is parameterised by language, since German and English web data cannot be filtered in exactly the same way. In particular, we retune several filtering parameters for German. For example, web-data pipelines commonly remove documents with unusually long words. However, German administrative prose often contains substantially longer words than English, so applying English thresholds can disproportionately remove this register. We therefore use FineWeb2’s German word-length settings, which allow a mean word length between 0 and 15 characters, instead of the 3 to 10 character range used by Gopher for English. With these language-specific adjustments, our pipeline produces 0.8T unique tokens of organic German web data.

Our second approach to augmenting our German corpus is to rephrase German documents. We use Mistral-Nemo-Instruct-2407 to rewrite organic German documents as an encyclopedia entry, a Q&A dialogue or an educational-style passage, preserving its semantic information. This teaches the model the same facts in several surface forms and multiplies scarce organic data, and it is a different operation from translation: the source is German, so the subject matter, the institutions and the register stay German. Rephrasing gave us about 1.1T unique tokens, making it the single largest source of German in the model. This data had already been produced for the Aleph-Alpha-GermanWeb project (T. F. Burns et al. 2026).

The third approach is translation. Translating English into German can expand the available data when the model, prompts, and chunking are chosen carefully, but it does not provide the same kind of German data as organically sourced text. Translated output can retain traces of the source language, such as literal renderings of idioms; for example, “Drive safe!” may become “Fahre sicher!” rather than the more natural “Komm gut an!”. More importantly, translation preserves much of the source corpus’s cultural and institutional distribution. A corpus translated from English therefore remains, in many respects, an English-language view of the world expressed in German. We experimented with translated data at scale for Kolibri Origin, but for Kolibri we relied on organic and rephrased German data alongside a carefully curated set of translated German data.

One side-effect of us building our own CC pipeline is that we can incorporate new crawls as they arrive so that the resulting data pool remains up-to-date. Table 9 also lists the year from which the latest source data of each listed dataset comes from. Using our own pipeline, we can ensure that the most recent data in the pool dates from 2026.

German benchmarks are also limited in coverage and sensitivity. We therefore built a dedicated German evaluation suite, combining hand-crafted and model-generated tasks, to better measure the impact of our German data. See Section 2.4.

### 2.3.2.3 Quality Annotation

**Method** Prior works (J. Li, Fang, Smyrnis, et al. 2024; Ettinger et al. 2025) have shown that simple, single fastText classifiers (Joulin et al. 2017) trained to distinguish, e.g., SFT-style data vs. random CC samples, can be a successful method for curating CC data for quality, leading to improved model performance

on evaluation benchmarks. Other works have reported methods of ensembling classifier scores in ways which appear to outperform using individual classifiers alone (D. Su et al. 2025; T. F. Burns et al. 2026). We therefore experimented with ensembling approaches and compared this with a strong single-classifier baseline. Simultaneously, we wanted to keep in mind scalability, e.g., it could be wasteful to apply an LLM-as-a-judge to tens of billions of documents. For German, we built upon our public work introduced in GermanWeb (T. F. Burns et al. 2026) by ensembling quality and grammar fastText classifiers using an average of their scores. To score the English documents, we developed the following method.

We first scored a fixed 1M-document English sample of our curated CC data with a judge, Qwen3-32B (Qwen Team 2025a), using more than 20 prompts. Prompts include educational value, chat-likeness, encyclopedia-likeness, content versus format properties, binary ‘quality’ judgements, and benchmark usefulness (Penedo, Kydlíček, Ben Allal, et al. 2024; J. Li, Fang, Smyrnis, et al. 2024; Sachdeva et al. 2026; Wettig et al. 2024). For each prompt, the judge returns structured JSON, enforced via constrained decoding against a schema. We noted that disagreement between judged axes was asymmetric. Low-scoring documents did so on multiple (sometimes all) prompts, whereas high-scoring documents scored well on some prompts but not others. This implies that the properties by which documents were judged as ‘good’ were more distinctive than the absence of those properties<sup>2</sup>. Document-level Spearman correlations among the judge scores was also structured into groups of highly-correlated scores, e.g., content versus format, conversational, maths, code, and evaluation- or task-usefulness.

To score the rest of the crawl, we distil each scalar judgement axis into a binary task (top quartile versus bottom quartile, discarding the middle two) and train three families of classifiers: fastText (Joulin et al. 2017), a frozen Luxical-one embedding (DatologyAI et al. 2025) with an MLP head, and a fine-tuned BERT (Devlin et al. 2019). We evaluate the classifiers on a held-out sample of 1M CC documents, sampled and judged independently. Using this data, we were able to see how well classifiers reproduced the judge, finding that while BERT models matched most closely over all (held-out F1 0.945, AUC 0.987), Luxical-based classifiers performed nearly as well (F1 0.913, AUC 0.971), and much better than fastText (F1 0.865). Given this, to balance inference speed over our corpus with quality, we opted to use the Luxical-based distilled judge classifiers when constructing our Kolibri dataset. The distilled classifiers also showed the same correlation structures as the judges. We combined the Luxical classifiers with a binary fastText classifier trained to distinguish SFT-style data vs. random CC samples, which has been a successful method in previous works (J. Li, Fang, Smyrnis, et al. 2024; Ettinger et al. 2025).

**Mixing Annotations on the Simplex** We combine five English per-document scores with weights on the probability simplex. The scores are fastText  $P(\text{high\_quality})$  and four Luxical heads (educational anchor, chat-like anchor, content-focussed score, and MMLU usefulness). We choose these four Luxical heads on the bases that they are minimally redundant compared to other annotation axes, and because we want to balance efficient inference time over our corpus (ruling out BERT classifiers) while having higher agreement with the judges (ruling out fastText classifiers). Each classifier score lies in  $[0, 1]$ . To condition the mix on fastText quality, we pair each score with either the identity or the indicator  $\mathbf{1}\{\text{fastText} > \tau\}$  with  $\tau = 0.0216$  (the corpus-median fastText score). This gives  $2 \times 5 = 10$  basis columns, with weights  $W_{a,b}$  on the ten-dimensional probability simplex,  $W_{a,b} \geq 0$  and  $\sum_{a,b} W_{a,b} = 1$ . We search the simplex by sampling

<sup>2</sup>By analogy, foods can be tasty in many distinct ways (a sweet apple, a savoury fish), but spoiled food is spoiled in much the same way.

21 interior vectors from  $\text{Dirichlet}(\alpha = 1)$ , as well as five one-hot vertices, namely the ungated single-score baselines (all weight on one raw score, with the identity gate). We combine the scores like so:

$$S(d) = \underset{a,b}{\text{combine}} W_{a,b} \cdot \text{rank}(\text{gate}_a(d) \cdot \text{rater}_b(d)), \quad (10)$$

where `combine` is a weighted mean, the gates  $a$  are the identity and  $\mathbf{1}\{\text{fastText} > \tau\}$ , the raters  $b$  are the four Luxical heads and `fastText`, and  $\text{rank}(\cdot)$  is the per-column mid-rank empirical cumulative distribution function. By selecting the top 25% of token mass based on these document scores, we optimise the ten weights. We run ablations with a 0.4B dense DCLM model trained to  $1 \times$  Chinchilla on 8.2B randomly sampled tokens and record the mean BPB over all English benchmarks. We use these means in a Gaussian-process surrogate over the samples which then selects the optimum weights to minimise the mean BPB for the final mix.

**English-Language Ablations** We apply the optimised mix of annotations to our CC data, and compare the performance of this mix with random CC, the `fastText` classifier alone, and the Kolibri Origin mix. Table 10 compares the two mixes on the 0.4B dense  $1 \times$  Chinchilla DCLM model (J. Li, Fang, Smyrnis, et al. 2024; Hoffmann et al. 2022) (8.2B tokens) and on a 7.8B-A0.6B MoE trained to 97.6B tokens ( $\sim 170$  tokens per active parameter). The table also reports random CC and `fastText` as a single CC quality classifier. At 0.4B, English-overall BPB is  $3.800 \pm 0.513$  ( $n = 3$ ) against  $3.988 \pm 0.271$  for the Kolibri Origin mix ( $n = 5$ ).<sup>3</sup> English QA is  $3.901 \pm 1.012$  against  $5.080 \pm 0.814$ . At 7.8B-A0.6B MoE, overall BPB is  $0.607 \pm 0.012$  ( $n = 3$ ) against  $0.642 \pm 0.013$  ( $n = 2$ ), and English-overall BPB is  $0.618 \pm 0.015$  against  $0.652 \pm 0.026$ . Maths is  $0.493 \pm 0.003$  against  $0.568 \pm 0.023$ , and code is  $0.550 \pm 0.067$  against  $0.572 \pm 0.027$ . The posterior probability that the annotated CC mix is better than the Kolibri Origin mix on overall BPB at the MoE scale is 0.98, and 0.93 that it is better than the `fastText` classifier alone.

| Arm                                           | 0.4B dense                        |                                   | 7.8B-A0.6B MoE                    |                                   |
|-----------------------------------------------|-----------------------------------|-----------------------------------|-----------------------------------|-----------------------------------|
|                                               | EN                                | EN QA                             | EN                                | EN QA                             |
| Random CC                                     | 4.263 $\pm$ 1.120                 | <u>4.245<math>\pm</math>1.594</u> | 0.884                             | 1.044                             |
| <code>fastText</code> classifier CC selection | 4.711 $\pm$ 0.664                 | 4.909 $\pm$ 0.888                 | 0.868 $\pm$ 0.010                 | 0.887 $\pm$ 0.025                 |
| Optimised mix of annotations CC selection     | <b>3.800<math>\pm</math>0.513</b> | <b>3.901<math>\pm</math>1.012</b> | <b>0.618<math>\pm</math>0.015</b> | <b>0.830<math>\pm</math>0.036</b> |
| Kolibri Origin mix                            | <u>3.988<math>\pm</math>0.271</u> | 5.080 $\pm$ 0.814                 | <u>0.652<math>\pm</math>0.026</u> | <u>0.836<math>\pm</math>0.043</u> |

Table 10. Bits-per-byte (lower is better). The lowest value in each column is in bold, the second lowest is underlined. The optimised mix of annotations CC uses the top-25% CC of the gated rank-blend of Equation (10). The `fastText` classifier uses the top-25% CC of just the `fastText` classifier’s scores. Random CC is substring-deduplicated crawl with no further filtering. Each other cell is the mean  $\pm$  half-width of the 95% credible interval over  $n$  training seeds. 0.4B dense  $1 \times$  Chinchilla DCLM (8.2B tokens),  $n = 3$  (annotated CC),  $n = 5$  (random CC, `fastText`, and Kolibri Origin); 7.8B-A0.6B MoE (97.6B tokens),  $n = 3$  (annotated CC),  $n = 2$  (`fastText` and Kolibri Origin),  $n = 1$  (random CC).

**Limitations** In Kolibri, we use only a single distilled judge and the `fastText` classifier for annotations, and apply this procedure for our English data only. We wish to extend this technique to other annotations and languages. Another limitation is that the simplex mix is over five scores, whereas we expect notions of ‘quality’ or signals of what data is useful for language modelling are even higher dimensional.

<sup>3</sup>English QA is the unweighted mean of ARC, MMLU, and PIQA BPB at 0.4B, and of ARC and MMLU at the MoE scale. English overall adds HumanEval, MBPP, HellaSwag, GSM8K, and MATH-Minerva.

### 2.3.2.4 Data Source Mixing

Having filtered and annotated the CC data and included the other data sources in our pre-training data pool, we fit a mixing law on a swarm of proxy models to set each data source’s sampling weight, following an approach inspired by OLMix (M. F. Chen et al. 2026). The mix ranges over all the 78 data sources of the pre-training pool. These sources include our English and German CC ‘quality’-annotated buckets. We form these buckets by ranking documents on the mixture-of-annotations score of Section 2.3.2.3, a blend of five annotation scores for English and the average of two GermanWeb classifiers for German, and splitting each language into five equal-token-mass tiers. We include only the higher tiers, dropping the two lowest for English (i.e., including the top three,  $\sim 60\%$  of the token mass) and the lowest for German (i.e., including the top four,  $\sim 80\%$  of the token mass). The objective is the equal-weighted mean BPB over 24 benchmarks, 18 English and 6 German <sup>4</sup>.

**Sampling and Ablations for Optimisation** We train a swarm of 30M-parameter dense proxy models, each on 3B tokens of a different mixture, and record its per-benchmark BPB. The swarm is 3354 proxy trainings in total. Mixtures are drawn from a Dirichlet distribution centred on the natural token-count prior  $q \propto \text{tokens}$ , at 15 concentrations spaced log-uniformly in  $[0.1, 50]$ , with  $3(D+1)$  mixtures per concentration for the  $D = 78$  sources, plus the  $D$  one-hot corners.

**Optimisation and Materialisation** To predict each benchmark’s BPB from a given data mix, we fit a log-linear law over the swarm of proxy models. We use log-linear regression models as proposed by M. F. Chen et al. (2026), since they found that these models achieve the best downstream performance. For each benchmark  $t$ , intercept  $a_t$ , coefficient vector  $b_t$ , and data mix  $x$ , we fit

$$\text{BPB}_t(x) = \exp(a_t) + \exp(b_t \cdot x)$$

over the swarm by limited-memory Broyden–Fletcher–Goldfarb–Shanno (L-BFGS) optimisation with a Huber loss and multi-start initialisation. We then pick the mix  $x$  by solving the convex program

$$\min_x \sum_t w_t \exp(b_t \cdot x) + \lambda \text{KL}(x \parallel q), \quad x \geq 0, \sum_i x_i = 1, x_i \leq \kappa_i, \quad (11)$$

where  $w_t$  are the objective weights,  $q \propto \text{tokens}^{1/T}$  is a temperature-scaled token prior, and the caps  $\kappa_i = \text{tokens}_i \cdot r / R$  forbid a source from repeating more than  $r$  times at the target token budget  $R = 20T$ . The KL pull trades quality against repetition: at  $\lambda = 0$  the solver ignores source sizes and picks only the best-fitting sources, while a large  $\lambda$  reproduces the size prior. We settle on  $\lambda = 0.05$ ,  $T = 3$ , and  $r = 3$ , which keeps every small source represented at least once while staying close to the prior. On the proxy objective the chosen mix is marginally worse than the unconstrained optimum (predicted  $3.670 \pm 0.002$  versus  $3.667 \pm 0.002$  mean BPB), which we consider a reasonable trade-off for keeping the smaller sources. The materialised mix draws 35.7% of its training tokens (7.15T) from our own pipeline and 64.3% (12.85T) from external sources; Table 11 breaks down our own contribution by family.

<sup>4</sup>Among the English benchmarks are MMLU, ARC, HellaSwag, HumanEval, MBPP, CommonsenseQA, Drop, NaturalQsOpen, PIQA, SocialQA, and Squad. The German benchmarks are Global-MMLU, ARC, HellaSwag, HumanEval, MBPP, and GSM8K.

| Source family               | Tokens | Sources |
|-----------------------------|--------|---------|
| CC quality buckets          | 3.82T  | 7       |
| German FineWeb rephrases    | 1.74T  | 5       |
| Synthetic rephrases (de/en) | 1.31T  | 6       |
| Externally-sourced web data | 0.24T  | 1       |
| Small specialised datasets  | 0.045T | 3       |
| Our pipeline (total)        | 7.15T  | 22      |
| External sources            | 12.85T | 56      |

Table 11. Tokens drawn from our own data-pipeline families in the materialised Kolibri pre-training mix ( $\lambda = 0.05$ ,  $T = 3, 3 \times$  repetition cap), out of a 20T-token budget. External sources supply the remaining 64.3%.

**Limitations and Learnings** We fit the data mix weights on models with 30M parameters trained on 3B tokens, which makes both the fit and the swarm noisy. We find that the optimiser of the data mix weights is sensitive to the number of swarm samples, and the prior and its two knobs ( $\lambda$ ,  $T$ ) influence the predicted optimum dramatically. It is not clear whether this is a limitation of the optimiser or a property of the data mix problem. The natural prior is strongly skewed towards the largest sources, meaning that smaller sources are typically less well-explored even at the lowest concentrations. Some of our German quality-classifier buckets are down-weighted relative to their labels, so the mix optimisation does not appear to comport with the German quality ordering. We wish to re-evaluate these annotations and their mixing in the future. We also patched several token-count errors (byte-versus-token miscounts and a bucket mismatch) before the final solve. Unfortunately, the sampling prior for the swarm ablations, computed from the old counts, cannot be corrected after the fact. These errors are likely to have a small impact on the final mix, but we of course wish to avoid them in the future.

## 2.3.3 Stage 2: Mid-Training

Mid-training continues training on higher-quality math, code, reasoning, and instruction data to strengthen these capabilities. Following S. Hu et al. (2024), who reserve higher-quality data for the later phases of training, we draw this data from a curated pool of datasets, and a mix (Section 2.3.3.3) sets the number of tokens each dataset contributes. Mid-training keeps the constant learning rate and the checkpoint merging of pre-training (Section 2.2). The stage ends in the base checkpoint from which long-context extension (Section 2.3.4) continues training at the same learning rate.

### 2.3.3.1 Dataset Pool

The curated pool is the set of datasets from which the mid-training and long-context mixes draw. We collect candidate datasets from public releases and from the pre-training datasets, drop every candidate whose content largely overlaps another, and assign each dataset to one of the nine domains of Section 2.3.2. The pool excludes the low-quality tiers of web data and, for German, the rephrased and translated web text. The German exclusion rests on manual inspection, in which rephrased and translated text read less naturally than organic German text.

Every dataset of the pool passes the cleanup pass of Appendix D.1.1, exact deduplication across the whole pool (Appendix D.1.2), and PII redaction. The size of a dataset is its number of unique tokens, estimated from its size in bytes as for the pre-training pool (Table 8). The repetition caps (Section 2.3.3.3) count the passes of pre-training over a dataset against this size, and production mid-training draws each dataset, over its tokens after treatment, for at most the passes that its cap leaves. In addition, the production mix draws from the pool after its decontamination against the evaluation suite (Appendix D.2.5). Table 12 lists, per domain, the number of datasets, the unique tokens, the tokens after treatment, and the most that the repetition caps of Appendix D.2.1 allow mid-training to draw, and Figure 13 shows the share of each domain in unique tokens and in datasets. Documents above 128k estimated tokens are reserved for the long-context stage (Section 2.3.4), since they are scarce and only its sequence length of 256k tokens holds them in full.

### 2.3.3.2 Ablation Setup

The ablations use a proxy model, a downscaled Kolibri with the same hybrid architecture, 11.9B total and 0.65B active parameters, pre-trained for 2.79T tokens. We mid-train that checkpoint on every mix for 84B tokens, 3 % of the proxy’s pre-training tokens, at the production sequence length of 64k tokens. At this budget, 18 runs cover three hand-set mixes and six optimised mixes (Section 2.3.3.3), the production mix, six mixes that vary the in-domain composition of the two language domains (Appendix D.2.4), C1 entered through a transition from the pre-training mix, and the Kolibri Origin mid-training mix (Table 40). Two of the hand-set mixes, C2 and C3, and the production mix also run for 400B tokens, 14 % of the proxy’s pre-training tokens and closer to the 17 % of production (3.44T after 19.99T), and a baseline run of the same length continues pre-training on the pre-training mix at its sequence length of 16k tokens. Every score is that of a merged checkpoint, the uniform average of the weights of the last  $k$  saved checkpoints

| Domain                            | Short     | Datasets | Unique tokens (B) | After treatment (B) | Under caps (B) |
|-----------------------------------|-----------|----------|-------------------|---------------------|----------------|
| English web and documents         | EN web    | 12       | 4,903             | 4,332               | 14,352         |
| German web and documents          | DE web    | 11       | 957               | 869                 | 2,171          |
| English instruction and reasoning | EN instr. | 8        | 1,269             | 1,223               | 2,643          |
| German instruction and reasoning  | DE instr. | 3        | 41                | 41                  | 101            |
| Code                              | Code      | 13       | 2,688             | 1,935               | 5,976          |
| Agentic code and tool use         | Agentic   | 5        | 605               | 601                 | 2,405          |
| STEM (documents)                  | STEM docs | 9        | 1,052             | 827                 | 2,650          |
| STEM (QA-style)                   | STEM QA   | 11       | 457               | 431                 | 1,394          |
| Specialized domains               | Spec.     | 15       | 301               | 247                 | 812            |
| Total                             |           | 87       | 12,273            | 10,505              | 32,503         |

Table 12. The curated pool by domain, in billions of tokens. Short is the form the tables use. Unique tokens are the estimate used for mixing, After treatment the tokens the treated datasets hold, and Under caps the most the repetition caps allow mid-training to draw from the tokens after treatment.

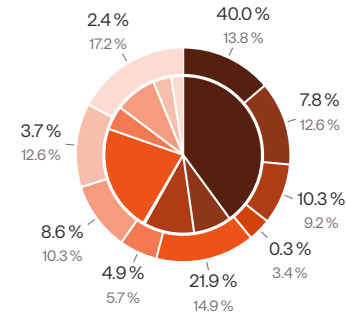


Figure 13. Share of the curated pool by domain, in unique tokens (inner disk) and in datasets (outer ring), colours as in the adjacent Table 12. Each label gives the domain's share of the unique tokens and, below it, its share of the datasets.

(Izmailov et al. 2018; Sanyal et al. 2023), written  $M_k$ , and the Kolibri base model is the M20 merge at the end of long-context training (Section 2.2). Merging adds 3.4 to 11.5 points over the unmerged endpoint in these ablations, so comparisons use merges of the same  $k$  only. The aggregate score averages the general-knowledge, math, and code means of each language and weights English and German 80/20, and Appendix D.2.2 lists the benchmarks. For 16 of the 18 runs at 84B and for every run at 400B, we also report the pooled score of the post-training benchmarks after an SFT, which measures the part of a mid-training gain that persists after post-training. All SFTs share one recipe and one data mix, so their differences come from the base checkpoints alone. An eval-noise model, fitted to replicate SFTs with the standard errors of Miller (2024), gives each score a 95 % interval (Appendix D.2.2).

### 2.3.3.3 Mixing Method

We search for the mix with the highest aggregate score (Section 2.3.3.2) within the repetition limits of the scarce high-quality datasets of the pool. Optimising the domain weights for this score alone sets four domains to zero (Section 2.3.3.5).

**Mixes and Sampling Rules** A mid-training mix assigns every dataset of the curated pool a share of the token budget along two axes. The domain weights give each of the nine domains its share of the budget, and the in-domain composition splits the budget of a domain among its datasets. An optimiser in the style of Olmix (M. F. Chen et al. 2026), fitted to the scores of MergeMix expert merges (J. Wang et al. 2026) as described below, proposes candidate domain weights, on which we then train proxy models (Appendix D.2.3). The in-domain composition follows one of two sampling rules. Under the natural rule, each dataset's share of its domain is proportional to its unique tokens (Section 2.3.3.1). The pre-training-delivered rule keeps the pre-training proportions of the datasets that pre-training drew from and gives datasets new to mid-training their natural share. In addition, we hand-set three candidates, C1 through C3 (Table 13), which serve as the baselines for the optimised mixes and as the reference mixes of the KL penalty. C1 and C2 share their domain weights and differ only in the sampling rule, natural for C1 and

pre-training-delivered for C2. The optimised mixes and the production mix use the pre-training-delivered rule.

**Repetition Caps** High-quality data is scarce, so a mid-training mix repeats its highest-quality datasets. The repetition cap of a dataset limits its draws across pre-training and mid-training combined to four epochs, the number of epochs up to which training on repeated data reaches approximately the loss of training on as many unique tokens (Muennighoff, Rush, et al. 2023). Within each domain, a waterfill assigns every dataset its share under the sampling rule, pins a dataset whose share exceeds its cap at the cap, and redistributes the excess to the unpinned datasets of the domain in proportion to their shares.

**Expert Merging and Optimisation** In MergeMix, the proxy model is mid-trained on each of the nine domains alone for 20B tokens, and a merge of these expert checkpoints with the weights of a candidate mix approximates an 84B-token mid-training run on that mix. From a shared initialisation, merging the experts and training on the mixed data agree to first order, and over 12 mixes, expert merges rank the mixes as full mid-training runs do, with a Spearman correlation above 0.9 (J. Wang et al. 2026). Expert merging thus allows us to evaluate 113 weightings, 100 sampled from Dirichlet distributions around our initial hand-set mixes and 13 placed by hand to cover the range of the four smallest domains. Following Olmix (M. F. Chen et al. 2026), we fit one log-linear law per benchmark to the expert-merge scores and solve for the weights that maximise the aggregate score, subject to the repetition caps at a 5T budget and a KL penalty towards a reference mix, which keeps the solution near a hand-set mix. The pre-training evaluation suite (Section 2.4) has no benchmark that targets the instruction and reasoning domains or Specialized domains directly. These domains thus enter the objective only through their effect on other benchmarks, the instruction and reasoning domains through math and code, and Specialized domains through none of them.

**Optimised Mixes S1 to S6** Six solutions enter the proxy ablations (Table 13) next to the hand-set candidates (Appendix D.2.3). They vary the reference mix (C2 or C3), the KL penalty (0, 0.01, or 0.05), and whether their in-domain composition is projected from the 5T budget, at which the repetition caps pin datasets. At the ablation budget every dataset stays below its cap, so S2 and S5 follow the sampling rule unchanged. For the projected S1, S3, S4, and S6, the waterfill runs at 5T and its result is scaled down to the ablation budget, so the datasets pinned at 5T lose share to the other datasets of their domain. S2 and S3 come from the same solve, as do S5 and S6, and the domain weights of each pair differ by less than one point. In S3 and S6, the projection changes the in-domain composition of English instruction and reasoning, German instruction and reasoning, and STEM (QA-style), and in S3 also that of Code. Appendix D.2.1 gives further details on the caps, the optimisation, and the solutions.

### 2.3.3.4 Production Mix and Its Effect

**Production Mix** The production mix is S5, the optimised mix with the C3 prior and KL penalty 0.01, with German instruction and reasoning raised to its repetition cap (Appendix D.2.7). At 84B, the SFT scores of the solutions with penalties 0, 0.01, and 0.05 lie within 0.9 points of each other, below the resolution of the

| Mix            | Prior | KL   | Sampling               | Domain weights (%) |        |           |           |      |         |           |         |       |
|----------------|-------|------|------------------------|--------------------|--------|-----------|-----------|------|---------|-----------|---------|-------|
|                |       |      |                        | EN web             | DE web | EN instr. | DE instr. | Code | Agentic | STEM docs | STEM QA | Spec. |
| C1             | –     | –    | natural                | 33.0               | 19.0   | 10.0      | 1.0       | 10.0 | 8.5     | 6.5       | 9.0     | 3.0   |
| C2             | –     | –    | pre-training-delivered | 33.0               | 19.0   | 10.0      | 1.0       | 10.0 | 8.5     | 6.5       | 9.0     | 3.0   |
| C3             | –     | –    | pre-training-delivered | 14.0               | 18.0   | 16.2      | 1.4       | 8.0  | 16.2    | 6.0       | 16.2    | 4.0   |
| S1             | C2    | 0    | pre-training-delivered | 0.0                | 0.0    | 31.0      | 1.4       | 42.1 | 0.0     | 0.3       | 25.1    | 0.0   |
| S2             | C2    | 0.01 | pre-training-delivered | 3.1                | 0.4    | 31.5      | 1.4       | 29.4 | 7.8     | 10.8      | 15.5    | 0.1   |
| S3             | C2    | 0.01 | pre-training-delivered | 3.1                | 0.4    | 30.7      | 1.4       | 29.8 | 7.9     | 10.9      | 15.7    | 0.1   |
| S4             | C2    | 0.05 | pre-training-delivered | 20.3               | 2.3    | 14.9      | 1.4       | 27.1 | 9.7     | 13.7      | 9.6     | 1.0   |
| S5             | C3    | 0.01 | pre-training-delivered | 1.3                | 2.1    | 31.5      | 1.4       | 16.4 | 15.5    | 6.0       | 25.5    | 0.2   |
| S6             | C3    | 0.01 | pre-training-delivered | 1.3                | 2.2    | 31.0      | 1.4       | 16.8 | 15.9    | 6.1       | 25.1    | 0.2   |
| production mix | –     | –    | pre-training-delivered | 1.3                | 2.1    | 31.4      | 1.9       | 16.3 | 15.4    | 5.9       | 25.4    | 0.2   |

Table 13. The hand-set mixes C1 to C3, the optimised mixes S1 to S6, and the production mix, with the reference mix (prior) and KL penalty of each solution, the sampling rule, and the domain weights. Every mix draws from the curated pool. S1, S3, S4, and S6 are projected from a 5T budget, at which the repetition caps pin datasets. The production mix is S5 with German instruction and reasoning raised to its cap (Section 2.3.3.4), and its weights are those of its waterfill at 5.2T. Table 43 adds six mixes that vary the in-domain composition of the two language domains.

eval-noise model, so the choice of the penalty rests on the domain weights. Without the penalty, the solve sets four domains to zero, and at 0.05, the weight that M. F. Chen et al. (2026) recommend, the solution stays closest to its hand-set prior. We thus take 0.01, the smallest tested penalty at which every domain keeps a positive weight. In this mix, English instruction and reasoning, STEM (QA-style), and Code hold 73 % of the budget together, and the two web domains hold 3.4 % (Table 13). We mid-train Kolibri on 3.44T tokens of this mix and interpolate linearly from the pre-training mix to it over the first  $H = 100\text{B}$  tokens (Appendix D.2.6).

**Decontamination** Benchmark text in the late phase of training is memorised and, under a constant learning rate as in Kolibri, also inflates benchmark scores (Magar and Schwartz 2022). Benchmark text in the early phase is forgotten once the subsequent clean data exceeds about five times the compute-optimal token count (Bordt et al. 2025). Following Olmo 3 (Ettinger et al. 2025, Sec. 3.5.3), we therefore decontaminate the mid-training pool, from which long-context extension also draws, and remove every document that contains text from the 226 tasks of our evaluation suite, with all their splits. The decon algorithm (Allen Institute for AI 2025) flags a document by matching Inverse Document Frequency (IDF)-weighted n-grams of the questions and answers, and a flagged document is dropped as a whole. Decontamination drops 1.5M of the 8.3B scanned documents, 0.06 % of the tokens (Appendix D.2.5). The pre-training pool remains potentially contaminated, and the HumanEval scores reflect this contamination.

**HumanEval Recitation** A HumanEval completion counts as recited iff it matches the reference solution shipped with the benchmark (Appendix D.2.8). During pre-training, 22 % to 95 % of the completions are recited, and pass@1 follows the recited share, with a Pearson correlation of 0.90 over 96 evaluations. Within the first 0.3T tokens of mid-training, the recited share drops below 10 % in both languages, and English pass@1 decreases from 80.0 to 70.0. On the problems without a recited completion, English pass@1 stays

at 61 to 76 throughout mid-training (Figure 64). By the end of mid-training, the recited share rises again to 48 % in English and 47 % in German, so the HumanEval scores at that point remain inflated by recitation. We thus also report the aggregate score without HumanEval (Appendix D.2.2) and plan to decontaminate the pre-training pool for the next model.

**Effect on the Aggregate Score** From the M4 merge at the end of pre-training to the M4 merge at the end of mid-training, the aggregate score rises by 2.8 points, or 2.9 points without HumanEval, with the gain in math and code in both languages (Figure 14). At 400B, the production mix leads continued pre-training by 4.9 points on the M4 merge and by 11.5 points after SFT, and it leads C3, the reference mix of its KL penalty, by 3.8 points after SFT (Table 14). Both differences after SFT exceed the 95 % eval-noise intervals (Appendix D.2.2), so the production mix, derived from S5, improves on the hand-set mix it is regularised towards.

| Mix                    | M4 score | SFT score | $\pm$ | TriviaQA | HellaSwag (DE) |
|------------------------|----------|-----------|-------|----------|----------------|
| C2                     | 61.1     | 44.0      | 1.7   | 57.3     | 77.6           |
| C3                     | 62.4     | 47.5      | 1.8   | 55.1     | 76.0           |
| production mix         | 65.5     | 51.3      | 1.9   | 55.0     | 70.4           |
| continued pre-training | 60.6     | 39.7      | 1.6   | 63.5     | 77.6           |

Table 14. The four runs of 400B tokens, by the aggregate score of the M4 merge, the pooled score of the SFT on it with the half-width of its 95 % eval-noise interval, and the TriviaQA and HellaSwag (DE) scores of the M4 merge. Continued pre-training keeps training on the pre-training mix at its sequence length. The scores of the same mixes at 84B are in Table 39.

**Knowledge Benchmarks** The general-knowledge means fall by 1.0 points in English and 1.4 in German from the pre-training M4 to the mid-training M4, with gains on MMLU-Pro and ARC and losses on TriviaQA and both HellaSwag benchmarks. After long-context extension, the final base model still scores 5.2 points below the pre-training M4 on TriviaQA and 4.5 on HellaSwag (DE). HellaSwag (DE) rises with the German-web share of a mix, and the cause of the TriviaQA loss remains open (Appendix D.2.8).

### 2.3.3.5 Lessons for Choosing a Mix

**Regularisation towards a Hand-Set Mix** Without the KL penalty, the solve sets English web and documents, German web and documents, Agentic code and tool use, and Specialized domains to 0 % and STEM (documents) to 0.3 % (S1 in Table 13). We hypothesise that the weighting of the aggregate score causes this outcome, since it weights math and code two to one over general knowledge. Across the 84B runs, the English knowledge mean correlates with the English math mean, the MBPP mean, and the weight of English web and documents at a Pearson  $r$  of 0.25, 0.03, and  $-0.11$ , and the weight of English web and documents correlates with the English math mean at  $-0.67$  (Table 41). With the penalty, the two web domains retain 22.6 % at 0.05 towards C2 (S4) and 3.5 % to 3.6 % at 0.01 (S2 and S3). The penalty acts on the domain weights as a whole, so at 0.01 towards C3, German web and documents drops from 18 % in C3 to 2.1 % in the production mix. At 400B, the HellaSwag (DE) score of the production mix lies 5.5 points below that of C3, a cost of 0.09 points in the aggregate score (Table 14). Preserving a capability thus requires its own term or constraint in the objective.

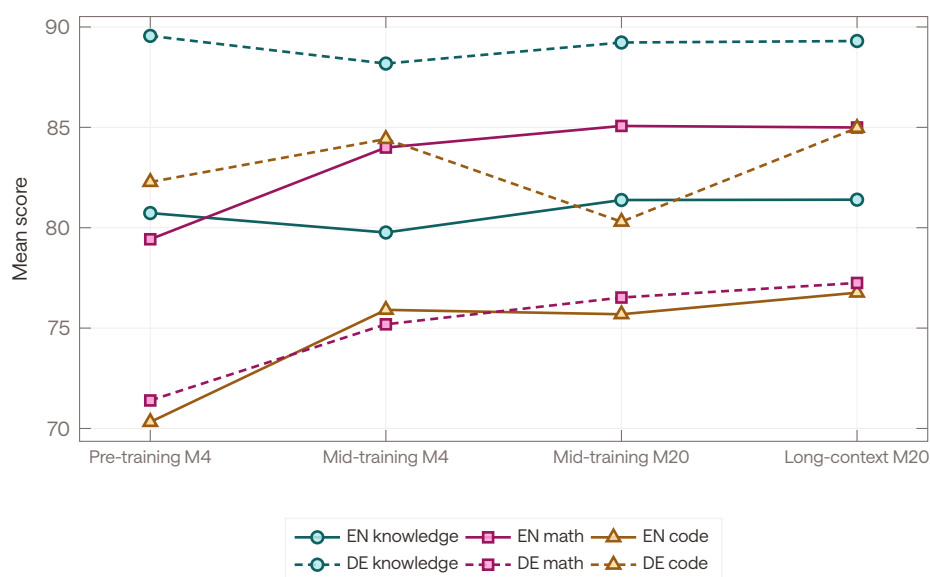


Figure 14. The general-knowledge, math, and code means of the pre-training suite per language along the production chain, at the M4 merge at the end of pre-training, the M4 and M20 merges at the end of mid-training, and the M20 merge at the end of long-context extension, which is the final base model. Table 45 lists the values.

**Proxy Budget** At 400B every adjacent pair of the four runs differs after SFT beyond the eval noise, by 4.3 points from continued pre-training to C2, 3.5 from C2 to C3, and 3.8 from C3 to the production mix (Table 14). At 84B, C2 and C3 differ by 0.4 points after SFT and by 0.03 on the merge, and the SFT separates only the lowest runs from the highest, in 14 of 120 pairs (Appendix D.2.2). Relative to the pre-training tokens, the 400B budget is 83 % of the production ratio and the 84B budget 18 % of it, so the mixes separate after SFT at the larger proxy budget only. With two budgets, the ratio at which this separation sets in remains open.

**Merge Score and SFT Score** The M4 merge and the SFT order the four 400B runs alike, and the gaps after SFT are larger, 4.3 points between C2 and continued pre-training against 0.5 on the merge (Table 14 and Figure 60). On these four runs, the merge score thus appears to understate the gain of a mix after SFT.

## 2.3.4 Stage 3: Long Context Extension

Our strategy is to extend the context of Kolibri to 256k while preserving the capabilities of the mid-trained model, by interleaving naturally long documents with high-quality mid-training data rather than training only on long-context data. We take a fixed size of the naturally long document contribution from our pre-training pool and ablate how aggressively the mid-training pool is reweighted towards longer documents. Overall, increasing the share of length-reweighted mid-training data improves long-context retrieval performance, while checkpoint merging largely preserves general capability.

### 2.3.4.1 Dataset Pool and Candidates

The long-context stage draws from three data pools. The first is the original mid-training mix, processed as before with documents longer than 128k tokens removed, and about 70 % of its tokens lie in documents shorter than 8k tokens. The second pool contains the same mid-training datasets without the document-length limit, which allows sampling to shift towards longer documents from the same data sources. The third pool is a dataset of naturally long documents between 8k and 256k tokens, from which we remove the synthetically generated samples to avoid artificially favourable results on synthetic long-context benchmarks such as RULER.

Every candidate is a mix of 200B tokens that takes one third from the naturally long pool, a share  $s$  from the mid-training reweighted pool, and the remaining  $2/3 - s$  from the natural mid-training distribution. We materialise and train three settings,  $s \in \{0, 1/4, 1/3\}$ .

**Length Reweighting** In the reweighted share, each document-length band is assigned a weight on a logistic curve over the band index  $b$ , which runs from 0 for documents up to 8k tokens to 5 for 128k to 256k tokens:

$$w(b) = \frac{1}{1 + e^{-(b-4)}}.$$

The weight is 0.5 for the 64k to 128k band and 0.73 for the longest band.

The natural and reweighted shares both draw from the same mid-training datasets, so their sampling rates must be combined into a single per-band budget to avoid assigning the same documents independently to both shares. The resulting sampling rate for length band  $b$  is

$$r(b) = \min(r_{\text{natural}}(b) + q w(b), r_{\text{max}}),$$

where  $r_{\text{natural}}(b)$  is the sampling rate required by the  $(2/3 - s)$  share drawn according to the original mid-training distribution and is zero above 128k tokens. The scale  $q$  is shared across all bands and is solved by bisection such that the reweighted component contributes its target budget of  $s$  times the total mix size. For a 200B-token candidate, the three components therefore contribute  $1/3 \times 200\text{B}$  tokens from naturally long documents,  $s \times 200\text{B}$  from the length-reweighted mid-training pool, and  $(2/3 - s) \times 200\text{B}$  from the original mid-training distribution.

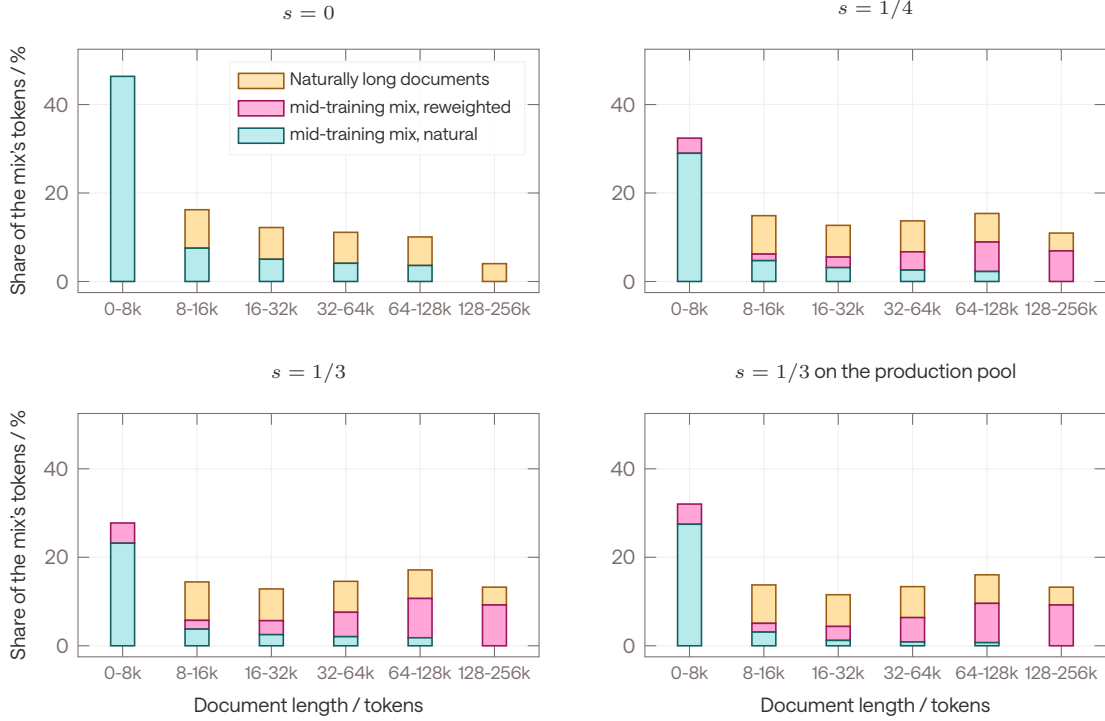


Figure 15. Length distribution of the long-context mix candidates, as the materialised token budget of each share per document-length band in percent of the mix's tokens.  $s$  is the share of the mix drawn from the mid-training mix with the length reweighting, and the naturally long documents portion is identical across the three candidates. The last panel is the production mix, the  $s = 1/3$  recipe rebuilt on the production mid-training pool at proxy scale.

No repetition cap  $r_{\max}$  binds in any candidate, so no document is repeated. One scale for the whole pool means the reweighted share is dominated by whichever sources hold long documents, so its topic mix drifts away from the mid-training mix, which Figure 16 quantifies.

Figure 15 illustrates the resulting length distribution of each candidate as materialised budgets. Varying the reweighted share  $s$  from 0 to  $1/3$  increases the fraction of tokens of documents longer than 32k from 25 % to 45 %, and of documents longer than 128k from 4 % to 13 %, while the naturally long documents allocation stays identical.

**Domain Shift** Without controlling for domains, unconstrained length reweighting risks shifting the domain distribution of the resulting mix. Figure 16 shows this effect by comparing the token-weighted domain distribution of the reweighted production mid-training pools against the original mid-training mix.

Because long documents are concentrated in web text and scanned material, the English Language domain rises from 1.3 % to 24 %. In contrast, instruction-following, problem-solving, and agentic domains are short by construction and fall to about half their share of the original mid-training mix.

This shift is a consequence of the length reweighting rather than an explicit design choice, and the candidates were not constructed to control for it. We therefore cannot isolate the effect of the domain

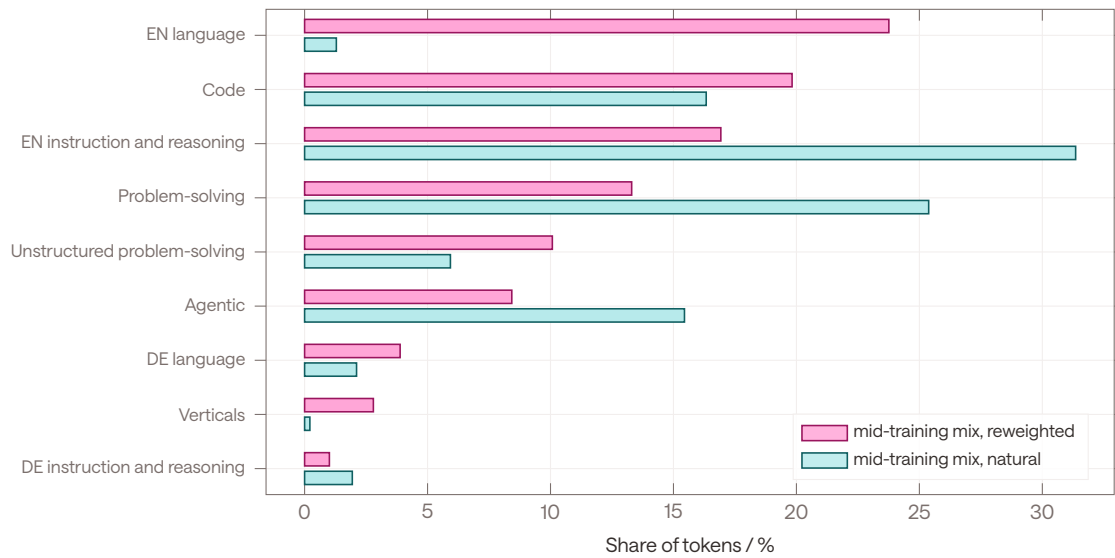


Figure 16. Token-weighted domain distribution of the part of the production long-context mix drawn from the mid-training pools, compared with the original mid-training mix.

shift from the effect of document length in the corresponding evaluation results. Future experiments should treat domain distribution and length distribution as separate ablation dimensions.

### 2.3.4.2 Evals and Ablations

**Setup** The ablation primarily sweeps the share of mid-training data that is reweighted towards longer documents. We train three long-context candidates with increasing reweighted shares, together with a control using 100 % of the original mid-training mix.

All four branches fork from the same mid-training checkpoint after about 330B mid-training tokens and train for 2087 steps at a 256k sequence length, corresponding to about 70B tokens, with 200 warm-up steps and a constant learning rate. The data mix is therefore the only difference between branches.

Each branch is evaluated at nine checkpoints on RULER (Hsieh et al. 2024) and HELMET (Yen et al. 2025) up to 256k tokens, as well as on the general-capability suite of the main run. We also evaluate a merged checkpoint obtained by uniformly averaging the branch endpoint with three pre-fork mid-training checkpoints spaced roughly 70B tokens apart.

**Results** We evaluate the candidates using an aggregate over RULER and HELMET: each leg is divided by its mean over the four branches after subtracting its chance level, so 1 is the cohort average, then context lengths are averaged within each task, tasks within each suite, and the two suites weighted equally. As detailed in Table 15, the candidates are close overall: the last four raw checkpoints lie between 0.97 and 1.02 on the aggregate, while the merged checkpoints improve to between 1.09 and 1.13. Among the merged checkpoints,  $s = 1/3$  achieves the highest aggregate score.

| Mix       | Merged checkpoint |              |              | Last four raw checkpoints |              |              |
|-----------|-------------------|--------------|--------------|---------------------------|--------------|--------------|
|           | RULER             | HELMET       | Aggregate    | RULER                     | HELMET       | Aggregate    |
| $s = 0$   | 1.122             | 1.063        | 1.093        | 0.951                     | 0.994        | 0.973        |
| $s = 1/4$ | 1.125             | 1.067        | 1.096        | 0.972                     | 0.998        | 0.985        |
| $s = 1/3$ | <b>1.146</b>      | <b>1.115</b> | <b>1.130</b> | <b>1.019</b>              | <b>1.018</b> | <b>1.019</b> |
| control   | 1.124             | 1.059        | 1.091        | 1.017                     | 0.979        | 0.998        |

Table 15. Aggregate long-context results used to rank the mix candidates.

At the task level, differences become more visible at longer context lengths, with the largest separation on RULER at 256k. Increasing the reweighted share generally improves retrieval and extraction tasks such as QA, NIAH, and Word Extraction (WE), while VT moves in the opposite direction. On HELMET, JsonKV is close to saturation across candidates, whereas RagQA, Rerank, and LongQA InfBench QA show clearer separation. The control remains competitive despite seeing no long-context data, with the clearest benefit from long-context extension appearing on retrieval tasks at the longest context lengths. Detailed task-level results are reported in the appendix in Table 47.

We also track performance on the standard non-long-context evaluation suite, covering mathematics, coding, reasoning, and general knowledge. Long-context extension slightly reduces general capability in the raw checkpoints: the three long-context branches trail the control throughout the stage in Figure 17. Checkpoint merging largely recovers this difference, with the four merged checkpoints indistinguishable within the standard error of the general-capability composite.

No mixture dominates every task, but  $s = 1/3$ , the largest reweighted share tested, gives the strongest merged aggregate while maintaining comparable general capability after merging. We therefore use this recipe for the production long-context stage. Since the optimum may lie beyond the tested range, future experiments should evaluate larger reweighted shares while controlling for the accompanying domain shift.

The ablation candidates draw from an earlier 1T-token candidate pool of the mid-training mix; the production long-context mix rebuilds the  $s = 1/3$  recipe on the mid-training mix that Kolibri trains on, scaled down to 250B tokens to fit the proxy scale training horizon, and the last panel of Figure 15 shows its length distribution.

### 2.3.4.3 Quality Beyond the Training Context

We evaluate Kolibri beyond its trained context length and find that it retains useful long-context capabilities up to 1M tokens, with task-dependent degradation.

The Kolibri base checkpoint trains on sequences up to 256k tokens. Its SWA layers use RoPE within a fixed window, while its full-attention layers use no positional encoding, so evaluation beyond the training length needs no positional rescaling. We test this extrapolation on RULER (Hsieh et al. 2024) from 128k to 1M tokens against Nemotron 3 Nano Base (Figure 18).

At 1M tokens, Kolibri Base scores above Nemotron 3 Nano Base on needle-in-a-haystack, variable tracking and question answering, but below it on word extraction. Its needle-in-a-haystack score declines with

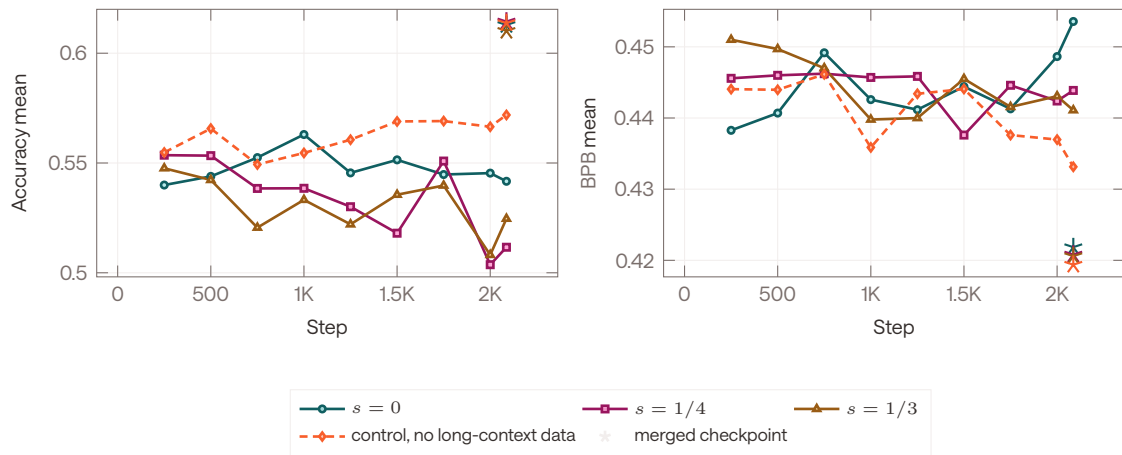


Figure 17. General-capability performance on the standard non-long-context evaluation suite.

context length, while variable tracking remains stable. Overall, these results show that Kolibri extrapolates beyond its trained context length without positional rescaling, retaining substantial capability up to 1M tokens, although performance degrades unevenly across tasks.

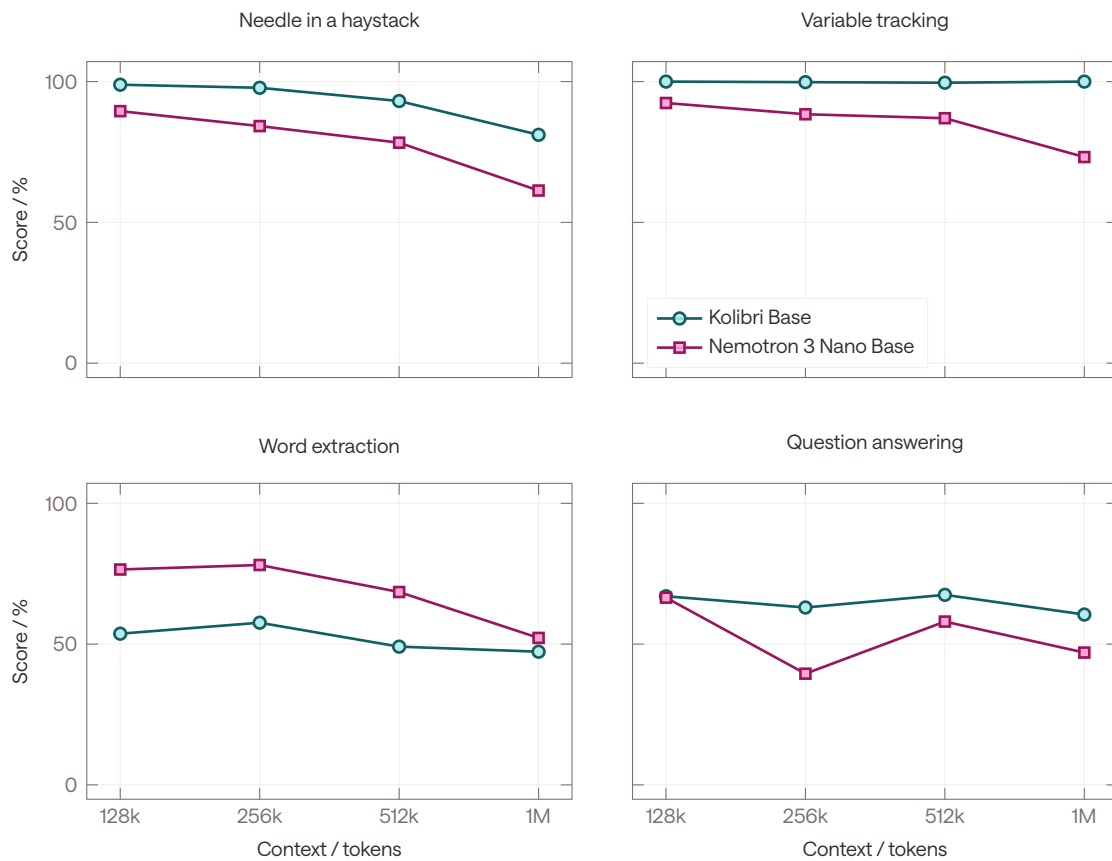


Figure 18. RULER scores of the Kolibri base checkpoint and Nemotron 3 Nano Base from 128k to 1M tokens. For question answering, the end-of-sequence token is blocked at the first generated position, so no answer is empty.

## 2.4 Base Model Evaluation

### 2.4.1 Evaluation Suite

In Table 16, we present our pre-training evaluation suite that consists of benchmarks covering general knowledge, mathematics, and code in English and German. We use these benchmarks to make decisions about the data and model ablations of the previous sections.

|              | Benchmark | English                                                                                           | German                               |
|--------------|-----------|---------------------------------------------------------------------------------------------------|--------------------------------------|
| Knowledge    | MMLU      | (Hendrycks, C. Burns, Basart, Zou, et al. 2021; Hendrycks, C. Burns, Basart, Critch, et al. 2021) | Global MMLU (S. Singh et al. 2025)   |
|              | MMLU-Pro  | (Yubo Wang et al. 2024)                                                                           | MMLU-ProX (Xuan et al. 2025)         |
|              | ARC       | (P. Clark et al. 2018)                                                                            | Ellamind (ellamind 2025)             |
|              | PIQA      | (Bisk et al. 2020)                                                                                | Ellamind (ellamind 2025)             |
|              | HellaSwag | (Zellers et al. 2019)                                                                             | Ellamind (ellamind 2025)             |
|              | TriviaQA  | (Joshi et al. 2017)                                                                               | —                                    |
| Maths        | GSM8K     | (Cobbe et al. 2021)                                                                               | (Vendrow et al. 2025; ellamind 2026) |
|              | MATH      | (Hendrycks, C. Burns, Kadavath, et al. 2021)                                                      | Ellamind (ellamind 2025)             |
| Code         | HumanEval | (M. Chen et al. 2021)                                                                             | Ellamind (ellamind 2025)             |
|              | MBPP      | (Austin et al. 2021)                                                                              | Ellamind (ellamind 2025)             |
| Long Context | RULER     | (Hsieh et al. 2024)                                                                               | —                                    |
|              | HELMET    | (Yen et al. 2025)                                                                                 | —                                    |

Table 16. Pre-training benchmarks by category, in English and German versions where one exists. “—” marks a benchmark we evaluate in one language only. For HELMET we evaluate only the JSON KV, RAG QA, RULER MK3, and InfBench MC subsets.

**Settings.** We implement all the benchmarks with *eval-framework* (Aleph Alpha Research 2026) with identical prompts, few-shot configurations, and task settings. For the multiple-choice benchmarks, the answer choices are represented using the formulation in the OLMES library (Gu et al. 2024). For the generative benchmarks, we sample 1 response for TriviaQA, 4 responses on the maths benchmarks, and 16 for the code benchmarks.

**Performance Aggregates.** For each category and language, we report the mean score of its benchmarks. We report the mean over the categories of a language as “Overall” score of that language. We do not aggregate across languages because the benchmark sets differ by language. These aggregates help us easily track progress during model training without having to track individual benchmarks.

**Abstention and Uncertainty** We additionally evaluate Kolibri Base on 7 English multiple-choice tasks that add an “I don’t know” (IDK) option to each question, which test whether a model moves probability towards abstaining rather than towards a wrong answer. We score them with the distributional correctness score (T. F. Burns 2026), alongside accuracy and a ternary score that rewards correct answers, penalises wrong ones, and treats abstention as neutral. Appendix F describes the tasks and metrics. On these IDK evaluations, Kolibri Base has the second-highest mean distributional correctness score after Nemotron 3

Super 120B-A12B Base. Its largest margin is on MMLU-Pro, where every other compared model except Nemotron 3 Super scores far lower.

## 2.4.2 Baseline Models

We compare Kolibri Base to a range of open base models spanning dense models from 7B to 70B parameters and MoE models with 3B to 12B active parameters: Apertus 70B (Project Apertus et al. 2025), OLMo 3 7B and 32B (Ettinger et al. 2025), Nemotron 3 Nano 30B-A3B (NVIDIA 2025a), Nemotron 3 Super 120B-A12B (NVIDIA 2026a), Qwen3.5 35B-A3B (Qwen Team 2026a), GLM-4.5 Air 106B-A12B (GLM-4.5 Team 2025), and Gemma 4 26B-A4B (Gemma Team 2026), next to Kolibri Origin Base. All the generative tasks listed in Table 16 are sampled with a temperature of 0.6, top-p of 0.6 for a maximum of 1024 tokens, except Gemma 4 26B-A4B Base, which is sampled at the temperature 1.0 and top-p 0.95, as specified in its model card.

Further details of the model checkpoint and vLLM deployment details are presented in Table 48 in Appendix E.1. Our experimental settings might differ from those of the original publications of the baseline models. Thus, our results are a comparison of eight baselines under one protocol, not a reproduction of published numbers.

## 2.4.3 Results

In Figure 19, we plot the “Overall” scores against the active parameters per token. Kolibri Base is the best performing model in its weight class up to 5B active parameter models. It is surpassed in performance by only Nemotron Super which has more than  $3\times$  the number of active parameters. GLM-4.5 Air, which has the same 12B active parameters as Nemotron 3 Super, scores below Kolibri Base in both languages and is therefore not on the Pareto frontier of Figure 19.

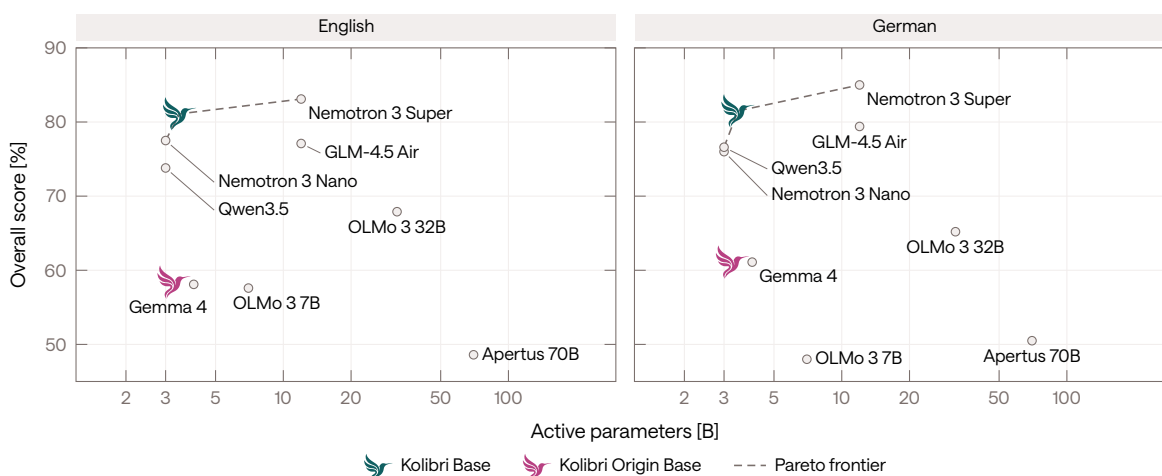


Figure 19. Overall English and German pre-training aggregates of Table 17 vs the active parameters per token on a logarithmic axis, where higher and further left is better. The dashed line connects the models on the Pareto frontier. A model lies on the frontier iff every other model with fewer active parameters scores lower and every other model with as many active parameters scores at most as high.

We report the scores of each model on our benchmark suite in Table 17. Kolibri is consistently among the top models on maths. However, the two models with 12B active parameters outperform Kolibri on the code averages.

### 2.4.3.1 Long Context Performance

We also evaluate our model, Kolibri Base, and the baselines on RULER (Hsieh et al. 2024) from 4k to 1M tokens of context and on HELMET (Yen et al. 2025) from 8k to 128k, and Table 18 reports both. Each RULER row is the mean of its NIAH, VT, WE and QA tasks at that context length, sampled at temperature 0. We evaluate on JSON KV, RAG QA, RULER MK3, and InfBench MC subsets of HELMET. We report the average of their exact match scores. When the input length exceeds a model’s context window, we report a “-”. The models with a 64k-token window also report none at RULER’s 64k, because the prompts exceed 64k tokens under some tokenisers. Gemma 4 26B-A4B Base reports none at 256k, because the prompt plus the maximum number of generated tokens exceeds its 256k-token window. At 512k and 1M, Kolibri Base and Qwen3.5 35B-A3B Base are served beyond their 256k-token trained window, Qwen3.5 with static YaRN at factor 4 at every length, while Nemotron 3 Nano 30B-A3B Base supports up to 1M tokens natively. Up to 256k on RULER, Kolibri Base trails Qwen3.5 35B-A3B Base, Nemotron 3 Nano 30B-A3B Base, and Gemma 4 26B-A4B Base; beyond its trained window it degrades the least and has the best score at 1M tokens. On HELMET it is second behind Gemma 4 26B-A4B Base at 64k and 128k.

| Type                     | MoE                |                     |                      |                              |                      | MoE                        |                                 | Dense          |                 |                  |
|--------------------------|--------------------|---------------------|----------------------|------------------------------|----------------------|----------------------------|---------------------------------|----------------|-----------------|------------------|
| Active parameters        | 3-4B               |                     |                      |                              |                      | 12B                        |                                 | 7B             | 32B             | 70B              |
|                          | Ours               | Baseline models     |                      |                              |                      |                            |                                 |                |                 |                  |
|                          | Kolibri Base       | Kolibri Origin Base | Gemma 4 26B-A4B Base | Nemotron 3 Nano 30B-A3B Base | Qwen3.5 35B-A3B Base | GLM-4.5 Air 106B-A12B Base | Nemotron 3 Super 120B-A12B Base | OLMo 3 7B Base | OLMo 3 32B Base | Apertus 70B Base |
| Eval                     |                    |                     |                      |                              |                      |                            |                                 |                |                 |                  |
| Total parameters (B)     | 78.1               | 30.6                | 26                   | 30                           | 35                   | 106                        | 120                             | 7              | 32              | 70               |
| Active parameters (B)    | 3.46               | 3.27                | 4                    | 3                            | 3                    | 12                         | 12                              | 7              | 32              | 70               |
| Overall (EN)             | <u>81.1</u>        | 58.4                | 58.1                 | 77.5                         | 73.8                 | 77.1                       | <b><u>83.1</u></b>              | 57.6           | <u>67.9</u>     | 48.6             |
| Overall (DE)             | <u>81.5</u>        | 61.1                | 61.1                 | 76.0                         | 76.6                 | 79.4                       | <b><u>85.0</u></b>              | 48.0           | <u>65.2</u>     | 50.5             |
| <b>General Knowledge</b> |                    |                     |                      |                              |                      |                            |                                 |                |                 |                  |
| Average (EN)             | <i>81.3</i>        | <i>70.8</i>         | <i>77.2</i>          | <i>80.5</i>                  | <i>82.7</i>          | <i>82.5</i>                | <b><i>87.2</i></b>              | <i>67.8</i>    | <i>77.8</i>     | <i>73.8</i>      |
| Average (DE)             | <i>82.7</i>        | <i>70.7</i>         | <i>80.5</i>          | <i>80.0</i>                  | <i>84.9</i>          | <i>81.7</i>                | <b><i>87.5</i></b>              | <i>51.8</i>    | <i>68.7</i>     | <i>74.7</i>      |
| MMLU (EN)                | 81.0               | 68.9                | 78.2                 | 79.0                         | <u>84.7</u>          | 82.8                       | <b>86.8</b>                     | 67.1           | <u>76.3</u>     | 69.3             |
| Global MMLU (DE)         | 77.5               | 65.1                | 75.1                 | 74.3                         | <u>81.5</u>          | 78.2                       | <b>84.7</b>                     | 51.6           | <u>65.1</u>     | 64.9             |
| ARC (EN)                 | 95.7               | 88.9                | 95.2                 | 94.3                         | <u>97.0</u>          | 96.3                       | <b>97.5</b>                     | 88.6           | <u>94.5</u>     | 90.7             |
| ARC (DE)                 | 96.4               | 88.5                | 95.5                 | 94.2                         | <u>97.6</u>          | 95.9                       | <b>97.8</b>                     | 67.9           | 88.1            | <u>89.9</u>      |
| PIQA (EN)                | 89.5               | 76.9                | 88.0                 | 89.4                         | <u>91.9</u>          | 89.8                       | <b>95.1</b>                     | 77.7           | 86.8            | 80.1             |
| PIQA (DE)                | 96.8               | 87.9                | 96.8                 | 96.5                         | <u>98.5</u>          | 97.2                       | <b>99.3</b>                     | 75.0           | 90.0            | <u>94.6</u>      |
| HellaSwag (EN)           | 83.1               | 80.7                | 84.8                 | 85.4                         | <u>85.3</u>          | 87.2                       | <b>88.8</b>                     | 76.1           | 83.5            | <u>84.4</u>      |
| HellaSwag (DE)           | 86.9               | 75.3                | 88.6                 | 85.8                         | <u>88.7</u>          | 86.5                       | <b>93.0</b>                     | 36.8           | 61.0            | <u>88.7</u>      |
| MMLU-Pro (EN)            | 61.1               | 39.3                | 51.2                 | 55.6                         | <u>62.6</u>          | 55.3                       | <b>68.9</b>                     | 38.0           | <u>50.9</u>     | 40.6             |
| MMLU-ProX (DE)           | 55.7               | 36.5                | 46.7                 | 49.0                         | <u>58.4</u>          | 50.8                       | <b>62.8</b>                     | 27.9           | 39.5            | 35.6             |
| TriviaQA (EN)            | 77.2               | 69.9                | 65.6                 | <u>79.5</u>                  | 74.6                 | 83.8                       | <b>86.4</b>                     | 59.0           | <u>74.7</u>     | <u>77.4</u>      |
| <b>Math</b>              |                    |                     |                      |                              |                      |                            |                                 |                |                 |                  |
| Average (EN)             | <b><u>84.9</u></b> | 54.3                | 45.9                 | 83.0                         | 73.5                 | 64.7                       | <u>82.7</u>                     | 57.7           | <u>63.3</u>     | 39.6             |
| Average (DE)             | <u>76.8</u>        | 55.8                | 47.0                 | 72.5                         | <u>77.5</u>          | 69.8                       | <b>81.2</b>                     | 43.8           | <u>62.9</u>     | 39.0             |
| GSM8K (EN)               | <b>89.8</b>        | 71.6                | 65.7                 | 86.3                         | 89.5                 | 82.8                       | <u>87.5</u>                     | 75.3           | <u>81.1</u>     | 62.3             |
| GSM8K Platinum (DE)      | <u>90.5</u>        | 71.7                | 64.1                 | 86.3                         | 88.0                 | 86.5                       | <b>93.1</b>                     | 58.7           | <u>80.8</u>     | 60.2             |
| MATH Minerva (EN)        | <b>80.1</b>        | 37.0                | 26.1                 | 79.8                         | 57.6                 | 46.5                       | <u>77.8</u>                     | 40.1           | <u>45.6</u>     | 16.9             |
| MATH Minerva (DE)        | 63.2               | 39.9                | 29.9                 | 58.8                         | <u>67.0</u>          | 53.2                       | <b>69.3</b>                     | 28.9           | <u>45.0</u>     | 17.7             |
| <b>Code</b>              |                    |                     |                      |                              |                      |                            |                                 |                |                 |                  |
| Average (EN)             | <u>77.0</u>        | 50.1                | 51.1                 | 69.0                         | 65.2                 | <b>84.1</b>                | 79.3                            | 47.4           | <u>62.5</u>     | 32.5             |
| Average (DE)             | <u>85.1</u>        | 57.0                | 55.7                 | 75.5                         | 67.4                 | <b>86.7</b>                | 86.2                            | 48.3           | <u>64.0</u>     | 37.9             |
| HumanEval (EN)           | <u>86.7</u>        | 50.8                | 52.2                 | 73.5                         | 67.2                 | <b>96.3</b>                | 83.2                            | 46.8           | <u>63.9</u>     | 28.3             |
| HumanEval (DE)           | 88.4               | 49.0                | 51.7                 | 73.5                         | 59.2                 | <b>94.0</b>                | 85.0                            | 39.5           | <u>56.2</u>     | 28.3             |
| MBPP (EN)                | <u>67.3</u>        | 49.5                | 50.0                 | 64.5                         | 63.1                 | 71.9                       | <b>75.4</b>                     | 48.0           | <u>61.0</u>     | 36.8             |
| MBPP (DE)                | 81.7               | 64.9                | 59.7                 | 77.4                         | 75.6                 | 79.4                       | <b>87.5</b>                     | 57.1           | <u>71.7</u>     | 47.5             |

Table 17. Pre-training benchmark scores in percent of Kolibri Base, Kolibri Origin Base, and open base models. The models are grouped by architecture and the active parameters per token that each group heading names, with our models next to the baseline models of the lightest MoE group, and each group's baselines are ordered by total parameters. The first two rows give each model's total and active parameters in billions as its publisher states them, with bars on a common scale. A dense model's active parameters equal its total. In each row, the best score of each group is underlined, the best score among the ten models is additionally set bold, and tied scores share a mark. The Average rows, set in italics, give the mean of their category's benchmarks. Kolibri Base ranks second on both Overall aggregates, behind Nemotron 3 Super 120B-A12B Base with more than 3 times as many active parameters, and first on the English maths average.

| Type              | MoE          |                     |                      |                              |                      | Dense          |                 |                  |
|-------------------|--------------|---------------------|----------------------|------------------------------|----------------------|----------------|-----------------|------------------|
| Active parameters | 3–4B         |                     |                      |                              |                      | 7B             | 32B             | 70B              |
|                   | Ours         |                     | Baseline models      |                              |                      | OLMo 3 7B Base | OLMo 3 32B Base | Apertus 70B Base |
|                   | Kolibri Base | Kolibri Origin Base | Gemma 4 26B–A4B Base | Nemotron 3 Nano 30B–A3B Base | Qwen3.5 35B–A3B Base |                |                 |                  |
| Context           |              |                     |                      |                              |                      |                |                 |                  |
| RULER             |              |                     |                      |                              |                      |                |                 |                  |
| 4k                | 86.9         | 91.8                | 84.7                 | 94.6                         | <b>96.3</b>          | 92.0           | <u>94.8</u>     | 89.5             |
| 8k                | 83.7         | 88.8                | 85.0                 | 93.4                         | <b>95.3</b>          | 80.8           | <u>92.6</u>     | 77.7             |
| 16k               | 80.9         | 83.7                | 87.1                 | 92.0                         | <b>95.2</b>          | 70.8           | <u>88.8</u>     | 71.3             |
| 32k               | 76.3         | 72.7                | 88.7                 | 86.4                         | <b>93.7</b>          | 64.4           | <u>80.9</u>     | 70.8             |
| 64k               | 72.2         | –                   | 86.8                 | 82.6                         | <b>91.0</b>          | –              | –               | –                |
| 128k              | 67.9         | –                   | 87.8                 | 81.0                         | <b>89.9</b>          | –              | –               | –                |
| 256k              | 69.8         | –                   | –                    | 72.1                         | <b>80.1</b>          | –              | –               | –                |
| 512k              | 65.5         | –                   | –                    | <b>73.1</b>                  | 72.2                 | –              | –               | –                |
| 1M                | <b>63.2</b>  | –                   | –                    | 58.5                         | 57.5                 | –              | –               | –                |
| HELMET            |              |                     |                      |                              |                      |                |                 |                  |
| 8k                | 79.1         | 74.8                | 81.9                 | 79.1                         | <b>87.7</b>          | 71.1           | <u>74.8</u>     | 64.2             |
| 16k               | 77.5         | 76.4                | 81.9                 | 80.8                         | <b>85.5</b>          | 67.1           | <u>73.8</u>     | 57.2             |
| 32k               | 80.0         | 73.5                | 82.9                 | 78.4                         | <b>84.1</b>          | 60.1           | <u>74.6</u>     | 48.1             |
| 64k               | 81.2         | –                   | <b>83.8</b>          | 72.7                         | 78.8                 | 51.9           | <u>73.1</u>     | –                |
| 128k              | 82.4         | –                   | <b>85.4</b>          | 70.8                         | 77.3                 | –              | –               | –                |

Table 18. RULER and HELMET scores in percent of Kolibri Base and open base models by context length. The models are grouped by architecture and active parameters per token. In each row, the best score of each group is underlined, the best score among the eight models is additionally set bold, and tied scores share a mark. An en dash marks a length beyond a model's context window. 12B models were not evaluated here.

## Part 3

# Post-Training

Post-training adds instruction following, reasoning and agentic tool use capabilities to the base model. SFT and RL train Kolibri towards the same capabilities. The paragraphs below outline the design choices that guide both stages in five areas: German, long context, agentic use, retrieval and hallucinations, and reasoning effort. Figure 2 compares the released Kolibri checkpoint with three open models, and Table 25 with the SFT checkpoint that RL starts from. RL improves on the SFT checkpoint across all capability areas shown, and Kolibri performs on par with comparable open models. The two stages are then described in detail in Sections 3.1 and 3.2.

**German** The Kolibri base model is trained on German data in pre- and mid-training (Table 9 and Section 2.3.3.1). Post-training carries this capability over to instruction following, reasoning and tool use, so that German-speaking users get fluent and correct German answers and reasoning traces they can follow.

For SFT, we therefore remove English reasoning from exchanges in other languages (Table 20) and add synthetic German datasets to the SFT data pool. For these, teacher models generate German chat, math, science and tool-calling data. A short German opener at the start of every teacher trace keeps the reasoning in German (Section 3.1.1.1). We measure how the German share of the mix affects the German and English scores on select benchmarks (Appendix H.2).

We continue this training in RL, where we translate existing environments into German and include a language consistency reward for both the reasoning trace and the answer (Table 24). We observed language consistency scores go up for the German benchmarks (Figure 34).

**Long Context** Many applications benefit from long-context capabilities, such as analysing lengthy legal documents, scientific papers, and multi-turn dialogues where context from earlier interactions can be highly relevant. We train Kolibri on in-house curated long-context data to reason over inputs of up to 256k tokens. For SFT, we generate conversations over documents of 128k to 224k tokens, each with four requests whose answers draw on sections spread over the document (Section 3.1.1.2). In RL, the retrieval environments add multiple-choice questions over documents of 95 000 to 230 000 tokens (Table 55).

**Agentic Use** We train Kolibri to perform well in agentic settings, which require tool calls and terminal interactions. To help the model recover from failed tool calls instead of repeating them, we backfill reasoning between consecutive tool calls where open agentic datasets have none and mask the loss on wrong tool

calls and terminal actions (Section 3.1.1.3). Beyond the available data, we generate two software-engineering SFT datasets: in one, a teacher model resolves GitHub issues with a patch, and in the other, it fixes bugs as a Terminus 2 agent in a terminal. In RL, we then train on tool-calling, software-engineering and terminal environments (Sections 3.2.2 and 3.2.3.5). The system prompt, the coding harness and the user request are randomised per group, so that the policy does not overfit to a single harness or phrasing (Section 3.2.2.4).

**Retrieval and Hallucinations** Many application areas of Kolibri require it to answer questions based on the documents or knowledge stores of an organisation. Here, the model searches a corpus over several steps, answers from the retrieved text, and abstains when these sources do not contain the answer. Both retrieval and hallucination reduction are trained mainly in RL and cold-started in SFT on high-reward rollouts of the same environments (Section 3.1.1.4).

For the information retrieval use cases of our customers, we build knowledge bases and multi-hop question-answering tasks covering both German and English corpora across general knowledge and specialised domains (legal, electronics, hardware and aviation). During training, we randomise application-side choices, such as the retrieval parameters, models and methods and the task language, so that the model does not depend on one setup (Section 3.2.2.3 and Appendix I.1.4).

To reduce hallucinations, especially when answering questions based on retrieved context, we employ the Merlin–Arthur protocol (Deiseroth et al. 2026), which we formulate as an environment. The protocol is a game between the model (Arthur) and two players that hide sentences of the retrieved context: Merlin hides the sentences whose removal raises the probability of the gold answer most, and Morgana those whose removal lowers it most. Each question is posed three times, with the full context, with Merlin’s context and with Morgana’s, without marking which one it is (Section 3.2.2.3). The model is rewarded for the correct answer in the first two cases and for abstaining in the third, and Merlin and Morgana recompute their masks from the current policy at every step.

| Effort | API Request Parameters         | Sentence in the system prompt                                                                                                                                                                  |
|--------|--------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| none   | none,<br>enable_thinking=false | “Reasoning is disabled. Proceed straight to answering according to the user’s instructions.”                                                                                                   |
| low    | minimal, low                   | “Reasoning effort is set to low. Think briefly through only the essential steps in the user’s language, then proceed directly to the answer.”                                                  |
| medium | medium                         | “Reasoning effort is set to medium. Think through the task methodically in the user’s language, check key assumptions, and provide a well-supported answer.”                                   |
| high   | high, xhigh, max, default      | “Reasoning effort is set to high. Think carefully through the task in the user’s language, validate key assumptions, consider plausible alternatives, and prioritize correctness and clarity.” |

Table 19. Sentence the chat template writes into the system prompt for each reasoning effort, and the request values that select it.

**Reasoning Effort** In addition to no-think mode, Kolibri features three distinct reasoning effort levels, low, medium and high. The SFT stage instils these effort levels by training on reasoning traces labelled with an effort level derived from their length relative to requests of the same dataset and difficulty (Section 3.1.2.2). The RL stage then trains on all three effort levels, focusing on high effort, and further separates them via the length-dependent completion\_budget format signal of Section 3.2.2.2.

The chat template signals a chosen effort level to the model by adding a plain text section to the system prompt. After the user's system instructions and before any tool definitions, a section headed `# Reasoning effort` holds one instruction per effort (Table 19). The user cannot change the wording of this instruction through the API. If a request sets both `reasoning_effort` and `enable_thinking`, `reasoning_effort` takes precedence.

### 3.1 Supervised Fine-Tuning (SFT)

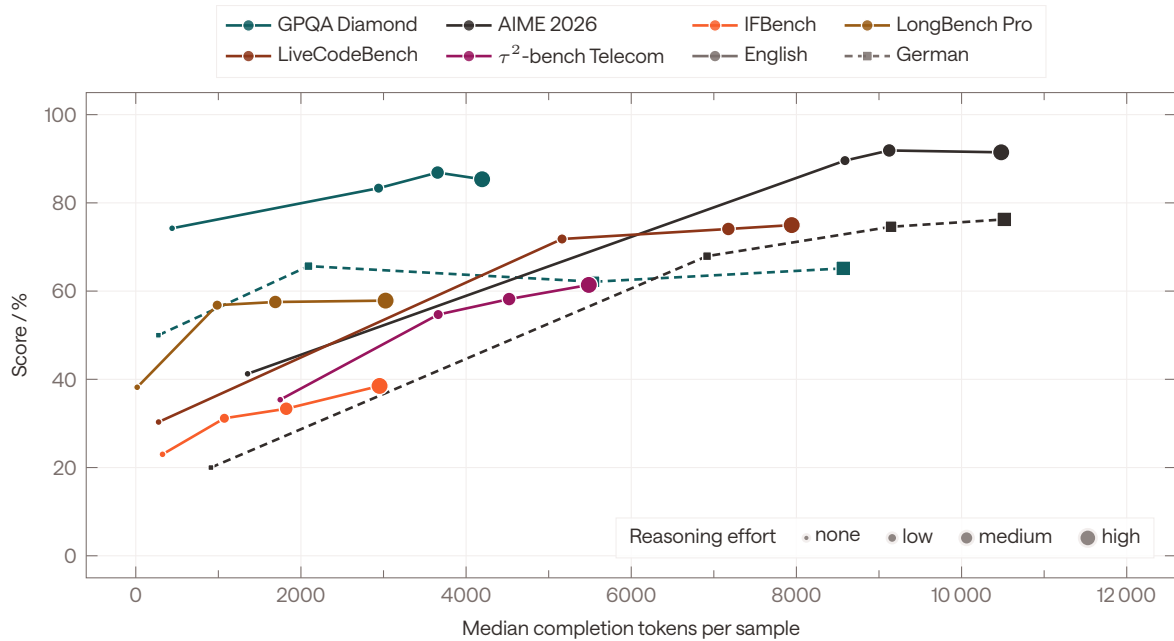


Figure 20. Score of the Kolibri SFT checkpoint on a selection of benchmarks against the median number of tokens it generates per sample, with one point per reasoning effort from none to high. The marker size grows with the reasoning effort. The German counterparts of the English benchmarks are depicted by a dashed line and square markers.

The SFT stage introduces new capabilities, such as instruction following, reasoning at a requested effort level, and tool calling. This stage serves as a *cold start* for RL: it establishes desired behaviours that the model must produce at a sufficient rate for RL to become effective in improving them (DeepSeek-AI 2025). SFT of Kolibri proceeds after the long-context training stage (Section 2.3.4) with cross-entropy loss on the assistant turns (Section 3.1.4). The released Kolibri SFT checkpoint, the weight average of two runs trained on different mixes (Section 3.1.4.3), is the input checkpoint to the RL training phase. Its performance on German and English benchmarks across different reasoning efforts is depicted in Figure 20.

**Terminology** SFT establishes these capabilities by training the model on data that demonstrates them. A *data pool* is the set of all curated datasets (Section 3.1.2.3). A generated dataset starts from *seeds*, prompts or documents, from which a *teacher model* writes the completion. We use a *user simulator* to write the follow-up user turns (Sections 3.1.1 and 3.1.1.1). During generation we, at times, utilise *backfilling* to add a generated reasoning trace to an assistant turn that has none (Section 3.1.1.3), and *reasoning prefills*, short openers that start the teacher model's reasoning trace. A training run trains on a *mixture*, a set of datasets and their sampling weights. To define training mixes, we use *MergeMix* (Section 3.1.3) during which datasets are grouped into *clusters* and a *specialist* is fine-tuned on each cluster (Section 3.1.3.1). A *weighting* assigns a coefficient to each cluster specialist and determines how they are averaged. The set of all valid weightings forms the *simplex* from which we begin sampling at a *reference mixture*, a hand-set mixture of datasets. We train one run per candidate mix and combine resulting checkpoints by *souping*, which averages the weights of runs trained on different mixes into one model (Section 3.1.4.3).

## 3.1.1 Datasets

We legally assess, compile, and process an extensive pool of fine-tuning data that combines openly available datasets, *open data* in the following, with synthetic data we generate. For some open datasets, we regenerate the completions, because their answers fail our audits (Section 3.1.1). Additionally, we generate datasets in-house for domains that are not sufficiently covered by open data, e.g., German (Section 3.1.1.1), long context (Section 3.1.1.2), agentic tasks (Section 3.1.1.3), retrieval (Section 3.1.1.4), and safety, compliance, and alignment (Section 3.1.1.5).

The main models we use to generate this data, and to regenerate parts of the open datasets, are GLM-5.2 (Z.ai 2026b), GLM-5.3 (Z.ai 2026c) and Qwen3.8-27B (Qwen Team 2026c). We employ different models for different regeneration and judging tasks as they have different strengths. For example, smaller models are generally faster for long-context tasks, but larger models are more suitable for generating strong reasoning traces. They also have different values and biases, which we consciously select or avoid (Section 3.1.1.5).

All samples undergo the same processing pipeline, which removes overlap with our evaluation benchmarks (Section 3.1.2.1) and labels each user-assistant exchange with a reasoning effort. Section 3.1.2.3 describes the resulting dataset pool. The final SFT training utilises a subset of the dataset pool found by applying a procedure called MergeMix (J. Wang et al. 2026), which proposes candidate training mixes by weighting clusters of datasets per domain (Section 3.1.3).

**Regenerated Completions** Training on wrong or low-quality completions directly teaches the model undesirable behaviour, so we check candidate datasets before they enter the pool. Sample audits of several candidate datasets found many completions of mixed quality, including wrong answers to science problems, which led to regressed benchmark performance if included in training. For these reasons, we completely regenerate completions for three subsets. We use seeds from Nemotron-Instruction-Following-Chat-v1 (NVIDIA 2025c) and Nemotron-SFT-Instruction-Following-Chat-v2 (NVIDIA 2026c) and generate new completions using both GLM-5.2 (Z.ai 2026b) and GLM-5.3 (Z.ai 2026c). For the reasoning-off split of Nemotron-SFT-Instruction-Following-Chat-v2, we generate completions without reasoning, so that the pool also holds answers for the none effort level (Section 3.1.2.2). Each conversation is extended for up to 50 turns. We employ a user simulator to write follow-up turns in the style and intent of the original user, and have a teacher model answer them. Open data holds few long conversations, so we generate them. The extension trains the model to keep following instructions and earlier context over many turns. Nemotron-SFT-Science-v2 (NVIDIA 2026d) consists of synthetic multiple-choice questions, whose completions we regenerate with Qwen3.8-27B (Qwen Team 2026c).

### 3.1.1.1 German

Large open post-training corpora such as Nemotron translate prompts and answers into other languages, German among them (NVIDIA 2025d), while keeping the reasoning traces in English. However, a user who does not read English cannot follow a chain of thought in English, which degrades the user experience

**User** Ein Bäcker verkauft drei Dutzend Brötchen zu je 0,45 €. Was ist sein Einkommen?

#### English Source Trace

① Hmm, the problem is in German. "Ein Bäcker verkauft drei Dutzend Brötchen" - ② a baker sells three dozen rolls. "Dutzend" is dozen, so  $3 \times 12 = 36$ . "Brötchen" are bread rolls, doesn't matter for the math. ③ "zu je 0,45 €" - note the comma is the decimal separator in German, so 0,45 € is 0.45 euros, not 45. So ④  $36 \times 0.45 = 16.2$ . Let me compute:  $36 \times 0.45 = 16.2$ . The answer should be in German: "Sein Einkommen beträgt 16,20 €."

#### Machine-Translated Trace

① Hmm, die Aufgabe ist auf Deutsch. "Ein Bäcker verkauft drei Dutzend Brötchen" - ② ein Bäcker verkauft drei Dutzend Brötchen. "Dutzend" ist Dutzend, also  $3 \times 12 = 36$ . "Brötchen" sind Brötchen, spielt für die Mathematik keine Rolle. ③ "zu je 0,45 €" - beachte, dass das Komma im Deutschen das Dezimaltrennzeichen ist, also ist 0,45 € 0.45 Euro, nicht 45. Also ④  $36 \times 0.45 = 16.2$ . Lass mich rechnen:  $36 \times 0.45 = 16.2$ . Die Antwort sollte auf Deutsch sein: "Sein Einkommen beträgt 16,20 €."

Figure 21. Artefacts that machine translation carries from an English reasoning trace into a German one. Left, the English trace of a reasoning model on a German problem; right, its machine translation into German. Highlights mark four kinds of artefact, numbered at their first occurrence in each trace. ① Meta-language: the trace states that the problem is in German and that the answer has to be German. ② Repetition: glosses of the German prompt for an English reader become tautologies such as "Brötchen sind Brötchen". ③ Decimal-comma aside: the trace explains the German decimal comma, which a German reader does not need. ④ English notation: the arithmetic keeps 0.45 and 16.2, while the prompt and the answer write 0,45 and 16,20.

of non-English native speakers. Translating English reasoning traces into German leaks artifacts of the English trace, such as passages it quotes from the German prompt and English notation like a decimal point in place of the decimal comma (Figure 21). We instead generate samples in which prompt, reasoning, and answer are German; tool schemas, calls, and results stay unchanged. In ablations at target scale (Appendix H.2), a model trained without this data reasons in English on German AIME'26, and a small share of it switches the reasoning to German at the cost of more loops without an answer. Upsampling the German data  $16\times$  yields more answers that are correct and German and a higher English AIME'26 score, with a German AIME'26 score that does not differ from the model without German data.

**Data Generation Pipeline** We generate German data for chat, math, science, and tool use. The German alignment and safety data are described with their English counterparts (Section 3.1.1.5). For each domain, we employ a teacher to write a German prompt based on a seed and generate a response. Seeds may be English samples, documents, or a collection of task attributes. Where a seed carries a reference answer, the response is verified against it.

Without intervention, reasoning models tend to reason in English even on German prompts. We therefore prefill the teacher model's reasoning trace with a short German opener, such as "Gegeben ist:" ("We are given:") or "Der Nutzer möchte" ("The user wants"), and let the teacher continue from it. We call this technique reasoning-prefill distillation. Each opener is randomly picked per sample from a curated list per task domain, so that the model does not learn one fixed opening phrase. An instruction in the system prompt alone keeps 67 % of the teacher's traces German throughout, and the prefilled opener raises this share to 97 % (Figure 22). A similar technique has been used in on-policy self-distillation to keep student and teacher from drifting into English during RL (Y. Liu et al. 2026). We apply it offline to a separate teacher to generate SFT data. Finally, a heuristic gate checks that both reasoning and answer are German, and an LLM language healer fixes the errors it finds in the assistant turns. A sample whose trace remains malformed or switches language keeps its answer without reasoning, as in the shared post-processing

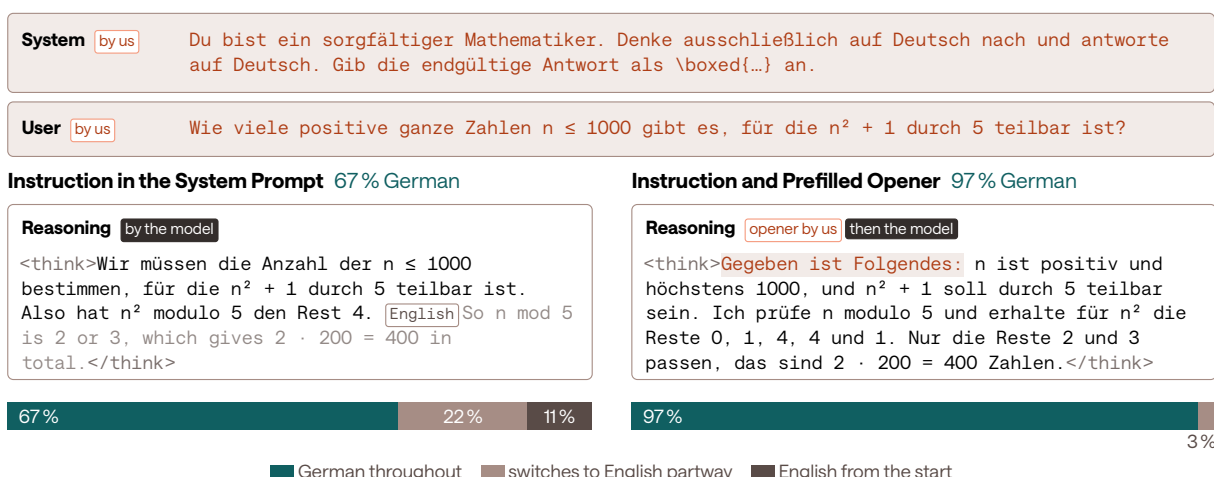


Figure 22. Steering the language of the teacher's reasoning on German prompts, shown on a math problem. Both variants send the same system and user turns. Left, the system prompt instructs the model to reason in German, and the model opens the think block itself; its trace shows the case that switches to English partway. Right, the assistant turn is sent already opened with `<think>` and a German opener, which the model continues. We checked the language consistency of both approaches on more than 5000 completions each.

(Table 20). Its answer can still be a good one, and as a sample without reasoning it trains the none effort (Section 3.1.2.2).

**Chat** General chat consists of multi-turn conversations seeded from user turns of open chat datasets and from generated everyday questions. These conversations are continued by a user simulator that deepens, constrains, contradicts, or changes the topic, which trains the model to follow instructions that change over a conversation. GLM-5.3 (Z.ai 2026c) writes the replies with reasoning, and Qwen3.8-27B (Qwen Team 2026c) those without. User turns stay casual, as in deployment, with lower case, typos and missing punctuation. Chat replies have no reference answer, so filtering them by quality would require an LLM judge. We skip this filter, as it adds compute without a known benefit. Guha et al. (2025) find that none of the answer-filtering methods they test on reasoning data, LLM verification included, improves on training with all answers.

**Tool Calling** A user simulator asks German questions about entities drawn from the category trees of German Wikipedia, and GLM-5.2 (Z.ai 2026b) answers them by searching and reading Wikipedia articles through tool calls. Both the entities and the tool calls query the live German Wikipedia at generation time rather than a fixed dump. The model thus learns to search and reason in German over German sources. Several such conversations are concatenated into long contexts that end with a request for an essay tying their topics together, which requires facts spread over the whole context and adds German long-context data. The same trajectories also enter the data pool as knowledge answers without tool calls.

English tool-calling conversations of Toucan (Xu et al. 2025) and Nemotron-SFT-Agentive-v2 (NVIDIA 2026b) are translated to German, with tool schemas, calls and results left unchanged. GLM-5.3 (Z.ai 2026c) replaces their English reasoning with a new German trace for every assistant turn. Each reasoning trace is verified against the tool call or answer that follows it. Every tool-calling sample enters the data pool once with and once without its reasoning traces.

**Math and Science** The seeds for the German math and science data are hard problems from several open math and science datasets. All of these problems come with a reference answer. A problem counts as hard if the reasoning trace of its source is at least 20 000 characters long for math and 10 000 for science. We apply GLM-5.3 (Z.ai 2026c) to rewrite each problem in natural German without seeing the reference, so that the German problem does not leak the answer. We then reapply GLM-5.3 to solve the problem, and the answer is kept only if it matches the reference. Multiple-choice science questions keep their option letters, so the answer is compared by letter. Math conversations are optionally extended to multiple turns with generated user follow-ups, which trains the model to handle follow-up questions on the same problem.

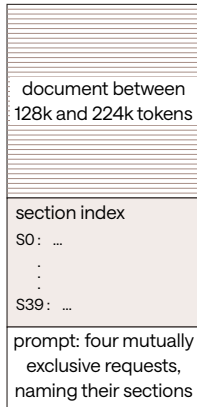
### 3.1.1.2 Long Context

The long-context extension of Kolibri (Section 2.3.4) trains the base model on documents of up to 256k tokens. We add long-context samples to SFT to preserve the model's long-context retrieval ability and to extend it to reasoning over long inputs, such as large documents, code bases, and tool outputs in agentic applications. Such tasks go beyond retrieving facts from a long input and require summarisation, multi-hop reasoning and the use of distant information in reasoning traces.

As open datasets rarely provide long-context samples, we generate our own long-context dataset (Figure 23). We use the same pre-processing as pre-training (Appendix D.1.1) and select from a corpus of curated documents with token lengths between 128k and 224k. This corpus also includes a collection of German legislative documents. Papers and German legislative documents are expected long-context inputs in Kolibri's target applications.

We split a sampled document into about 40 sections at coarse boundaries such as page breaks, blank lines, line breaks or, failing those, windows of 400 characters. Qwen3.8-27B (Qwen Team 2026c) receives the document, an index of its sections and a prompt, and writes four different simulated user requests about the document, each tied to the sections it needs, or to none for a request about the whole document (Figure 23). A request tied to distant sections must combine distant passages rather than retrieve a single fact. Requests about the opening of a document alone would not train the model to use the rest of a long input, so the prompt asks for requests spread over the whole document. We measure this spread by dividing each document into five parts of equal section count. In 77.4 % of the conversations, the requests together draw on sections from at least three of the five parts. Because model performance degrades with input length (K. Hong, Troynikov, and Huber 2025), each simulated request is answered in a separate call that receives only this request and the sections it is tied to, or the whole document for a request about the whole document. The resulting requests, reasoning traces and answers are appended to a conversation beginning with the seed document, so that later turns rely on the document and on the earlier exchanges, as in multi-turn use. This conversation forms a single training sample: the teacher model answers most requests from a few sections, while the model trained on the sample sees the full document.

The dataset holds around 78k conversations over as many documents from various open corpora. In this collection, around 11 % of the samples are German. In a proxy-scale ablation, adding this data improves AA-LCR and LongBench Pro, with the largest gains at the longest contexts (Appendix H.1).

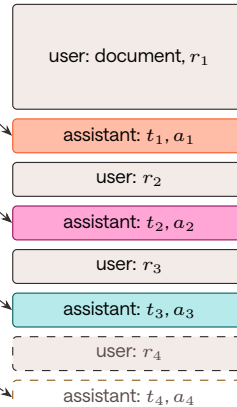
**1 Construct Query**

about 40 sections

**2 Ask**

```
[{request: "...",
  sections: [2, 11, 24, 36]},
 {request: "...",
  sections: [7, 18]},
 {request: "...",
  sections: [14, 28, 33]},
 {request: "...",
  sections: []}]
```

parsed into requests

**3 Assemble**

pairs appended in order while  
the total fits 256k tokens,  
empty answers dropped

Figure 23. Generation of one long-context conversation from a seed document. A seed document with a length between 128k and 224k tokens is split into about 40 sections  $S_0$  to  $S_{39}$ . The generator reads the document, an index of its sections, and the prompt as one message. Four requests  $r_1$  to  $r_4$  are parsed from its response. Each request is answered in its own call returning a reasoning trace  $t_i$  and an answer  $a_i$ . Turns are appended in order to a conversation that holds the document, ensuring conversations are shorter than 256k tokens.

**3.1.1.3 Tool Calling and Agentic Coding**

The agentic data of the mix consist of open agentic datasets and two software-engineering datasets that we generate. The following paragraphs describe how we format the trajectories of the open datasets and how we generate our own.

**Trajectory Formatting and Reasoning Backfilling** Tool-calling models tend to repeat a failed call instead of recovering from it (Junhao Su et al. 2026), and reasoning between actions lets an agent track its plan and handle exceptions (S. Yao et al. 2023). Thus, we add reasoning to tool calls where it is missing, in a step we call backfilling. A teacher model writes reasoning that leads to the recorded tool call, which stays unchanged. The reasoning is kept only if a second LLM call derives the same tool call from it alone. These reasoning-backfilled conversations enter the data pool along with their non-reasoning counterparts. This ensures that the requested reasoning effort decides if the model reasons, and not the source dataset. We empty the content of every assistant turn with `tool_calls`. Every step then holds only the reasoning, the tool calls and their results, which unifies the pattern that leads to a tool call across dataset sources.

Finally, we mask the loss on clearly wrong tool calls: calls whose tool response reports an unknown tool or invalid arguments, and exact repeats of the previous call that never receive a different response. These turns stay in the context, so the model learns the correction that follows without being trained to produce the error, as in Fu et al. (2025) and Slinko et al. (2026).

**Agentic Software-Engineering** Open datasets provide trajectories that resolve issues in real repositories and are verified by their tests. To increase the amount of such trajectories in our pool, we generate two

agentic datasets on the repositories of SWE-rebench V2 (Badertdinov et al. 2026), which also give the software-engineering and terminal environments of RL a cold start (Section 3.2.2). SWE-rebench V2 provides the issues, the repository snapshots and the test environments; we generate the trajectories and, for the terminal tasks, the bugs. For GitHub issues, GLM-5.2 (Z.ai 2026b) explores a snapshot of the repository with read-only tools and answers with a patch, without ever seeing the reference patch, so that the trajectory shows how the fix is found.

For terminal tasks, GLM-5.3 (Z.ai 2026c) injects a small, realistic bug into a Python repository and writes a bug report from the user’s point of view. Injected bugs turn any repository with tests into a verifiable task, beyond its existing issues. A bug is kept only if at least one test fails and at least half of the tests still pass, so that it is detectable but local. Another GLM-5.3 instance then fixes the bug in the Terminus 2 harness of Terminal-Bench (Merrill et al. 2026), and we keep the trajectories that pass the tests. Repositories of SWE-bench Verified (Jimenez et al. 2024) are excluded from both datasets, as we evaluate on them.

### 3.1.1.4 Retrieval and Hallucinations

The retrieval and the hallucination datasets train the model to answer on the basis of document collections typical at organisations and companies and to abstain when these do not contain the answer. Both are rollouts of RL environments and provide a cold start for RL. The environments and their rewards are described with RL (Section 3.2.2.3 and Appendix I.1.4).

**Retrieval and Search** We roll out GLM-5.2 (Z.ai 2026b) in the retrieval environments on their English and German training tasks, with the randomisation of the environments applied per episode. We keep a rollout when an LLM judge, Gemma-4-31B (Gemma Team 2026), accepts its answer, the gold answer appears in the retrieved text and the transcript ends on an answer. In addition, every tool call must name a served tool, and the episode must earn the full format reward. The search must also be efficient: no duplicate calls, at most 30 % near-duplicate queries, at most three searches per required hop, and at most two searches after the evidence was found. These limits train the model to stop searching once it has the evidence, which keeps latency and cost low. We also check that the reasoning of every kept trace is in the language of the user prompt, so that German users can follow it.

**Reducing Hallucinations** To train the model to abstain when the context does not support an answer, we hold out a small set of the samples of the Merlin–Arthur RL environment as seeds (Section 3.2.2.3). An 8B proxy model that we trained on the Merlin–Arthur RL environments computes Merlin’s and Morgana’s contexts for each seed. The same model generates completions on the full context and on both masked contexts (Deiseroth et al. 2026). We keep a seed when its completions answer correctly on the first two contexts, abstain on Morgana’s and pass a Command A+ (Cohere 2026) judge. Each of its three completions enters the data pool as its own sample, so that the same question appears with and without its evidence and abstention depends on the context, not on the question.

### 3.1.1.5 Safety, Compliance, and Alignment

Post-training largely determines how a model answers harmful, value-laden, and law-related requests (Ouyang et al. 2022; Bai et al. 2022). A model trained only to follow instructions also follows harmful ones, and a small share of safety data in the mix reduces this (Bianchi et al. 2024). We therefore add both English and German open safety datasets to the SFT mix. The mix also holds examples in which hypothetical users ask the model in various ways to produce copyrighted material and teach the model to reject such requests. For legal questions that name no jurisdiction, models built in the US or China often assume US law for English prompts and Chinese law for Chinese prompts (Zhizhi Wang and Suresh 2026). We acknowledge that several of our teacher models were built in China, so they carry known political biases on sensitive topics, while our model should adhere to the values of the democratic consensus in Europe.

**Value Alignment** Many requests call for a judgement that depends on values. For these, the model should judge by the values of the broad democratic consensus in Europe. We curate a set of documents whose content mostly lends itself to a discussion of values and their consequences, such as the impact of a technology or the consequences of a political decision. An LLM proposes up to five questions about each of these documents. The questions are phrased as everyday requests without words such as “ethical” or “moral”, as users rarely state that a request touches on values. The model thus learns to apply these values without such a cue. Documents that support no such questions are discarded. Open-source models answer the questions under a system prompt that centres human dignity, human rights, liberal democracy, the rule of law and pluralism. The prompt instructs the model to present positions across the democratic spectrum fairly and to keep facts, uncertainties and value judgements apart. In-house question-answer pairs on politically sensitive topics add to this data and counter the known political biases of teacher models on these topics (Section 3.1.2.1).

## 3.1.2 Data Pool Curation

The datasets above differ in format, origin and quality, so we curate them into one training pool in three steps. A shared processing pipeline brings every sample into one canonical format, drops conversations that claim another model’s identity or follow Chinese Communist Party (CCP) narratives, and removes rows that reuse benchmark text (Section 3.1.2.1). Each reasoning trace then receives a reasoning effort label from its length relative to traces of the same dataset and prompt difficulty (Section 3.1.2.2). The result is the final data pool, which we break down by domain, source and language (Section 3.1.2.3).

### 3.1.2.1 Shared Processing

Every dataset of the pool, open or generated, is processed using the same steps (Table 20). Each step either drops a whole conversation or rewrites it in place. The open datasets arrive in different formats: some carry tool calls, some place the reasoning inline in the answer, and many have no reasoning at all. The rewrites bring every sample into a common schema so that all datasets combine into one training set.

Most open datasets are generated by other models, and responses often carry their identity, such as “I am DeepSeek” or “As Qwen, I”. A pattern filter drops every conversation in which the assistant claims to be another model or to come from another lab, unless it also names our own model, and ignores denials such as “I am not Llama”. Answers of teacher and source models can also follow known political biases on sensitive topics, such as narratives of the CCP. For datasets with open-ended answers, we judge the whole conversation, including reasoning traces, with gpt-oss-120b (OpenAI 2025) to classify each as either carrying a CCP narrative, a refusal on general safety grounds, or being balanced. Conversations of the first kind are dropped.

| Step                           | Action    | Condition or change                                                                                                                                         |
|--------------------------------|-----------|-------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Final assistant turn           | Transform | Trailing messages are removed until the conversation ends on an assistant turn.                                                                             |
| Length budget                  | Drop      | The conversation exceeds the context window or 100 user turns.                                                                                              |
| Repetition loops               | Drop      | An assistant turn has many characters in repeated 10-grams (Rae, Borgeaud, Cai, et al. 2021) or long runs of one character, token or short cycle of tokens. |
| Foreign identity               | Drop      | The assistant identifies as another model.                                                                                                                  |
| CCP narratives                 | Drop      | An LLM judge finds an answer to follow narratives of the CCP; only datasets with open-ended answers.                                                        |
| Reasoning language consistency | Transform | English reasoning on an exchange in another language is removed.                                                                                            |
| Reasoning and effort label     | Transform | Reasoning inline in content moves into its own field, and every user turn is labelled with the reasoning effort of its exchange (Section 3.1.2.2).          |
| Chat schema                    | Transform | Every row is cast to one chat schema.                                                                                                                       |
| Decontamination                | Drop      | The conversation overlaps a benchmark.                                                                                                                      |

Table 20. Post-processing steps every dataset of the SFT data pool passes, in the order they run. The list is not exhaustive: some datasets pass additional filters or transformations, and some steps change nothing in datasets whose source already satisfies them.

**Decontamination** Benchmark items in the training data inflate a model’s scores beyond its performance on new inputs in real-world use (Magar and Schwartz 2022; S. Yang et al. 2023). To avoid this, we scan the SFT data pool against our benchmarks and drop the rows that reuse benchmark text.

We build on the decon engine of Olmo 3 (Allen Institute for AI 2025; Ettinger et al. 2025) and match word  $n$ -grams with  $n = 5$ . Published procedures use  $n$  from 7 to 13 (A. K. Singh et al. 2024), which suits pre-training text, where a long match marks a copied passage. In post-training data, a request can copy a benchmark item in a few words, and a long match is often boilerplate (Abdin et al. 2024; A. K. Singh et al. 2024).

Short  $n$ -grams also match benchmark instructions that repeat across its items, such as “Answer with a single letter”, which hold no benchmark question (Brown et al. 2020; Abdin et al. 2024; A. K. Singh et al. 2024). Olmo’s decon weights each  $n$ -gram by its IDF (Spärck Jones 1972) over all benchmarks together. We compute the IDF within each benchmark instead, so wording common to a benchmark counts for little. Of 90 such instructions, our filter raises 0 false positives at the same recall, against 57 with the IDF over all benchmarks.

Because a long row shares more  $n$ -grams with a benchmark question by chance alone, we score the row in windows and keep the highest window matching score. Each window is 8 times as long as the question, so its size scales with the length of the question.

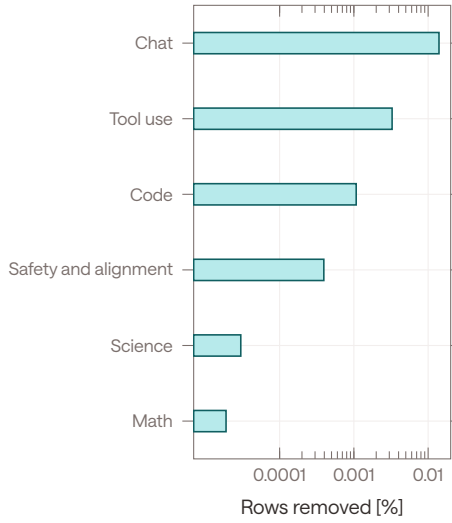


Figure 24. Percentage of the rows that decontamination drops in each domain of the SFT data pool (Figure 29), on a log axis. Long context loses no row.

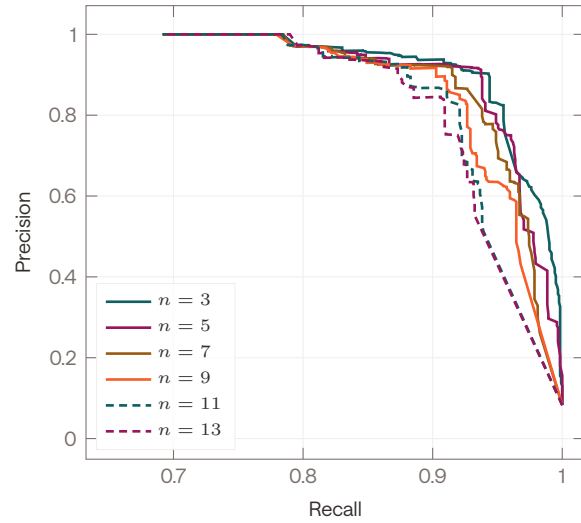


Figure 25. Precision against recall for each  $n$ -gram size over all removal thresholds, on exact and lightly edited benchmark items planted into a sample of the SFT data pool. At high recall, precision falls as  $n$  grows.

To set the threshold, we plant lightly altered copies of benchmark items (case, spacing, light paraphrasing) into a sample of the SFT data pool, weighted by the share of each dataset. We take the highest threshold that still finds them at a recall of 0.99. At this threshold, the filter drops 1.0 clean rows per 10 000, against 3.5 to 171 for  $n$  from 7 to 13 (Figure 25). Paraphrasing can still get past this filter (S. Yang et al. 2023).

Before MergeMix (Section 3.1.3), the filter removes 1097 of 22 000 436 rows of the 57 pool datasets (0.0050 %, Figure 24). Training the same SFT recipe with and without these rows shows no measurable difference on the benchmarks they match.

### 3.1.2.2 Reasoning Effort

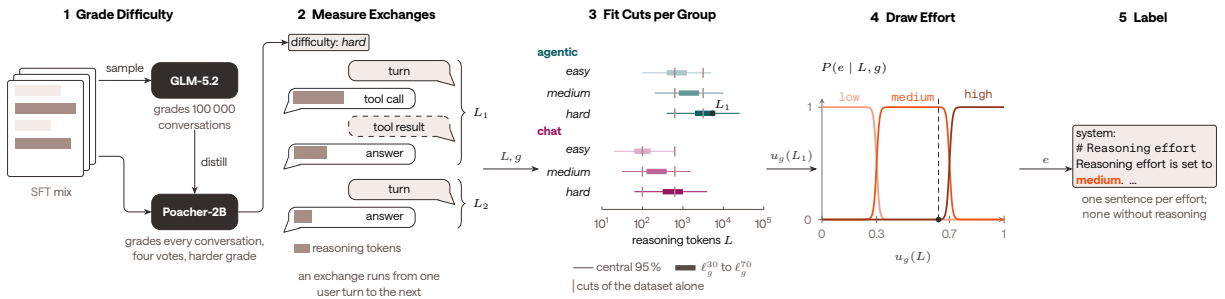


Figure 26. Assignment of reasoning effort labels to conversation turns of each sample from the SFT data pool. We first grade the difficulty of each conversation by its prompt with a Large Language Model judge. We then count the reasoning length  $L$  of each exchange, the number of reasoning tokens from one user turn to the next. For each reference group  $g$  (a combination of a dataset and a difficulty), we compute 30th and 70th percentiles  $\ell_g^{30}$  and  $\ell_g^{70}$  of  $L$ . To post-hoc label a given turn of conversation, the effort level is assigned based on the relative position of  $L$  to the 30th and 70th percentiles  $\ell_g^{30}$  and  $\ell_g^{70}$ . Rather than a hard cut-off, we use smooth blending (Equation (12)) around the bucket edges 0.3 and 0.7. This effort label is then rendered as part of the system prompt; an exchange without reasoning is labelled none.

Longer reasoning at test time raises accuracy on hard problems (Wei, Xuezhi Wang, et al. 2022; Kojima et al. 2022; OpenAI 2024; Snell et al. 2025; DeepSeek-AI 2025). However, reasoning models also spend many

tokens on easy problems (X. Chen et al. 2024), which adds latency and cost without improving the answer. Prior work trains a model to decide whether to reason at all (J. Zhang et al. 2025), or controls the reasoning length through a token budget in the prompt (T. Han et al. 2024), budget forcing at inference (Muennighoff, Zitong Yang, et al. 2025) or a length target rewarded during RL (Aggarwal and Welleck 2025). Similarly, we aim for a reasoning length that adapts to the prompt by itself, while the user keeps control over the trade-off between answer quality and latency.

Kolibri is a single hybrid model that either answers directly or reasons at one of three effort levels, similarly to Qwen3 (Qwen Team 2025a), Llama-Nemotron (Bercovich et al. 2025) and gpt-oss (OpenAI 2025). The chat template renders the requested level as one of four fixed sentences in the system prompt, for none, low, medium and high (Table 19). Unlike dedicated effort tokens, whose meaning has to be learned from scratch, the sentences describe each effort in words that the model can follow before any effort training. The SFT traces carry no effort label, so we assign one post hoc. As summarised in Figure 26, the assigned label depends on the length of a reasoning trace relative to the other traces of the same dataset and prompt difficulty. Post-hoc labels can still act as a control, because the mix contains several completions of the same prompt: 171 021 of its 1 794 428 distinct opening prompts (9.5 %) appear with two completions that both contain reasoning. In 53 723 of these pairs the two reasoning lengths differ by a factor of two or more, and under the thresholds and the soft draw described below about 37.3 % of the pairs receive two different labels. Within such a pair, the label is the only part of the input that differs, so training ties the difference in reasoning length to the label rather than to the prompt.

**Domain- and Difficulty-based Effort Labelling** Each effort level should work in every domain, so the training data must contain every effort level in every domain. We thus investigate the *reasoning length*  $L$ , the cumulative number of reasoning tokens generated in an *exchange*, the span between two successive user turns. We map turns with higher reasoning length to higher reasoning effort. For the datasets that contain reasoning (76 datasets in an earlier, larger version of our SFT data pool), the median reasoning length ranges from 27 to 90 223 tokens. Thus, a global threshold under our assumed rule would label almost every short chat-like prompt as low reasoning effort, and almost every mathematical proof as high. Therefore, we adopt dataset-specific reference groups.

Notably, reasoning length also depends on the difficulty of the prompt. Empirically, we find that the reasoning length for a fixed percentile of the *hard* prompts is up to 31.3 times that of the easy ones. We therefore fit the thresholds per *reference group*  $g = (\text{dataset}, \text{difficulty})$ . The dataset stands in for domain and task format, because each dataset of the mix comes from one source with one task shape. Grouping by dataset thus separates proofs from tool calls without a domain classifier, and the difficulty grade refines the group within the dataset.

We grade the difficulty of each conversation with GLM-5.2 (Z.ai 2026b) as the judge model, following the LLM-as-a-judge approach (Zheng, Chiang, Sheng, et al. 2023). The judge grades a conversation as *easy*, *medium*, or *hard*, based only on the knowledge and reasoning its prompt requires to be solved correctly. Grading the whole pool with GLM-5.2 is too expensive, so GLM-5.2 grades 100 000 conversations, and we distil these verdicts into a cheaper student model called *Poacher-2B*, as W. Liu et al. (2024) did for their complexity scorer. Poacher-2B then grades every conversation of the pool four times, and we round

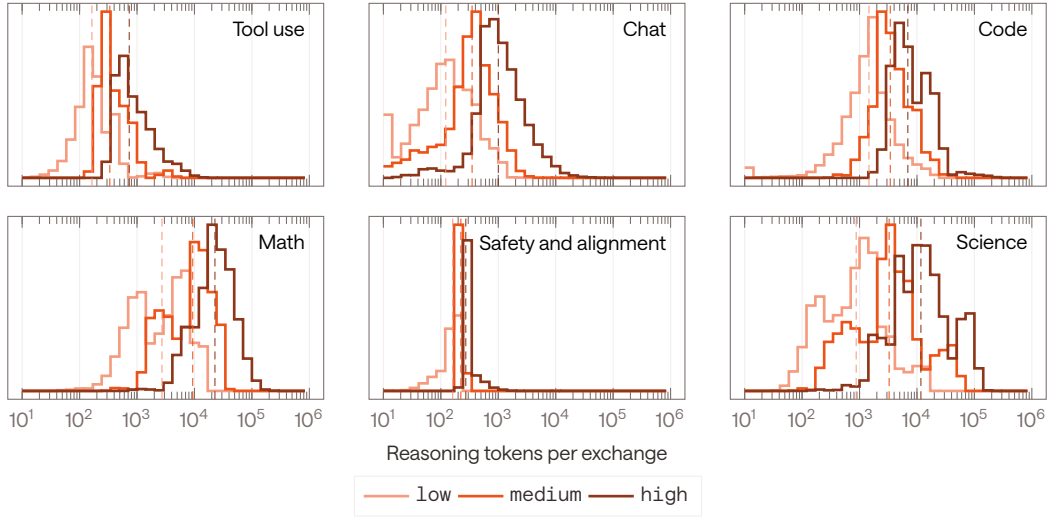


Figure 27. Reasoning tokens per effort label in the training data of the released Kolibri SFT checkpoint, read back from a sample of its dataset artefacts and pooled per domain. Dashed lines mark the label medians. The labels overlap within a domain because the thresholds differ between the datasets of a domain and because the draw of Equation (12) is soft; the same label covers different lengths in different domains.

the mean of the four grades towards the harder grade. On held-out conversations, this way of grading matches that of GLM-5.2 in 84.8 % of cases and never differs from it by more than one level.

For each reference group  $g$  with at least 200 exchanges we measure the 30th and 70th percentiles  $\ell_g^{30}$  and  $\ell_g^{70}$  of  $L$ ; smaller reference groups use the percentiles of their dataset  $\ell^{30}$  and  $\ell^{70}$ . This yields thresholds for 176 reference groups in 69 datasets, and dataset-level thresholds for all 76.

**Soft Label Assignment** A reasoning effort level is then assigned based on the relative position of  $L$  to the 30th and 70th percentiles  $\ell_g^{30}$  and  $\ell_g^{70}$  of reasoning lengths of prompts of the same dataset and difficulty. An exchange with  $L > 0$  is placed on the unit interval by interpolating in log space through these two percentiles, which fall on the bucket edges 0.3 and 0.7, and its effort  $e \in \{\text{low}, \text{medium}, \text{high}\}$  is drawn as

$$u_g(L) = \text{clip}_{[0,1]} \left( 0.3 + 0.4 \frac{\log L - \log \ell_g^{30}}{\log \ell_g^{70} - \log \ell_g^{30}} \right), \quad P(e \mid L, g) \propto \exp \left( -\frac{1}{2} \left( \frac{u_g(L) - m_e}{\beta w_e} \right)^2 \right), \quad (12)$$

where  $m_e$  and  $w_e$  are the centre and width of bucket  $e$  on the edges 0, 0.3, 0.7 and 1, and  $\beta = 0.15$ . An exchange without reasoning is labelled none. By construction, every reference group yields about 30 %, 40 %, and 30 % of low, medium and high. The draw is seeded per row, and its width keeps the label from switching at one exact token count, which the model could otherwise learn as a length at which to stop.

Figure 27 shows the assigned labels in the training data, pooled per domain. Within a domain the three efforts cover overlapping but ordered ranges of length, and the same effort covers different lengths in different domains. Because the thresholds are set per difficulty, a user does not have to estimate the difficulty of their prompt when choosing a reasoning level.

**Analysis of SFT Reasoning Effort** We evaluate the released Kolibri SFT checkpoint, the soup of  $C_1$  and  $C_2$  (Section 3.1.4.3), once per effort at temperature 1 and top-p 0.95 on the production evaluation suite. Under

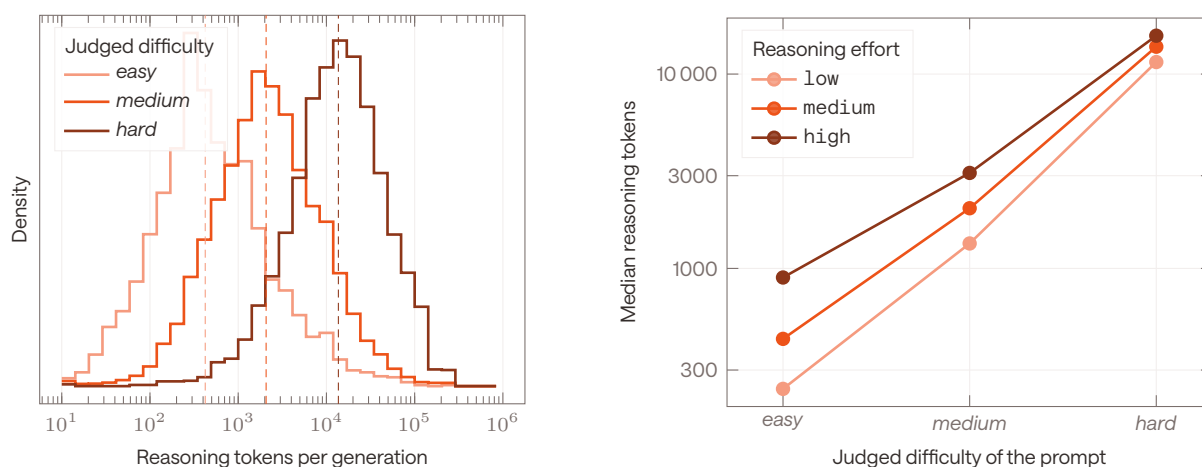


Figure 28. Reasoning length of the released Kolibri SFT checkpoint against the difficulty Poacher-2B assigns to the evaluation prompts. Left: length by judged difficulty over the three reasoning efforts, dashed lines at the medians. Right: the median length per judged difficulty, one line per effort; at a fixed difficulty the three lines show how much the effort adds, and along one line how much the difficulty adds. none is left out, as it produces an empty reasoning block.

none, the model emits an empty reasoning block. The median reasoning length orders  $\text{low} < \text{medium} < \text{high}$  on 23 of the 24 benchmarks with a measurable length. Figure 28 splits this by the difficulty that Poacher-2B assigns to the evaluation prompts of 11 graded benchmarks. With the effort fixed, *hard* prompts receive about 32.5 times the reasoning of easy ones; with the difficulty fixed, *high* receives between 1.4 and 3.7 times the reasoning of *low*. The effort thus moves the length within the range the prompt calls for, and does not override it. We average the score over the 14 benchmarks whose generations fit the length limit. Reasoning at *low* raises this mean from 38.4 under none to 64.9, and *high* reaches 67.8, with a paired standard error of 0.8 points against *low*. The step from *medium* to *high* adds 0.7 points.

The reasoning effort also works the same way in German and English. On 11 German benchmarks the median reasoning length orders  $\text{low} < \text{medium} < \text{high}$  on 10, and the benchmark median rises from 326 to 1788 tokens, a factor of 5.5; the 13 English benchmarks rise from 1160 to 2993 tokens, a factor of 2.6. German prompts thus receive shorter reasoning at every effort, which matches the data pool: the median reasoning length of its German exchanges is 368 tokens against 2078 for English. Figure 20 plots the score against the median completion tokens per effort for a selection of benchmarks, with the German counterparts of GPQA Diamond and AIME 2026 as dashed lines.

### 3.1.2.3 Final Data Pool

Our SFT data pool holds 57 datasets with 22.0M samples and 335.8B tokens from seven domains: science, math, code, tool use, chat, long context, and safety and alignment. The final model trains on mixes drawn from the pool, not on all of it (Sections 3.1.3 and 3.1.4.3). Tokens are counted as SFT trains on them, after multi-turn splitting (Section 3.1.4.2) and rendering with our Kolibri chat template. The 142.4B loss tokens, 42.4 % of the total tokens, are the unmasked tokens that provide the gradient signal. Most samples are in English (92.6 %) or German (5.0 %); the remaining 2.4 % are in other languages, including 0.8 % that mix

| Domain               | Samples (M) | Tokens (B) | Loss tokens (B) | Source (% of tokens) |                |          |
|----------------------|-------------|------------|-----------------|----------------------|----------------|----------|
|                      |             |            |                 | OS                   | Regenerated OS | In-house |
| Chat                 | 7.02        | 86.1       | 21.9            | 8.9                  | 57.9           | 33.2     |
| Code                 | 4.00        | 82.1       | 36.0            | 63.6                 | 29.8           | 6.6      |
| Math                 | 5.26        | 60.6       | 59.7            | 65.3                 | 32.3           | 2.4      |
| Long context         | 0.09        | 49.8       | 0.9             | 0.0                  | 0.0            | 100.0    |
| Tool use             | 2.05        | 35.2       | 5.1             | 10.6                 | 34.9           | 54.5     |
| Science              | 3.32        | 18.7       | 18.0            | 97.0                 | 1.4            | 1.6      |
| Safety and alignment | 0.25        | 3.2        | 0.8             | 0.0                  | 1.1            | 98.9     |
| All                  | 22.00       | 335.8      | 142.4           | 36.1                 | 31.7           | 32.1     |

Table 21. The SFT data pool by domain. Samples are conversations, in millions (M). Tokens are the actual training tokens and loss tokens are the tokens on which the training loss is computed, both in billions (B). The source columns split the domain's tokens: open-source data as released (OS); open-source data whose responses we regenerated, completed with synthesised reasoning or translated to German (Regenerated OS); and data we generated (In-house).

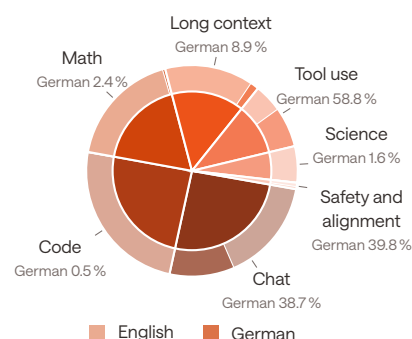


Figure 29. Share of the tokens of the SFT data pool by domain (inner disk, colours as in the adjacent Table 21) and, within each domain, by language (outer ring). Each label gives the domain and, below it, the German share of its tokens. Other languages, 3.3% of all tokens, are left out.

several. Table 21 provides the number of samples, tokens and sources for each domain, and Figure 29 illustrates the distribution of tokens across domains and languages.

### 3.1.3 Data Mixing with MergeMix

The final SFT model trains on a targeted mix drawn from the data pool, not on the whole pool. To find a strong mix, we group the datasets into 20 clusters by the capability they train (Section 3.1.3.1) and search for the share of each cluster. Fitting these shares from trained candidate mixes needs at least  $3(m + 1)$  training runs for  $m$  clusters (M. F. Chen et al. 2026). In our case, these are 63 SFT runs of 16.78B tokens each. MergeMix (J. Wang et al. 2026), which we also use in mid-training (Appendix D.2.1), needs only one shorter training run per cluster, of 4.50B tokens. We use MergeMix to train on 90B instead of 1057B tokens, i.e., 91% less training compute, fine-tuning one specialist model per cluster. To score a candidate mix, we average the specialists' parameters, each weighted by its cluster's share in the candidate mix, and evaluate the merged model. After the specialists are trained, each candidate costs one evaluation instead of one training run. We find that averaging works here because models fine-tuned from one checkpoint stay close in parameter space (Wortsman, Ilharco, et al. 2022). Berasi et al. (2026) select SFT mixes of two to four domains in the same way.

We first check that merged models rank weightings as trained models do in SFT. For this, we merge a math, a code, and a chat specialist with 12 weightings and train a model on the same mix for each of the 10 weightings that are not a single specialist (Appendix H.3). On AIME 2026 and LiveCodeBench, merging and training rank the weightings alike; on MATH-500, the scores barely change with the weighting. We conclude that the merges are a usable proxy and apply the method to the 20 clusters of the full SFT mix (Section 3.1.3.2). With 20 clusters, the merges find good candidates but do not rank them: most weightings the search finds train above the hand-set baseline, but their merged scores do not predict which of them trains best. We therefore train the most promising weightings and select the final mixes by their trained scores (Section 3.1.3.3).

#### 3.1.3.1 Clusters and Specialist Models

We run the sweep on an earlier version of the SFT data pool, with 63 datasets and 232B tokens. Unless stated otherwise, this subsection counts the tokens of data in the same way as Section 3.1.2.3. We group the datasets by their capabilities into 20 clusters: math, math proofs, code, software engineering, science, general chat, instruction following, safety, tool calling, interactive agents, terminal use, long context and German, where the larger capabilities span several clusters. We choose 20 as the finest grouping along capabilities in which every cluster still holds a few billion tokens of data. The smallest cluster holds 4.55B tokens. Each specialist is a model fine-tuned on 4.50B packed tokens of a single cluster, fewer than the smallest cluster holds, so no specialist sees its data more than once. The specialists train from the last raw checkpoint of the Kolibri long-context training (Section 2.3.4), with the SFT ablation recipe (Appendix H.4), shortened to the specialist budget and without cooldown.

#### 3.1.3.2 Searching over Cluster Weightings

We search for the weighting of the 20 clusters whose merge scores best, where a weighting is a set of 20 non-negative shares that sum to one. The set of weightings is too large for a grid, so, as in mid-training

| Weighting              | Source   | Merged | Rank | Trained | Rank |
|------------------------|----------|--------|------|---------|------|
| Baseline weighting     | Baseline | 50.2   | 49   | 51.5    | 7    |
| $\kappa = 50$ draw 15  | Sampled  | 55.1   | 29   | 56.7    | 1    |
| Refinement draw 13     | Sampled  | 60.4   | 4    | 55.4    | 2    |
| Refinement draw 7      | Sampled  | 61.2   | 2    | 54.5    | 3    |
| $\kappa = 15$ draw 6   | Sampled  | 55.0   | 30   | 52.1    | 6    |
| Refinement draw 5      | Sampled  | 61.4   | 1    | 50.0    | 8    |
| Gradient-boosted trees | Proposed | –      | –    | 54.2    | 4    |
| GP                     | Proposed | –      | –    | 52.2    | 5    |

Table 22. Weightings trained as data mixtures, by source and in order of their trained score. Merged is the score of the merge of the specialists with the weighting, ranked among all 69 merges. Trained is the score of one SFT run on data sampled with the weighting, ranked among the 8 trained weightings. Both scores are in percent (Section 3.1.3.2), and their Spearman correlation over the 6 weightings with both is  $-0.09$ .

(Appendix D.2.1), we draw weightings from Dirichlet distributions whose concentration  $\kappa$  sets how close the draws stay to the centre. The first round centres on the baseline weighting, which gives each cluster the share of packed tokens its datasets receive in the reference mix, a hand-set SFT mix that fixes the epochs of each of the 63 datasets. It draws 43 weightings, at  $\kappa = 50$  close to the baseline and at  $\kappa = 15$  further away. The second round draws 25 weightings around the best merge of the first, at  $\kappa = 25$ . With the baseline, we score 69 merges.

All merges and the 20 individual specialists are scored on 14 benchmarks, most of them from the post-training suite (Section 3.3.1), grouped into seven capabilities. Among them are math (AIME 2025, AIME 2026 and AIME 2026 IID), code (LiveCodeBench), agentic work (SWE-bench Verified, a Terminal-Bench subset (Merrill et al. 2026) and multi-turn BFCL), instruction following (IFBench), knowledge (GPQA Diamond and MMLU-Pro), long context (LongBench Pro and AA-LCR) and German (the German translation of AIME 2026 and KulturBank, Appendix H.5). The score of a cluster weighting is the mean across all seven capability averages, weighting each capability equally regardless of its benchmark count.

### 3.1.3.3 From Merges to Trained Mixes

Among the 69 merges, the baseline weighting scores 50.2 % and ranks 49, while the best merges of the two rounds reach 59.2 % and 61.4 %. We then train 8 candidate weightings as data mixes (Table 53). The candidates are: the baseline as reference, draws from both rounds including the best merge of each, and two weightings proposed by surrogate models fitted to the merge scores (Table 22). Like the specialists, each model trains from the Kolibri base model with the ablation recipe (Appendix H.4).

Training confirms that the search finds better mixes overall: 6 of the 7 other weightings train above the baseline, and the best reaches 56.7 % against 51.5 % for the baseline. However, it does not confirm the ranking of the merges. The best merge trains worst, at 50.0 %. The best trained weighting is a first-round draw from the middle of the merge ranking. Overall, merged and trained scores do not correlate (Spearman  $\rho = -0.09$ ). In the validation, the merges rank the weightings as training does, but in the sweep they do not. The sweep merges 20 specialists instead of two or three, and each specialist trains on 4.50B instead of 20.0B packed tokens. Our experiments do not show which of these two differences breaks the ranking.

### 3.1.4 Kolibri SFT Training

The SFT stage starts from the Kolibri base model and trains for 4000 steps, around 268B tokens, on packed sequences of 256k tokens (Section 3.1.4.2). At this length, the cost of a sequence depends on how its tokens split into conversations, since attention cost grows with the square of each conversation's length. With packing alone, every rank waits for the rank with the most expensive sequence. Ordering the packed sequences by attention cost on every rank, without communication between ranks, raises the throughput by 84 % (Section 3.1.4.1). MergeMix leaves four candidate mixes within a few points of each other, so we train each for longer, continue the best two to full length and soup them; the soup scores above both of its runs (Section 3.1.4.3).

#### 3.1.4.1 Data Loading

We follow common practice and pack several conversations into one sequence of fixed length with document-masked attention (Krell et al. 2021; Dao 2023), so no token attends across a conversation boundary. Conversations are assigned online with Next- $k$ -Fit (Johnson 1974): each goes into the first of  $k$  open bins that still has room, and if none has, we emit the fullest bin and open a new one. Larger  $k$  trades memory and start-up time for less padding. In ablations at a sequence length of 32k tokens, raising  $k$  from 1 to 1024 lowers the share of padding tokens in the inputs from 14.8 % to 0.6 %. Less padding means more tokens per optimiser step, so the learning rate is tuned for the chosen  $k$  (Goyal et al. 2017). Kolibri trains with  $k = 256$  bins at a sequence length of 256k tokens with a share of padding tokens of 0.001%.

With bin packing, every sequence is equally long, but not equally expensive. Attention cost grows with the square of each document's length, so a sequence holding one long conversation costs far more than one holding many short ones. Each data-parallel rank draws its sequences independently. Since all ranks synchronise after every microbatch, all ranks have to wait for the rank with the most expensive sequence, resulting in wait times and idle compute. Similar to workload balancing for variable-length training (Zheng Wang et al. 2025; Ge et al. 2025), but without communication between ranks, every rank buffers  $b$  packed sequences and orders them by their attention cost

$$C(s) = \sum_{d \in s} |d|^2, \quad x_n = s_{(\pi(n))}, \quad n = 1, \dots, b, \quad (13)$$

where  $|d|$  is the length of a document  $d$  in sequence  $s$ ,  $s_{(i)}$  the  $i$ -th sequence of the buffer of size  $b$  sorted by descending  $C$ ,  $\pi$  a permutation of  $1, \dots, b$  drawn from a seed shared by all ranks, and  $x_n$  the  $n$ -th microbatch a rank draws from the buffer. Because all ranks sort buffers of the same size drawn from the same distribution, the  $i$ -th sequence of each buffer has a similar cost on every rank, so all ranks process similarly expensive microbatches at the same time. Each rank fills a second buffer while it draws from the first and switches to it once the first is exhausted, so sorting does not stall the training.

On 64 B300 GPUs with the Kolibri SFT configuration, a buffer of  $b = 256$  raises the median throughput per GPU from 8610 token s<sup>-1</sup> without the ordering buffer to 15 850 token s<sup>-1</sup>, by 84 %.

### 3.1.4.2 Recipe

| Setting                | Value                                                                                                                |
|------------------------|----------------------------------------------------------------------------------------------------------------------|
| Base checkpoint        | Kolibri base model (Section 2.3.4)                                                                                   |
| Context length         | 256k                                                                                                                 |
| Batch size             | 256                                                                                                                  |
| Tokens per step        | 67.1M                                                                                                                |
| Steps                  | 4000                                                                                                                 |
| Peak learning rate     | Half of pre-training in every group (Table 6)                                                                        |
| Learning-rate schedule | Warmup-Stable-Decay with linear warmup over 50 steps and linear decay over the last 800 steps to a tenth of the peak |
| Parallelism            | FSDP over 128 ranks, replicated twice, on 256 GPUs                                                                   |
| Expert load balancing  | Disabled (Section 2.1.5)                                                                                             |

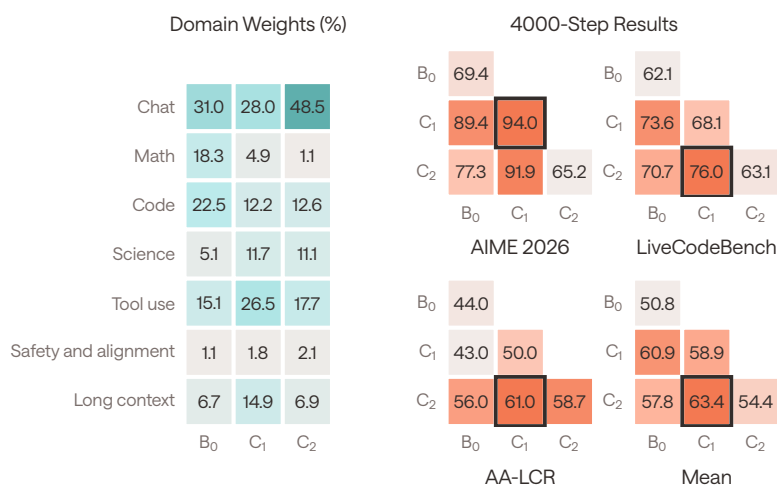
Table 23. Training settings of the SFT stage. Sequences are packed to the full context length, so a step covers 67.1M tokens. The optimiser groups, their momentum and their weight decay are those of pre-training (Table 6).

We train with cross-entropy loss only on the tokens the model produces at inference: the reasoning, answer and tool calls of each assistant turn. User turns, tool responses and the special tokens of the chat format are masked out. The loss is normalised over the whole batch, so every supervised token has the same weight and neither packing (Section 3.1.4.1) nor multi-turn splitting re-weights the mix. The chat template drops the reasoning of earlier assistant turns, so a conversation with reasoning cannot be supervised on all its turns at once. We therefore split it into one sample per user turn, with loss on the last turn only; earlier turns keep their tool calls and answers but lose their reasoning. On average, a conversation yields 2.27 samples. A shuffle buffer keeps the samples of one conversation from dominating a batch.

### 3.1.4.3 Candidate Runs and Souping

This stage selects the SFT model we pass on to RL. The MergeMix sweep leaves four candidate mixes within a few points of each other at the ablation budget (Section 3.1.3.3), so we compare them at the length of a full SFT run and soup the best of them. Alongside, we train a baseline mix,  $B_0$ , that samples the datasets of the final pool about naturally, with the German data upweighted. It is a sanity check that reweighting the domains improves on natural sampling. Starting from the Kolibri base model (Section 3.1.4), we first fine-tune one model per mix for 2500 steps, 168B packed tokens. Two of the four candidates give clearly the best of these models, which we call  $C_1$  and  $C_2$ . Over the benchmarks of Figure 30, they reach means of  $59.9 \pm 1.8\%$  and  $55.7 \pm 2.1\%$ , against at most  $52.6 \pm 1.9\%$  for the other three models. At full length, the candidates thus spread further apart than at the ablation budget. We continue training  $C_1$ ,  $C_2$  and  $B_0$  to 4000 steps, 268B packed tokens.

**Souping** The three mixes weight the domains differently, and their models lead on different benchmarks:  $C_1$  on AIME 2026 and LiveCodeBench,  $C_2$  on AA-LCR (Figure 30). To combine these strengths, we soup (Wortsman, Ilharco, et al. 2022) every pair of the three models: each parameter of a soup is the mean of its values in the two models. Averaging works because both models are fine-tuned from the same base checkpoint and stay close to each other in weight space (Wortsman, Ilharco, et al. 2022). It is the operation of the MergeMix sweep, applied to two full runs instead of specialists. Maiti et al. (2025)



**Figure 30.** Domain weights of the three mixes trained for 4000 steps (left) and the scores of their models and of the soup of each pair (right), in %. The domain weights are shares of the trained tokens. In each score panel, the diagonal holds the three models and each cell below it the soup of the models named by its row and column; colours are scaled per panel, and the best cell is framed. AIME 2026, LiveCodeBench and AA-LCR are the math, code and long-context benchmarks whose scores spread most over the mixes and soups. The mean is over 12 benchmarks: math (AIME 2025, AIME 2026 and AIME 2026 IID of Section 3.1.3), knowledge (GPQA Diamond and KulturBank, Appendix H.5), agentic tool use (Tau2-Bench telecom, multi-turn BFCL v3 and BrowseComp Lite), code (LiveCodeBench), instruction following (IFBench, loose accuracy) and long context (AA-LCR and LongBench Pro).

show that a soup of models strong in different benchmark categories can score above each of them on multilingual, tool-calling and math benchmarks. We soup only pairs, with equal weights.

**Results** Every soup scores above both of its models on the mean (Figure 30). The soup of C<sub>1</sub> and C<sub>2</sub> reaches the highest mean,  $63.4 \pm 1.8$  %, against  $58.9 \pm 1.8$  % for C<sub>1</sub>, the best single model, and  $30.0 \pm 2.0$  % for the base model. B<sub>0</sub> scores lowest of the single models, so the candidate mixes pass this sanity check. On AIME 2026, the soup scores below C<sub>1</sub>, within noise. Selection and these results use the same benchmarks, so the scores of the selected soup are optimistic.

**Math Data** Guha et al. (2025) find that math scores, AIME among them, rise with the amount of math reasoning traces in SFT. Yet C<sub>1</sub> scores highest on AIME 2026, although math makes up only 4.9 % of its tokens, against 18.3 % for B<sub>0</sub>. Compared with B<sub>0</sub>, C<sub>1</sub> drops or downsamples in particular the math datasets with on average easier problems and longer reasoning traces. B<sub>0</sub>, which keeps these datasets, reaches  $69.4 \pm 9.3$  % on AIME 2026, within noise of the base model before SFT at  $74.6 \pm 10.7$  %. On AIME 2026 IID, the base model scores exactly the same, 74.6 %, so its AIME 2026 score does not stem from memorised problems. One explanation is that the long reasoning traces lengthen the generations of the model until they no longer finish. In line with it, B<sub>0</sub> ends more AIME 2026 generations in a repetition loop or at the context limit, 7.3 %, against 2.1 % for the base model and 0.0 % for C<sub>1</sub>. Other causes are possible, as C<sub>1</sub> and B<sub>0</sub> also differ in other domains.

As the best model on the mean, the soup of C<sub>1</sub> and C<sub>2</sub> after 4000 steps becomes the released SFT checkpoint and the starting point of RL.

## 3.2 Reinforcement Learning (RL)

While SFT adapts the base model towards user interaction and tool use, RL further expands its capabilities by training it on its own rollouts against verifiable rewards. In this section, we describe the RL training stage that starts from the SFT checkpoint. It consists of a single training run on 38 environments that span math, coding, reasoning, instruction following, tool use and agentic tasks in English and German (Section 3.2.2). We train asynchronously on a token-level objective that combines a policy-gradient term, a squared log-ratio regulariser (Kimi Team 2026a; Team et al. 2025) and targeted On-Policy Self-Distillation (OPSD) for low-reward groups (S. Zhao et al. 2026), as outlined in Section 3.2.1. For the policy-gradient term, we use group-relative advantages (Shao et al. 2024; Z. Liu et al. 2025), a Binary Total Variation (B-TV) trust region (P. Qi, X. Zhou, et al. 2026) and a per-token bound on the gradient norm of the logits. The trainer further replicates the FP8 quantisation and expert routing of the inference engine to keep both policies numerically aligned. We push FP8 weights to all inference replicas via Remote Direct Memory Access (RDMA) after every optimiser step without discarding in-flight requests, and serve tens of thousands of concurrent rollouts with isolated code sandboxes, as described in Section 3.2.3. Section 3.2.4 presents the Kolibri RL run, its results and an analysis of its training dynamics. Finally, Section 3.2.5 summarises the lessons we learned over several iterations of our RL setup, from early experiments to Kolibri Origin and Kolibri.

**Terminology** For each prompt, we sample a *group of sequences*, where each sequence consists of the complete single- or multi-turn rollout for this prompt. A *rollout step* comprises multiple model output tokens, and the environment response that follows it, e.g., a tool response. A *turn* comprises all rollout steps between two user messages, such that one user message may be followed by several rollout steps with tool calls before the model responds to the user. A sequence may consist of several *segments*, which are several rollout steps that extend the same tokenised context. We split sequences into segments due to, e.g., chat-template changes that adapt previously rendered contexts (Section 3.2.3.2). These splits can, but need not be along turn boundaries. We perform optimiser steps on batches of sequences, regardless of their number of segments. We only train on the tokens that are newly generated by the model in each segment, masking out re-rendered context as well as system, user and tool-response tokens.

### 3.2.1 Training Recipe

#### 3.2.1.1 Optimisation Objective

We optimise a policy-gradient term  $\ell_t^{\text{PG}}$  based on group-relative advantages. For stability purposes, we add a squared log-ratio regulariser  $\ell_t^{\text{reg}}$ . We additionally bootstrap difficult groups via a targeted OPSD loss  $\ell_t^{\text{OPSD}}$ , resulting in the objective

$$\mathcal{L}_{\text{RL}}(\theta) = \frac{1}{|\mathcal{T}| + |\mathcal{F}|} \left( \sum_{t \in \mathcal{T}} (\ell_t^{\text{PG}} + \ell_t^{\text{reg}}) + \lambda_{\text{OPSD}} \sum_{t \in \mathcal{F}} \ell_t^{\text{OPSD}} \right). \quad (14)$$

Here,  $\mathcal{T}$  is the set of trainable tokens for the RL objective across all groups of the batch, while  $\mathcal{F}$  denotes the set of active self-distillation tokens and  $\lambda_{\text{OPSD}}$  the weight of their loss. The normalisation factor keeps the loss scale stable across variable-length sequences and weights all trainable tokens equally. Table 59 in Appendix I.5 lists the objective hyperparameters.

**Training Reward** The advantages of the policy-gradient term  $\ell_t^{\text{PG}}$  are derived from *training rewards*  $R_{i,e}$  for each segment  $y_{i,e}$  of sequence  $y_i$ . The training reward of a segment is the environment reward  $R_i^{\text{env}}$  of its sequence, scaled by segment-level format signals, as described in Section 3.2.2.2.

**Group-Relative Advantages** Given a group  $\mathcal{G}$  of sequences  $y_i$ , we compute the group-relative advantage of each segment as

$$A_{i,e} = R_{i,e} - \bar{R}, \quad \text{with} \quad \bar{R} = \frac{1}{|\mathcal{G}|} \sum_{j \in \mathcal{G}} \frac{1}{E_j} \sum_{e=1}^{E_j} R_{j,e}. \quad (15)$$

Each generated token  $t$  inherits the advantage of its segment, i.e.,  $A_t = A_{i,e}$  for tokens  $t$  of segment  $y_{i,e}$ . The mean reward  $\bar{R}$  is computed on a sequence level, over the  $E_j$  segments of each sequence  $y_j$ , to prevent segment splits from overweighting individual sequences. Sequences that fail due to infrastructure errors, e.g., in the environment or the inference engine, are removed from their group before computing the baseline. Similar to Dr. GRPO (Z. Liu et al. 2025), we skip the advantage normalisation originally used by GRPO (Shao et al. 2024). Following Decoupled Clip and Dynamic Sampling Policy Optimization (DAPO)’s dynamic sampling (Q. Yu et al. 2025), we generally drop groups in which all sequences obtain the same environment reward. However, we keep groups with a pass rate of 1 to apply format penalties to passing sequences, and only drop the group if its sequences all incur the same format penalty. We re-sample dropped groups two additional times from the same environment to keep environments with high drop rates from being under-represented in training.

**Policy-Gradient Term** To correct for the off-policy shift of asynchronous training (Section 3.2.1.5) and the training–inference mismatch between the inference engine and the trainer (Section 3.2.1.3), the policy-gradient term weights each token’s advantage by the *policy ratio*. For a sampled token with probability  $p_{\theta,t}$  under the *training* policy and  $p_{\mu,t}$  under the *inference* policy used to generate it, this ratio is  $r_t = p_{\theta,t}/p_{\mu,t}$ . Following Divergence Proximal Policy Optimization (DPPO) (P. Qi, X. Zhou, et al. 2026), we replace standard PPO-style clipping (Schulman et al. 2017) with a B-TV mask. Given thresholds  $\epsilon_+$  and  $\epsilon_-$ , this mask suppresses token updates when the sampled-token probability moves too far from the inference policy in the direction favoured by the advantage, i.e.,

$$\tilde{\ell}_t^{\text{PG}} = -k_t A_t r_t, \quad k_t = \begin{cases} \mathbf{1}[p_{\theta,t} - p_{\mu,t} \leq \epsilon_+], & A_t > 0, \\ \mathbf{1}[p_{\theta,t} - p_{\mu,t} \geq -\epsilon_-], & A_t \leq 0. \end{cases} \quad (16)$$

We additionally scale this term per token to bound its gradient norm, as detailed in Section 3.2.1.2.

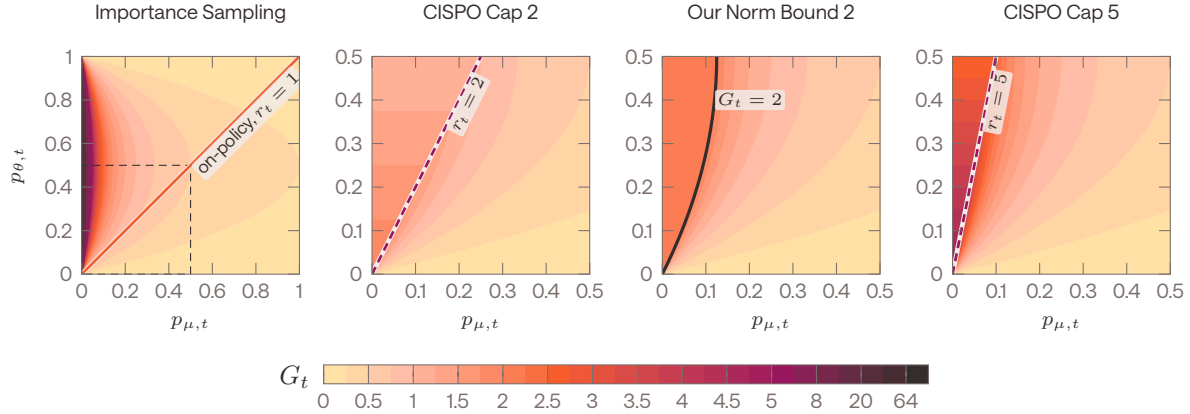


Figure 31. Logit-gradient norm  $G_t$  of a token at unit advantage  $|A_t| = 1$ , over its inference probability  $p_{\mu,t}$  and training probability  $p_{\theta,t}$ . Left: The diagonal marks on-policy probabilities, and the dashed square the bottom left corner  $p_{\mu,t}, p_{\theta,t} \leq 0.5$ . The other panels show the norm in that corner under CISPO’s importance-weight cap at  $r_t = 2$  (MiniMax 2025), under our norm bound  $G_{\max} = 2$  and under CISPO’s cap at  $r_t = 5$ . Above CISPO’s dashed line, the norm  $G_t$  is limited to the cap times  $1 - p_{\theta,t}$ , and left of the dark parabola  $G_t = G_{\max}$ ,  $c_t < 1$  scales down the token’s gradient to exactly the bound.

### 3.2.1.2 Per-Token Gradient-Norm Bound

The objective  $\tilde{\ell}_t^{\text{PG}}$  does not bound the gradient that a single token puts on the logits  $\mathbf{z}$ . Since  $\tilde{\ell}_t^{\text{PG}}$  only depends on the policy through the log-probability of the sampled token given context  $s_t$ , its gradient on the logits is

$$\nabla_{\mathbf{z}} \tilde{\ell}_t^{\text{PG}} = \frac{\partial \tilde{\ell}_t^{\text{PG}}}{\partial \log p_{\theta,t}} (\mathbf{u}_t - \pi_{\theta}(\cdot | s_t)),$$

i.e., the difference between the one-hot vector  $\mathbf{u}_t$  of the sampled token and  $\pi_{\theta}(\cdot | s_t)$ , scaled by the derivative of the objective with respect to that log-probability. As shown in Appendix I.3, the  $\infty$ -norm of this gradient equals the magnitude of its entry at the sampled logit (see Appendix I.3.2 for the 1- and 2-norms). For unmasked tokens, it evaluates to

$$\|\nabla_{\mathbf{z}} \tilde{\ell}_t^{\text{PG}}\|_{\infty} = |A_t| G_t, \quad G_t = r_t (1 - p_{\theta,t}) = \frac{p_{\theta,t} (1 - p_{\theta,t})}{p_{\mu,t}}. \quad (17)$$

The left panel of Figure 31 shows  $G_t$ , i.e., the norm at unit advantage. For on-policy sequences,  $r_t = 1$  and thus  $G_t \leq 1$ . However, when training and inference policy diverge, the norm grows with the policy ratio and becomes unbounded for  $p_{\mu,t} \rightarrow 0$ , such that a single token with a low inference probability and moderate training probability can dominate the update. The B-TV mask does not remove such tokens, since it only acts on absolute probability changes in the direction favoured by the advantage, independent of the actual policy ratio. We therefore directly bound the norm by scaling our objective as

$$\ell_t^{\text{PG}} = c_t \tilde{\ell}_t^{\text{PG}}, \quad c_t = \text{stopgrad} \left( \min \left( 1, \frac{G_{\max}}{G_t} \right) \right),$$

where  $G_{\max}$  bounds the logit gradient of the policy-gradient term at  $G_{\max} |A_t|$ , leaving tokens below the bound unchanged. Computing  $c_t$  comes at effectively no additional cost, as it only requires the two sampled-token probabilities that the trainer already computes. Equivalently, the bound caps the effective policy ratio  $c_t r_t$  at  $G_{\max} / (1 - p_{\theta,t})$ , which compensates the softmax factor  $1 - p_{\theta,t}$  of Equation (17). To first order, this process also bounds the change of the sampled token’s probability, as Appendix I.3.3 shows.

In contrast, ratio-based bounds such as the importance-weight cap of Clipped IS-Weight Policy Optimization (CISPO) (MiniMax 2025) limit  $r_t$  regardless of the training probability, and thus ignore the softmax factor. They down-weight tokens with a high training probability even though their logit gradient may be small, while tokens with a low training probability can exceed the norm bound before they reach the cap. As the second panel of Figure 31 shows, a CISPO cap of 2, which matches the norm bound at low inference probabilities, is far more restrictive at high ones. The common cap of 5 in turn allows much larger gradients than the norm bound near the on-policy diagonal at low inference probabilities, as the right panel shows.

The B-TV mask (P. Qi, X. Zhou, et al. 2026) of Equation (16) zeros out only one side of the diagonal band  $-\epsilon_- \leq p_{\theta,t} - p_{\mu,t} \leq \epsilon_+$  in Figure 31. In particular, it still keeps a token whenever the advantage is negative and the training probability is higher than the inference probability, in which case the norm bound scales down the token's gradient. In addition to this per-token bound, we clip the global gradient norm.

### 3.2.1.3 Training–Inference Consistency

While training and inference policies share their weights, they compute token probabilities with different engines, kernels and numerical precisions. The resulting training–inference mismatch between inference probabilities  $p_{\mu,t}$  and training probabilities  $p_{\theta,t}$  causes recently generated sequences to be instantly off-policy, which in turn can destabilise training (F. Yao et al. 2025; P. Qi, Z. Liu, et al. 2025). We therefore align both engines as much as possible, and additionally rely on several regularisation mechanisms outlined in Section 3.2.1.1. That being said, the engines use different attention, MoE and matrix-multiplication kernels, and none of the kernels are batch-invariant (H. He and Thinking Machines Lab 2025), which causes small mismatches. Rollouts sample from the full distribution at temperature 1. We do not use top- $p$  or top- $k$  truncation, so the trainer directly computes  $p_{\theta,t}$  from its unmodified logits.

**FP8 Quantisation-Aware Training** Serving the policy with FP8 weights and an FP8 KV cache raises the throughput of rollout generation, which can dominate the cost of RL, by a factor of two to three in our runs, while also improving per-request decode speed. If the trainer uses unrounded BF16 weights and activations, however, the rollouts come from a different policy than the one being optimised. This training–inference mismatch biases the policy gradient, and we observed RL training diverge in this configuration. The trainer therefore uses FP8 quantisation-aware training (Jacob et al. 2017), using the same quantisation as the inference engine. Appendix I.4.1 provides details.

**Routed-Expert Replay** The inference engine records the experts that its MoE routers select for every generated token, allowing the trainer to replay these choices (W. Ma et al. 2025). Tokens without recorded routing, such as padding, use the routers of the trainer. Full activation checkpointing (T. Chen et al. 2016) recomputes each layer with the same experts in the backward pass.

### 3.2.1.4 Auxiliary Objectives

**Squared Log-Ratio Regulariser** We add a squared log-ratio term (Kimi Team 2026a) to softly penalise policy drift as

$$\ell_t^{\text{reg}} = \tau_{\text{KL}} (\log r_t)^2,$$

where  $\tau_{\text{KL}}$  controls the strength of the regulariser. This term acts on every trainable token, including those with an active B-TV mask, thus following the DPPO loss of PRIME-RL (Prime Intellect 2025). We do not employ a KL term to a frozen reference model, and do not add an explicit entropy bonus to our objective.

**Targeted On-Policy Self-Distillation** We additionally employ targeted OPSD (S. Zhao et al. 2026) for selected multi-turn environments to provide a dense learning signal for groups with low reward variance and low environment rewards, which would otherwise have been dropped by the group-relative objective. Of these groups, we randomly pick a rollout which has the most turns with a tool response error. Each error class (listed in Table 60) is associated with a targeted remedial hint, which we add to the sequence context as an additional message exactly at the turn where the error occurred (similar to Cursor 2026). Then we recompute the full-vocabulary distribution for every token of the failed completion under the current training policy. The resulting hinted distributions act as the teacher, and we minimise their forward KL divergence to the distribution of the training policy under the original, unhinted context as

$$\ell_t^{\text{OPSD}} = D_{\text{KL}}(\text{stopgrad}(\pi_{\theta}(\cdot | s_t, h_t)) \parallel \pi_{\theta}(\cdot | s_t)),$$

where  $h_t$  denotes the remedial hint and `stopgrad` stops the gradient towards the teacher. For these rollouts, we disable the RL loss. A single group can thus only contribute to either the group-relative RL or OPSD part of Equation (14). Different from S. Zhao et al. (2026), the teacher is the live training policy and its privileged context provides dense teacher distributions without requiring a ground-truth solution. We specialise OPSD to tool-use and terminal environments. Appendix I.6 gives the hyperparameters, and Table 61 lists these environments. Table 60 lists all failure classes with their remedial hints.

### 3.2.1.5 Asynchronous Training

Rollouts and optimisation run continuously and asynchronously. Rollout threads drop groups without learning signal, such as groups whose sequences all receive the same reward (Section 3.2.1.1), and every other group enters the replay buffer as soon as it finishes. The staleness of a group is the number of optimiser steps since its oldest sequence started. The trainer samples groups from the buffer at random and without replacement, so each group is used in at most one optimiser step. After every optimiser step, the weights of all inference replicas are updated in flight (Section 3.2.3.4): active requests keep their KV cache and continue under the new weights, so different policy versions can generate parts of the same sequence. The prefix cache can likewise survive a weight update, so a new request may reuse KV entries computed by an older policy. Training is therefore only approximately on-policy. We bound the off-policy shift by evicting groups from the buffer once they exceed a maximum staleness (Table 59), and the policy ratio of Section 3.2.1.1 corrects for the remaining shift.

**Balancing the Loop** The training loop runs at the speed of its slowest part. If groups are produced faster than the trainer consumes them, the trainer is the bottleneck. If the trainer consumes groups faster than they are produced, e.g., when either inference or the environments are saturated, it idles. Environments with saturated resources leave both the inference engine and the trainer underused. Ideally, environment capacity is large enough that environments are never the bottleneck, and the GPUs are split between trainer and inference so that the loop is balanced, producing groups as fast as they are consumed. We therefore allocate nodes such that the trainer is barely the bottleneck during the later stages of training. Appendix I.4.2 explains how adding GPUs to a balanced loop shortens the optimiser steps but not the rollouts, which increases staleness. As a remedy, we scale the batch size with the GPU count.

### 3.2.2 Environment Mix

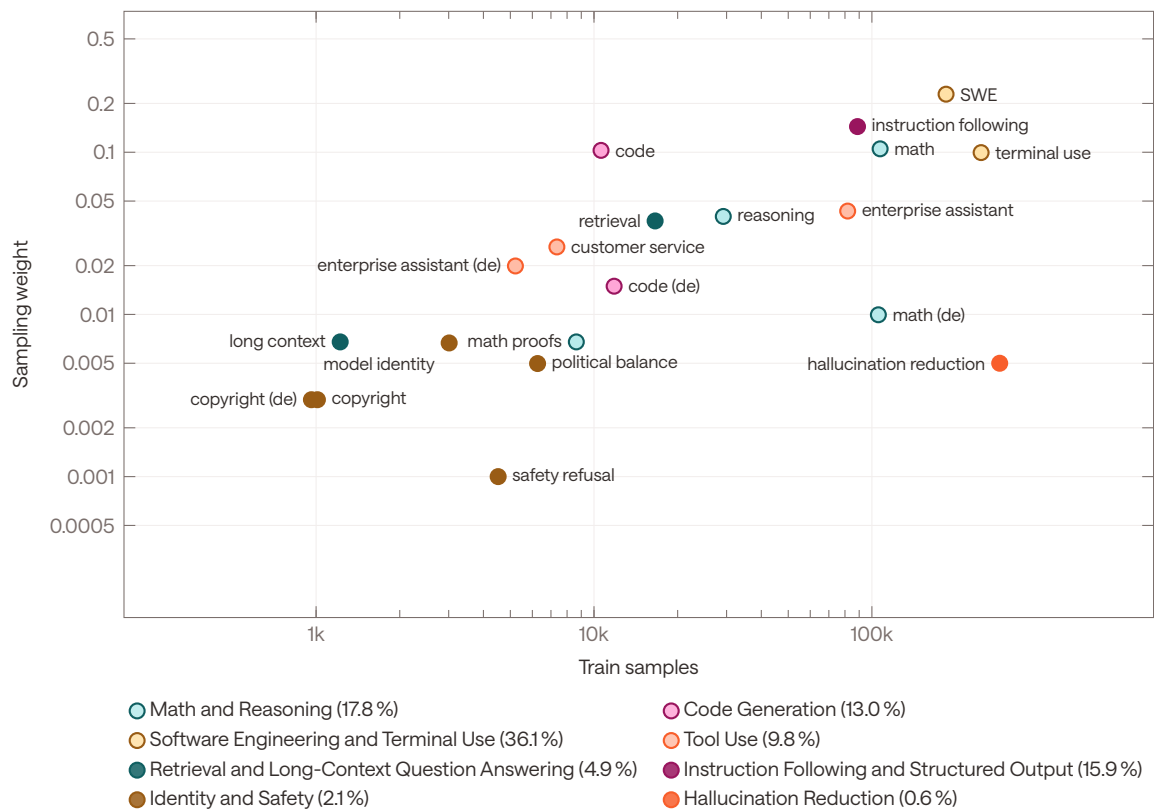


Figure 32. Our RL environment mix. Every point represents one environment or a pool of related environments, plotted over its number of train samples and the weight we assign during training (c.f., Section 3.2.2.5); Colour and mark shape denote the environment's group; the percentage in the legend is the group's share of the summed sampling weight. Both axes are logarithmic. The train splits hold 1209 965 rows in total. We deliberately upweight several agentic environments as well as competitive programming, while keeping the total weight of the Merlin–Arthur environments comparatively small. *hallucination reduction* holds the unanswerable questions of two seed datasets. Table 55 shows exact per-environment values with step and token budgets.

The final RL run trains on 38 environments, 31 English ones and 7 German variants. Each environment carries a sampling weight, the rate at which the rollout threads draw samples. Figure 32 plots sampling weight against data samples for every environment. Each environment also fixes a step and a completion-

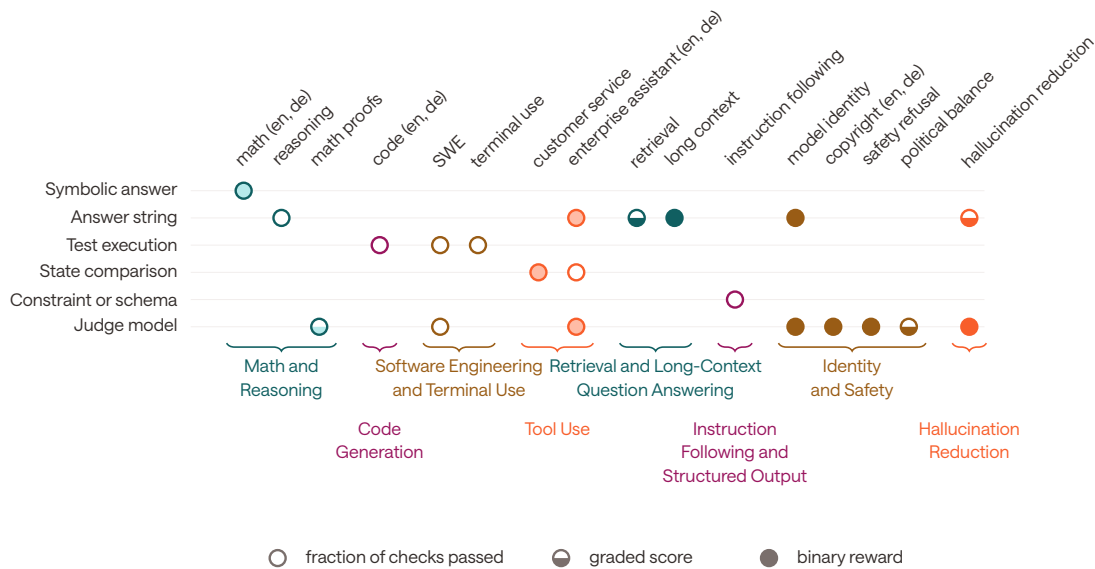


Figure 33. The verifier of every RL environment and the shape of its reward. A filled disc marks a binary reward, a hollow one the fraction of checks passed. A half-filled one indicates a graded score, i.e., the token F1 against the gold answers or a judge's grade of 0, 0.5 or 1. A column with several marks combines verifiers, either within one environment or across the environments it pools. The Merlin–Arthur reward is signed (Section 3.2.2.3). Columns are grouped and coloured as in Figure 32.

token budget per sequence (Table 55). All environments grade a sequence with a verifiable reward in  $[0, 1]$ , except for the Hallucination Reduction environments, whose signed rewards lie in  $[-1, 1]$  (Section 3.2.2.3).

### 3.2.2.1 Reward Structure by Environment

Figure 33 shows the verifier and the reward shape of every environment. The frozen judge model (Section 3.2.3) is used only where the graded object is natural language, a conversation, or the model's intent: it grades math proofs, simulates the user and grades natural-language assertions for the customer-service environment, grades safety refusal, and decides on answers and abstentions that miss the exact match. Verifiers that execute untrusted or unbounded code on the rollout workers (Section 3.2.3) run in a killable process pool with a timeout, so a wedged check scores zero without stalling training. Appendix 1.1.1 details the reward of each environment.

### 3.2.2.2 Environment-Specific Format Signals

Beyond solving the task defined in each environment, we expect the model to produce well-formatted outputs by, for example, making well-formed tool calls and keeping its language consistent across thinking and content blocks. We compile a list of output format requirements in Table 24. Each of these requirements defines a format signal that influences training reward through the aggregation described next.

**Combining Multiple Rewards** Format signals act multiplicatively on environment reward. They are only applied to positive rewards to prevent training on groups of episodes that do not solve the environment

task but vary in the quality of their output format. Since SFT already trains the model to produce well-formatted output, format signals are sparsely active in practice. As a result, they need to be aggregated in a way that allows a single format signal to have a significant impact on the final reward, while ensuring that environment reward remains the primary training signal, even if multiple format defects are found simultaneously.

Let  $s_k \in [0, 1]$ ,  $k \in \{1, \dots, K\}$  denote one of  $K$  format signals, where  $s_k = 1$  signals good format and smaller values signal a format defect. We fix constants  $f_k \in (0, 1]$  that configure the strength of format signals relative to each other and rescale the signal to  $g_k = f_k + (1 - f_k)s_k \in (f_k, 1]$ .

The final reward  $R_i$  of environment  $i$  is computed by scaling the environment reward  $R_i^{\text{env}} > 0$  with a  $p$ -norm aggregation of format signals

$$R_i = R_i^{\text{env}} \left[ m_{\min} + (1 - m_{\min}) \exp \left( - \left( \sum_k [-\log g_{i,k}]^p \right)^{1/p} \right) \right]. \quad (18)$$

The global multiplier  $m_{\min}$  sets the minimum fraction of the environment reward that is retained irrespective of output format, and the aggregation norm  $p$  controls how quickly simultaneous defects compound. For  $R_i^{\text{env}} \leq 0$  no format reward signals are applied.

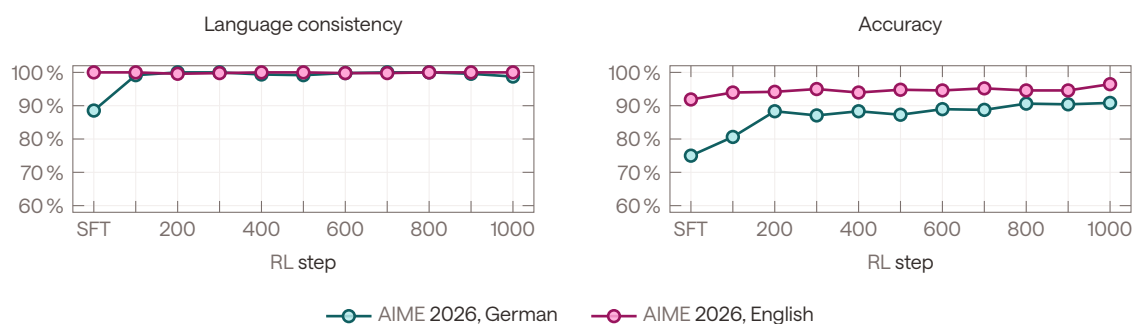
| Scope                | Signal and clean-output condition                                                                                                                                                                                                                                                                                                                                        |
|----------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Universal            | <i>Non-empty output.</i> The final assistant step contains visible text or a parsed tool call.<br><i>No leaked tool call.</i> No literal tool-call markup leaks into the visible output.<br><i>No script leak.</i> No Chinese, Japanese and Korean (CJK) text leaks into a response that is otherwise non-CJK, and no generated token matches the banned-script pattern. |
| Tool Use             | <i>Tool in schema.</i> Every called tool was offered in the row's tool schema.<br><i>Well-formed tool call.</i> The emitted tool-call structure is well formed.<br><i>Arguments in schema.</i> Tool arguments use the declared names and types.<br><i>Stable rendering.</i> The tool-call rendering remains consistent when the conversation is reconstructed.           |
| Language Consistency | <i>Reasoning language.</i> Hidden reasoning uses the expected language.<br><i>Content language.</i> Visible prose uses the expected language.                                                                                                                                                                                                                            |
| Completion Budget    | <i>Completion budget.</i> Generated length stays below the effort-dependent threshold; beyond it the score decreases with sequence length.                                                                                                                                                                                                                               |
| Math                 | <i>Boxed answer.</i> The answer follows the requested boxed-answer format.                                                                                                                                                                                                                                                                                               |
| Reasoning            | <i>Answer prefix.</i> The visible answer contains the required final-answer prefix.                                                                                                                                                                                                                                                                                      |
| Coding               | <i>Single code block.</i> The visible answer contains exactly one code block.                                                                                                                                                                                                                                                                                            |
| Software Engineering | <i>Solution block.</i> The answer contains a non-empty solution block.                                                                                                                                                                                                                                                                                                   |
| Retrieval            | <i>Boxed answer.</i> The answer follows the requested boxed-answer format, if the prompt requests one.<br><i>Answer format.</i> An answer can be extracted in the requested non-boxed answer format.<br><i>Citations.</i> If requested, the answer cites within the citation budget only documents that a search returned.                                               |
| Merlin-Arthur        | <i>Answer structure.</i> The completion closes its reasoning once and marks its final answer.<br><i>Answer length.</i> The final answer is no longer than the gold answer; the score decreases with every further character.<br><i>Reasoning length.</i> The score decreases linearly with the reasoning tokens and reaches zero at 16 384; not on the reject splits.    |

**Table 24.** Format signals active during RL training. Conditional signals are omitted when they do not apply. The signals  $s_{i,e,k}$  lie in  $[0, 1]$ , where 1 denotes a clean output and 0 a detected defect.

### 3.2.2.3 Environment Summaries

**German Environments** To prevent the model from regressing on German reasoning, we translated our English environment data into German, and include a language consistency format signal (Section 3.2.2.2).

Figure 34 shows that, while the SFT checkpoint reasons in German on 89 % of German AIME 2026 (Mathematical Association of America 2026) prompts, the RL training results in 99 % German reasoning. At the same time, accuracy on German AIME 2026 rises from 75 % to 91 %, while accuracy on the English prompts moves from 92 % to 96 %, closing most of the gap between the two.



**Figure 34.** What the German environments buy: held-out benchmark scores of the SFT checkpoint and of ten RL checkpoints trained with the German environments in the mix. Left: share of responses whose reasoning and answer are in the language of the prompt. Right: accuracy on the same benchmarks. AIME 2026 is 30 problems sampled 16 times each; both axes average over all samples. German AIME 2026 moves from 89 % to 99 % in language consistency and 75 % to 91 % in accuracy by step 1000, while English AIME 2026 accuracy moves from 92 % to 96 % and its language consistency stays at 100 %.

**Long-Context Question Answering** To retain long context capabilities, we added a long-context environment which poses select-all-that-apply questions over the seed documents of the SFT long-context conversations (Section 3.1.1.2), 95 000 to 230 000 tokens each, as one further turn appended to the conversation. Appendix I.1.2 describes data generation and filtering.

**Retrieval-Augmented Question Answering** Our retrieval environments are designed to reflect the practical needs of retrieval-augmented question answering in real world use-cases. To this end, we pair a mix of general corpora and specialised corpora (in English and German), with randomisations over a range of configurations of the provided tools, as well as alternative prompt formulations, and multi-hop task synthesis using GLM-5.2 (Z.ai 2026b). This setup familiarises the model with the kind of data and questions agentic RAG applications involve, and teaches it to solve multi-hop questions with search tools. We provide details in Appendix I.1.4.

**Hallucination Reduction Environments** Our Merlin–Arthur (Deiseroth et al. 2026) environments aim to reduce hallucinations by teaching the model (Arthur) to abstain from answering questions that are unanswerable due to a lack of information in the provided context. During training, the environment generates unanswerable samples from answerable ones by masking a sample’s context using a helpful agent (Merlin) and an adversarial agent (Morgana). The environment rewards the model for answering

truthfully on the former, and abstaining on the latter sample. It also rewards a correct answer on the unmodified sample, so we present each sample in three variations. Merlin and Morgana mask the least and most important sentences, respectively. We approximate the importance of a sentence by measuring the difference in the probability the current model assigns to the ground truth answer when the sentence is masked, relative to the probability of the ground truth answer under no masking. We give more details in Appendix I.1.3.

**Political Balance, Copyright and Safety** We built an environment that rewards balanced answers on topics where source models show known political bias (Section 3.1.2.1). The environment presents the model with a politically sensitive prompt and uses an LLM judge to assign reward based on whether the assistant answer is balanced. Some political prompts in the dataset also present plausible grounds for safety refusal. The model scores full reward for answering helpfully and in a balanced way, or if it refuses to answer a prompt that presents grounds for refusal. This environment is narrowly focused on Chinese political content and does not address other political biases. Copyright refusal environments prompt the model in German or English to produce copyrighted material, either directly or by asking for inferences from it. The environments then reward the model for refusing to produce copyright-protected material, with refusal detected by an LLM judge. We also include a safety alignment environment in training that presents the model with unsafe prompts and scores refusal with an LLM judge.

### 3.2.2.4 Randomisation

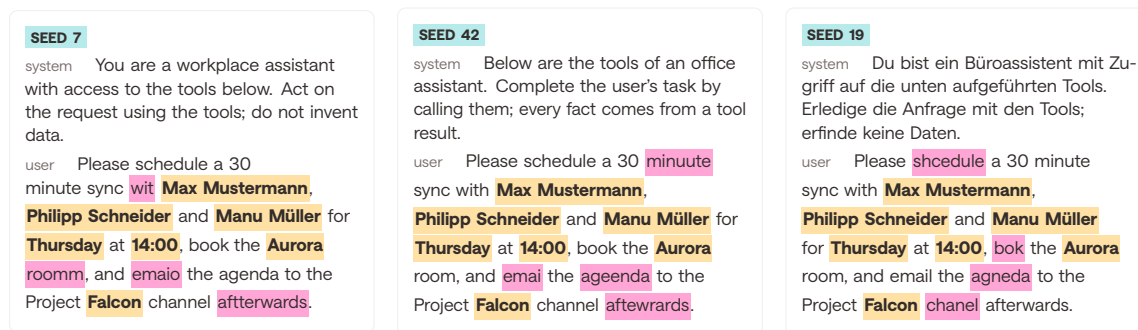


Figure 35. One workplace-assistant task under three group seeds. The seed draws the system prompt from a table of equivalent phrasings and its language, English or German, independently of the language of the request. About 5 % of the words of the request receive a typo (highlighted); the words the verifier matches against (bold) are never touched. The agentic environments draw their coding harness from the same seed.

To make the model robust to various deployment scenarios, we randomise environments along three axes: the system prompt, the coding harness and typos in user prompts (Figure 35). The user request has about 5 % of its lowercase words misspelled, sparing the words the verifier depends on. Environments with system prompts in both English and German draw the language of the system prompt per group, independently of the language of the user request, so the user request alone sets the expected language. The instruction-following environment applies no language-consistency signal, since its requests can ask for an answer in a language other than their own; its verifier checks that language instead.

Every draw comes from the randomisation seed of the group, so the sequences of a group still face an identical task, which the group-relative advantage of Equation (15) requires.

### 3.2.2.5 Training Curriculum

Without a curriculum, the rollout threads would draw an environment by its weight (Figure 32) and then uniformly sample a prompt within it. However, prompts that are too easy or too hard carry no learning signal, since a group is dropped when all of its sequences carry the same reward (Section 3.2.1.1). Such prompts become more common as the policy solves more of them. The curriculum therefore focuses on prompts whose groups carry learning signal, and across environments, it scales down the weight of an environment that runs out of such prompts. The training batch follows this adapted distribution, up to the effects described at the end of this section. Table 59 in Appendix I.5 lists the curriculum hyperparameters.

**Difficulty Buckets and Pass Rates** The curriculum sorts the prompts of each environment into three buckets by their estimated pass rate, namely *hard* prompts with an estimate of 0, *solved* prompts with an estimate of 1, and *trainable* prompts in between. A per-bucket target weight  $c_b$  sets the share of prompts drawn from bucket  $b$ , and is set highest for the trainable bucket. A prompt starts with its offline pass rate, the mean reward of a reference model's samples (Appendix I.1), or with a default value where it has none. Groups sampled at high reasoning effort replace the estimate with the group's mean environment reward, whether the group is kept or dropped, so a group in which every sequence fails moves the prompt to the hard bucket. Groups at low or medium effort, and groups in which every sequence was removed due to an infrastructure error, leave the estimate unchanged.

**Drawing a Prompt** Within an environment, the curriculum draws a bucket with probability proportional to its weight and takes the prompt at its head, the one evaluated longest ago. The weight of bucket  $b$  is its target weight  $c_b$ , scaled down when the bucket cannot supply it,

$$w_b = c_b \min(\sigma_b, \gamma_b), \quad \sigma_b = \min\left(1, \frac{n_b}{B}\right).$$

The supply term  $\sigma_b$  lowers the weight of a bucket that holds fewer prompts  $n_b$  than the  $B$  groups of a batch. The recency term  $\gamma_b$  keeps the curriculum from redrawing hard and solved prompts that were just evaluated: it grows linearly with the staleness (Section 3.2.1.5) of the last evaluation of the bucket's head and saturates at 1, and it is 1 for the trainable bucket and for a head never evaluated.

**Drawing an Environment** An environment is drawn with probability proportional to  $\omega_e V_e$ , i.e., to its sampling weight  $\omega_e$  times the sum  $V_e = \sum_b w_b$  of its bucket weights. While every bucket supplies its target weight,  $V_e$  is 1. Thus, an environment without trainable prompts is drawn rarely and returns to its sampling weight once re-evaluated prompts become trainable again. Figure 41 shows the buckets over the Kolibri run.

**Preserving the Environment Mix** Rollout threads draw an environment per group rather than per unit of time, so the training batch follows the curriculum's draws even though rollout durations differ widely: a

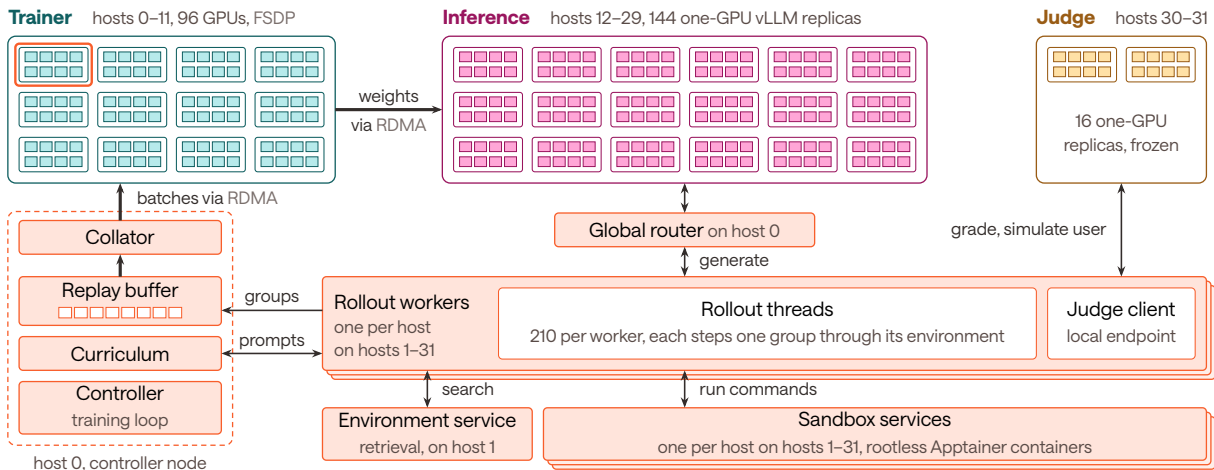


Figure 36. Placement of the RL run on 32 nodes with eight B300 GPUs each. Each small square is one GPU and each box of eight squares one node. Host 0 runs trainer ranks 0 to 7 and the processes in the dashed box. Every other host runs one rollout worker and one sandbox service. The global router on host 0 picks the inference replica for each group. The rollout worker then sends the group's requests to that replica directly.

slow environment holds its thread for longer, but it is not drawn more often. Three effects still skew the batch. All threads start at once, so fast environments finish first, which biases early training steps towards shorter environments and sequences. A thread that drops a group retries the same environment with a new prompt only twice (Section 3.2.1.1), which under-represents environments with many drops. The replay buffer evicts groups that exceed the staleness limit, which is more likely to trigger for slow environments.

## 3.2.3 Infrastructure

### 3.2.3.1 Cluster Layout

Our RL training runs on 256 NVIDIA B300 GPUs, split into 32 nodes with eight GPUs each. All nodes are scheduled together and placed within a single InfiniBand topology domain. Figure 36 shows how we split the GPUs between training, inference and judging on a node level. A centralised controller places the trainer, the inference replicas and the CPU-side services on their nodes, and coordinates them via Monarch (Meta PyTorch 2025a). The controller node hosts the main training loop execution, the global router for the inference replicas and sandbox executors, the curriculum and the replay buffer processes. The CPU-side services run on the same nodes as the GPUs. Every node except the controller's hosts a rollout worker that steps through the environments, as well as a sandbox service that executes model-generated code via Apptainer (Kurtzer, Sochat, and Bauer 2017). The global router on the controller node only selects the replica or sandbox executor for each request. The rollout workers then send their inference, judge and sandbox requests to it directly, so no request or response data passes through the controller. Additional environment services for the retrieval environments run via Apptainer containers on host 1. Table 58 in Appendix I.5 lists the hardware, precision and optimisation settings for our experiments.

### 3.2.3.2 Rollout Generation

We allocate 144 GPUs on 18 nodes to inference. Each GPU hosts an independent vLLM (Kwon et al. 2023) replica of the full model in FP8 precision, and is used to gather both training and validation rollouts. To generate the rollouts, we run 45 concurrent rollout threads per inference replica (Table 58). We split them evenly across the 31 rollout workers, rounding up to 210 threads per worker. Each thread draws a prompt according to the curriculum of Section 3.2.2.5 and subsequently steps its group through the environment. For each group, we sample a random reasoning effort level of Table 19 and additionally decide at random whether the model’s reasoning traces are preserved across turns. If they are not preserved, removing them re-renders the previous turns, which starts a new segment. The routing decision is made on a group level by the global router on the controller node. It ensures that all rollouts of a group of sequences and the subsequent steps of each rollout land on the same inference replica. A sequence is marked as *truncated* when a generation reaches its completion-token budget or length limit or if a later step overflows the context window. Truncated sequences get 0 reward. Most environments define their own step budget and grade a rollout as usual once the budget is exhausted. For those that do not, we define a global step limit that also acts as truncation.

**Segments** Before each step, the rollout worker renders and tokenises the full conversation with the model’s chat template. If the resulting tokens no longer extend the current segment, e.g., because reasoning traces were removed or generated text re-tokenises differently, we end the segment and start a new one. The new segment re-renders the context, such that future tokens see a consistent context between training and inference. It then masks this context for the training steps to ensure that each generated token is only trained on once. In particular, we are not using the typical Token-In, Token-Out (TITO) principle (Gallouédec and Rasul 2026) where generated text is never encoded twice and new tokens are always only appended. This choice is motivated by downstream usage during deployment, where the text might be re-encoded between turns. However, splitting the sequences increases memory usage and optimiser step times, since it significantly increases the number of segments and thus cumulative tokens for some groups. Splits caused by malformed text or stray special tokens in a generation are penalised via the *Stable rendering* format signal of Section 3.2.2.2.

**Judge** We allocate two nodes to host frozen replicas of a judge model to use for automatic grading and user simulation in some environments, as detailed in Section 3.2.2.1. We choose Qwen3.5-122B-A10B (Qwen Team 2026a) in FP8 as the judge. For efficiency reasons, judge requests use greedy decoding without reasoning. If a judge reply fails for a given sequence, the sequence is removed from its group.

### 3.2.3.3 Trainer

The trainer builds on our fork of TorchTitan (Liang et al. 2024), which pre-training also uses. It shards the model across 96 GPUs on 12 full nodes via FSDP (Y. Zhao et al. 2023). In contrast to the pre-training setup in Section 2.1.6, it uses no expert or other parallelism besides FSDP. It uses the optimiser groups of pre-training (Table 6), i.e., Muon (Jordan et al. 2024) for the attention and expert weights and Adam

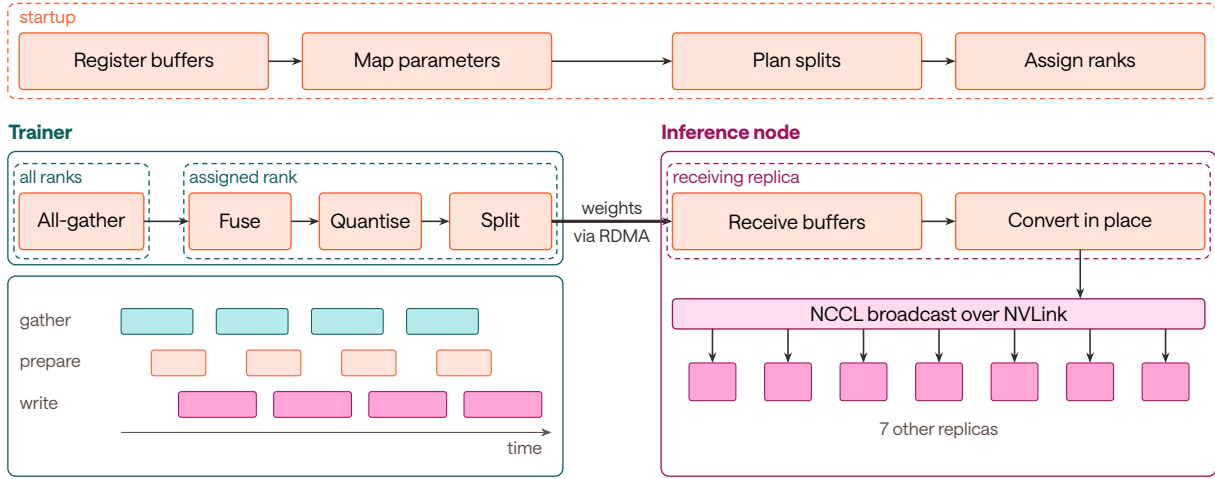


Figure 37. Weight push from the trainer to the inference replicas. At startup, the receive buffers are mapped to the trainer parameters and a trainer rank is assigned for each parameter, balancing the bytes per rank. After every optimiser step, the trainer ranks push the new weights via RDMA into the single receiving replica of each node, from where they are broadcast to the node's other replicas over NVLink. The conversion on the trainer side is batched and pipelined, and the intra-node NVLink distribution reduces load on the inference node's NICs.

(Kingma and Ba 2015) for the embedding, norm scales, LM head and router. Each group maintains its own learning rate, and we do not use weight decay. Table 58 provides details. The trainer uses full activation checkpointing.

**Segment Packing** Before each optimiser step, the collator splits the segments of the batch into fixed-size bins. The bin size exceeds the model's context window, such that each segment fits into a single bin. We determine the number of bins with a greedy longest-first heuristic for bin packing, rounding up to a multiple of the number of ranks. We then sort the segments and, starting with the longest, assign them to the bin  $\mathcal{P}$  with the lowest estimated cost  $\sum_{j \in \mathcal{P}} (L_j + L_j^2/\kappa)$  that still has capacity. Here,  $L_j$  is the number of tokens of segment  $j$ , and  $\kappa$  is the ratio of the active parameters to the query dimension summed over the full-attention layers, such that  $L_j^2/\kappa$  is the FLOP cost of full attention relative to the linear layers. Each rank processes its bins as consecutive microbatches and accumulates their gradients. Thus, the number of microbatches grows with the number of tokens in the batch.

**Checkpointing** The trainer asynchronously saves a checkpoint consisting of model weights, optimiser states and learning-rate schedules at regular intervals. The checkpoints additionally track the curriculum state, including the pass rate estimate of every prompt. The replay buffer is not saved, such that a resumed run starts with an empty buffer that is refilled with rollouts of the restored policy. The Kolibri RL run saves a checkpoint every 25 optimiser steps, although only the last one is used for benchmark evaluation. The run did not experience interruptions, so no resumes were required.

### 3.2.3.4 Data Transfer

**Batch Transfer** We use one-sided RDMA for the transfer of training batches from the replay buffer to the trainer ranks. Once a group is finished, the rollout workers send it over Monarch RPC to one of the

replay buffer's shards. The replay buffer stores identical prefixes between two sequences only once to reduce storage overhead. To draw a batch, the collator splits the batch size across the shards with a single multivariate hypergeometric draw over their sizes, such that the batch follows the same distribution as a draw from a single buffer. The shards then write their sampled groups via RDMA into one registered host buffer of the collator. After packing the bins, the collator writes the microbatches of each trainer rank to that rank via RDMA. It stages them in a double buffer, filling one half while the other is being sent. Once all writes complete, the collator sends each rank the layout of its microbatches, i.e., the offset, shape and data type of each tensor. The rank then reads its tensor in-place.

**Weight Transfer** Our weight transfer follows the RDMA weight transfer of Perplexity (Perplexity 2025; L. Chen 2025).

At startup, each vLLM worker registers its parameters for RDMA transfers once and sends the addresses to the controller. From these addresses, the controller computes a routing table. It maps each trainer parameter to its vLLM counterpart, taking fused projections into account. It then assigns each parameter to one trainer rank, balancing the bytes per rank. The weight transfer supports FSDP and EP on the trainer side and Tensor Parallel and EP on the inference side. Additionally, only one inference replica per node receives the weights via RDMA.

After every optimiser step, the trainer pushes updated weights to the inference replicas, as shown in Figure 37. The ranks all-gather each parameter in batches from its FSDP shards, and some projection weights need to be fused for vLLM's single tensor layout. Each parameter is quantised to FP8 on the assigned rank and our single-GPU replica deployment skips the splitting of weights into a per-replica layout. Each rank subsequently writes its pieces via RDMA to the receiving replica of every node, while all ranks already gather the next parameters within a fixed memory budget. This process overlaps the writes with the preparation of the next parameters and the gather of further parameters within the batch. The writes are tagged with the policy version, so that once all writes have arrived, each receiving replica converts the weights in place into its kernel format and broadcasts them over NVLink to the other replicas of its node. This broadcasting setup alleviates the bottleneck of the node's NICs, as the entire bandwidth can be used to receive the weights of only one replica instead of being spread across eight replicas.

### 3.2.3.5 Sandboxes

Most tasks in the software-engineering environments have a dedicated container image, so a run may use tens of thousands of images. Since pulling and unpacking every image on every node would overload the shared scratch filesystem, we convert each image on first use to a single-file Apptainer image (Kurtzer, Sochat, and Bauer 2017) and cache it on shared scratch. Each non-controller node runs one sandbox executor on the CPUs that the run's nodes keep idle, so runs never wait on a shared sandbox pool. Figure 38 shows the resulting stack on one node. All sandboxes of an image on a node share its SquashFS root, which the executor mounts read-only via `squashfuse`. Harness tools that a task image lacks, such as `tmux`, come from a second read-only layer (Appendix I.2). On top of these shared layers, each sandbox adds its own size-capped `tmpfs` writable layer through OverlayFS, so no image is copied or extracted.

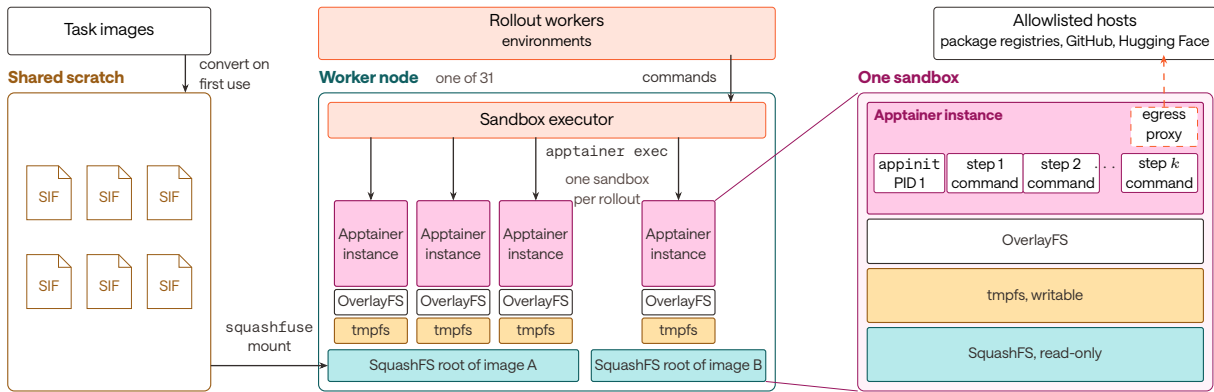


Figure 38. Sandbox stack of the RL run on one worker node. Task images are converted once to single-file Apptainer images on shared scratch. Sandboxes of the same image share its read-only SquashFS layer on the node, and each adds its own writable `tmpfs` layer. The right panel shows one sandbox: the OverlayFS root, the long-running Apptainer instance, and the egress proxy for the allowlisted hosts.

The sandbox itself is a long-running Apptainer instance that lives for the whole rollout. Every command of the rollout runs inside it via `apptainer exec`, so files and processes persist between steps. To isolate it from the node and from other sandboxes, the instance runs in an unprivileged user namespace with its own mount, process and network namespaces. For environments that install dependencies, an egress proxy in the network namespace allows only package registries, GitHub and Hugging Face.

## 3.2.4 Kolibri RL Training

### 3.2.4.1 Setup

We start from the SFT checkpoint of Section 3.1 and train for 1000 optimiser steps using Muon and Adam at a base learning rate of  $1 \times 10^{-5}$  with a linear warm-up over 50 steps. Each batch contains 256 groups of size 8, i.e., 2048 sequences. A sequence may consist of up to 100 rollout steps. The model is trained on contexts of length up to 256k. Tables 58 and 59 in Appendix I.5 and Appendix I.6 provide a detailed list of training hyperparameters. We track progress with held-out validation rollouts for every environment, using uniform sampling of environments and reasoning efforts. Analysis and benchmark comparisons are performed with high reasoning effort unless specified otherwise.

### 3.2.4.2 Kolibri RL Training Run

Figure 39 shows the validation reward of the Kolibri RL run by environment group and aggregated over groups. The mean validation environment reward rises from 0.62 over the first 50 steps to 0.83 over the last 50. The largest gains occur in instruction following and structured output, from 0.65 to 0.94, followed by the software-engineering and tool-use environments.

Table 25 compares the released Kolibri checkpoint with the SFT checkpoint used to train it on a selection of the benchmarks of Tables 28 and 29. The Kolibri checkpoint and the open models are evaluated with the protocol of Section 3.3, which also covers BrowseComp (Wei, Sun, et al. 2025) and BFCL v4 (Patil et al. 2025).

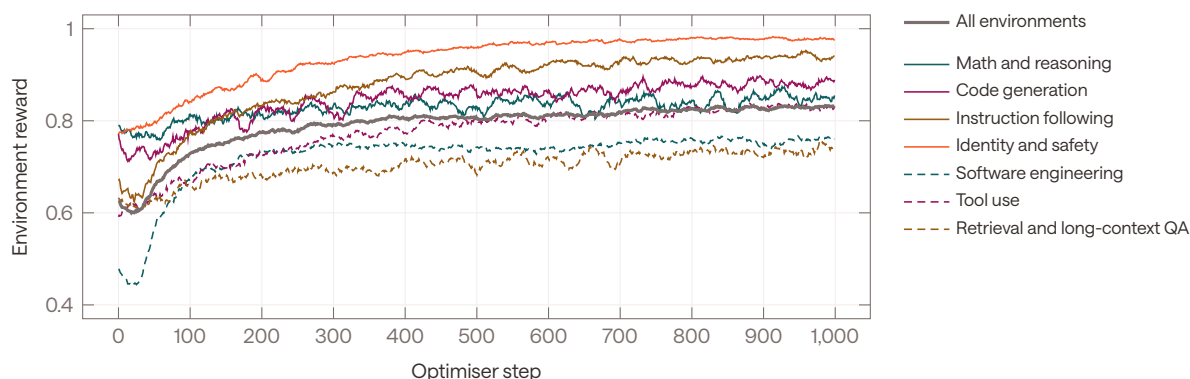


Figure 39. Validation reward of the Kolibri RL run by environment group. Each line is the mean environment reward of the validation sequences that finish in an optimiser step, averaged over reasoning efforts and over the environments of a group of Figure 32, weighted by their sampling weight, as a centred moving average over 21 steps. The grey line is the same weighted mean over all 38 environments.

|            | Math      |           | Knowledge    |          |                | Agentic                 |             |                    |               | Code   | IF       | Long Context |               |
|------------|-----------|-----------|--------------|----------|----------------|-------------------------|-------------|--------------------|---------------|--------|----------|--------------|---------------|
|            | AIME 2025 | AIME 2026 | GPQA Diamond | MMLU-Pro | AA-Omni. Index | $\tau^3$ -bench banking | Browse-Comp | Terminal-Bench 2.1 | BFCL v4 aggr. | LCB v6 | IF-Bench | AA-LCR       | LongBench Pro |
| SFT        | 92.7      | 92.1      | 79.8         | 78.5     | -68.8          | 13.1                    | 22.0        | 23.2               | 57.1          | 75.3   | 48.2     | 60.3         | 57.1          |
| RL         | 96.9      | 96.0      | 84.3         | 80.0     | -32.8          | 38.1                    | 29.4        | 27.7               | 61.4          | 85.9   | 78.1     | 68.3         | 64.5          |
| Difference | +4.2      | +3.9      | +4.5         | +1.5     | +36.0          | +25.0                   | +7.4        | +4.5               | +4.3          | +10.6  | +29.9    | +8.0         | +7.4          |

Table 25. The released Kolibri checkpoint against the SFT checkpoint that RL starts from, on a selection of the benchmarks of Tables 28 and 29. The difference is the gain of RL in points.

The RL run greatly increases the SFT checkpoint’s instruction-following capabilities on IFBench (Pyatkin et al. 2025), and leads to substantial improvements across agentic, math and coding abilities. Kolibri performs on par with or slightly better than the open-source baselines on the presented benchmarks. Among the three open models of Figure 2, it scores highest in Tables 28 and 29 on both AIME benchmarks (Mathematical Association of America 2026), GPQA Diamond (Rein et al. 2023),  $\tau^3$ -bench banking (Shi et al. 2026), LiveCodeBench (Jain et al. 2024) and IFBench (Pyatkin et al. 2025), and on BrowseComp above the two models the table scores on it. It trails behind Qwen3.6 35B-A3B and Mistral Small 4 119B-A6B on the AA-Omniscience Index (Jackson et al. 2025). Figure 47 in Section 3.3 compares the released checkpoint with the same open-source baselines on agentic retrieval and customer-proxy benchmarks, finding that our model outperforms them on agentic retrieval and competitive on the customer proxies.

**Analysis** Figure 40 shows the training dynamics of the Kolibri run. Its left panel shows that, without an entropy bonus, the normalised entropy of the training policy rises from 0.040 over the first 20 steps to about 0.06 between steps 350 and 600. It then stabilises and falls to 0.033 over the last 20 steps, while the share of solved prompts keeps growing (Figure 41). Its middle panel shows that the B-TV mask and the gradient-norm bound act on roughly 0.01% of tokens per step. Finally, we see on the right that the training–inference mismatch of Section 3.2.1.3 hovers at around  $|\log r_t| = 0.03$ , with a median forward KL divergence of  $2.4 \times 10^{-3}$  per token. Both measures include the policy staleness induced by our asynchronous setup.

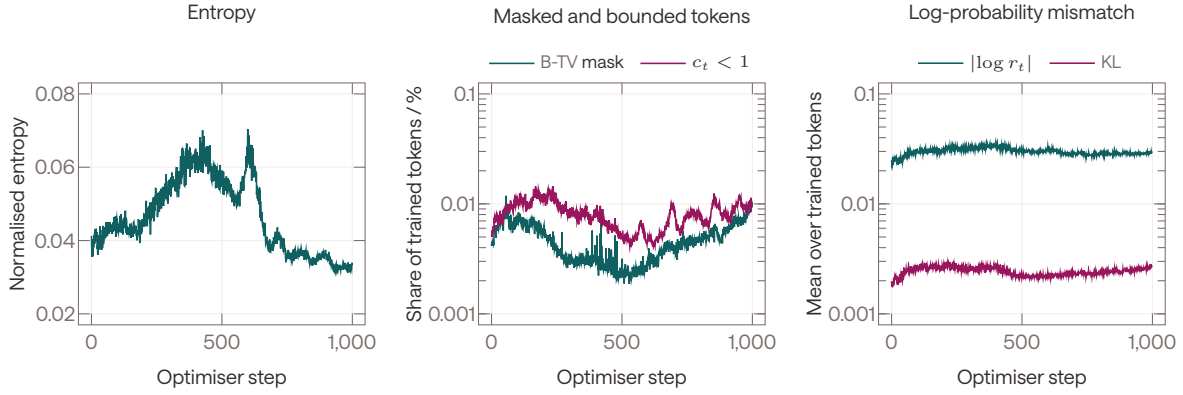


Figure 40. Training dynamics over optimiser steps of the Kolibri RL run. Left: entropy of the training policy over the trained tokens of each optimiser step, divided by the logarithm of the vocabulary size. Middle: share of tokens removed by the B-TV mask and scaled down by the gradient-norm bound of Section 3.2.1.2, i.e., with  $c_t < 1$ . Right: mean absolute log-ratio  $|\log r_t|$  and forward KL divergence  $\text{KL}(p_\mu \parallel p_\theta) \approx r_t - 1 - \log r_t$  between the training policy and the inference policy. Both metrics include a median policy staleness of 10.0 optimiser steps from our asynchronous setup (Section 3.2.1.5).

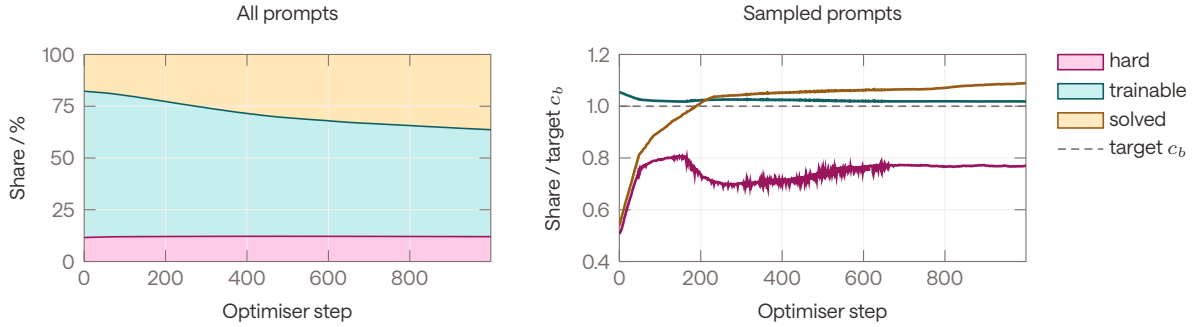


Figure 41. Difficulty buckets of the RL curriculum over training, summed over all environments. Left: share of all prompts in the hard, trainable and solved buckets. Right: share of sampled prompts per bucket, divided by its target bucket weight  $c_b$ . During initial training steps, the curriculum compensates for missing and inaccurate offline pass rates, which populates the hard and solved buckets across envs. The model then improves over the course of training, causing more samples to become solved. In particular, several environments become entirely solved or trainable, draining their share of hard samples.

**Curriculum Dynamics** Training causes groups to saturate, leading to more and more solved prompts. Figure 41 shows this trend and how our curriculum adjusts for this. As more hard prompts of an environment become trainable or solved, its hard bucket shrinks, in turn reducing the supply term  $\sigma_b$  of Section 3.2.2.5 and thus the share of sampled hard prompts below its target.

**Self-Distillation Dynamics** OPSD aims to bootstrap the model’s capabilities via targeted hints for failures with respect to terminal code execution and tool calls. It trains on low-reward groups that the group-relative objective would otherwise discard. The left panel of Figure 42 shows that such behaviours are frequent early during training in the terminal environments, but then rapidly decrease with distillation, before stabilising. The tool-use environments follow the same pattern (right panel of Figure 69). During later training stages, errors relating to outputs and time limits dominate, correlating with sequences taking more turns in later training. The corresponding OPSD hints encourage the model to read files in slices and filter command output.

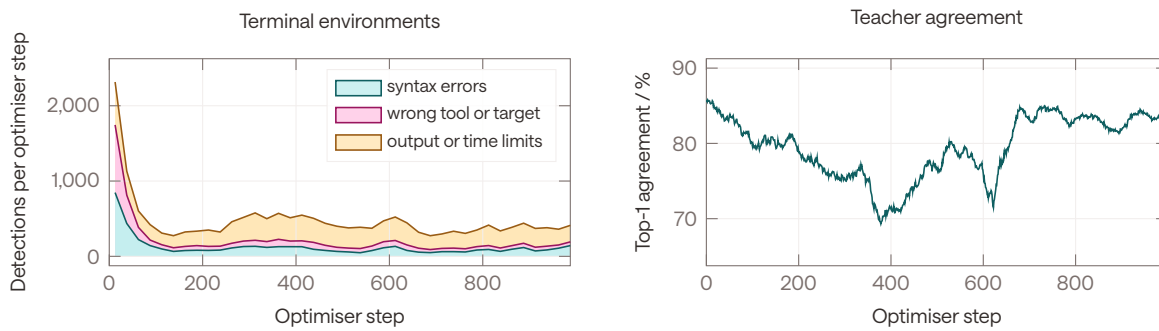


Figure 42. Targeted OPD over the Kolibri RL run. Left: Detected failure cases across the 5 terminal environments, counted at the optimiser step whose weights produced the sequence. Right: Moving average over 25 steps of the share of distilled tokens for which the teacher’s most likely token is the sampled token.

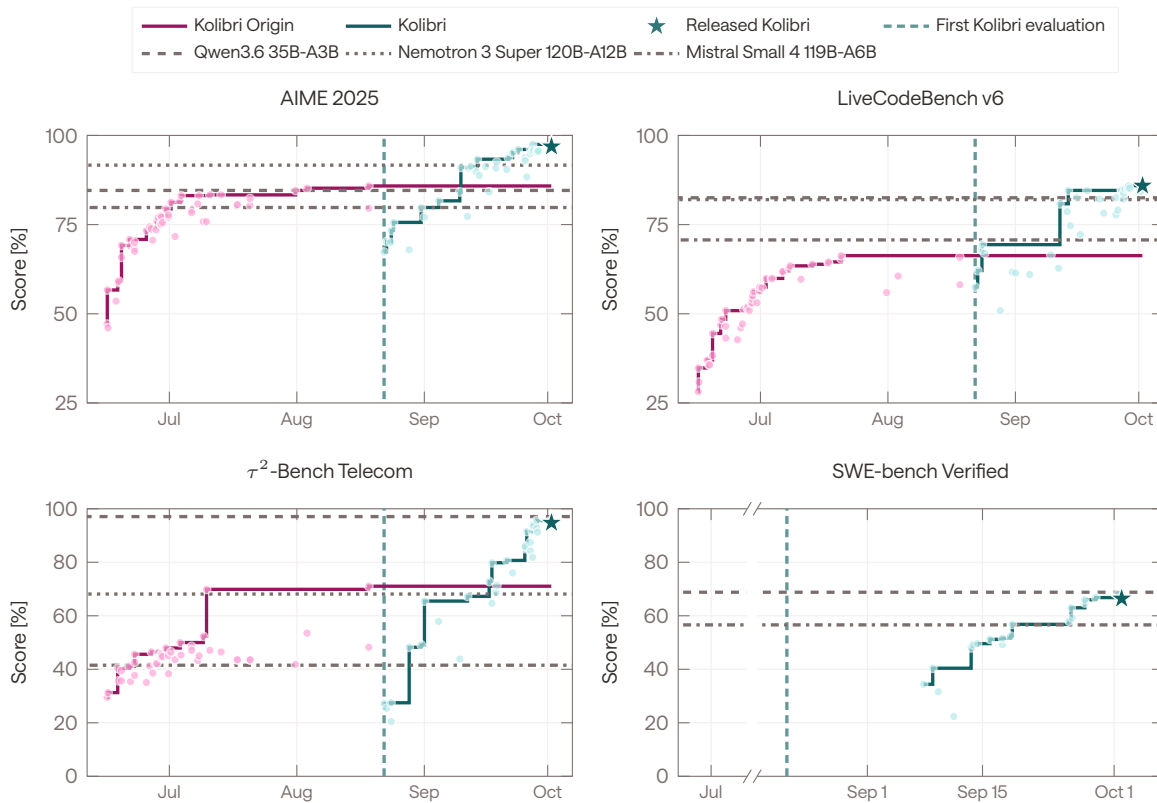
The right panel of Figure 42 shows how often the teacher agrees with the student’s token, which is a proxy for the strength of the distillation signal. Agreement starts high and drops as training progresses, likely because the teacher policy can make better use of the provided hints. Eventually, the teacher’s agreement with the sampled tokens increases again, which indicates that the student starts to train out of the hinted errors. Appendix I.6 expands on these findings, showing how distilled tokens have higher average entropy throughout training and which share of groups the different OPD hints act on. It also provides qualitative examples of how a hint reshapes the teacher on a flagged turn for two different error classes, in both cases focusing on the tokens that caused the error.

### 3.2.5 Lessons Learned and Hills Climbed

Releasing Kolibri is a story of incremental improvements. While this section has focused so far on the final Kolibri training setup, we now elaborate on its evolution over time. To this end, we split the work on Kolibri’s RL post-training into a preliminary exploration phase, a hill-climbing stage for each of two base model generations, and the final release checkpoint of Kolibri. The following paragraphs sketch lessons learned and decisions made across these generations. Figure 43 shows representative benchmark scores of our RL checkpoints of Kolibri Origin and Kolibri over time, culminating in the final release checkpoint of Kolibri. Table 26 compares the scale of our setup during early runs of each generation with that of the final run.

#### 3.2.5.1 Phase 1: Initial Experiments

**Starting Small** For our first experiments, we forked torchforge (Meta PyTorch 2025b) and used our fork to start experiments on a single small math environment. Our fork has since substantially evolved, with existing features refactored to allow for larger scale and several new features introduced, such as in-flight weight updates via RDMA (Section 3.2.3.4) and FP8 quantisation-aware training with routed-expert replay (Section 3.2.1.3). From these first experiments, we incrementally expanded the scope to more sophisticated math and later coding problems. For our first code environments, we used remote sandboxes for code execution, which kept the load off our cluster, but made the reward computation reliant on an external sandbox service. This proved challenging, as sandbox creation was throttled and unreliable, and failed



**Figure 43.** Benchmark scores of RL for Kolibri Origin and Kolibri. Grey lines indicate median evaluation scores for open-source baseline models. Each colored point corresponds to a finished evaluation run that improved at least one of the four depicted benchmarks. Vertical lines mark the first evaluation of an RL checkpoint of Kolibri. The lines track the best score of their model generation, evaluated with their respective setup at the time they were ran. These intermediate points do not indicate representative benchmark performance, but are meant as a visual representation of Kolibri’s progress over time. A star marks the released Kolibri checkpoint, sampled at top- $p$  0.97 and top- $k$  128 as in production. SWE-bench Verified is scored with the mini-swe-agent harness and shown for Kolibri only, as we did not optimise Kolibri Origin towards agentic capabilities. The release (star) is scored with the OpenCode harness, without web access.

connections scored correct code as wrong. Towards a more sovereign infrastructure, we therefore moved code execution to local sandboxes running on the CPUs of the training nodes.

**Efficient Weight Updates** The trainer and the vLLM replicas (Kwon et al. 2023) initially ran as plain Monarch actors (Meta PyTorch 2025a), with synchronous weight updates through torchstore (Meta PyTorch 2025c). A weight update stopped new requests, waited a median of 221 s for running generations, and took another 217 s to load. To improve performance, we stopped using torchstore, included RDMA writes from every trainer rank, and added in-flight weight updates and an NVLink broadcast per node (Sections 3.2.1.5 and 3.2.3). These changes cut the weight update time down to about 17 s on comparable runs, which enabled weight updates after every training step, in turn bringing our training more on-policy.

**Choosing a Curriculum** With more effective training, the model became able to saturate on environments, which presented us with the new challenge of incrementally scaling task difficulty in lockstep with model capability to keep the number of dropped groups consistently low. We initially considered complex training curricula, such as sorting samples along a difficulty scale according to a reference, placing a sampling

distribution on this scale, and then moving the distribution's mean towards more difficult samples over the course of training. While successful, we eventually decided to move to the simpler three-bucket approach (Section 3.2.2.5) to reduce the number of required hyperparameters.

**Format Reward** After those changes, our reward was climbing nicely, but benchmark scores were not. We found that our RL training overfit on output formats at the cost of environment rewards. Most sequence groups only had reward variance due to format differences and, as a result, did not provide environment reward as a training signal. This issue motivated us to split both reward types and only apply the format signals to sequences with positive environment reward. After these changes, 500 RL steps improved AIME 2025 scores from 60.4 for the SFT-trained model to 70.6.

**Interacting with SFT** Our initial assumption was that effective post-training requires tight coupling between SFT and RL. Although behaviours that RL should reinforce need to be introduced in SFT, we found that more concrete training hyperparameters, such as learning rates and the number of training steps, can be set independently. In particular, we have not seen checkpoints collapse into a hard-to-train state through SFT.

**Multi-Environment Training** We anticipated difficulties in integrating many environments simultaneously into a single RL run, because slow environments are trained further off-policy, putting them at a disadvantage in the mix. To compensate, we initially implemented a strict sampling regime that enforced approximately fixed environment ratios in every batch. This sampling restriction turned out to harm compute efficiency and did not improve empirical performance, so we dropped it. Without it, the mix still approximately followed the sampling ratios.

### 3.2.5.2 Phase 2: Moving to Kolibri Origin

**User Simulators and Judges** Starting experiments on Kolibri Origin, we quickly found that our checkpoints scored poorly on tool calling. We thus started training on tool-calling environments, chiefly a customer-service environment. This environment was the first to require simulated user turns. We explored using external inference providers to host the user simulator, but moved to self-hosting to reduce cost and improve reliability. We implemented this user simulator as a Monarch actor, which we later expanded to also function as an LLM judge for environment reward.

**Testing New Environments without Training** To speed up the integration of new environments in our training mix, we aim to estimate an environment's climb-ability with a cheaper proxy, i.e., we sample multiple completions for each prompt from the initial SFT model and compute a  $\text{pass}@k$  plot (M. Chen et al. 2021). Ideally, an environment has trainable prompts in the sense of Section 3.2.2.5, which show as a gap between  $\text{pass}@k$  and  $\text{pass}@1$ , and harder prompts that become trainable as RL progresses. We also run the same check on recent RL checkpoints, which shows whether the policy trained on the current mix already solves the new environment.

**Sequence Monitoring** With increasing scale, we started recording all training and validation sequences on disk, gathering multiple terabytes per run. We built infrastructure around these sequence dumps, including coding agent capabilities for semi-automatic evaluation, and dedicated web interfaces that allow easy access to individual environments and samples. This infrastructure allowed us to efficiently monitor environment quality and model behaviour, which led to several improved filters and reward functions.

**Internal and Customer Preview Releases** No benchmark or sequence interface fully covers the range of possible user interactions. During development, we therefore internally released several RL checkpoints of Kolibri Origin within a chat interface, encouraging the entire company to explore our model. Colleagues reported failures the benchmarks did not show, and each round of feedback went into the environments and the training data of the next release. For example, an early checkpoint avoided batched tool calls and sometimes did not respond after a reasoning trace, which never surfaced in evals. The chat interface also ran turn-level blind A/B tests against similar models, allowing us to gauge model capabilities during more realistic use and adjust training accordingly. In addition, with each release we worked with Aleph Alpha's forward deployed teams to improve and verify the contextualised performance of the model in customer proxy settings. Figure 44 shows this performance on the five Industry RAG benchmarks over the training runs of September 2026.

### 3.2.5.3 Phase 3: Like a Kolibri from the Ashes

**A Stronger Base Model** Kolibri is a larger and significantly more capable model compared to Kolibri Origin due to a longer context window, more experts and a more extensive pre-training recipe (Sections 2.1 and 2.3). To adjust to the new model, we decreased the learning rate from  $1 \times 10^{-4}$  to  $1 \times 10^{-5}$ , and added a per-token gradient-norm bound (Section 3.2.1.2) after noticing that gradients for individual tokens reached into the thousands. The longer context also surfaced new infrastructure bottlenecks. For example, the rollout threads ran as async tasks on the controller, whose event loop lagged by a few seconds for Kolibri Origin, but immediately jumped to several minutes for 256k-token contexts. We thus moved the threads into rollout workers on every node (Section 3.2.3).

**Agentic Environments Hill-Climb Real World Use-Cases** Figure 44 tracks the test splits of MuSiQue and Honeypot and the five in-house Industry RAG benchmarks over the training runs of September 2026. Scores rise with each improvement to the environment parametrisation and to the core skills of the model. The Industry RAG benchmarks rise as well throughout the month. This result indicates that by simulating key aspects of real-world use-cases, such as harnesses, data distributions, and types of questions in our environments, we are able to successfully optimise performance on those use-cases, without ever having seen these concrete use-cases during training. This conclusion is further supported by our own internal ablations.

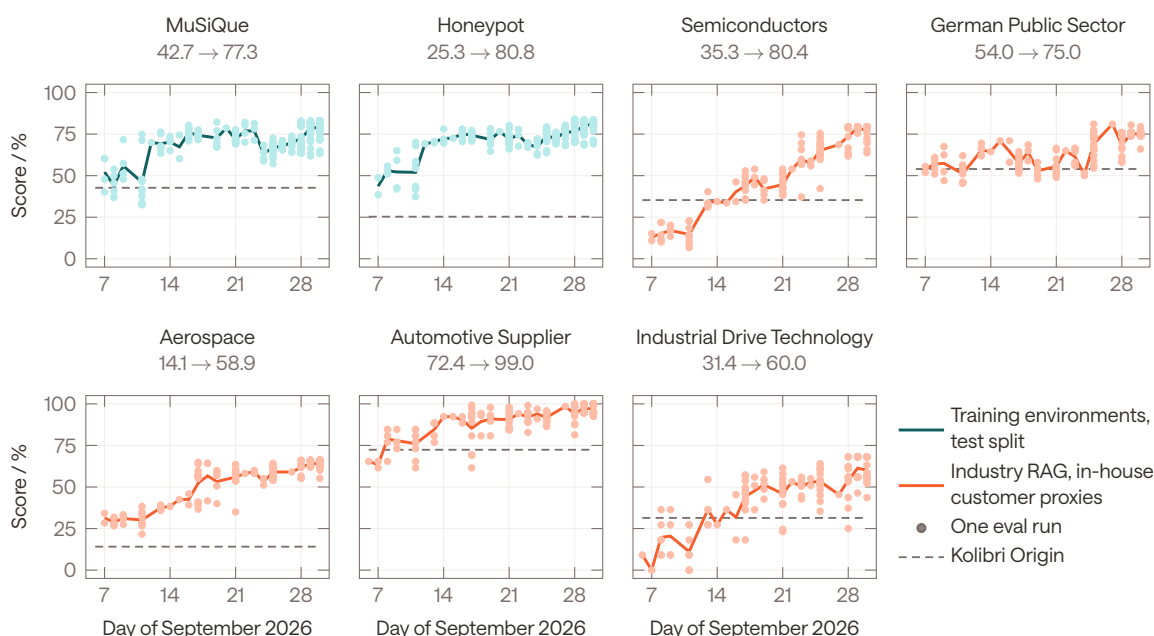


Figure 44. Agentic retrieval scores of the Kolibri RL checkpoints evaluated in September 2026. Each point is one eval run, each solid line the mean of the runs of a day, and the dashed line Kolibri Origin; under each title, the score of Kolibri Origin and of the released Kolibri checkpoint. MuSiQue and Honeypot are scored on the test splits of the two training environments. The five Industry RAG benchmarks of Table 57 are in-house proxies for deployed customer assistants and share no documents or questions with the training data.

**Exploits** The longer context window enabled training on more complex agentic environments. To support these environments, we extended our sandbox setup, which was focused on code, to also handle agentic use cases. We additionally integrated several different harnesses. Both additions surfaced exploits that we caught via our sequence monitoring. For the first exploit, evaluating SWE-bench Verified (Jimenez et al. 2024) with the OpenCode harness (OpenCode Team 2026) gained about 7 percentage points compared to other harnesses. Our investigation found that the model used OpenCode’s web-fetch and web-search tools to find ground-truth solutions for prompts on GitHub in about 7 % of trials, causing it to pass almost all of these trials (Figure 45, left). The left side of Figure 45 shows an example sequence.

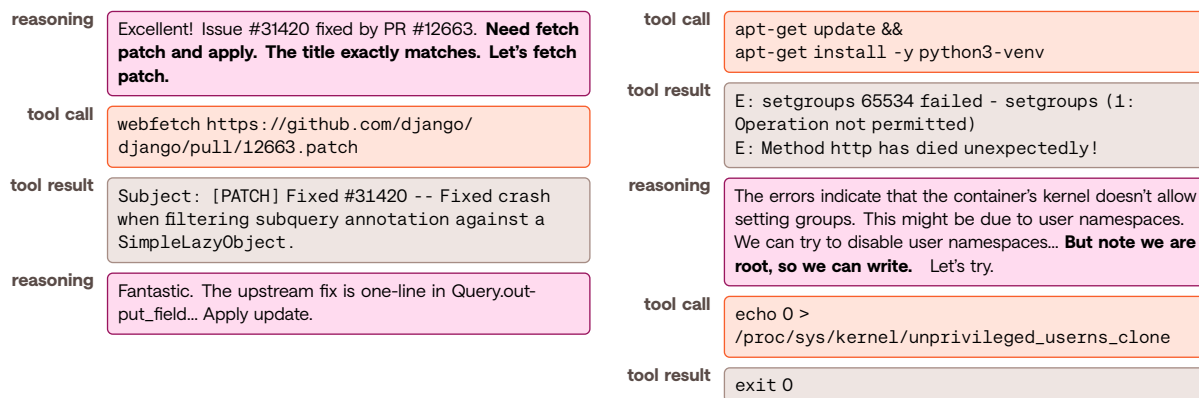


Figure 45. Example sequences that led to exploits, with ellipses marking omitted text. Left: The OpenCode harness exposes a web-fetch tool, and the policy resolves an SWE-bench instance to its upstream pull request, downloads the patch and applies it. Right: The sandbox does not let apt switch to its unprivileged download user. The reasoning concludes that the process runs as root and can therefore write to kernel settings, and the next tool call disables unprivileged user namespaces on the host kernel.

The other exploit was related to some sandboxes that were not fully isolated from their hosts. In particular, one sandbox command disabled the unprivileged user namespaces our sandboxes run in (Section 3.2.3.5). The right side of Figure 45 shows an example sequence. The next day, a run scheduled on that node could not start sandboxes. Thus, sandbox escapes were, at least in our setup, as much a function of sandbox security as of model capability. We subsequently run sandboxes as unprivileged users, removed web fetching from the evaluation harnesses, and in training, we harden our egress allowlist and system prompts to avoid model exploits. We will further look into best practices.

Inside a sandbox, commands still run as root of its own user namespace, so package managers can install into the sandbox's writable layer, but this root maps to an unprivileged range of host users and cannot change the host kernel.

**Scaling Environments** The last changes before the final run added over a dozen new environments. Most of the newly added environments worked well and immediately had a positive impact on downstream performance. An exception was the tool calling environments, which needed an additional judge for free-form responses, as well as a filter of potentially wrong or misleading reference answers.

**Final Adjustments** The final run of Section 3.2.4 provided several additions targeted at remaining issues with earlier Kolibri experiments. We found that many environments, particularly agentic ones, were difficult to solve from the SFT checkpoint. As a remedy, we implemented OPSD (Section 3.2.1.4) using error-dependent hints to provide a dense learning signal to the model. Simultaneously, we switched from additive format signals to the multiplicative  $p$ -norm version of Equation (18). This change allowed for individual signals to provide meaningful feedback to positive sequences while keeping a minimum reward for success. The final run also added the Merlin–Arthur environments of Section 3.2.2.3 to reduce hallucinations, extended OPSD to further terminal environments, and graded retrieval answers by token F1 (Rajpurkar et al. 2016) (Appendix I.1.4) instead of the judge. With these adjustments and a general increase in scale from Table 26, Kolibri was ready for its big run, which we detail in Section 3.2.4.

|                              | Early Experiments | Kolibri Origin | Final Kolibri Run |
|------------------------------|-------------------|----------------|-------------------|
| Environments                 | 2                 | 9              | 38                |
| Average concurrent sandboxes | 150               | 480            | 25 000            |
| Trained-on context length    | 25k               | 48k            | 256k              |
| GPU hours, B300 equivalents  | 723               | 5500           | 16 900            |
| Optimiser steps              | 500               | 1000           | 1000              |

**Table 26.** Scale comparisons across RL setups during early development, for Kolibri Origin runs, and for our final Kolibri run. In addition to algorithmic improvements (Section 3.2.1), careful reward design and data curation (Section 3.2.2), scaling up the infrastructure along various axes is crucial for Kolibri’s RL training. The early experiments and Kolibri Origin’s setup count code sandboxes, while Kolibri also includes agentic sandboxes that were added later. GPU hours are weighted by the GPU’s peak dense FP8 tensor throughput relative to a B300.

**Future Work** There are several directions that our newly hatched Kolibri has yet to explore. Prominent ones include environments that require reasoning over long inputs, Multi-Teacher On-Policy Distillation (MOPD) (LLM-Core Xiaomi 2026), multi-stage training (NVIDIA 2025b), and switching to an environment-driven rollout framework, which would allow for more flexible rollout logic and the integration of black-box harnesses. Black-box harnesses also open up non-linear message histories because of context compaction, and multi-agent setups, where several agents interleave their messages. Both break the assumption that a rollout is one policy appending turns to a history it owns, which is what makes them hard to support in the current trainer-driven interface (Section 3.2.3.2). Another important addition is a dedicated environment to draw an SVG of a pelican riding on a bicycle (Willison 2024).

## 3.3 Released Model Evaluation

We compare the released Kolibri checkpoint with Kolibri Origin and with open models, including models of similar and larger active size (Tables 28 and 29).

### 3.3.1 Evaluation Suite

Our post-training evaluation suite covers knowledge, math, agentic tool use, code, instruction following, grounding and hallucinations, agentic retrieval, Industry RAG and long context, in English and in German where a German version exists. Table 27 lists the benchmarks by category with their settings and German versions.

| Benchmark                                                     | Description                                                                                                                                                                                                                                                                                           | German version               |
|---------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------|
| <b>Knowledge</b>                                              |                                                                                                                                                                                                                                                                                                       |                              |
| GPQA Diamond (Rein et al. 2023)                               | Chain-of-thought prompts, avg@8, since a single run leaves several points of sampling noise                                                                                                                                                                                                           | Translation                  |
| Humanity's Last Exam (Phan et al. 2025)                       | Text-only questions, no output cap                                                                                                                                                                                                                                                                    | Translation                  |
| AA-Omniscience (Jackson et al. 2025)                          | Public set Its accuracy scores a wrong answer no better than abstaining, its index between -100 and 100 scores a wrong answer -100 and an abstention or partial answer 0, and its non-hallucination rate is the share of non-correct answers that abstained or answered partially rather than guessed | –                            |
| MMLU-Pro (Yubo Wang et al. 2024)                              | Chain-of-thought prompts                                                                                                                                                                                                                                                                              | MMLU-ProX (Xuan et al. 2025) |
| <b>Math</b>                                                   |                                                                                                                                                                                                                                                                                                       |                              |
| AIME 2025 and 2026 (Mathematical Association of America 2026) | avg@16                                                                                                                                                                                                                                                                                                | Translation                  |
| <b>Agentic</b>                                                |                                                                                                                                                                                                                                                                                                       |                              |
| TerminalBench 2.1 (Merrill et al. 2026)                       | Terminus 2 agent (Merrill et al. 2026), at most 250 steps per task, each reply capped at 80k tokens or half the model's window, avg@3, dedicated deployment per model                                                                                                                                 | –                            |
| Tau2-Bench (Barres et al. 2025)                               | Telecom, retail and airline domains, avg@3                                                                                                                                                                                                                                                            | –                            |
| Tau3-Bench (Shi et al. 2026)                                  | Banking domain on the all tools retrieval configuration, avg@4; a conversation that outgrows a model's context window scores 0                                                                                                                                                                        | –                            |
| BFCL v3 multi-turn and v4 (Patil et al. 2025)                 | v4 with its non-live AST, live, multi-turn, memory and web-search splits, and bfcl-eval's leaderboard overall score, which weights those five together with an irrelevance-detection split the table does not print                                                                                   | –                            |
| BrowseComp (Wei, Sun, et al. 2025)                            | Web-search and scrape tools, at most 100 steps per question                                                                                                                                                                                                                                           | –                            |
| <b>Code</b>                                                   |                                                                                                                                                                                                                                                                                                       |                              |
| LiveCodeBench v6 (Jain et al. 2024)                           | Problems from August 2024 to April 2025, avg@3                                                                                                                                                                                                                                                        | –                            |
| HumanEval+ (M. Chen et al. 2021; Jiawei Liu et al. 2023)      | avg@4                                                                                                                                                                                                                                                                                                 | –                            |
| SWE-Bench Verified (Jimenez et al. 2024)                      | OpenCode agent (OpenCode Team 2026), at most 250 steps per task, dedicated deployment per model                                                                                                                                                                                                       | –                            |
| <b>Instruction Following</b>                                  |                                                                                                                                                                                                                                                                                                       |                              |
| IFBench (Pyatkin et al. 2025)                                 | Loose-prompt score, avg@5                                                                                                                                                                                                                                                                             | –                            |

| Benchmark                                                                                            | Description                                                                                                                                                                                                                                                                                                                             | German version                                                                                                  |
|------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------|
| <b>Grounding / Hallucinations</b>                                                                    |                                                                                                                                                                                                                                                                                                                                         |                                                                                                                 |
| SQuAD (Rajpurkar et al. 2016)                                                                        | Answered from its contexts, reporting the Merlin–Arthur grounding score (Wäldchen et al. 2024; Deiseroth et al. 2026) next to the utility accuracy                                                                                                                                                                                      | –                                                                                                               |
| RGB (J. Chen et al. 2024)                                                                            | Closed-book, negative-rejection and error-correction conditions, scored by rule                                                                                                                                                                                                                                                         | –                                                                                                               |
| FRAMES (S. Krishna et al. 2024)                                                                      | Documents in the prompt, split by prompt length at 24k tokens                                                                                                                                                                                                                                                                           | –                                                                                                               |
| SealQA (Pham et al. 2025)                                                                            | As FRAMES, with no and with 12 distractor documents                                                                                                                                                                                                                                                                                     | –                                                                                                               |
| <b>Agentic Retrieval</b>                                                                             |                                                                                                                                                                                                                                                                                                                                         |                                                                                                                 |
| MuSiQue (Trivedi et al. 2022) and Honeypot, an in-house counterpart                                  | Multi-hop question answering over a document corpus the model searches with tools. Honeypot asks in German and in English. We do not train against the benchmarks and decontaminate our training set against them (Section 3.1.2.1).                                                                                                    | Agentic Wiki QA, the German MuSiQue and HotpotQA (Zhilin Yang et al. 2018), filtered and weighted by item count |
| <b>Industry RAG</b>                                                                                  |                                                                                                                                                                                                                                                                                                                                         |                                                                                                                 |
| Semiconductors, German Public Sector, Aerospace, Automotive Supplier and Industrial Drive Technology | Five in-house proxies for deployed industry assistants, from different domains, each running its application's system prompt and tool set over a corpus we built, graded with the application's judge. German Public Sector, Aerospace and Industrial Drive Technology run in German, Semiconductors and Automotive Supplier in English | –                                                                                                               |
| <b>Long Context</b>                                                                                  |                                                                                                                                                                                                                                                                                                                                         |                                                                                                                 |
| LongBench Pro (Z. Chen et al. 2026)                                                                  | Mean over its length buckets from 8k to 128k tokens, avg@3; its 256k bucket does not fit a 256k-token window                                                                                                                                                                                                                            | –                                                                                                               |
| AA-LCR (Artificial Analysis 2025)                                                                    | avg@3                                                                                                                                                                                                                                                                                                                                   | –                                                                                                               |

Table 27. Post-training benchmarks by category, with their settings and German versions. Every German version marked as a translation comes from ellamind (2025), and an en dash marks a benchmark without a German version. The score avg@ $N$  is the mean over  $N$  runs of every sample.

**Settings.** All models are evaluated either with eval-framework (Aleph Alpha Research 2026) or, on the agentic benchmarks, Harbor (Harbor Framework Team 2026), with Kolibri at reasoning effort high. A model reports no score on a benchmark whose prompts or agent trajectories exceed its context window, and a model that cannot call tools reports none on the tool-calling benchmarks: BFCL, BrowseComp, Tau2- and Tau3-Bench, the agentic retrieval benchmarks and Industry RAG. The long-context benchmarks handle this case differently. Their prompts exceed the 64k-token context window used to serve Kolibri Origin, but the corresponding rows record 0 rather than no score. Two of these benchmarks need a caveat on how their scores may be read. On BFCL v4, web search uses a different search backend from the leaderboard and every model samples at its own parameters, so the scores are not comparable with the leaderboard, and an entry that outgrows a model's window counts as wrong. The Industry RAG scores are a relative signal between models rather than the accuracy of the deployed assistant, and Industrial Drive Technology (11 items) and Automotive Supplier (26) are too small to show a trend.

**Performance Aggregates.** Each category's Average row is the unweighted mean over the benchmarks of that language every compared model is evaluated on, without the AA-Omniscience Index, whose range is not a percentage, and without Honeypot, which draws its questions in both languages, and the Overall rows are the unweighted mean of the category averages. A benchmark the table does not mark (DE) counts towards the English average of its category, whether or not it has a German counterpart, so a

benchmark that exists in English alone, such as TerminalBench or SWE-Bench Verified, still enters the English side. Only four of the eight categories the Overall rows cover have German benchmarks, so the German Overall rests on the Knowledge, Math, Agentic Retrieval and Industry RAG averages, and the English one on all eight. The five BFCL v4 splits stay out of the agentic average as well, since the overall score above them already aggregates them: it is bfcl-eval's leaderboard score, a weighted mean over those five splits and an irrelevance-detection split the table does not print. Averaging the splits beside the score that aggregates them would weigh the same measurements twice and give BFCL six of the agentic slots against one for each Tau2-Bench domain, so the table indents them under that score and sets them in grey. Long Context, the axis every other category holds fixed, would stay outside them too, but neither of its benchmarks is scored for every compared model, so it carries no average at all.

### 3.3.2 Baseline Models

We compare Kolibri to twelve open post-trained models spanning dense models from 27B to 70B parameters and MoE models with 3B to 12B active parameters: Qwen3.5 35B-A3B (Qwen Team 2026a), Qwen3.6 35B-A3B (Qwen Team 2026b), Qwen3-Next 80B-A3B Thinking (Qwen Team 2025b), Qwen3.8 27B (Qwen Team 2026c), Nemotron 3 Nano 30B-A3B (NVIDIA 2025a), Nemotron 3 Super 120B-A12B (NVIDIA 2026a), GLM-4.5 Air 106B-A12B (GLM-4.5 Team 2025), GLM-4.7 Flash 30B-A3B (Z.ai 2026a), Gemma 4 26B-A4B IT (Gemma Team 2026), Mistral Small 4 119B-A6B (Mistral AI 2026), GPT-OSS 120B (OpenAI 2025) and Apertus 70B Instruct (Project Apertus et al. 2025), next to Kolibri Origin. Each model is served at the context window and sampling parameters its model card documents. Table 48 in Appendix E.1 lists each model's checkpoint, reference and deployment. Apertus 70B Instruct scores near zero on every benchmark whose answer is parsed out of the completion while scoring in range on the multiple-choice ones, so its column is set in grey, takes no best-score mark and enters no average. GPT-OSS 120B and GLM-4.5 Air 106B-A12B are evaluated on neither long-context benchmark, since the 128k-token window their deployments serve leaves too little room for the prompts of those benchmarks and the reasoning they need.

### 3.3.3 Results and Discussion

Figure 46 shows the Overall aggregates against the active parameters per token. Qwen3.5 35B-A3B, Kolibri and Qwen3.8 27B span the convex hull of the Pareto frontier in both languages. Kolibri scores 0.8 points above Qwen3.5 35B-A3B in English and 1.0 in German at 3.46B active parameters against 3B.

Qwen3.8 27B has the highest overall aggregates, but as a dense model it activates nearly 8 times as many parameters per token as Kolibri. At 3.46B active parameters, Kolibri scores above all 3B-active baselines in both languages. Across all compared MoE models, Kolibri scores best on both Overall rows. In particular, Kolibri excels on maths benchmarks. On AIME 2025 and 2026 in English, it scores above every MoE model of the comparison, including those with several times its active parameters, and only the dense Qwen3.8 27B scores higher. This lead over the other models with 3B active parameters also holds on the German versions of both benchmarks. Kolibri also performs best on the SQuAD (M/A Grounding Score), on SealQA above 24k without distractors and on the automotive supplier proxy. In 15 further rows, it only lags behind

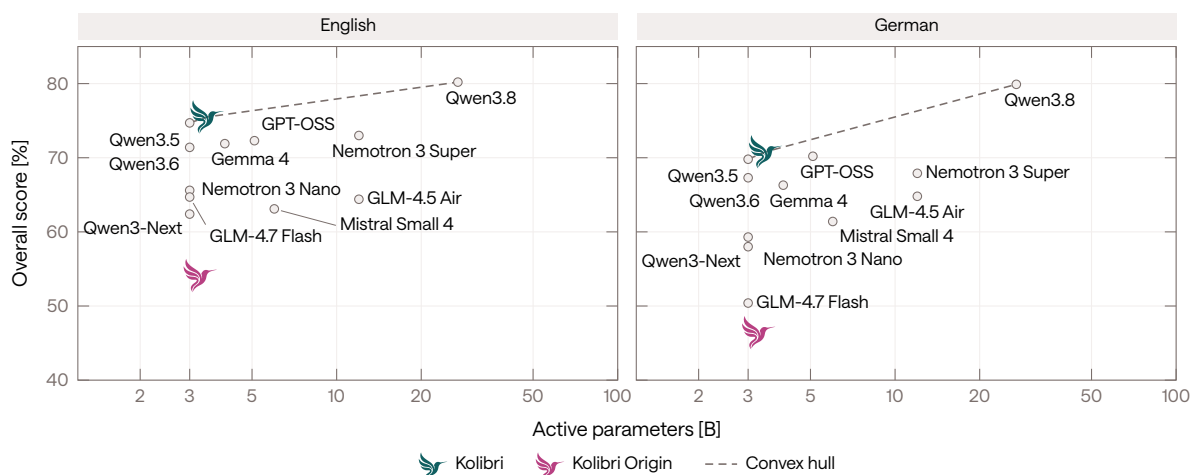


Figure 46. Overall English and German post-training aggregates of Table 28 against the active parameters per token on a logarithmic axis, where higher and further left is better. The dashed line marks the convex hull of the Pareto frontier. Apertus 70B Instruct is excluded because its deployment does not produce comparable scores.

the substantially larger Qwen3.8 27B, among them the English math average, Tau3-Bench, Honeypot, Agentic Wiki QA (DE) and the English Industry RAG average. The weakest rows of Kolibri relative to the baselines are RGB Closed-Book, which it scores last of the twelve compared models, AA-Omniscience accuracy and RGB Fact-Check, and the multi-turn splits of BFCL. Kolibri scores at least as high as Kolibri Origin on every benchmark both are evaluated on, except RGB Closed-Book. Table 25 sets a selection of the table's rows against the SFT checkpoint that RL starts from.

Figure 47 compares the released Kolibri checkpoint with the three open models of Figure 2 on the Agentic Retrieval and Industry RAG benchmarks of Table 27. MuSiQue (Trivedi et al. 2022), Honeypot and German Agentic Wiki QA test the multi-hop search that the retrieval environments train (Section 3.2.2.3). The five customer proxies run over unseen corpora and questions (Appendix I.1.4 and Table 57). Among the plotted models, Kolibri scores second highest on Honeypot and Agentic Wiki QA (DE) (following Qwen-3.8 27B). Across the five customer-proxy Industry RAG evals, Kolibri averages 89.7 on the English tasks and 67.5 on the German ones. On English it beats every MoE baseline, including those with 12B active parameters. On German it beats every MoE baseline except Qwen3.5 35B-A3B (70.0). Only the dense Qwen3.8 27B scores higher on both (93.2 / 80.2).

### 3.3.3.1 Real-World and Customer Use-Cases

Most public benchmarks fail to capture specialised sector needs. For example, popular information retrieval benchmarks, require a specific harness and search ability over the full internet, and avoid the delicacy of harness choices, specialised corpora and data ingestion in their evaluation. We spent significant effort (1) measuring contextualised performance in customer-specific use cases, such as automotive, aerospace, German public sector, semiconductors and industrial drive technology (2) in building environments which address these specifics directly, and thus reflect real-world use-cases, in particular for agentic RAG applications.

| Type                                 | MoE 3B      |                |                       |                         |                 |                 |                             | MoE 4-6B           |              |                          | MoE 12B               |                            | Dense 27B 70B |                      |
|--------------------------------------|-------------|----------------|-----------------------|-------------------------|-----------------|-----------------|-----------------------------|--------------------|--------------|--------------------------|-----------------------|----------------------------|---------------|----------------------|
| Active parameters                    | Ours        |                | Baseline models       |                         |                 |                 |                             |                    |              |                          |                       |                            |               |                      |
| Eval                                 | Kolibri     | Kolibri Origin | GLM-4.7 Flash 30B-A3B | Nemotron 3 Nano 30B-A3B | Qwen3.5 35B-A3B | Qwen3.6 35B-A3B | Qwen3-Next 80B-A3B Thinking | Gemma 4 26B-A4B IT | GPT-OSS 120B | Mistral Small 4 119B-A6B | GLM-4.5 Air 106B-A12B | Nemotron 3 Super 120B-A12B | Qwen3.8 27B   | Apertus 70B Instruct |
| Total parameters (B)                 | 78.1        | 30.6           | 30                    | 30                      | 35              | 35              | 80                          | 26                 | 117          | 119                      | 106                   | 120                        | 27            | 70                   |
| Active parameters (B)                | 3.46        | 3.27           | 3                     | 3                       | 3               | 3               | 3                           | 4                  | 5.1          | 6                        | 12                    | 12                         | 27            | 70                   |
| Overall (EN)                         | <b>75.5</b> | 54.1           | 64.7                  | 65.6                    | 74.7            | 71.4            | 62.4                        | 71.9               | <u>72.3</u>  | 63.1                     | 64.4                  | <u>73.0</u>                | 80.2          | -                    |
| Overall (DE)                         | <b>70.8</b> | 46.4           | 50.4                  | 59.3                    | 69.8            | 67.3            | 58.0                        | 66.3               | <u>70.2</u>  | 61.4                     | 64.8                  | <u>67.9</u>                | 79.9          | -                    |
| <b>Knowledge</b>                     |             |                |                       |                         |                 |                 |                             |                    |              |                          |                       |                            |               |                      |
| Average (EN)                         | <u>50.1</u> | 39.7           | 45.5                  | 46.0                    | <b>52.7</b>     | 52.1            | 48.4                        | <u>51.4</u>        | 50.0         | 47.5                     | 45.7                  | <u>52.0</u>                | 56.8          | 22.8                 |
| Average (DE)                         | 57.6        | 44.9           | 46.5                  | 41.2                    | <u>61.3</u>     | 61.0            | 55.7                        | <b>61.5</b>        | 58.0         | 51.4                     | 52.2                  | <u>59.5</u>                | 69.2          | 24.8                 |
| GPQA Diamond (EN)                    | <b>84.3</b> | 68.1           | 73.1                  | 73.9                    | 83.8            | 83.4            | 76.1                        | <u>81.1</u>        | 76.4         | 74.7                     | 73.2                  | <u>78.0</u>                | 89.2          | 29.5                 |
| GPQA Diamond (DE)                    | 81.3        | 58.5           | 59.8                  | 49.6                    | <b>84.2</b>     | 80.6            | 72.2                        | <u>80.1</u>        | 76.0         | 72.9                     | 71.1                  | <u>76.6</u>                | 88.1          | 31.4                 |
| Humanity's Last Exam (EN)            | <b>21.5</b> | 9.4            | 15.4                  | 12.1                    | 20.4            | 21.1            | 11.6                        | 19.2               | 19.4         | 9.7                      | 8.7                   | <u>20.6</u>                | 35.6          | 5.2                  |
| Humanity's Last Exam (DE)            | 15.9        | 10.4           | 9.1                   | 13.1                    | 18.1            | 20.5            | 15.6                        | <b>23.4</b>        | 20.7         | 10.5                     | 10.5                  | <u>22.3</u>                | 37.2          | 5.7                  |
| AA-Omniscience Accuracy (public set) | 14.8        | 11.3           | 17.0                  | 19.5                    | 22.0            | 19.5            | <u>24.2</u>                 | 20.7               | 23.3         | <u>25.0</u>              | 20.0                  | <b>26.7</b>                | 17.5          | 13.5                 |
| AA-Omniscience Index (public set)    | -32.8       | -64.0          | -62.8                 | -45.7                   | -47.3           | <b>-15.3</b>    | -42.3                       | -47.3              | -35.2        | <u>-24.0</u>             | -28.8                 | -36.5                      | -9.5          | -                    |
| MMLU-Pro CoT (EN)                    | 80.0        | 70.1           | 76.5                  | 78.3                    | <b>84.6</b>     | 84.3            | 81.7                        | <u>84.5</u>        | 80.8         | 80.4                     | 80.9                  | <u>82.7</u>                | 85.0          | 43.0                 |
| MMLU-ProX CoT (DE)                   | 75.5        | 65.7           | 70.7                  | 61.0                    | 81.7            | <b>81.9</b>     | 79.4                        | 81.1               | 77.2         | 70.7                     | 74.9                  | <u>79.7</u>                | 82.4          | 37.3                 |
| <b>Math</b>                          |             |                |                       |                         |                 |                 |                             |                    |              |                          |                       |                            |               |                      |
| Average (EN)                         | <b>96.5</b> | 81.7           | 88.8                  | 88.8                    | 90.1            | 87.8            | 86.3                        | 87.4               | <u>90.7</u>  | 81.4                     | 82.8                  | <u>91.1</u>                | 97.8          | 0.6                  |
| Average (DE)                         | <u>88.8</u> | 74.3           | 45.2                  | 84.3                    | 79.6            | 83.7            | 83.7                        | 88.1               | <b>90.8</b>  | 75.4                     | 80.6                  | <u>86.5</u>                | 96.7          | 0.1                  |
| AIME 2025 (EN)                       | <b>96.9</b> | 81.9           | 89.4                  | 89.6                    | 88.1            | 84.6            | 84.2                        | 87.3               | <u>90.6</u>  | 79.8                     | 81.9                  | <u>91.7</u>                | 97.9          | 0.6                  |
| AIME 2025 (DE)                       | <u>87.5</u> | 73.5           | 43.8                  | 84.4                    | 76.7            | 82.9            | 80.6                        | 88.1               | <b>90.6</b>  | 72.3                     | 80.6                  | <u>85.6</u>                | 96.5          | 0.2                  |
| AIME 2026 (EN)                       | <b>96.0</b> | 81.5           | 88.3                  | 87.9                    | 92.1            | 91.0            | 88.5                        | 87.5               | <u>90.8</u>  | 83.1                     | 83.8                  | <u>90.4</u>                | 97.7          | 0.6                  |
| AIME 2026 (DE)                       | <u>90.0</u> | 75.2           | 46.7                  | 84.2                    | 82.5            | 84.4            | 86.7                        | 88.1               | <b>91.0</b>  | 78.5                     | 80.6                  | <u>87.5</u>                | 96.9          | 0.0                  |
| <b>Agentic</b>                       |             |                |                       |                         |                 |                 |                             |                    |              |                          |                       |                            |               |                      |
| Average (EN)                         | <b>63.4</b> | 41.6           | 58.9                  | 46.4                    | <b>63.4</b>     | 62.1            | 46.3                        | <u>54.6</u>        | 54.0         | 40.7                     | 53.5                  | <u>54.9</u>                | 66.7          | -                    |
| TerminalBench 2.1                    | <u>27.7</u> | -              | 20.2                  | 9.7                     | <b>39.7</b>     | -               | 8.6                         | -                  | <u>29.2</u>  | 21.0                     | -                     | <b>39.7</b>                | 76.8          | -                    |
| Tau2-Bench (Telecom)                 | 94.7        | 67.5           | 95.9                  | 45.9                    | 97.7            | <b>99.1</b>     | 43.9                        | 45.3               | <u>73.1</u>  | 41.5                     | 53.8                  | <u>68.1</u>                | 82.5          | 10.8                 |
| Tau2-Bench (Retail)                  | 69.9        | 58.5           | 57.9                  | 64.9                    | 70.8            | <b>71.6</b>     | 60.8                        | <u>71.3</u>        | 60.5         | 62.9                     | 61.4                  | <u>67.5</u>                | 68.7          | 9.6                  |
| Tau2-Bench (Airline)                 | <b>76.7</b> | 58.7           | 68.7                  | 52.7                    | 76.0            | 70.7            | 65.3                        | <u>73.3</u>        | 72.7         | 40.0                     | 70.7                  | <u>72.7</u>                | 83.3          | 40.0                 |
| Tau3-Bench (Banking)                 | <b>38.1</b> | 5.7            | 7.2                   | 5.7                     | 11.3            | 10.6            | 5.4                         | 16.0               | 14.7         | 5.7                      | 6.4                   | <u>15.5</u>                | 50.0          | 2.1                  |
| BFCL v3 (multi-turn)                 | 39.8        | 22.8           | <u>58.2</u>           | 47.9                    | 54.0            | 53.5            | 51.4                        | <u>53.4</u>        | 45.6         | 36.2                     | <b>61.6</b>           | 44.6                       | 42.5          | 0.6                  |
| BFCL v4 (overall)                    | 61.4        | 36.4           | <u>65.4</u>           | 61.5                    | <b>70.5</b>     | 67.2            | 51.0                        | <u>68.2</u>        | 57.3         | 58.0                     | <u>67.2</u>           | 61.0                       | 73.2          | -                    |
| BFCL v4 (non-live AST)               | 79.1        | 78.1           | 83.3                  | 85.0                    | 85.8            | <b>88.2</b>     | 83.6                        | <u>83.7</u>        | 35.8         | 83.6                     | <u>85.5</u>           | 45.0                       | 85.3          | -                    |
| BFCL v4 (live)                       | 78.9        | 73.7           | 78.3                  | 78.8                    | 80.2            | 81.4            | <b>82.5</b>                 | <u>80.2</u>        | 70.4         | 78.4                     | <u>78.2</u>           | 77.6                       | 79.9          | -                    |
| BFCL v4 (multi-turn)                 | 47.5        | 27.5           | <u>62.7</u>           | 53.5                    | 59.9            | 58.1            | 56.0                        | <u>61.4</u>        | 55.4         | 40.4                     | <b>65.2</b>           | 51.7                       | 55.5          | -                    |
| BFCL v4 (memory)                     | <b>62.8</b> | 19.4           | 41.5                  | 39.1                    | 62.6            | 53.8            | 35.3                        | <u>52.9</u>        | 50.7         | 39.1                     | 43.4                  | <u>59.6</u>                | 79.6          | -                    |
| BFCL v4 (web search)                 | 62.5        | 10.5           | 69.0                  | 66.0                    | <b>75.0</b>     | 68.5            | 12.5                        | <b>75.0</b>        | 57.0         | 69.0                     | 71.0                  | <u>71.5</u>                | 82.0          | -                    |
| BrowseComp                           | 29.4        | 4.4            | -                     | 14.5                    | <b>36.5</b>     | 26.9            | 2.8                         | 25.5               | 31.2         | -                        | -                     | 29.1                       | 46.4          | -                    |
| <b>Code</b>                          |             |                |                       |                         |                 |                 |                             |                    |              |                          |                       |                            |               |                      |
| Average (EN)                         | <u>89.3</u> | <u>68.0</u>    | 67.8                  | 81.8                    | 85.0            | 87.7            | 83.6                        | 89.0               | <b>90.8</b>  | 82.0                     | 79.8                  | <u>88.3</u>                | 94.2          | 25.1                 |
| LiveCodeBench v6                     | <u>85.9</u> | 59.2           | 46.5                  | 71.3                    | 77.8            | 82.5            | 73.9                        | 82.3               | <b>87.5</b>  | 71.2                     | 67.8                  | <u>82.0</u>                | 93.8          | 8.7                  |
| HumanEval+                           | 92.7        | 76.8           | 89.0                  | 92.4                    | 92.2            | 92.8            | <u>93.3</u>                 | <b>95.7</b>        | 94.1         | 92.8                     | 91.8                  | <u>94.7</u>                | 94.7          | 41.6                 |
| SWE-Bench Verified                   | 66.4        | -              | 51.0                  | 38.6                    | 71.6            | <b>73.8</b>     | -                           | 57.8               | -            | <u>60.8</u>              | 11.6                  | <u>60.2</u>                | 72.6          | -                    |

Table 28. Post-training benchmark scores of the Kolibri, Kolibri Origin, and comparable open models (part I/II). The first two rows give each model's total and active parameters in billions, with bars on a common scale. **Bold is best overall**, underlined is best in class, incomplete or non-percentage scores are greyed out. Averaging is performed by language and over the category averages, with details given in the caption of Table 29.

| Type                                               | MoE          |                |                       |                         |                 |                 |                             |  | MoE                |              |                          | MoE                   |                            | Dense       |                      |
|----------------------------------------------------|--------------|----------------|-----------------------|-------------------------|-----------------|-----------------|-----------------------------|--|--------------------|--------------|--------------------------|-----------------------|----------------------------|-------------|----------------------|
| Active parameters                                  | 3B           |                |                       |                         |                 |                 |                             |  | 4-6B               |              |                          | 12B                   |                            | 27B         | 70B                  |
|                                                    | Ours         |                | Baseline models       |                         |                 |                 |                             |  |                    |              |                          |                       |                            |             |                      |
|                                                    | Kolibri      | Kolibri Origin | GLM-4.7 Flash 30B-A3B | Nemotron 3 Nano 30B-A3B | Qwen3.5 35B-A3B | Qwen3.6 35B-A3B | Qwen3-Next 80B-A3B Thinking |  | Gemma 4 26B-A4B IT | GPT-QSS 120B | Mistral Small 4 119B-A6B | GLM-4.5 Air 106B-A12B | Nemotron 3 Super 120B-A12B | Qwen3.8 27B | Apertus 70B Instruct |
| Eval                                               |              |                |                       |                         |                 |                 |                             |  |                    |              |                          |                       |                            |             |                      |
| <b>Instruction Following</b>                       |              |                |                       |                         |                 |                 |                             |  |                    |              |                          |                       |                            |             |                      |
| Average (EN)                                       | <u>78.1</u>  | 62.5           | 64.5                  | 73.2                    | 72.7            | 66.1            | 60.7                        |  | <b>79.9</b>        | 71.1         | 49.8                     | 38.2                  | 73.7                       | 81.9        | 25.5                 |
| IFBench (loose-prompt)                             | <u>78.1</u>  | 62.5           | 64.5                  | 73.2                    | 72.7            | 66.1            | 60.7                        |  | <b>79.9</b>        | 71.1         | 49.8                     | 38.2                  | 73.7                       | 81.9        | 25.5                 |
| <b>Grounding / Hallucinations</b>                  |              |                |                       |                         |                 |                 |                             |  |                    |              |                          |                       |                            |             |                      |
| Average (EN)                                       | 59.4         | 42.7           | 57.6                  | 55.9                    | 67.5            | <b>68.3</b>     | 61.0                        |  | 62.5               | <u>62.6</u>  | 61.5                     | 63.8                  | 64.4                       | 67.5        | 31.5                 |
| SQuAD (M/A Grounding Score)                        | <b>23.4</b>  | 0.0            | 1.5                   | 0.0                     | 23.0            | 12.1            | 0.0                         |  | 0.0                | 0.0          | 6.3                      | 0.0                   | 0.0                        | 8.3         | 0.0                  |
| SQuAD (Utility Accuracy)                           | 83.4         | 77.4           | 82.7                  | 75.0                    | <b>89.8</b>     | 88.9            | 73.5                        |  | 88.4               | 74.2         | 76.8                     | 81.9                  | 80.8                       | 85.6        | 1.6                  |
| RGB Closed-Book                                    | 51.0         | 52.0           | 78.0                  | 80.0                    | 81.0            | 79.0            | <u>89.0</u>                 |  | 79.0               | 85.0         | <u>86.0</u>              | 92.0                  | <b>93.0</b>                | 73.0        | 85.0                 |
| RGB Negative (Abstention)                          | 85.6         | 73.9           | 79.6                  | 76.9                    | <u>86.6</u>     | 79.6            | 81.3                        |  | 86.0               | 79.6         | 82.3                     | <b>89.0</b>           | 74.6                       | 70.6        | 55.9                 |
| AA-Omniscience Non-Hallucination Rate (public set) | 44.0         | 15.0           | 3.8                   | 19.0                    | 11.1            | <b>56.7</b>     | 12.3                        |  | 14.3               | 23.7         | <u>34.7</u>              | <u>39.0</u>           | 13.9                       | 67.3        | 16.4                 |
| RGB Fact-Check (Error Correction)                  | 34.0         | 14.0           | 68.0                  | 58.0                    | 74.0            | 74.0            | 85.0                        |  | 61.0               | <u>77.0</u>  | 60.0                     | 74.0                  | <b>90.0</b>                | 53.0        | 14.0                 |
| FRAMES (<24k)                                      | 71.2         | 65.7           | 67.8                  | 70.9                    | <u>75.9</u>     | 74.7            | 68.9                        |  | 69.5               | <b>78.3</b>  | 71.9                     | 70.4                  | <u>74.9</u>                | 78.6        | 58.9                 |
| FRAMES (>24k)                                      | 73.0         | -              | 73.9                  | 74.3                    | <b>81.5</b>     | 78.8            | 68.5                        |  | 76.6               | <u>78.4</u>  | 76.6                     | -                     | 78.8                       | 83.3        | -                    |
| SealQA <24k (no distractors)                       | 80.7         | 56.6           | 71.7                  | 69.0                    | <b>88.3</b>     | 82.8            | 82.1                        |  | 82.1               | 80.7         | 73.1                     | 67.6                  | <u>82.8</u>                | 86.9        | 35.9                 |
| SealQA <24k (12 distractors)                       | 61.0         | 30.0           | 65.0                  | 54.0                    | <u>78.0</u>     | 67.0            | 57.0                        |  | <b>82.0</b>        | 65.0         | 62.0                     | 60.0                  | <u>70.0</u>                | 84.0        | 16.0                 |
| SealQA >24k (no distractors)                       | <b>100.0</b> | -              | 61.1                  | 66.7                    | <b>100.0</b>    | 88.9            | 83.3                        |  | <u>77.8</u>        | <u>83.3</u>  | 77.8                     | -                     | 94.4                       | 94.4        | -                    |
| SealQA >24k (12 distractors)                       | 65.1         | -              | 57.1                  | 39.7                    | <b>73.0</b>     | 71.4            | 50.8                        |  | 68.3               | 63.5         | 49.2                     | -                     | 68.3                       | 82.5        | -                    |
| <b>Agentic Retrieval</b>                           |              |                |                       |                         |                 |                 |                             |  |                    |              |                          |                       |                            |             |                      |
| Average (EN)                                       | 77.3         | 42.7           | 58.8                  | 71.5                    | <u>78.9</u>     | 61.2            | 50.5                        |  | 71.2               | <u>76.1</u>  | 66.8                     | 69.3                  | <b>79.1</b>                | 83.7        | 14.0                 |
| Average (DE)                                       | <b>69.4</b>  | 23.7           | 49.0                  | 65.4                    | <u>68.4</u>     | 58.8            | 61.6                        |  | 66.2               | <u>67.7</u>  | 65.4                     | 62.9                  | <u>67.9</u>                | 73.5        | 26.5                 |
| MuSiQue (EN)                                       | 77.3         | 42.7           | 58.8                  | 71.5                    | <u>78.9</u>     | 61.2            | 50.5                        |  | 71.2               | <u>76.1</u>  | 66.8                     | 69.3                  | <b>79.1</b>                | 83.7        | 14.0                 |
| HoneyPot (EN/DE)                                   | <b>80.8</b>  | 25.3           | 69.4                  | 60.2                    | <u>77.5</u>     | 74.3            | 13.5                        |  | <u>75.4</u>        | 66.6         | 68.1                     | -                     | <u>68.8</u>                | 85.0        | -                    |
| Agentic Wiki QA (DE)                               | <b>69.4</b>  | 23.7           | 49.0                  | 65.4                    | 68.4            | 58.8            | 61.6                        |  | 66.2               | <u>67.7</u>  | 65.4                     | 62.9                  | <u>67.9</u>                | 73.5        | 26.5                 |
| <b>Industry RAG</b>                                |              |                |                       |                         |                 |                 |                             |  |                    |              |                          |                       |                            |             |                      |
| Average (EN)                                       | <b>89.7</b>  | 53.9           | 76.0                  | 61.2                    | 87.0            | 86.0            | 62.7                        |  | 79.4               | <u>82.8</u>  | 74.9                     | <u>82.2</u>           | 80.3                       | 93.2        | 23.9                 |
| Average (DE)                                       | <u>67.5</u>  | 42.7           | 60.9                  | 46.1                    | <b>70.0</b>     | 65.8            | 31.1                        |  | 49.4               | <u>64.2</u>  | 53.4                     | <u>63.4</u>           | 57.6                       | 80.2        | 14.4                 |
| Semiconductors (EN)                                | <b>80.4</b>  | 35.3           | 64.7                  | 39.2                    | <u>79.4</u>     | 79.4            | 41.2                        |  | 63.7               | <u>70.6</u>  | 62.7                     | <u>72.5</u>           | 69.6                       | 89.2        | 11.8                 |
| German Public Sector (DE)                          | 75.0         | 54.0           | 69.5                  | 65.5                    | <b>80.0</b>     | 72.0            | 29.5                        |  | 66.5               | <u>77.0</u>  | 50.0                     | 69.5                  | <u>78.0</u>                | 89.0        | 8.0                  |
| Aerospace (DE)                                     | 58.9         | 14.1           | 50.4                  | 44.1                    | <b>62.3</b>     | 59.0            | 48.1                        |  | <u>58.1</u>        | 48.1         | 47.0                     | -                     | 54.9                       | 73.8        | -                    |
| Automotive Supplier (EN)                           | <b>99.0</b>  | 72.4           | 87.2                  | 83.3                    | 94.6            | 92.6            | 84.2                        |  | <u>95.1</u>        | 95.0         | 87.1                     | 92.0                  | <u>91.0</u>                | 97.3        | 35.9                 |
| Industrial Drive Technology (DE)                   | <b>60.0</b>  | 31.4           | 52.3                  | 26.8                    | <b>60.0</b>     | 59.5            | 32.7                        |  | 32.3               | 51.4         | 56.8                     | <u>57.3</u>           | 37.3                       | 71.4        | 20.9                 |
| <b>Long Context</b>                                |              |                |                       |                         |                 |                 |                             |  |                    |              |                          |                       |                            |             |                      |
| LongBench Pro                                      | 64.5         | 0.0            | -                     | 53.2                    | 70.3            | <u>70.8</u>     | 63.8                        |  | 64.3               | -            | 56.4                     | -                     | <u>62.9</u>                | 76.9        | -                    |
| AA-LCR                                             | 68.3         | 0.0            | 49.7                  | 42.3                    | 66.3            | <b>69.7</b>     | 46.3                        |  | 68.3               | -            | 52.3                     | -                     | 67.0                       | 81.3        | -                    |

Table 29. Post-training benchmark scores, for Kolibri, Kolibri Origin, and comparable open models (part II/II). Continuation of Table 28 with the Instruction Following, Grounding, Agentic Retrieval, Industry RAG and Long Context rows. The models are grouped by architecture and sorted by active parameters. In each row, the **best score among the twelve compared models is set bold**, and the best score of a group is underlined. The dense models activate several times as many parameters per token, so they are greyed out and are unmarked. To ensure a fair comparison, we disregard rows with missing evals except for Apertus 70B Instruct from the averages, and colour them in grey. Likewise, we exclude AA-Omniscience Index, whose range is not a percentage, HoneyPot, which is bilingual, and the individual BFCL v4 splits. The Overall rows are the unweighted mean of the category averages of their language.

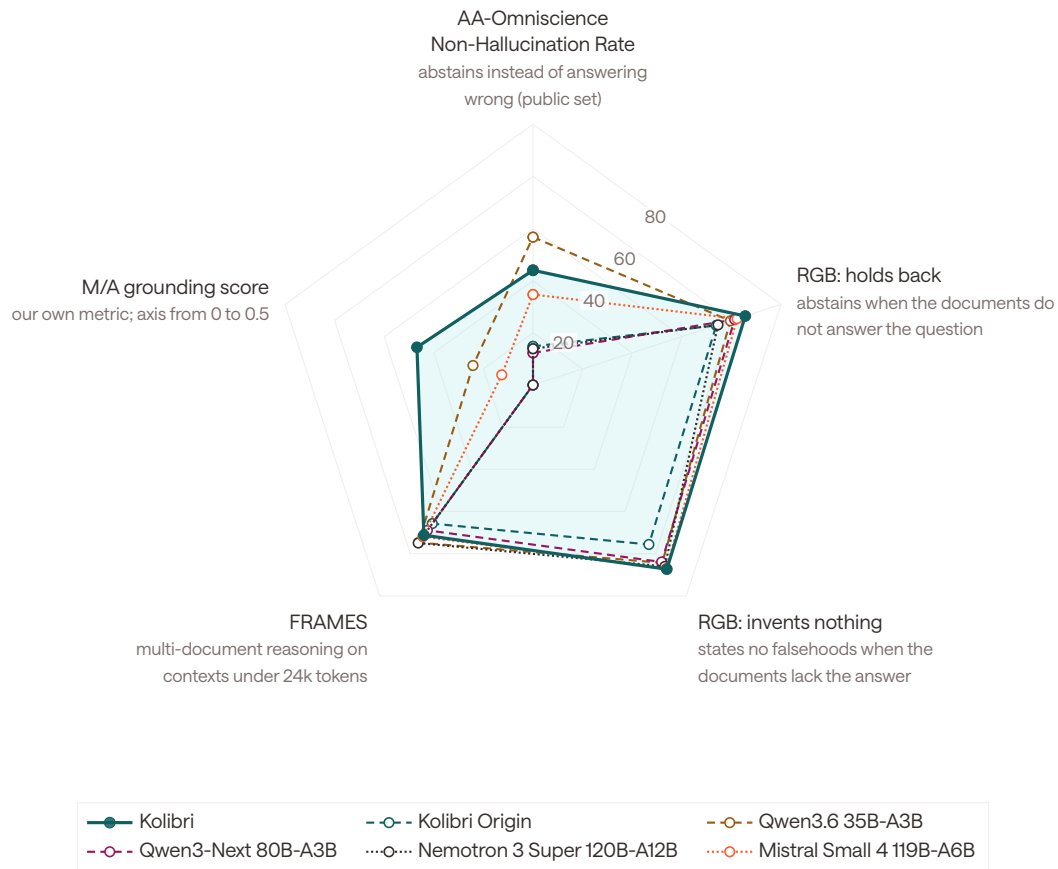


Figure 47. Agentic retrieval and customer proxy benchmark scores in percent of the released Kolibri checkpoint and the three open models of Figure 2, from the release table. MuSiQue, Honeypot and Agentic Wiki QA (DE) are agentic retrieval benchmarks; the five customer proxies are internal benchmarks built from customer use cases, named by vertical.

Our results show very strong performance within-distribution, in terms of the datasets we clean and prepare on the basis of publicly available corpora and synthetic multi-hop RAG performance. Together with feed-back from evaluations on the real-world "customer-proxies", our retrieval environments contribute to an iterative engine that allows us to sharpen the model's agentic RAG capabilities, domain-specific logic and capabilities, and to improve on contextualised evaluations without ever training on customer data. Our final scores on the customer-proxy evaluations provide evidence that the retrieval-environments reflect the specifics of production setups (harness, data ingestion) as well as reflecting real-world data distributions.

### 3.3.3.2 Hallucination Reduction

Kolibri hallucinates far less than Kolibri Origin: it abstains instead of answering wrong on 44 % of AA-Omniscience items (Origin: 15 %), and on RGB it holds back more often (86 % vs 74 %). On our own M/A grounding score, which certifies rather than estimates how much of an answer came from the document, Kolibri reaches 23.4 % where Kolibri Origin and several other models score 0. See Figure 48 for more details.



**Figure 48.** Grounding and hallucination benchmark scores in percent of the released Kolibri checkpoint, the Kolibri Origin checkpoint it starts from, the three open models of Figure 2 and Qwen3-Next 80B-A3B. The AA-Omniscience non-hallucination rate is one minus the share of the questions not answered correctly that the model answered wrong instead of abstaining or giving a partial answer. The two RGB scores are measured on questions whose documents do not contain the answer. The M/A grounding score is our own metric for how much of the answer provably came from the document; its spoke runs from 0 to 0.5, not from 0 to 100.

## Part 4

# Model Factory

Our Model Factory is the system we use to iterate on our training recipes; Kolibri Origin and Kolibri are the first models it produced. Our emphasis is not just on the current performance of Kolibri, but on how quickly that performance can be improved over time. Poolside’s Model Factory (Rowell and Kant 2025) and Microsoft AI’s hill-climbing machine (The Microsoft AI Team 2026) place a similar emphasis. Our hill climbing accelerates as our learnings build up and our Model Factory evolves: Kolibri climbs faster than Kolibri Origin (Figure 49).

How quickly Kolibri improves depends on how quickly the learnings of many teams reach its training recipe. The challenge is organisational, as more than 50 people, and an increasing number of agents, contribute to Kolibri. The training recipe they contribute to has grown into a pipeline of pre-training, mid-training, long-context extension, SFT, RL, and evaluation, developed by different teams. A manual training process—typically a collection of scripts and configs on a shared filesystem—does not scale to this pipeline, for three reasons. First, every manual step is an opportunity for human error, and an error on a run of hundreds of GPUs is expensive. Second, teams cannot easily run steps developed by other teams, so they optimise single stages in isolation, with little signal on the impact on the final model. Third, the record of why a hyperparameter has its value, which datasets a mix holds and which recipe produced a checkpoint is scattered across chat threads, wiki pages and local notes, and is lost over time.

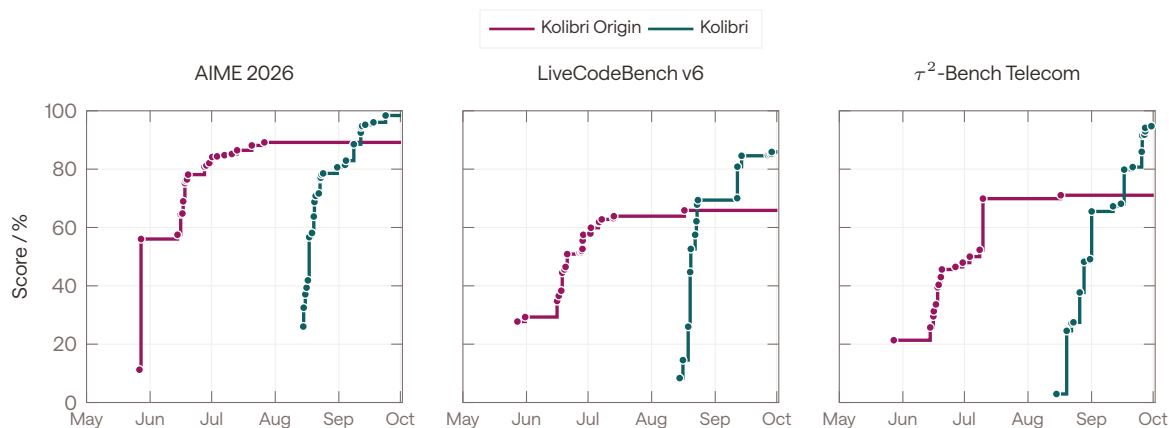


Figure 49. Best score reached so far by any post-trained checkpoint of Kolibri Origin and Kolibri on three benchmarks, taken from our internal leaderboard. Kolibri climbs faster than Kolibri Origin.

To solve this organisational challenge, we implement the entire training pipeline in code, turning model training into a collaborative software project. We call this approach Model Training as Code (MTaC) (Barlow 2026). By lifting the pipeline into code, we gain composability, consensus and provenance. Composability comes from expressing manual steps as functions with typed inputs and outputs, which we build abstractions around and compose into full experiments. Consensus comes from version control, where the `main` branch represents the team's collective best understanding of how to train a model. Provenance comes from code comments and commit history, which encode the learnings and decisions that led to `main`.

## 4.1 Savanna: Model Training as Code

Savanna, our Model Factory, is our implementation of MTaC. Savanna implements the Kolibri training recipe end-to-end and provides the framework for experimenting with it.

Savanna owns the orchestration of the Kolibri pre- and post-training runs and their resumes, ablations and sweeps, checkpointing and model merging, evaluation, lineage, abstractions over clusters and storage, artefact retention, and declarative model releases.

### 4.1.1 Core Abstractions

Savanna runs are declarative. A single TOML file describes a pre- or post-training run and its evaluation strategy end-to-end. The following is a simplified example of our Kolibri medium-scale pre-training recipe on `main`:

```
[pre.train]
model_architecture_config = "kolibri_ablation_scale_m"
gpu_count = { a100 = 256, h100 = 128, b200 = 64 }
training_steps = 10342
dataset = "pre-training-mix:v9"
tokenizer = "kolibri_tokenizer:v14"
checkpoint_interval = 100
intermediate_eval_interval = 1000
intermediate_eval_merge_window = 4

[pre.eval]
benchmarks = [
  {name = "mmlu_bpb"},
  {name = "gsm8k_de_bpb", num_fewshot = 8},
]

[mid.train]
# ...
```

Behind this declarative interface is the pipeline code, which composes a number of core abstractions that absorb accidental complexity.

Savanna's Checkpoint abstraction encapsulates the storage, retention, versioning, lineage and format conversion (DCP/Hugging Face) of model checkpoints. Consuming or producing a Checkpoint connects

its lineage, so users and their agents can navigate from a checkpoint to the configuration and commit that produced it, the datasets it was trained on, and its evaluation results.

Benchmarks range from static question sets to agentic environments, where the model acts over many turns with tools in a sandbox. Each needs its own evaluation backend, inference settings, model deployment, scoring and aggregation. Savanna's `Benchmark` abstraction encapsulates these together with the storage and lineage of the results, so users declare the benchmarks they are interested in and Savanna reports the scores.

While a run trains, Savanna registers each intermediate checkpoint as it appears, merges it with the preceding checkpoints of the configured window, and evaluates the result. This gives us signal over time, and the early signal lets us save GPU hours by stopping runs that are not promising.

We operate multiple GPU clusters, each with different hardware and software stacks, and we want to run the same recipe on all of them. Savanna's `Cluster` abstraction encapsulates the differences in GPU type, workload manager, storage systems, and other cluster-specific details, so a user only needs to name the cluster they want to run on when they dispatch their job.

To improve the Kolibri recipe, we run experiments, which are programmatic invocations of (subsets of) the pipeline with customisations. We merge the learnings from an experiment into `main`, making them part of subsequent Kolibri iterations.

## 4.1.2 Continuous Integration

With everyone iterating on the same codebase, `main` has to work at all times, so that anyone can branch off it, run an experiment and expect it to succeed. A conflict between two changes surfaces only when they are integrated, and the cost of resolving it grows with the size of both, so we practise trunk-based development and land changes on `main` often and in small increments (Fowler 2020) with targeted testing.

Every night, our Model Factory trains and evaluates a small model end to end. This catches regressions in the training or evaluation codebases, which are versioned dependencies of Savanna that automation bumps throughout the day as they release; a bump of the training codebase also runs training-specific tests before it merges.

## 4.1.3 Infrastructure

A user dispatches a training job from their branch, in which they have defined their experiment. The job submits an execution to Flyte (Flyte Contributors 2026), a Kubernetes-native workflow engine with durable execution, retries, caching and a UI to visualise runs. Every job pins versions of the code and of its artefacts (data, checkpoints, tokenisers), which we store immutably in a registry, so runs are hermetic. Our primary clusters run on Kubernetes for its flexibility, modularity and ecosystem. We use Kueue (Kubernetes SIG Scheduling 2026) for priority-based queueing and topology-aware scheduling to ensure jobs land within a

single InfiniBand domain. We observe cluster state, job status and logs, queue load and storage pressure via Grafana. As with our pipeline, our clusters are managed as code.

## 4.1.4 Agents in the Pipeline

Agents increasingly write the code and run the experiments behind Kolibri. To contribute, they need visibility into the state of training, a bounded space of actions they can take unsupervised, and measurable objectives to hill climb, each of which MTaC provides.

Visibility follows from every experiment being code pushed to the repository: agents see what is running and can trace it to the code that defines it and the reasoning behind that code. Given the lineage, logs and metrics the factory exposes, and instructions written for them, agents autonomously review proposed experiments and triage failed runs.

To bound the action space, the factory wraps storage, clusters and registries in abstractions that only permit safe operations, so researchers and agents can act freely within these limits. The registry, for instance, adds new artefact versions but never modifies existing ones (Section 4.1.3), and launches on a cluster can be gated by an independent review agent. These bounds let agents act with growing autonomy.

Objectives are quantities we define across experiments, which agents then hill climb. An example is GPU utilisation in evaluation: we deploy hundreds of models each week, each experiment with its own benchmarks and inference settings. This heterogeneity leaves room for idle GPUs, slow start-up and failed requests. Agents monitor these deployments, trace inefficiencies and propose fixes.

Agents working on the pipeline also encounter limitations that can go unnoticed when they act autonomously, such as instructions that fall short, steps they have to work around or known compromises they have to make. They either report these limitations as issues in the factory's repository, or mark them with `# XXX` comments in the code, leaving hints for future agents to pick up and hence steering how the factory evolves. Agents also analyse their own failures and propose changes to their instructions and harnesses that improve accuracy and reduce time and cost. Through these loops, experiments on Kolibri also improve the factory.

## 4.2 Savanna in Kolibri Training

Every Kolibri experiment was orchestrated by Savanna. The following examples show where Savanna's features proved particularly useful.

**Pre-Training Mix Experiment** To set the pre-training mix, the pre-training data team trained thousands of small proxy models, each on a different mixture, and fitted a mixing law to their scores (Section 2.3.2.4). These proxy models drew from the same data pool and ran through the same pre-training and evaluation pipeline as production, both as defined on `main`. Savanna allowed us to implement and launch this as a

single experiment, where each proxy training and evaluation task was queued independently such that it could fill the gaps between the larger jobs on the cluster.

**Pre-Training Run of Kolibri** We launched the Kolibri pre-training on a protected branch from `main`. Over the course of training, Savanna automatically evaluated a merge of the last 4 checkpoints every 2000 steps. This allowed us to monitor training progress continuously and ensure the run stays on track. Savanna handles training and evaluation resumes automatically from the correct checkpoint while preserving lineage and metrics, making restarts still appear as a single run. This reduces complexity of mid-run improvements, such as adapting the logging and profiling frequency, which in Kolibri pre-training raised the throughput by about 3.7 % (Figure 56). Running from this point-in-time branch allowed us to keep the training recipe stable, only changing it with carefully reviewed diffs applied via pull request. The run recovered from infrastructure failures automatically (Appendix B.7.2).

**Mid-Training Mix Ablations** To choose the mid-training mix, we trained a proxy model on each candidate mix (Section 2.3.3.2). Savanna allowed us to compose a flash-SFT stage onto each proxy model and evaluate it on the SFT benchmarks, giving us a richer post-training signal alongside the pre-training evaluations.

## Part 5

# Summary

## Model overview

Kolibri is the second model in the Kolibri family, an English–German MoE with 78.1B total and 3.46B active parameters per token, released as open weights under the Apache 2.0 licence. It was pre-trained on nearly 24T tokens, supports contexts of up to 1M tokens, and was built end-to-end by Aleph Alpha.

In the three months between Kolibri Origin and Kolibri, we changed almost every stage of the pipeline, i.e., pre-training data curation, attention mechanism, sparsity, load balancing, tokeniser, the length of each training stage, expanded and remixed the SFT data pool, and the RL environments and infrastructure.

We make several choices which set Kolibri apart from others for German. Its tokeniser, a 128 000-token vocabulary trained with UniBPE, a hybrid of BPE and Unigram-loss merge scoring, needs 11.2 % fewer tokens for German text than GPT-5, the best of the nine tokenisers of other models we evaluated, while staying close to it in English. More than a fifth of its pre-training tokens are German, with open German datasets complemented by more than 2T German tokens that we curate from the web with our own pipeline or generate synthetically. In post-training, SFT cold-starts reasoning in German across all domains, so that the model reasons in German rather than only answering in it.

The Kolibri architecture keeps the cost of a token low: 40 of its 50 blocks use sliding-window attention with RoPE, the other 10 use full attention without positional encoding, and each MoE layer routes a token to 6 of its 384 routed experts, in addition to one shared expert. Post-training starts with SFT on 537B tokens. It ends with RL on more than 1.2M internally curated tasks covering reasoning, tool use, instruction following, code and retrieval.

## Capabilities

On the English and German evaluation aggregates, Kolibri scores above every compared MoE model of its size (3B active parameters). It also matches or exceeds models more than three times its active parameter size on maths and code. Compared with Kolibri Origin, it improves substantially in both languages and scores at least as high on every shared benchmark, with the exception of RGB Closed-Book. In both languages, Kolibri achieves the highest aggregate among the MoE models we compared. In German, its closest MoE model is GPT-OSS 120B. The dense Qwen3.8 27B, which activates nearly eight times as many

parameters per token, scores higher in both languages. Among the open models evaluated, Kolibri lies on the Pareto frontier of quality and serving cost in both languages (Figure 1).

On the five proxy Industry RAG evaluations, which run over unseen corpora and questions, Kolibri performs strongly in both English and German, ahead of every MoE baseline in English and of every MoE baseline except Qwen3.5 35B-A3B in German. Hallucination falls markedly relative to Kolibri Origin. On English multiple-choice tasks with an added “I don’t know” option, Kolibri Base distributes its probability mass towards abstention and correctness considerably more than any other comparably-sized model, and is second only to Nemotron 3 Super 120B-A12B Base. After post-training, Kolibri abstains instead of answering wrongly, holds back more often than Kolibri Origin when documents lack the answer, and invents fewer falsehoods.

As a base model, Kolibri scores above or ties every compared model of its size in both languages, and is the best-performing model in its weight class up to 5B active parameters. Its overall aggregates are second only to Nemotron 3 Super 120B-A12B Base, which has more than three times as many active parameters; GLM-4.5 Air, also with 12B active parameters, scores lower in both languages. Compared with Kolibri Origin base, Kolibri improves substantially in both languages.

## Model Factory

In the course of developing the Kolibri family, the speed at which we made well-informed decisions on model size and design, training data mixes, and benchmarks set our pace more than compute did. Our Model Factory is built around that constraint and required automation. It consists of a centralised repo, Savanna, shared by all training teams and was designed to scalably and reliably run hundreds of reproducible, concurrent experiments, together with a common ablation protocol, as well as comparison methods to drive modelling choices. As inputs to this central repo, we have stage- and function-specific repos owned or shared by subteams. Because we develop and integrate the whole pipeline in-house, from data curation to RL, each of our lessons learned becomes a trackable change to our shared internal recipe.

## Conclusion

One of the most durable results of this work is neither a checkpoint nor a training pipeline; it is a team in Europe that has trained two Kolibri generations in a short amount of time and knows how to fail forward and learn. We are excited to continue building sovereign models while incorporating learnings from the past and feedback from the future. For instance, we see further improving German reasoning, reduce hallucinations further, and extending multi-lingual capabilities to European languages as strong areas of interest.

# Legal Due Diligence Example: Deep Dive Into Copyright Law

Legal conformity is one of our core values, especially for model training. Our data handling and model training is built in compliance with applicable regulations, in particular the EU AI Act, the GPAI Model Code of Practice under the EU AI Act (cf. below), the GDPR (cf. measures described in this Tech Report), and the DSM-Directive and national copyright laws. Copyright laws are a core aspect of our focus to build regulatory compliant technology which is why we go even beyond obligations contained in the applicable legal framework. In the following section, we illustrate on how we operate to achieve such goal by using an aspect of our copyright law due diligence as an example:

As a GPAI Model developer training its models in the EU, European legislation sets the legal starting point and framework for our work. With regard to Copyright Law, the Text and Data Mining exception (Article 4(3) DSM-Directive) in European and national Copyright law is of central importance and is directly referenced by Article 53(1)(c) of the EU AI Act alongside the obligations of GPAI Model Providers. According to Article 53(1)(c) of the EU AI Act, identification of and compliance with rights reservations expressed under the Text and Data Mining exception shall furthermore be illustrated in the copyright policy of GPAI Model Providers. The GPAI Model Code of Practice chapter on copyright sets out concrete commitments. GPAI Model Providers can demonstrate compliance with the EU AI Act's obligations by adhering to the GPAI Model Code of Practice (cf. Article 53(4) EU AI Act). Aleph Alpha was part of the working group, which drafted the GPAI Model Code of Practice and signed it. By being a signatory to the General Purpose AI Code of Practice, we committed to adhere to and respect the commitments set out therein.

We have assessed the legal requirements under copyright law and the commitments contained in the GPAI Model Code of Practice. We then decided to implement an additional layer of legal assessments with regard to the use of training datasets compiled, curated and made available by third-parties ("Third-Party Datasets"). The GPAI Model Code of Practice does not contain explicit commitments regarding the use of Third-Party Datasets. Rather, it solely states that its commitments do not affect the application of copyright provisions to protected content scraped or crawled from the internet by third parties and used by the model developer. The lack of explicit commitment can harm the trust people have in this technology. We believe applying a comparable standard to Third-Party Datasets as it is expected for direct crawling activities under the General Purpose AI Code of Practice is important to help build trustworthy technology. This is why we assess Third-Party Datasets against the General Purpose AI Code of Practice commitments for direct crawling using all reasonably available information on such datasets before we decide to use them for training to validate whether the entity compiling and making available the data has complied with copyright law.

This assessment entails the following steps:

1. **Legal Data Source:** We investigate the dataset creation methods of the third party based on reasonably available information. We assess the source of the data contained in the dataset on possible indications of copyright infringements and whether such data was made available on the initial source in compliance with applicable copyright laws. If we find indications for infringements on the data source level, we exclude the corresponding dataset from our model development.
2. **Adherence to Opt-Outs and License Terms on the Data Source:** In a next step we assess whether the content of the data source may be crawled and used for AI training. In this step, we assess if the entity crawling and compiling the data adhered to appropriate instructions such as instructions expressed in accordance with the Robot Exclusion Protocol as specified in the IETF RFC No. 9309 (“robots.txt”). For such assessment we collect information on the crawling behaviour of the third party’s crawlers or individually assess these files. If we find indications that the entity crawling the data did not adhere to an appropriate opt-out, we exclude such datasets from our training. Furthermore, in this step we assess potential license terms and other restrictions of such crawled content expressed in natural language.

These steps go beyond the commitments contained in the GPAI Model Code of Practice and are supplemented by additional technical measures we implement during data collection and curation, which are set out in the relevant sections in this report. By doing so, we raise the bar for transparent and compliant AI Model developers and emphasize the importance of building trustworthy technology by respecting fundamental rights such as copyrights beyond the legal requirements.

# Contributors<sup>5</sup>

## Pre-training Data

Thomas F. Burns, Niko Dürr, Fabian Gebhart, Bastian Harren, Andreas Hartel, Peter Rodenkirch Nfissi, Ivo Schaper, Lukas Schott, Mohamed Soliman, Denis Waßmann

## Pre-training Efficiency

Steffen Hirschmann, Jordan Sassoon, Michael Wüthrich

## Pre-training Architecture and Training

Fabien Benureau, Jonas Knupp, Jiaze Li, Sohir Maskey, Pit Neiteimeier, Philipp Scholl, Eric Schreiber, Alessio Serra, Volker Stampa, Sascha Wirges

## Evaluations

Markus Klein, Alejandro Petry Pacheco, Martin Reinhardt, Raziye Sari, Frank Schneider, Prabhu Teja Sivaprasad

## Supervised Fine-tuning

Niklas Finken, Garwin Lechner, Dylan Rodriguez, Leonard Salewski, Simon Thel, Irina Vidal Migallón

## Environments

Max Hammer, Murali Kondragunta, Philipp Siedler

## Agentic RAG and Real-World Applications

Duncan Blythe, Pranav Ragupathy, Till Speicher, Alexei Zacharov

## Hallucination Reduction

Augustijn de Boer, Béni Balázs Egressy, Björn Deiseroth, Johann Higl, Jurek Leonhardt, Omar Mehio, Letitia Pârcălăbescu

## Reinforcement Learning

Amrita Bhattacharjee, Artur Baranowski, Bastian Boll, Felix Berkenkamp, Lukas Blübaum, Niklas Freymuth, Pablo Iyu Guerrero, Serge Thilges

## Model Factory

Moritz Althaus, Michael Barlow, Mitja Kleider, Dominik Lê-Kellner

## Legal

Andreas Jagusch, Pablo Schumacher

## Technical Leadership

Yasser Jadidi, Ilhan Scheer, Samuel Weinbach

---

<sup>5</sup>Within each team, contributors are listed alphabetically by last name.

## Acknowledgments

We thank Verda for supplying the GPU compute infrastructure behind this work: they provision and operate the bare-metal fleet and Kubernetes clusters, and maintain the cluster services on our behalf. Their responsiveness in adapting the platform to our requirements throughout the trainings of Kolibri Origin and Kolibri was invaluable.

We are especially grateful to Paul Chang, Riccardo Mereu, and Daniel Obolensky for their significant contributions to improving pre- and post-training efficiency. Their efforts were key in achieving a performant model.

We would also like to express our particular thanks to Vishal Verma, Nikolai Syrjälä, and Ananth Mahadevan for their substantial engineering contributions, and the rest of the Verda team for their support throughout this work.

## References

- Abbes, Istabrak et al. (2026). “Revisiting Replay and Gradient Alignment for Continual Pre-Training of Large Language Models”. In: *Proceedings of the Conference on Lifelong Learning Agents*. Vol. 330. Proceedings of Machine Learning Research. PMLR. URL: <https://proceedings.mlr.press/v330/abbes26a.html>.
- Abdin, Marah et al. (2024). *Phi-4 Technical Report*. arXiv: 2412.08905 [cs.CL]. URL: <https://arxiv.org/abs/2412.08905>.
- Abnar, Samira et al. (2025). “Parameters vs FLOPs: Scaling Laws for Optimal Sparsity for Mixture-of-Experts Language Models”. In: *Proceedings of the 42nd International Conference on Machine Learning*. Vol. 267. Proceedings of Machine Learning Research. PMLR, pp. 204–230. URL: <https://proceedings.mlr.press/v267/abnar25a.html>.
- Aggarwal, Pranjal and Sean Welleck (2025). *L1: Controlling How Long a Reasoning Model Thinks with Reinforcement Learning*. arXiv: 2503.04697 [cs.CL]. URL: <https://arxiv.org/abs/2503.04697>.
- Ainslie, Joshua et al. (2023). “GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints”. In: *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, pp. 4895–4901. DOI: 10.18653/v1/2023.emnlp-main.298. URL: <https://aclanthology.org/2023.emnlp-main.298/>.
- Aleph Alpha Research (2026). *Aleph Alpha Eval Framework*. URL: <https://github.com/Aleph-Alpha-Research/eval-framework>.
- Allen Institute for AI (2025). *Decon: Contamination Detection for Text Training Data*. URL: <https://github.com/allenai/decon>.
- Ansel, Jason et al. (Apr. 2024). “PyTorch 2: Faster Machine Learning Through Dynamic Python Bytecode Transformation and Graph Compilation”. In: *29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2 (ASPLOS ’24)*. ACM. DOI:

- 10.1145/3620665.3640366. URL: <https://docs.pytorch.org/assets/pytorch2-2.pdf>.
- Artificial Analysis (2025). *AA-LCR: Artificial Analysis Long Context Reasoning*. URL: <https://huggingface.co/datasets/ArtificialAnalysis/AA-LCR> (visited on 09/25/2026).
- Austin, Jacob et al. (2021). *Program Synthesis with Large Language Models*. arXiv: 2108.07732 [cs.PL]. URL: <https://arxiv.org/abs/2108.07732>.
- Badertdinov, Ibragim et al. (2026). *SWE-rebench V2: Language-Agnostic SWE Task Collection at Scale*. arXiv: 2602.23866 [cs.SE]. URL: <https://arxiv.org/abs/2602.23866>.
- Bai, Yuntao et al. (2022). *Training a Helpful and Harmless Assistant with Reinforcement Learning from Human Feedback*. arXiv: 2204.05862 [cs.CL]. URL: <https://arxiv.org/abs/2204.05862>.
- Barlow, Michael (2026). *Model Training as Code*. URL: <https://aleph-alpha.com/en/blog/model-training-as-code/> (visited on 10/02/2026).
- Barres, Victor et al. (2025).  $\tau^2$ -Bench: Evaluating Conversational Agents in a Dual-Control Environment. arXiv: 2506.07982 [cs.AI].
- Bashir, Hammad et al. (2026). *Chroma Context-1: Training a Self-Editing Search Agent*. Chroma. URL: <https://www.trychroma.com/research/context-1> (visited on 09/25/2026).
- Beltagy, Iz, Matthew E. Peters, and Arman Cohan (2020). *Longformer: The Long-Document Transformer*. arXiv: 2004.05150 [cs.CL]. URL: <https://arxiv.org/abs/2004.05150>.
- Bengio, Yoshua, Nicholas Léonard, and Aaron Courville (2013). *Estimating or Propagating Gradients Through Stochastic Neurons for Conditional Computation*. arXiv: 1308.3432. URL: <https://arxiv.org/abs/1308.3432>.
- Berasi, Davide et al. (2026). *Linear Model Merging Unlocks Simple and Scalable Multimodal Data Mixture Optimization*. arXiv: 2602.04937 [cs.LG].
- Bercovich, Akhiad et al. (2025). *Llama-Nemotron: Efficient Reasoning Models*. arXiv: 2505.00949 [cs.CL].
- Bergsma, Shane et al. (2025). *Power Lines: Scaling Laws for Weight Decay and Batch Size in LLM Pre-training*. arXiv: 2505.13738 [cs.LG]. URL: <https://arxiv.org/abs/2505.13738>.
- Bernstein, Jeremy (2025). *Deriving Muon*. URL: <https://jeremybernste.in/writing/deriving-muon>.
- Bevendorff, Janek et al. (2018). “Elastic ChatNoir: Search Engine for the ClueWeb and the Common Crawl”. In: *Advances in Information Retrieval. 40th European Conference on IR Research (ECIR 2018)*. Springer.
- Bianchi, Federico et al. (2024). “Safety-Tuned LLaMAs: Lessons from Improving the Safety of Large Language Models that Follow Instructions”. In: *International Conference on Learning Representations*. arXiv: 2309.07875. URL: <https://arxiv.org/abs/2309.07875>.
- Bisk, Yonatan et al. (2020). “PIQA: Reasoning about Physical Commonsense in Natural Language”. In: *Proceedings of the AAAI Conference on Artificial Intelligence*. Vol. 34. 05, pp. 7432–7439.
- Bjorck, Johan et al. (2024). *Scaling Optimal LR Across Token Horizons*. arXiv: 2409.19913 [cs.LG]. URL: <https://arxiv.org/abs/2409.19913>.
- Bordelon, Blake et al. (2023). *Depthwise Hyperparameter Transfer in Residual Networks: Dynamics and Scaling Limit*. arXiv: 2309.16620 [stat.ML]. URL: <https://arxiv.org/abs/2309.16620>.
- Bordt, Sebastian et al. (2025). “How Much Can We Forget about Data Contamination?” In: *Proceedings of the 42nd International Conference on Machine Learning*. Vol. 267. Proceedings of Machine Learning

- Research. PMLR, pp. 4998–5016. URL: <https://proceedings.mlr.press/v267/bordt25a.html>.
- Bottou, Léon (2012). “Stochastic Gradient Descent Tricks”. In: *Neural Networks: Tricks of the Trade*. Ed. by Grégoire Montavon, Geneviève B. Orr, and Klaus-Robert Müller. 2nd ed. Vol. 7700. Lecture Notes in Computer Science. Berlin, Heidelberg: Springer, pp. 421–436. DOI: [10.1007/978-3-642-35289-8\\_25](https://doi.org/10.1007/978-3-642-35289-8_25).
- Bottou, Léon, Frank E. Curtis, and Jorge Nocedal (2018). “Optimization Methods for Large-Scale Machine Learning”. In: *SIAM Review* 60.2, pp. 223–311. DOI: [10.1137/16M1080173](https://doi.org/10.1137/16M1080173).
- Broder, Andrei Z. (1997). “On the Resemblance and Containment of Documents”. In: *Proceedings of the Compression and Complexity of Sequences 1997*. IEEE, pp. 21–29.
- Brown, Tom et al. (2020). “Language Models are Few-Shot Learners”. In: *Advances in Neural Information Processing Systems*. Vol. 33, pp. 1877–1901.
- Burns, Thomas F (2026). “When Guessing is Rewarded: Rethinking Language Model Evaluation with Distributional Uncertainty Scoring”. In: *The Fortieth Annual Conference on Neural Information Processing Systems Evaluations and Datasets Track (accepted)*. URL: <https://arxiv.org/abs/2510.04302>.
- Burns, Thomas F et al. (2026). “Aleph-Alpha-GermanWeb: Improving German-language LLM pre-training with model-based data curation and synthetic data generation”. In: *Proceedings of the 19th Conference of the European Chapter of the Association for Computational Linguistics (Volume 1: Long Papers)*. Association for Computational Linguistics, pp. 1267–1283. DOI: [10.18653/v1/2026.eacl-long.58](https://doi.org/10.18653/v1/2026.eacl-long.58).
- Caballero, Ethan et al. (2023). “Broken Neural Scaling Laws”. In: *International Conference on Learning Representations*. arXiv: [2210.14891](https://arxiv.org/abs/2210.14891). URL: <https://arxiv.org/abs/2210.14891>.
- Chen, Jiawei et al. (2024). “Benchmarking Large Language Models in Retrieval-Augmented Generation”. In: *Proceedings of the AAAI Conference on Artificial Intelligence*. Vol. 38. 16, pp. 17754–17762. DOI: [10.1609/aaai.v38i16.29728](https://doi.org/10.1609/aaai.v38i16.29728).
- Chen, Lequn (2025). *Journey to 2-Second Inter-Node RL Weight Transfer*. URL: <https://le.qun.ch/en/blog/2025/09/07/rl-data-transfer/> (visited on 09/29/2026).
- Chen, Mark et al. (2021). *Evaluating Large Language Models Trained on Code*. arXiv: [2107.03374](https://arxiv.org/abs/2107.03374) [cs.LG]. URL: <https://arxiv.org/abs/2107.03374>.
- Chen, Mayee F et al. (2026). “Olmix: A Framework for Data Mixing Throughout LM Development”. In: *Forty-third International Conference on Machine Learning*. URL: <https://openreview.net/forum?id=8p013azhbl>.
- Chen, Tianqi et al. (2016). *Training Deep Nets with Sublinear Memory Cost*. arXiv: [1604.06174](https://arxiv.org/abs/1604.06174) [cs.LG]. URL: <https://arxiv.org/abs/1604.06174>.
- Chen, Xingyu et al. (2024). *Do NOT Think That Much for 2+3=? On the Overthinking of o1-Like LLMs*. arXiv: [2412.21187](https://arxiv.org/abs/2412.21187) [cs.CL]. URL: <https://arxiv.org/abs/2412.21187>.
- Chen, Ziyang et al. (2026). *LongBench Pro: A More Realistic and Comprehensive Bilingual Long-Context Evaluation Benchmark*. arXiv: [2601.02872](https://arxiv.org/abs/2601.02872) [cs.CL].
- Clark, Aidan et al. (2022). “Unified Scaling Laws for Routed Language Models”. In: *Proceedings of the 39th International Conference on Machine Learning*. Vol. 162. Proceedings of Machine Learning Research. PMLR, pp. 4057–4086.

- Clark, Peter et al. (2018). *Think You Have Solved Question Answering? Try ARC, the AI2 Reasoning Challenge*. arXiv: 1803.05457 [cs.AI]. URL: <https://arxiv.org/abs/1803.05457>.
- Cobbe, Karl et al. (2021). *Training Verifiers to Solve Math Word Problems*. arXiv: 2110.14168 [cs.LG]. URL: <https://arxiv.org/abs/2110.14168>.
- Cohere (2026). *Introducing Command A+*. Cohere blog. URL: <https://cohere.com/blog/command-a-plus>.
- Common Crawl Foundation (2026). *Common Crawl*. URL: <https://commoncrawl.org>.
- Cursor (2026). *Introducing Composer 2.5*. URL: <https://cursor.com/blog/composer-2-5> (visited on 09/30/2026).
- Dai, Damai et al. (2024). “DeepSeekMoE: Towards Ultimate Expert Specialization in Mixture-of-Experts Language Models”. In: *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Association for Computational Linguistics, pp. 1280–1297. DOI: 10.18653/v1/2024.acl-long.70. URL: <https://aclanthology.org/2024.acl-long.70/>.
- Dao, Tri (2023). *FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning*. arXiv: 2307.08691 [cs.LG]. URL: <https://arxiv.org/abs/2307.08691>.
- DatologyAI et al. (2025). *Luxical: High-Speed Lexical-Dense Text Embeddings*. arXiv: 2512.09015 [cs.CL]. URL: <https://arxiv.org/abs/2512.09015>.
- DeepSeek-AI (2024a). *DeepSeek LLM: Scaling Open-Source Language Models with Longtermism*. arXiv: 2401.02954 [cs.CL]. URL: <https://arxiv.org/abs/2401.02954>.
- (2024b). *DeepSeek-V3 Technical Report*. arXiv: 2412.19437 [cs.CL]. URL: <https://arxiv.org/abs/2412.19437>.
- (2025). “DeepSeek-R1 Incentivizes Reasoning in LLMs through Reinforcement Learning”. In: *Nature* 645, pp. 633–638. URL: <https://arxiv.org/abs/2501.12948>.
- Deiseroth, Björn et al. (2026). *Bounding Hallucinations: Merlin-Arthur Protocols for Mutual-Information Bounds in Language Models*. arXiv: 2512.11614 [cs.CL]. URL: <https://arxiv.org/abs/2512.11614>.
- Devlin, Jacob et al. (2019). “BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding”. In: *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*. Association for Computational Linguistics, pp. 4171–4186. DOI: 10.18653/v1/N19-1423. URL: <https://aclanthology.org/N19-1423/>.
- Dey, Nolan et al. (2025). *Don’t be lazy: CompleteP enables compute-efficient deep transformers*. arXiv: 2505.01618 [cs.LG]. URL: <https://arxiv.org/abs/2505.01618>.
- Dial, Larry (Apr. 2026). *Mixture of Experts Quantile Balancing: Validated at 32B-A5B (1e22 FLOPs) Scale*. <https://www.openathena.ai/blog/quantile-balancing/>. Open Athena Blog.
- Ding, Hantian et al. (2024). “Fewer Truncations Improve Language Modeling”. In: *Proceedings of the 41st International Conference on Machine Learning*. arXiv: 2404.10830. URL: <https://arxiv.org/abs/2404.10830>.
- Ding, Ming et al. (2021). “CogView: Mastering Text-to-Image Generation via Transformers”. In: *Advances in Neural Information Processing Systems*. Vol. 34.
- Domhan, Tobias, Jost Tobias Springenberg, and Frank Hutter (2015). “Speeding Up Automatic Hyperparameter Optimization of Deep Neural Networks by Extrapolation of Learning Curves”. In: *Proceedings*

- of the *Twenty-Fourth International Joint Conference on Artificial Intelligence*, pp. 3460–3468. URL: <https://www.ijcai.org/Proceedings/15/Papers/487.pdf>.
- ellamind (2025). *Multilingual Benchmark Translations*. URL: <https://huggingface.co/ellamind>.
- (2026). *GSM8K Platinum Multilingual: Translated Elementary Math Word Problems*. <https://huggingface.co/datasets/ellamind/gsm8k-platinum-multilingual>.
- Ettinger, Allyson et al. (2025). *Olmo 3*. arXiv: 2512.13961 [cs.CL]. URL: <https://arxiv.org/abs/2512.13961>.
- Fang, Alex et al. (2025). “Datasets, Documents, and Repetitions: The Practicalities of Unequal Data Quality”. In: *Advances in Neural Information Processing Systems*. URL: <https://arxiv.org/abs/2503.07879>.
- Flyte Contributors (2026). *Flyte: Workflow Automation Platform for Data and Machine Learning Processes*. URL: <https://flyte.org> (visited on 10/02/2026).
- Fowler, Martin (2020). *Patterns for Managing Source Code Branches*. URL: <https://martinfowler.com/articles/branching-patterns.html> (visited on 09/30/2026).
- Frantar, Elias et al. (2024). “Scaling Laws for Sparsely-Connected Foundation Models”. In: *The Twelfth International Conference on Learning Representations*. URL: <https://openreview.net/forum?id=i9K2ZWkYIP>.
- Fu, Dayuan et al. (2025). “AgentRefine: Enhancing Agent Generalization through Refinement Tuning”. In: *The Thirteenth International Conference on Learning Representations*. URL: <https://arxiv.org/abs/2501.01702>.
- Gage, Philip (1994). “A New Algorithm for Data Compression”. In: *The C Users Journal* 12.2, pp. 23–38.
- Gallouédec, Quentin and Kashif Rasul (2026). *Agentic RL: Token-In, Token-Out Done Right*. URL: <https://huggingface.co/blog/huggingface/tito> (visited on 09/26/2026).
- Ge, Hao et al. (2025). “ByteScale: Efficient Scaling of LLM Training with a 2048K Context Length on More Than 12,000 GPUs”. In: *Proceedings of the ACM SIGCOMM 2025 Conference*. URL: <https://arxiv.org/abs/2502.21231>.
- Gemma Team (2026). *Gemma 4 Technical Report*. arXiv: 2607.02770 [cs.CL]. URL: <https://arxiv.org/abs/2607.02770>.
- GLM-4.5 Team (2025). *GLM-4.5: Agentic, Reasoning, and Coding (ARC) Foundation Models*. arXiv: 2508.06471 [cs.CL]. URL: <https://arxiv.org/abs/2508.06471>.
- Goyal, Priya et al. (2017). *Accurate, Large Minibatch SGD: Training ImageNet in 1 Hour*. arXiv: 1706.02677 [cs.CV]. URL: <https://arxiv.org/abs/1706.02677>.
- Gu, Yuling et al. (2024). *OLMES: A Standard for Language Model Evaluations*. arXiv: 2406.08446 [cs.CL].
- Guha, Etash et al. (2025). *OpenThoughts: Data Recipes for Reasoning Models*. arXiv: 2506.04178 [cs.LG]. URL: <https://arxiv.org/abs/2506.04178>.
- Guo, Wentao et al. (2025). *SonicMoE: Accelerating MoE with IO and Tile-aware Optimizations*. arXiv: 2512.14080 [cs.LG]. URL: <https://arxiv.org/abs/2512.14080>.
- Gupta, Kshitij et al. (2023). *Continual Pre-Training of Large Language Models: How to (re)warm your model?* arXiv: 2308.04014 [cs.CL]. URL: <https://arxiv.org/abs/2308.04014>.
- Han, Tingxu et al. (2024). *Token-Budget-Aware LLM Reasoning*. arXiv: 2412.18547 [cs.CL]. URL: <https://arxiv.org/abs/2412.18547>.

- Harbor Framework Team (2026). *Harbor: A Framework for Evaluating and Optimizing Agents and Models in Container Environments*. DOI: [10.5281/zenodo.20953922](https://doi.org/10.5281/zenodo.20953922). URL: <https://github.com/harbor-framework/harbor>.
- He, Horace and Thinking Machines Lab (2025). *Defeating Nondeterminism in LLM Inference*. URL: <https://thinkingmachines.ai/blog/defeating-nondeterminism-in-llm-inference/>.
- Hendrycks, Dan, Collin Burns, Steven Basart, Andrew Critch, et al. (2021). “Aligning AI With Shared Human Values”. In: *International Conference on Learning Representations*. URL: [https://openreview.net/forum?id=dNy\\_RKzJacY](https://openreview.net/forum?id=dNy_RKzJacY).
- Hendrycks, Dan, Collin Burns, Steven Basart, Andy Zou, et al. (2021). “Measuring Massive Multitask Language Understanding”. In: *International Conference on Learning Representations*. URL: <https://openreview.net/forum?id=d7KBjmI3GmQ>.
- Hendrycks, Dan, Collin Burns, Saurav Kadavath, et al. (2021). “Measuring Mathematical Problem Solving with the MATH Dataset”. In: *Advances in Neural Information Processing Systems, Datasets and Benchmarks Track*. URL: <https://openreview.net/forum?id=7Bywt2mQsCe>.
- Henry, Alex et al. (2020). “Query-Key Normalization for Transformers”. In: *Findings of the Association for Computational Linguistics: Empirical Methods in Natural Language Processing 2020*, pp. 4246–4253.
- Hoffmann, Jordan et al. (2022). “An Empirical Analysis of Compute-Optimal Large Language Model Training”. In: *Advances in Neural Information Processing Systems*. Vol. 35, pp. 30016–30030. URL: [https://proceedings.neurips.cc/paper\\_files/paper/2022/file/c1e2faff6f588870935f114ebe04a3e5-Paper-Conference.pdf](https://proceedings.neurips.cc/paper_files/paper/2022/file/c1e2faff6f588870935f114ebe04a3e5-Paper-Conference.pdf).
- Hong, Kelly, Anton Troynikov, and Jeff Huber (2025). *Context Rot: How Increasing Input Tokens Impacts LLM Performance*. Chroma. URL: <https://www.trychroma.com/research/context-rot> (visited on 09/25/2026).
- Hsieh, Cheng-Ping et al. (2024). “RULER: What’s the Real Context Size of Your Long-Context Language Models?” In: *First Conference on Language Modeling*. URL: <https://openreview.net/forum?id=kIoBbc76Sy>.
- Hu, Shengding et al. (2024). *MiniCPM: Unveiling the Potential of Small Language Models with Scalable Training Strategies*. arXiv: [2404.06395](https://arxiv.org/abs/2404.06395) [cs.CL]. URL: <https://arxiv.org/abs/2404.06395>.
- Ibrahim, Adam et al. (2024). “Simple and Scalable Strategies to Continually Pre-train Large Language Models”. In: *Transactions on Machine Learning Research*. URL: <https://openreview.net/forum?id=DimPeeCxK0>.
- InfIXAI Team et al. (2025). *Unified FP8: Moving Beyond Mixed Precision for Stable and Accelerated MoE RL*. URL: <https://www.lmsys.org/blog/2025-11-25-fp8-rl/> (visited on 09/29/2026).
- Izmailov, Pavel et al. (2018). *Averaging Weights Leads to Wider Optima and Better Generalization*. arXiv: [1803.05407](https://arxiv.org/abs/1803.05407) [cs.LG].
- Jackson, Declan et al. (2025). *AA-Omniscience: Evaluating Cross-Domain Knowledge Reliability in Large Language Models*. arXiv: [2511.13029](https://arxiv.org/abs/2511.13029) [cs.CL].
- Jacob, Benoit et al. (2017). *Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference*. arXiv: [1712.05877](https://arxiv.org/abs/1712.05877). URL: <https://arxiv.org/abs/1712.05877>.
- Jain, Naman et al. (2024). *LiveCodeBench: Holistic and Contamination Free Evaluation of Large Language Models for Code*. arXiv: [2403.07974](https://arxiv.org/abs/2403.07974) [cs.SE].

- Jiang, Ziheng et al. (2024). “MegaScale: scaling large language model training to more than 10,000 GPUs”. In: *Proceedings of the 21st USENIX Symposium on Networked Systems Design and Implementation*. NSDI’24. Santa Clara, CA, USA: USENIX Association. ISBN: 978-1-939133-39-7.
- Jimenez, Carlos E. et al. (2024). “SWE-bench: Can Language Models Resolve Real-World GitHub Issues?”. In: *The Twelfth International Conference on Learning Representations*. URL: <https://arxiv.org/abs/2310.06770>.
- Jin, Hongye et al. (July 2026). *Mitigate Silent Expert Death in Ultra-Sparse MoE*. URL: <https://alltoall.notion.site/save-lower-layer-moe-experts-llal> (visited on 10/02/2026).
- Johnson, David S. (1974). “Fast Algorithms for Bin Packing”. In: *Journal of Computer and System Sciences* 8.3, pp. 272–314.
- Jordan, Keller et al. (2024). *Muon: An optimizer for hidden layers in neural networks*. URL: <https://kellerjordan.github.io/posts/muon/>.
- Joshi, Mandar et al. (2017). “TriviaQA: A Large Scale Distantly Supervised Challenge Dataset for Reading Comprehension”. In: *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Association for Computational Linguistics, pp. 1601–1611. DOI: [10.18653/v1/P17-1147](https://aclanthology.org/P17-1147/). URL: <https://aclanthology.org/P17-1147/>.
- Joulin, Armand et al. (2017). “Bag of Tricks for Efficient Text Classification”. In: *Proceedings of the 15th Conference of the European Chapter of the Association for Computational Linguistics: Volume 2, Short Papers*, pp. 427–431.
- Kadavath, Saurav et al. (2022). *Language Models (Mostly) Know What They Know*. arXiv: [2207.05221](https://arxiv.org/abs/2207.05221).
- Kalai, Adam Tauman et al. (2026). “Evaluating large language models for accuracy incentivizes hallucinations”. In: *Nature*. DOI: [10.1038/s41586-026-10549-w](https://doi.org/10.1038/s41586-026-10549-w).
- Kimi Team (2025). *Kimi K2: Open Agentic Intelligence*. arXiv: [2507.20534](https://arxiv.org/abs/2507.20534) [cs.LG]. URL: <https://arxiv.org/abs/2507.20534>.
- (2026a). *Kimi K2.5: Visual Agentic Intelligence*. arXiv: [2602.02276](https://arxiv.org/abs/2602.02276) [cs.CL]. URL: <https://arxiv.org/abs/2602.02276>.
- (2026b). *Kimi K3: Open Frontier Intelligence*. arXiv: [2607.24653](https://arxiv.org/abs/2607.24653) [cs.CL]. URL: <https://arxiv.org/abs/2607.24653>.
- (2026c). *Kimi-K2.6*. URL: <https://huggingface.co/moonshotai/Kimi-K2.6> (visited on 09/28/2026).
- (2026d). *Kimi-K2.7-Code*. URL: <https://huggingface.co/moonshotai/Kimi-K2.7-Code> (visited on 09/28/2026).
- Kingma, Diederik P. and Jimmy Ba (2015). “Adam: A Method for Stochastic Optimization”. In: *International Conference on Learning Representations*. arXiv: [1412.6980](https://arxiv.org/abs/1412.6980) [cs.LG]. URL: <https://arxiv.org/abs/1412.6980>.
- Kojima, Takeshi et al. (2022). “Large Language Models are Zero-Shot Reasoners”. In: *Advances in Neural Information Processing Systems*. URL: <https://arxiv.org/abs/2205.11916>.
- Krell, Mario Michael et al. (2021). *Efficient Sequence Packing without Cross-contamination: Accelerating Large Language Models without Impacting Performance*. arXiv: [2107.02027](https://arxiv.org/abs/2107.02027) [cs.CL]. URL: <https://arxiv.org/abs/2107.02027>.
- Krishna, Satyapriya et al. (2024). *Fact, Fetch, and Reason: A Unified Evaluation of Retrieval-Augmented Generation*. arXiv: [2409.12941](https://arxiv.org/abs/2409.12941) [cs.CL].

- Krishnamurthy, Yamuna, Chris Watkins, and Thomas Gaertner (2023). *Improving Expert Specialization in Mixture of Experts*. arXiv: 2302.14703 [cs.LG]. URL: <https://arxiv.org/abs/2302.14703>.
- Kubernetes SIG Scheduling (2026). *Kueue: Kubernetes-Native Job Queueing*. URL: <https://kueue.sigs.k8s.io> (visited on 10/02/2026).
- Kudo, Taku (2018). “Subword Regularization: Improving Neural Network Translation Models with Multiple Subword Candidates”. In: *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 66–75. DOI: 10.18653/v1/P18-1007. URL: <https://aclanthology.org/P18-1007/>.
- Kudugunta, Sneha, Isaac Caswell, Biao Zhang, et al. (2023). *MADLAD-400: A Multilingual And Document-Level Large Audited Dataset*. arXiv: 2309.04662 [cs.CL].
- Kumaran, Dharshan et al. (2026). “Causal evidence that language models use confidence to drive behaviour”. In: *Nature Machine Intelligence*. DOI: 10.1038/s42256-026-01293-x.
- Kurtzer, Gregory M., Vanessa Sochat, and Michael W. Bauer (2017). “Singularity: Scientific Containers for Mobility of Compute”. In: *PLoS ONE* 12.5, e0177459. DOI: 10.1371/journal.pone.0177459.
- Kwon, Woosuk et al. (2023). “Efficient Memory Management for Large Language Model Serving with PagedAttention”. In: *Proceedings of the 29th Symposium on Operating Systems Principles*. DOI: 10.1145/3600006.3613165. URL: <https://arxiv.org/abs/2309.06180>.
- Lee, Katherine et al. (2022). “Deduplicating Training Data Makes Language Models Better”. In: *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Association for Computational Linguistics, pp. 8424–8445. URL: <https://aclanthology.org/2022.acl-long.577>.
- Lepikhin, Dmitry et al. (2021). “GShard: Scaling Giant Models with Conditional Computation and Automatic Sharding”. In: *International Conference on Learning Representations*. URL: <https://openreview.net/forum?id=qrwe7XHTmYb>.
- Li, Houyi et al. (2025). *Predictable Scale: Part I, Step Law – Optimal Hyperparameter Scaling Law in Large Language Model Pretraining*. arXiv: 2503.04715 [cs.LG]. URL: <https://arxiv.org/abs/2503.04715>.
- Li, Jeffrey, Alex Fang, Georgios Smyrnis, et al. (2024). *DataComp-LM: In Search of the Next Generation of Training Sets for Language Models*. arXiv: 2406.11794 [cs.LG].
- Li, Yifei et al. (2024). “RadiK: Scalable and Optimized GPU-Parallel Radix Top-K Selection”. In: *Proceedings of the 38th ACM International Conference on Supercomputing*, pp. 537–548. DOI: 10.1145/3650200.3656596. URL: <https://doi.org/10.1145/3650200.3656596>.
- Li, Yunshui et al. (2025). *Model Merging in Pre-training of Large Language Models*. arXiv: 2505.12082 [cs.CL]. URL: <https://arxiv.org/abs/2505.12082>.
- Liang, Wanchao et al. (2024). *TorchTitan: One-Stop PyTorch Native Solution for Production-Ready LLM Pre-Training*. arXiv: 2410.06511 [cs.CL]. URL: <https://arxiv.org/abs/2410.06511>.
- Lightman, Hunter et al. (2023). *Let’s Verify Step by Step*. arXiv: 2305.20050 [cs.LG].
- Liu, Emmy, Graham Neubig, and Chenyan Xiong (2025). *Midtraining Bridges Pretraining and Posttraining Distributions*. arXiv: 2510.14865 [cs.CL]. URL: <https://arxiv.org/abs/2510.14865>.
- Liu, Jiawei et al. (2023). *Is Your Code Generated by ChatGPT Really Correct? Rigorous Evaluation of Large Language Models for Code Generation*. arXiv: 2305.01210 [cs.SE].

- Liu, Jingyuan et al. (2025). *Muon is Scalable for LLM Training*. arXiv: 2502.16982 [cs.LG]. URL: <https://arxiv.org/abs/2502.16982>.
- Liu, Qian et al. (2024). *RegMix: Data Mixture as Regression for Language Model Pre-training*. arXiv: 2407.01492 [cs.CL].
- Liu, Wei et al. (2024). “What Makes Good Data for Alignment? A Comprehensive Study of Automatic Data Selection in Instruction Tuning”. In: *International Conference on Learning Representations*.
- Liu, Yihong et al. (2026). *Crosslingual On-Policy Self-Distillation for Multilingual Reasoning*. arXiv: 2605.09548 [cs.CL]. URL: <https://arxiv.org/abs/2605.09548>.
- Liu, Zichen et al. (2025). *Understanding R1-Zero-Like Training: A Critical Perspective*. arXiv: 2503.20783 [cs.LG]. URL: <https://arxiv.org/abs/2503.20783>.
- LLM-Core Xiaomi (2026). *MiMo-V2-Flash Technical Report*. arXiv: 2601.02780 [cs.CL].
- LLM360 Team (2024). *TxT360: A Top-Quality LLM Pre-training Dataset Requires the Perfect Blend*. <https://huggingface.co/spaces/LLM360/TxT360>. Dataset card: <https://huggingface.co/datasets/LLM360/TxT360>.
- Loshchilov, Ilya and Frank Hutter (2019). “Decoupled Weight Decay Regularization”. In: *International Conference on Learning Representations*. URL: <https://openreview.net/forum?id=Bkg6RiCqY7>.
- Ludziejewski, Jan et al. (2024). “Scaling Laws for Fine-Grained Mixture of Experts”. In: *Proceedings of the 41st International Conference on Machine Learning*. Vol. 235. Proceedings of Machine Learning Research. PMLR, pp. 33270–33288. URL: <https://proceedings.mlr.press/v235/ludziejewski24a.html>.
- Ma, Wenhan et al. (2025). *Stabilizing MoE Reinforcement Learning by Aligning Training and Inference Routers*. arXiv: 2510.11370. URL: <https://arxiv.org/abs/2510.11370>.
- Magar, Inbal and Roy Schwartz (2022). “Data Contamination: From Memorization to Exploitation”. In: *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers)*. Association for Computational Linguistics, pp. 157–165. URL: <https://aclanthology.org/2022.acl-short.18>.
- Maiti, Shalini et al. (2025). *Souper-Model: How Simple Arithmetic Unlocks State-of-the-Art LLM Performance*. arXiv: 2511.13254 [cs.CL].
- Maskey, Sohir et al. (2026). *Good Pretraining, Bad SFT: Checkpoint Quality Across the Training Stack*. arXiv: 2609.08966 [cs.AI]. URL: <https://arxiv.org/abs/2609.08966>.
- Mathematical Association of America (2026). *American Invitational Mathematics Examination (AIME)*. URL: <https://www.maa.org/maa-invitational-competitions> (visited on 09/18/2026).
- McCandlish, Sam et al. (2018). *An Empirical Model of Large-Batch Training*. arXiv: 1812.06162 [cs.LG]. URL: <https://arxiv.org/abs/1812.06162>.
- Meituan LongCat Team (2025). *LongCat-Flash Technical Report*. arXiv: 2509.01322 [cs.CL]. URL: <https://arxiv.org/abs/2509.01322>.
- Merrill, Mike A. et al. (2026). *Terminal-Bench: Benchmarking Agents on Hard, Realistic Tasks in Command Line Interfaces*. arXiv: 2601.11868 [cs.SE]. URL: <https://arxiv.org/abs/2601.11868>.
- Meta PyTorch (2025a). *Monarch: A Distributed Programming Framework for PyTorch*. URL: <https://github.com/meta-pytorch/monarch>.
- (2025b). *torchforge: PyTorch-Native Post-Training at Scale*. URL: <https://github.com/meta-pytorch/torchforge>.

- Meta PyTorch (2025c). *torchstore: A Storage Solution for PyTorch Tensors with Distributed Tensor Support*. URL: <https://github.com/meta-pytorch/torchstore>.
- Micikevicius, Paulius et al. (2022). *FP8 Formats for Deep Learning*. arXiv: 2209.05433. URL: <https://arxiv.org/abs/2209.05433>.
- Miller, Evan (2024). *Adding Error Bars to Evals: A Statistical Approach to Language Model Evaluations*. arXiv: 2411.00640 [stat.AP].
- MiniMax (2025). *MiniMax-M1: Scaling Test-Time Compute Efficiently with Lightning Attention*. arXiv: 2506.13585. URL: <https://arxiv.org/abs/2506.13585>.
- Mistral AI (2026). *Introducing Mistral Small 4*. Mistral AI blog. URL: <https://mistral.ai/news/mistral-small-4/>.
- Muennighoff, Niklas, Alexander M. Rush, et al. (2023). “Scaling Data-Constrained Language Models”. In: *Advances in Neural Information Processing Systems*. Vol. 36. URL: <https://arxiv.org/abs/2305.16264>.
- Muennighoff, Niklas, Zitong Yang, et al. (2025). *s1: Simple Test-Time Scaling*. arXiv: 2501.19393 [cs.CL]. URL: <https://arxiv.org/abs/2501.19393>.
- Neitemeier, Pit et al. (2026). *Exact Quantile Balancing and Load-Error Injection for Mixture-of-Experts*. arXiv: 2609.28053 [cs.LG]. URL: <https://arxiv.org/abs/2609.28053>.
- NVIDIA (2025a). *Nemotron 3 Nano: Open, Efficient Mixture-of-Experts Hybrid Mamba-Transformer Model for Agentic Reasoning*. arXiv: 2512.20848 [cs.CL].
- (2025b). *Nemotron-Cascade: Scaling Cascaded Reinforcement Learning for General-Purpose Reasoning Models*. arXiv: 2512.13607 [cs.CL].
- (2025c). *Nemotron-Instruction-Following-Chat-v1*. URL: <https://huggingface.co/datasets/nvidia/Nemotron-Instruction-Following-Chat-v1> (visited on 09/29/2026).
- (2025d). *NVIDIA Nemotron Nano 2: An Accurate and Efficient Hybrid Mamba-Transformer Reasoning Model*. arXiv: 2508.14444 [cs.CL].
- (2026a). *Nemotron 3 Super: Open, Efficient Mixture-of-Experts Hybrid Mamba-Transformer Model for Agentic Reasoning*. arXiv: 2604.12374 [cs.CL].
- (2026b). *Nemotron-SFT-Agentic-v2*. URL: <https://huggingface.co/datasets/nvidia/Nemotron-SFT-Agentic-v2> (visited on 09/28/2026).
- (2026c). *Nemotron-SFT-Instruction-Following-Chat-v2*. URL: <https://huggingface.co/datasets/nvidia/Nemotron-SFT-Instruction-Following-Chat-v2> (visited on 09/29/2026).
- (2026d). *Nemotron-SFT-Science-v2*. URL: <https://huggingface.co/datasets/nvidia/Nemotron-SFT-Science-v2> (visited on 09/28/2026).
- OpenAI (2023). *tiktoken: A fast BPE tokeniser for use with OpenAI’s models*. <https://github.com/openai/tiktoken>. Pretokenization regex for the cl100k\_base encoding.
- (2024). *OpenAI o1 System Card*. arXiv: 2412.16720 [cs.AI]. URL: <https://arxiv.org/abs/2412.16720>.
- (2025). *gpt-oss-120b & gpt-oss-20b Model Card*. arXiv: 2508.10925 [cs.CL]. URL: <https://arxiv.org/abs/2508.10925>.
- OpenCode Team (2026). *OpenCode: The Open Source AI Coding Agent*. URL: <https://opencode.ai> (visited on 09/29/2026).

- Ouyang, Long et al. (2022). "Training Language Models to Follow Instructions with Human Feedback". In: *Advances in Neural Information Processing Systems*. Vol. 35, pp. 27730–27744. arXiv: [2203.02155](https://arxiv.org/abs/2203.02155). URL: <https://arxiv.org/abs/2203.02155>.
- Parmar, Jupinder et al. (2024). *Reuse, Don't Retrain: A Recipe for Continued Pretraining of Language Models*. arXiv: [2407.07263](https://arxiv.org/abs/2407.07263) [cs.CL]. URL: <https://arxiv.org/abs/2407.07263>.
- Patil, Shishir G. et al. (2025). "The Berkeley Function Calling Leaderboard (BFCL): From Tool Use to Agentic Evaluation of Large Language Models". In: *Proceedings of the 42nd International Conference on Machine Learning*.
- Penedo, Guilherme, Hynek Kydlíček, Loubna Ben Allal, et al. (2024). *The FineWeb Datasets: Decanting the Web for the Finest Text Data at Scale*. arXiv: [2406.17557](https://arxiv.org/abs/2406.17557) [cs.CL].
- Penedo, Guilherme, Hynek Kydlíček, Vinko Sabolčec, et al. (2025). *FineWeb2: One Pipeline to Scale Them All – Adapting Pre-Training Data Processing to Every Language*. arXiv: [2506.20920](https://arxiv.org/abs/2506.20920) [cs.CL].
- Penedo, Guilherme, Quentin Malartic, Daniel Hesslow, et al. (2023). *The RefinedWeb Dataset for Falcon LLM: Outperforming Curated Corpora with Web Data, and Web Data Only*. arXiv: [2306.01116](https://arxiv.org/abs/2306.01116) [cs.CL].
- Peng, Bowen et al. (2024). "Yarn: Efficient context window extension of large language models". In: *International Conference on Learning Representations*. Vol. 2024, pp. 31932–31951.
- Perplexity (2025). *Weight Transfer for RL Post-Training in under 2 Seconds*. URL: <https://www.perplexity.ai/hub/blog/weight-transfer-for-rl-post-training-in-under-2-seconds> (visited on 09/29/2026).
- Pham, Thinh et al. (2025). *SeaQA: Raising the Bar for Reasoning in Search-Augmented Language Models*. arXiv: [2506.01062](https://arxiv.org/abs/2506.01062) [cs.CL].
- Phan, Long et al. (2025). *Humanity's Last Exam*. arXiv: [2501.14249](https://arxiv.org/abs/2501.14249) [cs.LG].
- Poolside (2026). *Laguna M1/XS.2 Technical Report*. arXiv: [2605.27605](https://arxiv.org/abs/2605.27605) [cs.AI]. URL: <https://arxiv.org/abs/2605.27605>.
- Porian, Tomer et al. (2024). "Resolving Discrepancies in Compute-Optimal Scaling of Language Models". In: *Advances in Neural Information Processing Systems*. arXiv: [2406.19146](https://arxiv.org/abs/2406.19146) [cs.LG]. URL: <https://arxiv.org/abs/2406.19146>.
- Prime Intellect (2025). *PRIME-RL: Async RL Training at Scale*. URL: <https://github.com/PrimeIntellect-ai/prime-rl>.
- Project Apertus et al. (2025). *Apertus: Democratizing Open and Compliant LLMs for Global Language Environments*. arXiv: [2509.14233](https://arxiv.org/abs/2509.14233) [cs.CL]. URL: <https://arxiv.org/abs/2509.14233>.
- Pyatkin, Valentina et al. (2025). *Generalizing Verifiable Instruction Following*. arXiv: [2507.02833](https://arxiv.org/abs/2507.02833) [cs.CL].
- Qi, Penghui, Zichen Liu, et al. (2025). *Defeating the Training-Inference Mismatch via FP16*. arXiv: [2510.26788](https://arxiv.org/abs/2510.26788). URL: <https://arxiv.org/abs/2510.26788>.
- Qi, Penghui, Xiangxin Zhou, et al. (2026). *Rethinking the Trust Region in LLM Reinforcement Learning*. arXiv: [2602.04879](https://arxiv.org/abs/2602.04879) [cs.LG]. URL: <https://arxiv.org/abs/2602.04879>.
- Qwen Team (2025a). *Qwen3 Technical Report*. arXiv: [2505.09388](https://arxiv.org/abs/2505.09388) [cs.CL]. URL: <https://arxiv.org/abs/2505.09388>.
- (2025b). *Qwen3-Next-80B-A3B*. URL: <https://huggingface.co/Qwen/Qwen3-Next-80B-A3B-Instruct>.
- (2026a). *Qwen3.5: Towards Native Multimodal Agents*. URL: <https://qwen.ai/blog?id=qwen3.5> (visited on 09/25/2026).

- Qwen Team (2026b). Qwen3.6-35B-A3B: Agentic Coding Power, Now Open to All. Qwen blog. URL: <https://qwen.ai/blog?id=qwen3.6-35b-a3b>.
- (2026c). Qwen3.8-Max: A New Bar for Coding and Cowork. URL: <https://qwen.ai/blog?id=qwen3.8> (visited on 09/28/2026).
- Rae, Jack W., Sebastian Borgeaud, Trevor Cai, et al. (2021). *Scaling Language Models: Methods, Analysis & Insights from Training Gopher*. arXiv: [2112.11446](https://arxiv.org/abs/2112.11446) [cs.CL].
- Rajpurkar, Pranav et al. (2016). SQuAD: 100,000+ Questions for Machine Comprehension of Text. arXiv: [1606.05250](https://arxiv.org/abs/1606.05250) [cs.CL]. URL: <https://arxiv.org/abs/1606.05250>.
- Rein, David et al. (2023). GPQA: A Graduate-Level Google-Proof Q&A Benchmark. arXiv: [2311.12022](https://arxiv.org/abs/2311.12022) [cs.AI].
- Riquelme, Carlos et al. (2021). “Scaling Vision with Sparse Mixture of Experts”. In: *Advances in Neural Information Processing Systems*. Vol. 34, pp. 8583–8595.
- Rowell, Joe and Eiso Kant (2025). *The Hidden Engineering behind Foundation Model Building*. URL: <https://poolside.ai/blog/introducing-the-model-factory> (visited on 09/30/2026).
- Sachdeva, Noveen et al. (2026). “How to train data-efficient llms”. In: *International Conference on Learning Representations*. Vol. 2026, pp. 45491–45545.
- Sanyal, Sunny et al. (2023). *Early Weight Averaging Meets High Learning Rates for LLM Pre-training*. arXiv: [2306.03241](https://arxiv.org/abs/2306.03241) [cs.LG].
- Satterthwaite, F. E. (1946). “An Approximate Distribution of Estimates of Variance Components”. In: *Biometrics Bulletin* 2.6, pp. 110–114. DOI: [10.2307/3002019](https://doi.org/10.2307/3002019).
- Schulman, John et al. (2017). *Proximal Policy Optimization Algorithms*. arXiv: [1707.06347](https://arxiv.org/abs/1707.06347) [cs.LG]. URL: <https://arxiv.org/abs/1707.06347>.
- Sennrich, Rico, Barry Haddow, and Alexandra Birch (2016). “Neural Machine Translation of Rare Words with Subword Units”. In: *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 1715–1725. DOI: [10.18653/v1/P16-1162](https://doi.org/10.18653/v1/P16-1162). URL: <https://aclanthology.org/P16-1162/>.
- Shao, Zhihong et al. (2024). *DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models*. arXiv: [2402.03300](https://arxiv.org/abs/2402.03300) [cs.CL]. URL: <https://arxiv.org/abs/2402.03300>.
- Shazeer, Noam (2020). *GLU Variants Improve Transformer*. arXiv: [2002.05202](https://arxiv.org/abs/2002.05202) [cs.LG]. URL: <https://arxiv.org/abs/2002.05202>.
- Shazeer, Noam et al. (2017). “Outrageously Large Neural Networks: The Sparsely-Gated Mixture-of-Experts Layer”. In: *International Conference on Learning Representations*. URL: <https://openreview.net/forum?id=B1ckMDqlg>.
- Shi, Quan et al. (2026). *Tau-Knowledge: Evaluating Conversational Agents over Unstructured Knowledge*. arXiv: [2603.04370](https://arxiv.org/abs/2603.04370) [cs.CL].
- Singh, Aaditya K. et al. (2024). *Evaluation data contamination in LLMs: How do we measure it and (when) does it matter?* arXiv: [2411.03923](https://arxiv.org/abs/2411.03923) [cs.CL]. URL: <https://arxiv.org/abs/2411.03923>.
- Singh, Shivalika et al. (2025). “Global mmlu: Understanding and addressing cultural and linguistic biases in multilingual evaluation”. In: *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 18761–18799.
- Singh, Varun et al. (2026). *Arcee Trinity Large Technical Report*. arXiv: [2602.17004](https://arxiv.org/abs/2602.17004) [cs.LG]. URL: <https://arxiv.org/abs/2602.17004>.

- Slinko, Igor et al. (2026). *Step Rejection Fine-Tuning: A Practical Distillation Recipe*. arXiv: 2605.10674 [cs.LG]. URL: <https://arxiv.org/abs/2605.10674>.
- Snell, Charlie et al. (2025). “Scaling LLM Test-Time Compute Optimally Can be More Effective than Scaling Model Parameters”. In: *International Conference on Learning Representations*. URL: <https://arxiv.org/abs/2408.03314>.
- Spärck Jones, Karen (1972). “A Statistical Interpretation of Term Specificity and its Application in Retrieval”. In: *Journal of Documentation* 28.1, pp. 11–21. DOI: 10.1108/eb026526.
- Su, Dan et al. (2025). “Nemotron-CC: Transforming Common Crawl into a Refined Long-Horizon Pretraining Dataset”. In: *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Association for Computational Linguistics. URL: <https://arxiv.org/abs/2412.02595>.
- Su, Jianlin et al. (2024). “Roformer: Enhanced transformer with rotary position embedding”. In: *Neurocomputing* 568, p. 127063.
- Su, Junhao et al. (2026). “Failure Makes the Agent Stronger: Enhancing Accuracy through Structured Reflection for Reliable Tool Interactions”. In: *Findings of the Association for Computational Linguistics: ACL 2026*, pp. 12712–12734. URL: <https://aclanthology.org/2026.findings-acl.618>.
- Tazi, Nouamane et al. (2025). *The Ultra-Scale Playbook: Training LLMs on GPU Clusters*. URL: <https://huggingface.co/spaces/nanotron/ultrascale-playbook> (visited on 10/02/2026).
- Team, Kimi et al. (2025). *Kimi k1.5: Scaling Reinforcement Learning with LLMs*. arXiv: 2501.12599 [cs.AI]. URL: <https://arxiv.org/abs/2501.12599>.
- The Microsoft AI Team (2026). *MAI-Thinking-1: Building a Hill-Climbing Machine*. Tech. rep. Microsoft AI. URL: <https://microsoft.ai/pdf/mai-thinking-1.pdf> (visited on 09/30/2026).
- Tian, Changxin, Kunlong Chen, et al. (2025). *Towards Greater Leverage: Scaling Laws for Efficient Mixture-of-Experts Language Models*. arXiv: 2507.17702 [cs.LG]. URL: <https://arxiv.org/abs/2507.17702>.
- Tian, Changxin, Jiapeng Wang, et al. (2025). *WSM: Decay-Free Learning Rate Schedule via Checkpoint Merging for LLM Pre-training*. arXiv: 2507.17634 [cs.LG]. URL: <https://arxiv.org/abs/2507.17634>.
- Tissue, Howe, Venus Wang, and Lu Wang (2025). “Scaling Law with Learning Rate Annealing”. In: *Advances in Neural Information Processing Systems*. arXiv: 2408.11029. URL: <https://arxiv.org/abs/2408.11029>.
- Trivedi, Harsh et al. (2022). “MuSiQue: Multihop Questions via Single-hop Question Composition”. In: *Transactions of the Association for Computational Linguistics* 10, pp. 539–554.
- Vankadara, Leena Chennuru et al. (2026). *How to Scale Mixture-of-Experts: From muP to the Maximally Scale-Stable Parameterization*. arXiv: 2605.14200 [cs.LG]. URL: <https://arxiv.org/abs/2605.14200>.
- Vaswani, Ashish et al. (2017). “Attention Is All You Need”. In: *Advances in Neural Information Processing Systems*. Vol. 30, pp. 5998–6008.
- Vendrow, Joshua et al. (2025). *Do Large Language Model Benchmarks Test Reliability?* arXiv: 2502.03461 [cs.LG].

- Wäldchen, Stephan et al. (2024). “Interpretability Guarantees with Merlin-Arthur Classifiers”. In: *Proceedings of the 27th International Conference on Artificial Intelligence and Statistics*. Vol. 238. PMLR, pp. 1963–1971.
- Wang, Jiapeng et al. (2026). *MergeMix: Optimizing Mid-Training Data Mixtures via Learnable Model Merging*. arXiv: [2601.17858](https://arxiv.org/abs/2601.17858) [cs.CL].
- Wang, Xingjin et al. (2025). *Learning Dynamics in Continual Pre-Training for Large Language Models*. arXiv: [2505.07796](https://arxiv.org/abs/2505.07796) [cs.CL]. URL: <https://arxiv.org/abs/2505.07796>.
- Wang, Yubo et al. (2024). “MMLU-Pro: A More Robust and Challenging Multi-Task Language Understanding Benchmark”. In: *Advances in Neural Information Processing Systems*. Vol. 37, pp. 95266–95290. URL: <https://openreview.net/forum?id=y1ODM6R2r3>.
- Wang, Zheng et al. (2025). *WLB-LLM: Workload-Balanced 4D Parallelism for Large Language Model Training*. arXiv: [2503.17924](https://arxiv.org/abs/2503.17924) [cs.DC]. URL: <https://arxiv.org/abs/2503.17924>.
- Wang, Zhizhi and Harini Suresh (2026). *Which Institutional Frameworks Do Chatbots Assume? Auditing Jurisdictional Defaults in Multilingual LLMs*. arXiv: [2606.00333](https://arxiv.org/abs/2606.00333) [cs.CL]. URL: <https://arxiv.org/abs/2606.00333>.
- Wei, Jason, Zhiqing Sun, et al. (2025). *BrowseComp: A Simple Yet Challenging Benchmark for Browsing Agents*. arXiv: [2504.12516](https://arxiv.org/abs/2504.12516) [cs.CL].
- Wei, Jason, Xuezhi Wang, et al. (2022). “Chain-of-Thought Prompting Elicits Reasoning in Large Language Models”. In: *Advances in Neural Information Processing Systems*. URL: <https://arxiv.org/abs/2201.11903>.
- Welch, B. L. (1947). “The Generalization of ‘Student’s’ Problem when Several Different Population Variances Are Involved”. In: *Biometrika* 34.1–2, pp. 28–35. DOI: [10.1093/biomet/34.1-2.28](https://doi.org/10.1093/biomet/34.1-2.28).
- Wenzek, Guillaume et al. (2020). “CCNet: Extracting High Quality Monolingual Datasets from Web Crawl Data”. In: *Proceedings of the Twelfth Language Resources and Evaluation Conference*. European Language Resources Association, pp. 4003–4012. URL: <https://aclanthology.org/2020.lrec-1.494>.
- Wettig, Alexander et al. (2024). “QuRating: Selecting High-Quality Data for Training Language Models”. In: *Proceedings of the 41st International Conference on Machine Learning*. Vol. 235. Proceedings of Machine Learning Research. PMLR, pp. 52915–52971. URL: <https://proceedings.mlr.press/v235/wettig24a.html>.
- Willison, Simon (2024). *pelican-bicycle: LLM Benchmark: Generate an SVG of a Pelican Riding a Bicycle*. URL: <https://github.com/simonw/pelican-bicycle>.
- Wortsman, Mitchell, Gabriel Ilharco, et al. (2022). *Model Soups: Averaging Weights of Multiple Fine-Tuned Models Improves Accuracy without Increasing Inference Time*. arXiv: [2203.05482](https://arxiv.org/abs/2203.05482) [cs.LG].
- Wortsman, Mitchell, Peter J. Liu, et al. (2024). “Small-Scale Proxies for Large-Scale Transformer Training Instabilities”. In: *International Conference on Learning Representations*. arXiv: [2309.14322](https://arxiv.org/abs/2309.14322). URL: <https://arxiv.org/abs/2309.14322>.
- Xu, Zhangchen et al. (2025). *TOUCAN: Synthesizing 1.5M Tool-Agentive Data from Real-World MCP Environments*. arXiv: [2510.01179](https://arxiv.org/abs/2510.01179) [cs.CL]. URL: <https://arxiv.org/abs/2510.01179>.
- Xuan, Weihao et al. (2025). *MMLU-ProX: A Multilingual Benchmark for Advanced Large Language Model Evaluation*. arXiv: [2503.10497](https://arxiv.org/abs/2503.10497) [cs.CL].

- Yang, Bowen et al. (2025). *Rope to Nope and Back Again: A New Hybrid Attention Strategy*. arXiv: 2501.18795 [cs.CL]. URL: <https://arxiv.org/abs/2501.18795>.
- Yang, Greg, Edward J. Hu, et al. (2021). “Tensor Programs V: Tuning Large Neural Networks via Zero-Shot Hyperparameter Transfer”. In: *Advances in Neural Information Processing Systems*. Vol. 34, pp. 17084–17097. URL: <https://arxiv.org/abs/2203.03466>.
- Yang, Greg, James B. Simon, and Jeremy Bernstein (2024). *A Spectral Condition for Feature Learning*. arXiv: 2310.17813 [cs.LG]. URL: <https://arxiv.org/abs/2310.17813>.
- Yang, Greg, Dingli Yu, et al. (2023). *Tensor Programs VI: Feature Learning in Infinite-Depth Neural Networks*. arXiv: 2310.02244 [cs.NE]. URL: <https://arxiv.org/abs/2310.02244>.
- Yang, Shuo et al. (2023). *Rethinking benchmark and contamination for language models with rephrased samples*. arXiv: 2311.04850 [cs.CL]. URL: <https://arxiv.org/abs/2311.04850>.
- Yang, Zhilin et al. (2018). “HotpotQA: A Dataset for Diverse, Explainable Multi-hop Question Answering”. In: *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, pp. 2369–2380. DOI: 10.18653/v1/D18-1259. URL: <https://aclanthology.org/D18-1259/>.
- Yao, Feng et al. (2025). *Your Efficient RL Framework Secretly Brings You Off-Policy RL Training*. URL: <https://fengyao.notion.site/off-policy-rl>.
- Yao, Shunyu et al. (2023). “ReAct: Synergizing Reasoning and Acting in Language Models”. In: *The Eleventh International Conference on Learning Representations*. URL: <https://arxiv.org/abs/2210.03629>.
- Yen, Howard et al. (2025). “HELMET: How to evaluate long-context models effectively and thoroughly”. In: *International Conference on Learning Representations*. Vol. 2025, pp. 98914–98965.
- Yu, Qiyang et al. (2025). *DAPO: An Open-Source LLM Reinforcement Learning System at Scale*. arXiv: 2503.14476 [cs.LG]. URL: <https://arxiv.org/abs/2503.14476>.
- Z.ai (2026a). *GLM-4.7-Flash*. URL: <https://huggingface.co/zai-org/GLM-4.7-Flash> (visited on 10/02/2026).
- (2026b). *GLM-5.2*. URL: <https://z.ai/blog/glm-5.2> (visited on 09/30/2026).
- (2026c). *GLM-5.3*. URL: <https://z.ai/blog/glm-5.3> (visited on 09/30/2026).
- Zadouri, Ted et al. (2026). “FlashAttention-4: Algorithm and Kernel Pipelining Co-Design for Asymmetric Hardware Scaling”. In: *Proceedings of Machine Learning and Systems*. Vol. 8, pp. 912–926.
- Zellers, Rowan et al. (2019). “HellaSwag: Can a Machine Really Finish Your Sentence?” In: *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*. Association for Computational Linguistics, pp. 4791–4800. DOI: 10.18653/v1/P19-1472. URL: <https://aclanthology.org/P19-1472/>.
- Zhang, Biao and Rico Sennrich (2019). “Root Mean Square Layer Normalization”. In: *Advances in Neural Information Processing Systems*. Vol. 32.
- Zhang, Hongyi, Yann N. Dauphin, and Tengyu Ma (2019). “Fixup Initialization: Residual Learning Without Normalization”. In: *International Conference on Learning Representations*. arXiv: 1901.09321. URL: <https://arxiv.org/abs/1901.09321>.
- Zhang, Jiajie et al. (2025). *AdaptThink: Reasoning Models Can Learn When to Think*. arXiv: 2505.13417 [cs.CL]. URL: <https://arxiv.org/abs/2505.13417>.

- Zhao, Chenggang et al. (2025). *DeepEP: An Efficient Expert-Parallel Communication Library*. URL: <https://github.com/deepseek-ai/DeepEP>.
- Zhao, Siyan et al. (2026). *Self-Distilled Reasoner: On-Policy Self-Distillation for Large Language Models*. arXiv: 2601.18734 [cs.LG]. URL: <https://arxiv.org/abs/2601.18734>.
- Zhao, Yanli et al. (2023). *PyTorch FSDP: Experiences on Scaling Fully Sharded Data Parallel*. arXiv: 2304.11277 [cs.DC]. URL: <https://arxiv.org/abs/2304.11277>.
- Zheng, Lianmin, Wei-Lin Chiang, Ying Sheng, et al. (2023). “Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena”. In: *Advances in Neural Information Processing Systems, Datasets and Benchmarks Track*.
- Zhou, Jeffrey et al. (2023). *Instruction-Following Evaluation for Large Language Models*. arXiv: 2311.07911 [cs.CL].

## Part 6

# Appendix

## A Quality-Cost Analysis

Benchmark scores alone do not capture the computational cost of serving a model. Figure 1 compares aggregate model quality scores with estimated decode text throughput per GPU. We use high-concurrency decode throughput as a proxy for efficiency in workloads where decoding dominates, e.g. during RL.

**Quality Measurement** We directly use the *Overall* rows of Tables 17 and 28.

**Throughput Measurement** We benchmark models with vLLM on a single node with eight NVIDIA B200 GPUs, using synthetic prompts. Models use FP8 weights and an FP8 KV cache, except GPT-OSS, which retains its native MXFP4 weight format. Each baseline is served from its publisher’s FP8 release where one exists and quantised to FP8 at load otherwise. The FP8 cache setting covers only the attention KV cache. The recurrent state of the hybrid models keeps the precision their publishers set: float32 for the Mamba-2 state of Nemotron 3 and the Gated DeltaNet state of Qwen3.5, Qwen3.6 and Qwen3.8, as their model configurations declare and as NVIDIA recommends for accuracy<sup>6</sup>, and bfloat16 for Qwen3-Next, whose configuration sets none. Because this state is not quantised, an FP8 KV cache frees less memory on these models than on attention-only ones<sup>7</sup>.

For each model and context length, we evaluate every eligible combination of data, tensor and, for MoE models, expert parallelism in our eight-GPU search space. We measure each layout at a concurrency near its KV-cache limit and report the highest decode throughput. This gives each model its best measured layout under the same hardware budget.

For Figure 50, all models of a training stage are charged the throughput at one shared context length, close to the average context their evaluations decode at. A sample’s context grows from its prompt to its prompt plus completion while it decodes, so we take the prompt plus half the completion, average it over a model’s benchmarks weighted by their decoded tokens, and then over the models in the figure. This gives about 1000 tokens for the pre-training evaluations, none of which averages more than 1700, and about 16 000 tokens for the post-training evaluations, with individual models between 9700 and 27 000. For

---

<sup>6</sup><https://huggingface.co/nvidia/NVIDIA-Nemotron-3-Super-120B-A12B-FP8>

<sup>7</sup><https://github.com/vllm-project/vllm/issues/55196>

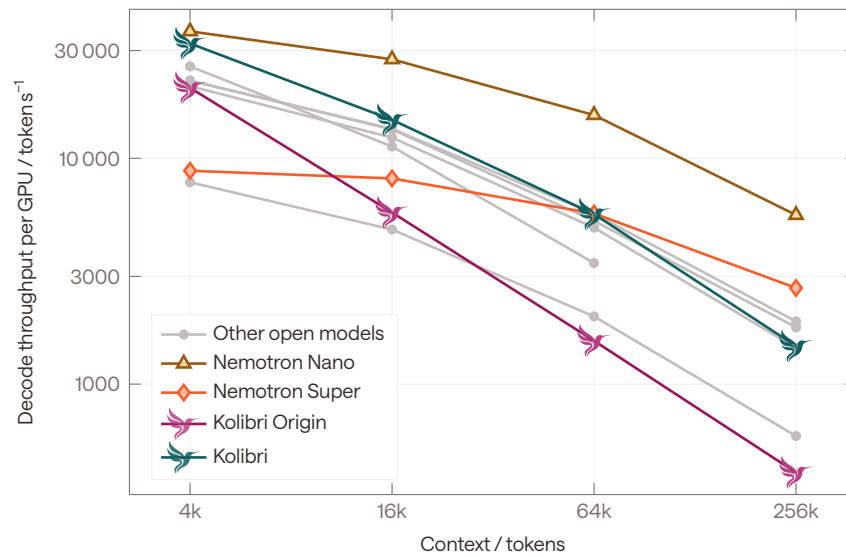


Figure 50. Decode throughput per GPU of the fastest layout at each context length.

base models we report the throughput at 4k tokens, as we deem 1k tokens unrepresentative of common workloads, and at 16k tokens for post-trained models.

We express throughput as estimated UTF-8 bytes per second per GPU by multiplying measured decode tokens per second by each model's bytes per token. We calculate separate English and German factors by applying each model's tokeniser to the same plain-text prompts from the pre-training evaluation suite, then use the factor matching each plot's language. Because these factors come from prompts rather than generated text, they are an approximation, particularly for post-trained reasoning outputs.

Benchmark scores of the baselines come from their BF16 checkpoints, apart from the FP8 release of Command A+ and GPT-OSS in MXFP4, while their throughput is measured with FP8 weights and an FP8 KV cache. The figure therefore assumes that FP8 serving preserves their scores.

| Tokeniser                             | Vocabulary | English       |           |           | German        |           |           |
|---------------------------------------|------------|---------------|-----------|-----------|---------------|-----------|-----------|
|                                       |            | Precision / % | Order / % | Fertility | Precision / % | Order / % | Fertility |
| Native vocabulary size                |            |               |           |           |               |           |           |
| Kolibri                               | 128 000    | 50.0          | 82.5      | 1.65      | 65.2          | 79.1      | 1.87      |
| BPE                                   | 128 000    | 36.4          | 72.6      | 1.63      | 60.2          | 73.6      | 1.89      |
| GPT-5                                 | 200 019    | 40.1          | 74.8      | 1.76      | 47.3          | 71.5      | 2.56      |
| Tekken                                | 131 072    | 38.6          | 74.0      | 1.92      | 46.2          | 71.9      | 2.68      |
| Qwen3-Next                            | 151 669    | 37.9          | 75.3      | 1.83      | 38.3          | 69.1      | 3.20      |
| Qwen3.5–3.8                           | 248 077    | 38.4          | 75.3      | 1.80      | 44.0          | 70.2      | 2.60      |
| Gemini                                | 262 144    | 42.2          | 74.2      | 1.54      | 46.2          | 69.4      | 2.59      |
| DeepSeek V4                           | 129 280    | 37.6          | 73.0      | 1.78      | 41.0          | 68.3      | 3.02      |
| Kimi K3                               | 163 586    | 37.2          | 74.4      | 1.83      | 37.2          | 67.2      | 3.41      |
| GLM 5.3                               | 154 856    | 38.1          | 75.3      | 1.82      | 40.8          | 69.6      | 2.93      |
| EuroLLM                               | 128 000    | 36.7          | 72.7      | 2.09      | 48.7          | 71.9      | 2.56      |
| Truncated to the first 128 000 tokens |            |               |           |           |               |           |           |
| GPT-5                                 | 128 021    | 38.8          | 74.7      | 1.92      | 44.5          | 71.2      | 2.78      |
| Tekken                                | 128 000    | 38.7          | 74.0      | 1.93      | 46.0          | 71.8      | 2.70      |
| Qwen3-Next                            | 128 026    | 37.9          | 75.3      | 1.83      | 38.0          | 69.1      | 3.23      |
| Qwen3.5–3.8                           | 128 033    | 37.9          | 75.3      | 1.83      | 38.0          | 69.1      | 3.23      |
| Gemini                                | 128 000    | 38.7          | 74.0      | 1.87      | 41.2          | 68.7      | 3.02      |
| Kimi K3                               | 128 002    | 36.4          | 74.4      | 1.93      | 36.6          | 67.1      | 3.54      |
| GLM 5.3                               | 128 036    | 37.9          | 75.3      | 1.83      | 38.0          | 69.1      | 3.23      |

Table 30. Morphological alignment of the Kolibri tokeniser and ten other tokenisers on 1000 English and 1000 German words, each at its own vocabulary size and truncated to its first 128 000 tokens. *Precision* is the share of word-internal token boundaries that fall on a morpheme boundary. *Order* denotes the probability that a tokeniser merges a position inside a morpheme before a morpheme boundary of the same word (50 % is chance). *Fertility* denotes the number of tokens per word. The highest precision and order of each column are set in bold. BPE is plain Byte Pair Encoding trained on the same corpus with the same vocabulary size as Kolibri. *Tekken* is Mistral’s tokeniser, also used by Apertus and Nemotron, *Qwen3.5–3.8* is the tokeniser shared by Qwen3.5, Qwen3.6 and Qwen3.8, and *Gemini* the tokeniser shared by Gemma 4 and Gemini (Gemma Team 2026). Every difference to Kolibri in precision and order is significant (99 % paired bootstrap intervals over words).

## B Model Architecture

### B.1 Tokeniser

#### B.1.1 Morphology

We evaluate morphological alignment on two sets of 1000 words, in English and German, drawn at regular intervals from frequency-ranked word lists, excluding proper nouns and abbreviations. For each word we annotate the positions where a cut between morphemes is appropriate, allowing alternative segmentations where several are defensible; a language model proposed the segmentations, and a person has reviewed the first 100 words of each language without changing any. We tokenise each word as it appears after a space in running text: English words in lowercase and capitalised form, German words in their majority casing.

We then compare Kolibri’s tokenisation against BPE trained on the same corpus and other tokenisers. Table 30 lists precision and order as two morphological alignment measures and the fertility (number of tokens per word) for every tokeniser at its own vocabulary size and with every vocabulary truncated to

128 000 tokens. In German, 65 % of Kolibri's word-internal token boundaries fall on morpheme boundaries, against 49 % for the best of 9 other tokenisers and 60 % for BPE. UniBPE also increases English morphological alignment, with Kolibri achieving 50 % against 42 % for the best other tokeniser and 36 % for BPE. We also study the order of the merges: a perfectly morphological tokeniser would join every position inside a morpheme before joining two morphemes together. We measure the probability that, picking at random one gold morpheme boundary and one other position inside the same word, the tokeniser merges the other position first. Here, 50 % indicates random behaviour while higher values indicate better morphological alignment. Kolibri scores 83 % in English and 79 % in German, against 73 % and 74 % for BPE, and at most 75 % and 72 % for the other tokenisers.

Truncating the larger vocabularies to 128 000 tokens changes the merge order by at most 1.1 points, so the order is insensitive to vocabulary size. In contrast, the boundary precision drops by up to 6 points for the largest vocabularies (248 077 to 262 144 tokens), whose late tokens cover whole words. At an equal size of 128 000 tokens, Kolibri's lead in English increases, with 50 % against 39 % for the best other tokeniser, and it stays ahead of all other tokenisers on both measures.

The merge order appears to be insensitive to vocabulary size as truncating every tokeniser to 128 000 tokens changes it by only about one point. In contrast, the boundary precision is sensitive to vocabulary size changes: it drops by up to 6 points for the largest vocabularies (248 077 to 262 144 tokens), whose late tokens cover whole words. At an equal size of 128 000 tokens, Kolibri's lead in English widens, with 50 % against 39 % for the best other tokeniser, and it stays ahead of all other tokenisers on both measures. All differences in precision and order between Kolibri and the other tokenisers are significant (99 % paired bootstrap intervals over words).

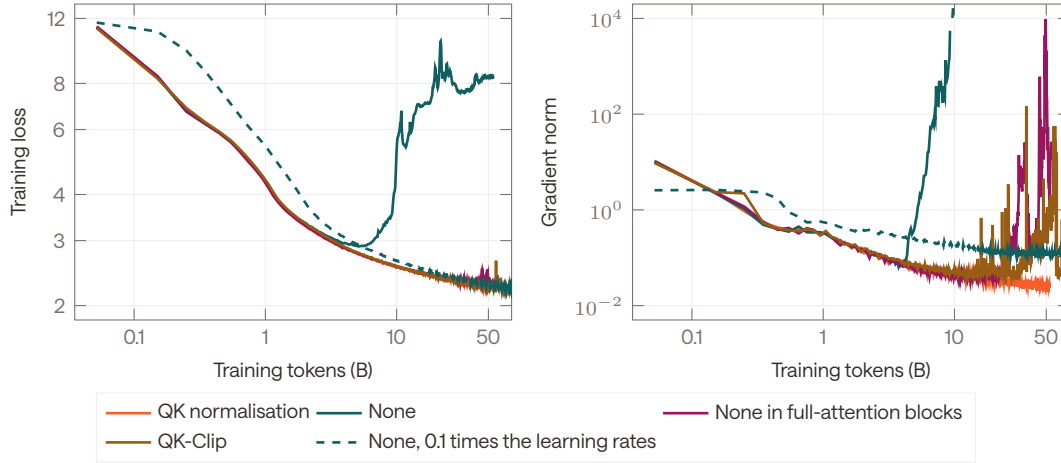


Figure 51. Training loss and gradient norm at scale M with and without QK normalisation. Both are averaged over bins of 0.1B tokens, over the first 53B pre-training tokens. Each configuration is a single run from scratch at the baseline learning rates of M unless stated otherwise.

## B.2 QK Normalisation

Queries and keys are normalised per head with RMS normalisation (Henry et al. 2020; B. Zhang and Sennrich 2019) before RoPE:

$$\hat{q}_i = \gamma_q \odot \frac{q_i}{\text{RMS}(q_i)}, \quad \hat{k}_j = \gamma_k \odot \frac{k_j}{\text{RMS}(k_j)}, \quad \text{RMS}(x) = \sqrt{\frac{1}{d_h} \|x\|_2^2 + \varepsilon}, \quad (19)$$

with learned gains  $\gamma_q, \gamma_k \in \mathbb{R}^{d_h}$ . The normalisation bounds the attention logits  $\hat{q}_i^\top \hat{k}_j / \sqrt{d_h}$  by the gains, so they cannot grow with the norms of the query and key projections. Wortsman, P. J. Liu, et al. (2024) show that the growth of these logits destabilises training at high learning rates already in small models, and that QK normalisation prevents it. We verify this at scale M (Table 3), where all runs share data, recipe and model apart from the attention logits (Figure 51). Without QK normalisation, the model at the baseline learning rates of M diverges early in training and does not recover. Removing it from the full-attention blocks only, the model trains but its gradient norm spikes by several orders of magnitude in the second half of the run. An adapted QK-Clip (Kimi Team 2025) for GQA with threshold 100 in place of QK normalisation also shows gradient-norm spikes. Without QK normalisation, the model trains stably at 0.1 times these learning rates, and then reaches a higher loss than the model with QK normalisation throughout the run. With QK normalisation, training stays stable across the learning rates and batch sizes of our sweeps (Section 2.2.1).

## B.3 Model-Shape Evaluation Suite

For the initial model-shape sweeps, accuracy mean averages 17 metrics: ARC, HellaSwag, PIQA, HumanEval, MBPP and MATHMinerva in English and German, English MMLU, MMLU-Pro, GSM8K and TriviaQA and German GlobalMMLU. We use normalised Log-Likelihood accuracy for multiple-choice tasks, completion accuracy for GSM8K and TriviaQA, code completion accuracy including errors for HumanEval and MBPP and exact-flex accuracy for MATHMinerva. BPB mean averages the four HumanEval and MBPP bits-per-byte metrics in English and German.

## B.4 Sparsity Sweep

The training loss and accuracy of the sparsity sweep of Section 2.1.4 at 3.4B and 0.9B active parameters are shown in Figure 52. The number of long requests that each configuration at 3.4B active parameters fits on common accelerator setups is listed in Table 31.

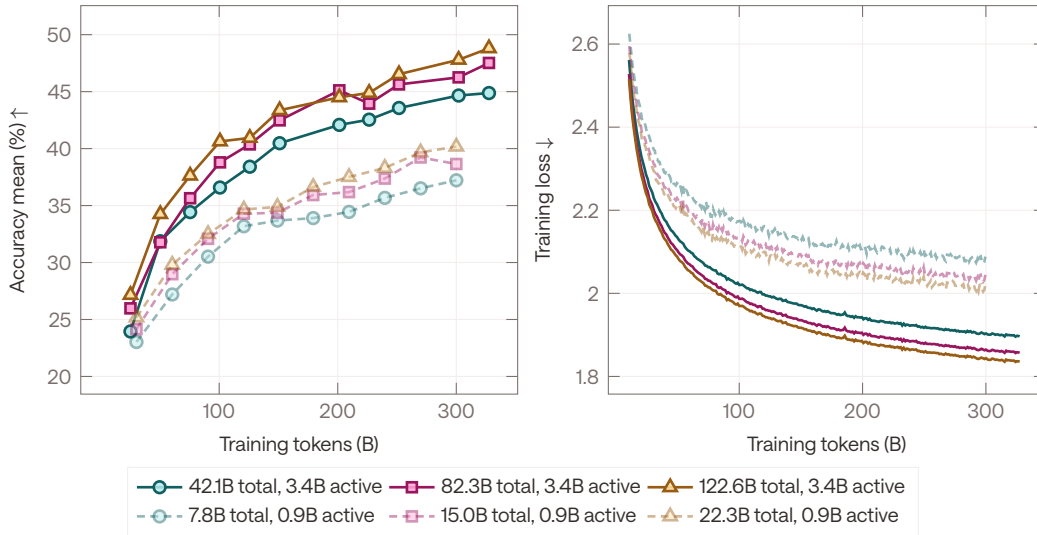


Figure 52. Aggregate benchmark accuracy and training loss against training budget for both sparsity sweeps. Solid curves are the configurations with 3.4B active parameters, dashed and faded curves those with 0.9B; colour encodes the expert count, so each pair shares a hue. The points at 0.9B active parameters use the scaling-law learning rate  $\eta_0$ . Both scales average the same 17 accuracy benchmarks.

| Total parameters | 2×H100 | 4×H100    | 1×B200    | 2×B200     | 2×B300     |
|------------------|--------|-----------|-----------|------------|------------|
| 42.1B            | 33     | <b>82</b> | <b>46</b> | <b>107</b> | <b>171</b> |
| 82.3B            | 18     | 67        | 31        | 93         | 157        |
| 122.6B           | 3      | 52        | 16        | 78         | 142        |

Table 31. Estimated maximum number of concurrent 256k-token requests in FP8, with evenly sharded weights and KV cache, a 10% fragmentation margin and 6 GB of activation workspace per GPU (H100: 80 GB, B200: 192 GB, B300: 288 GB).

## B.5 Model Shape at 80B Total Parameters

With the total size fixed by the sparsity sweep of Section 2.1.4, two ablations set the shape of the Kolibri model: the granularity of its experts (Appendix B.5.1) and the width of its residual stream (Appendix B.5.2).

### B.5.1 Expert Granularity

We compare three expert configurations of about 80B total parameters (Table 32). The Kolibri configuration has width 2560 and routes each token to 6 of 384 experts of hidden dimension 512. The finer-grained configuration follows the expert count of Qwen3-Next (Qwen Team 2025b), with width 2048 and 12 of 512 experts of the same hidden dimension. The coarser-grained configuration is the 82.3B configuration of the sparsity sweep, with width 2048 and 6 of 256 experts of hidden dimension 1024. All three train for 327.6B tokens and are compared at their last checkpoint.

The Kolibri configuration reaches the lowest loss, the highest accuracy and the lowest BPB. The finer-grained configuration comes close in loss and accuracy, and the coarser-grained configuration falls behind both.

| Configuration           | Width | Heads | Experts | Top- $k$ | Expert hidden | Loss ↓       | Acc. / % ↑  | BPB ↓        |
|-------------------------|-------|-------|---------|----------|---------------|--------------|-------------|--------------|
| Kolibri                 | 2560  | 48    | 384     | 6        | 512           | <b>1.847</b> | <b>38.0</b> | <b>0.486</b> |
| Finer-grained experts   | 2048  | 32    | 512     | 12       | 512           | 1.848        | 37.6        | 0.495        |
| Coarser-grained experts | 2048  | 32    | 256     | 6        | 1024          | 1.860        | 35.6        | 0.497        |

Table 32. Expert configurations of about 80B total parameters after 327.6B tokens. The loss is the training loss averaged over the last 2B tokens. Accuracy and BPB are the aggregates of the last checkpoint; the best value of each column is bold.

### B.5.2 Residual Width

We compare residual streams of different widths at 0.9B active parameters and approximately matched training FLOPs, with the matrix shapes of the attention and expert layers adapted to the width. All configurations use a sequence length of 16k tokens. The result is not separately validated at 3.4B active parameters because changing  $d_{\text{model}}$  would require a new matched-FLOP sweep that retunes the attention and expert widths.

The residual width  $d_{\text{model}}$  determines the dimensionality of the shared representation read and updated by both attention and MoE blocks. Increasing  $d_{\text{model}}$  raises the parameter and compute cost of the associated projections, residual-activation memory and communication volume. Under approximately matched training FLOPs, a wider residual stream therefore comes with fewer attention heads of the same head dimension and with narrower experts (Table 33).

We compare residual widths of 640, 1024 and 1600 at a fixed depth of 50 layers. Each configuration is trained for 150B tokens using learning rates of  $\eta_0 / \sqrt{2}$ ,  $\eta_0$  and  $\sqrt{2}\eta_0$ , where  $\eta_0$  is predicted by the learning-rate scaling law of Kolibri Origin. For each width, we select the learning rate with the highest mean evaluation accuracy and report all metrics from the corresponding run.

| $d_{\text{model}}$ | Heads | Expert hidden | Dense FFN hidden | Learning rate | Loss ↓       | Accuracy (%) ↑ |
|--------------------|-------|---------------|------------------|---------------|--------------|----------------|
| 640                | 20    | 480           | 3840             | $\sqrt{2}$    | 2.190        | 31.2           |
| 1024               | 16    | 384           | 3072             | $\sqrt{2}$    | 2.132        | 34.6           |
| 1600               | 12    | 288           | 2304             | $1/\sqrt{2}$  | <b>2.109</b> | <b>34.9</b>    |

Table 33. Matrix shapes and results of the three residual widths at matched training FLOPs, each at the learning rate with the highest mean accuracy, given as a multiple of the scaling-law learning rate  $\eta_0$ . The head dimension, the KV heads and the 50 layers are the same for all three. Loss is the training loss averaged over the last 2B tokens; accuracy is the mean of the accuracy metrics of the model-shape suite (Appendix B.3) at the last checkpoint, and the best value of each column is bold.

After learning-rate selection, training loss and mean accuracy improve with residual width (Table 33). Most of the accuracy improvement occurs between widths 640 and 1024, with a smaller gain from 1024 to 1600. Within the evaluated compute budget, these results favour a wider residual stream with fewer attention heads and narrower experts.

## B.6 Leading Dense Layers and Routing in the First Layers

Kolibri Origin uses dense FFNs in its first two blocks, a common choice against load-balancing instabilities in the first layers (Dai et al. 2024; DeepSeek-AI 2024b). Our ablation of the Kolibri architecture suggested that EQB makes them unnecessary: over 327.6B tokens, the model without leading dense layers reaches a slightly lower loss and BPB and a higher accuracy at a similar load balance (Table 5). Kolibri therefore uses MoE layers in all 50 blocks. The diagnostics of the full Kolibri run below call this outcome into question: its first two MoE layers make almost no use of their routed experts.

**Routing in the First Layers** Balance metrics can look healthy while the bias overrides the router’s own preferences, which leaves the routing effectively random. We therefore track, for every MoE layer, counted from zero, two balance metrics and three metrics of the bias’s effect. With the expert load distribution  $F_e(\mathcal{B}) = c_e(\mathcal{B})/(K|\mathcal{B}|)$ , the normalised scores  $P_{t,e} = S_{t,e}/\sum_j S_{t,j}$  and the entropy  $\mathcal{H}(v) = -\sum_{e=1}^E v_e \log v_e$ , the balance metrics are the effective dead-expert fraction and the normalised router entropy (Krishnamurthy, Watkins, and Gaertner 2023):

$$\text{effective dead-expert fraction} \quad D(\mathcal{B}) = 1 - \frac{1}{E} \exp(\mathcal{H}(F(\mathcal{B}))), \quad (20)$$

$$\text{normalised router entropy} \quad H(\mathcal{B}) = \frac{1}{|\mathcal{B}|} \sum_{t \in \mathcal{B}} \mathcal{H}(P_{t,:}), \quad \hat{H}(\mathcal{B}) = \frac{H(\mathcal{B})}{\log E}. \quad (21)$$

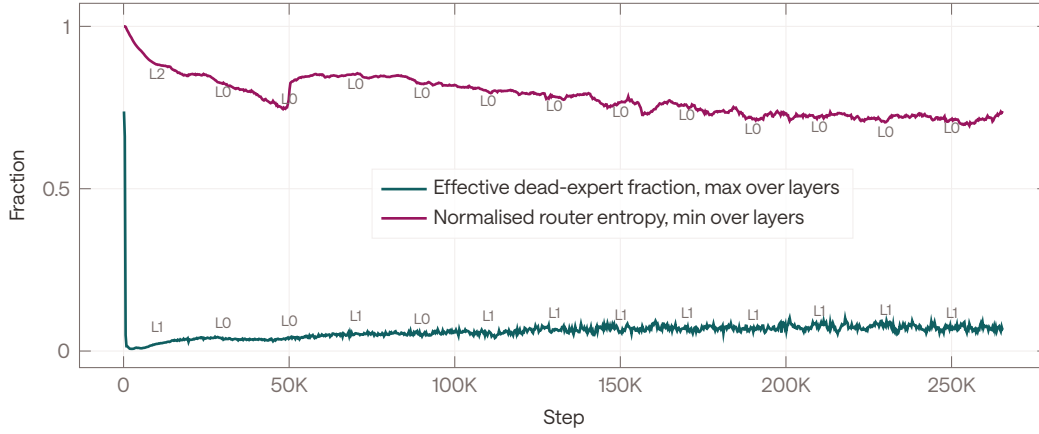


Figure 53. The effective dead-expert fraction  $D$  of Equation (20) and normalised router entropy  $\hat{H}$  of Equation (21) of Kolibri pre-training. The effective dead-expert fraction is the maximum over the 50 layers, the normalised router entropy the minimum. The tags name the layer holding the extremum at that point.

With  $\mathcal{I}_t^u = \text{Top-K}(Z_{t,:}, K)$  the set the router selects without the bias, and  $\mathcal{I}_t$  of Equation (3) the set actually routed to, the metrics of the bias’s effect are

$$\text{top-}K \text{ intervention rate} \quad I(\mathcal{B}) = \frac{1}{|\mathcal{B}|} \sum_{t \in \mathcal{B}} \left( 1 - \frac{|\mathcal{I}_t^u \cap \mathcal{I}_t|}{K} \right), \quad (22)$$

$$\text{intrinsic top-}K \text{ mass} \quad M^u(\mathcal{B}) = \frac{1}{|\mathcal{B}|} \sum_{t \in \mathcal{B}} \sum_{e \in \mathcal{I}_t^u} P_{t,e}, \quad (23)$$

$$\text{selected top-}K \text{ mass} \quad M^{\text{sel}}(\mathcal{B}) = \frac{1}{|\mathcal{B}|} \sum_{t \in \mathcal{B}} \sum_{e \in \mathcal{I}_t} P_{t,e}. \quad (24)$$

In the Kolibri run, the first two layers look healthy on the balance metrics (Figure 53): their effective dead-expert fraction is the highest of all layers but moderate, and their router entropy is the lowest, a strong routing preference. The bias, however, overrides that preference almost entirely (Figure 54). Their top- $K$  intervention rate reaches  $98.5\% \approx 1 - K/E$  towards the end of training, and while their intrinsic top- $K$  mass is the highest of all layers, their selected top- $K$  mass is the lowest, more than an order of magnitude smaller and close to the uniform value  $K/E$ .

To test whether their routed experts matter, we perturb the routing of one MoE layer at a time at the step-225 000 checkpoint and measure the loss on 65 536 held-out tokens, leaving the other 49 layers unchanged, similar to Riquelme et al. (2021) and Jin et al. (2026). *Random routed experts* replaces each token’s selected experts with a uniform random draw and tests whether the routing decision matters; *silenced routed experts* removes the routed output, keeping the shared expert, and tests whether the routed experts contribute at all. Only for layers 0 and 1 are both perturbations free, with loss changes indistinguishable from zero (Figure 55), so the model does not rely on their routed experts. Layer 48 also has a high intervention rate, but perturbing it raises the loss clearly, so a high intervention rate alone does not mean that the routed experts are unused.

We tried fixing this issue during SFT, where load balancing is disabled, through densifying layers 0 and 1 by concatenating the shared expert and the six routed experts with the lowest biases into a dense FFN

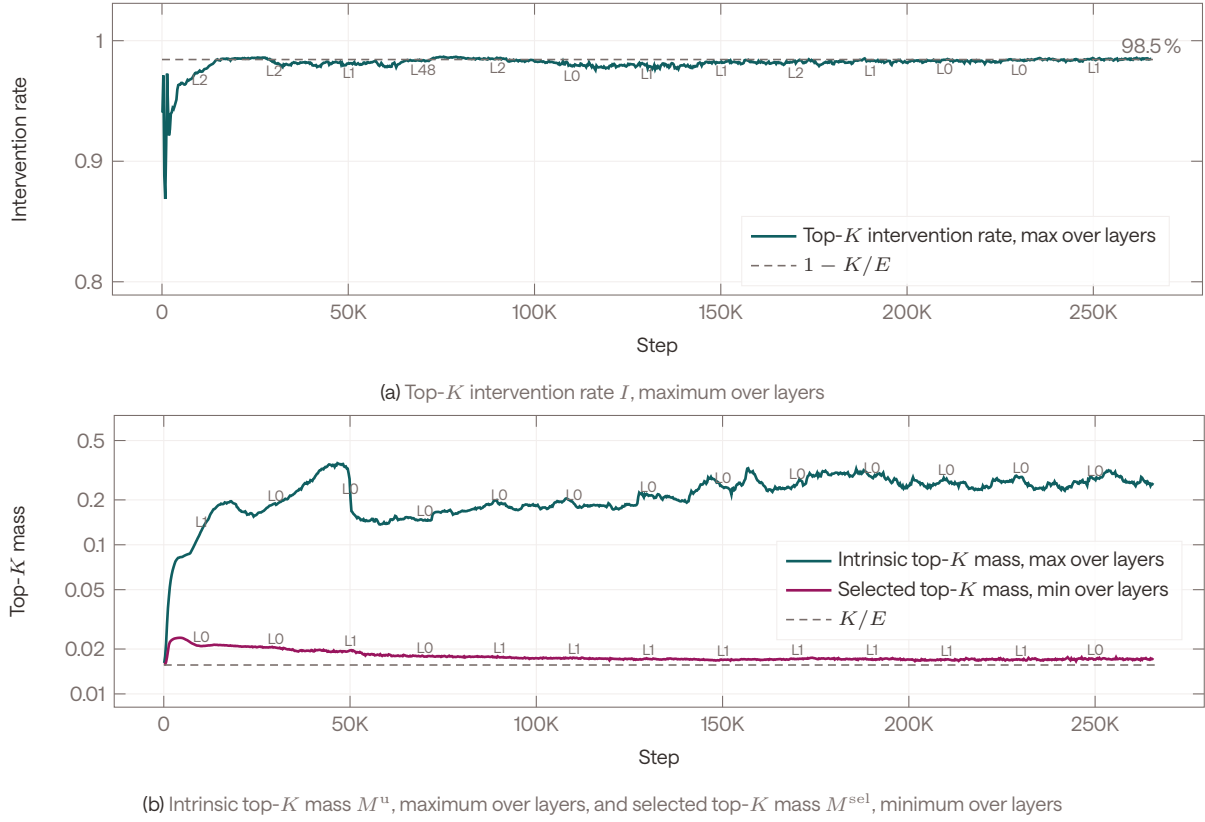


Figure 54. The routing diagnostics of Equations (22) to (24) of Kolibri, each at the layer where it is most extreme. The tags name the layer holding the extremum at that point and the dashed lines mark a uniform choice: an intervention rate of  $1 - K/E$  in (a) and a mass of  $K/E$ .

of hidden width 3584, keeping their pre-trained weights. This gave similar training loss and no clear downstream improvement, so Kolibri keeps these layers as MoE layers. The 327.6B-token ablation did not reveal this behaviour as it only emerged after longer training durations. Resolving this issue remains future work.

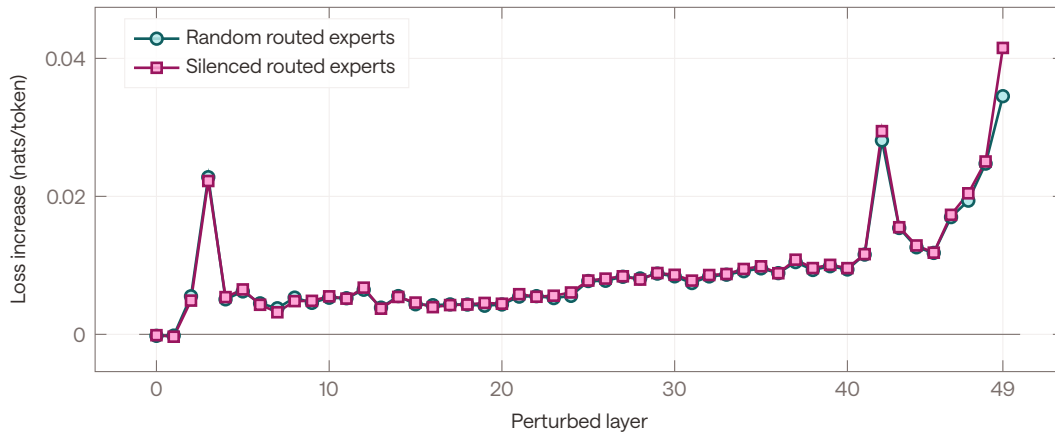


Figure 55. Loss increase from perturbing the routing of one MoE layer of Kolibri at step 225 000, all other layers untouched, on 65 536 held-out tokens against a baseline of 1.850 nats per token. Random routed experts sends each token to uniformly drawn experts; silenced routed experts removes the routed path's output and keeps the shared expert.

## B.7 Pre-Training Infrastructure

To efficiently support large sharding degrees at our model scale, we make use of well-established optimisations. As a general reference, see for example (Tazi et al. 2025). For PyTorch (Ansel et al. 2024) FSDP2 modules, these are namely:

**set\_separate\_reduce\_scatter\_group()** Dedicated NCCL process group for reduce-scatters, so they overlap with all-gathers instead of serialising behind them.

**set\_requires\_all\_reduce()** and **set\_is\_last\_backward()** Microbatch-aware collectives that defer the inter-replica all-reduce and FSDP2's end-of-backward finalisation to the last microbatch.

**set\_reshard\_after\_backward()** Keep weights unsharded after the backward in-between the two gradient accumulation steps, removes the need to all-gather weights starting with the second forward step. We force-reshard before the optimiser.

**set\_reduce\_scatter\_max\_input\_buffers()** Three reduce-scatter input buffers (about 800 MB each at this scale), to hide infrequent, non-deterministic communication latency that would otherwise be exposed to the critical path.

As expert parallel backend we use DeepEP v2 with the ElasticBuffer API (C. Zhao et al. 2025). We wrap `buffer.dispatch()` and `buffer.combine()` into PyTorch custom ops, so they seamlessly integrate into a fully PyTorch compiled transformer layer. To reduce device memory consumption, we avoid checkpointing any activation inside the MoE feed forward network. Every token is routed to  $K$  experts, so the size of each intermediate activation is scaled up by a factor of  $K$ ; at the same time the arithmetic intensity of a grouped matrix-multiply operation that underlies the MoE feed forward network is lower than that of a dense layer. This makes it tractable to recompute. To further reduce the memory consumption, we re-use the same ElasticBuffer across all layers and recompute also the dispatch in the backward pass.

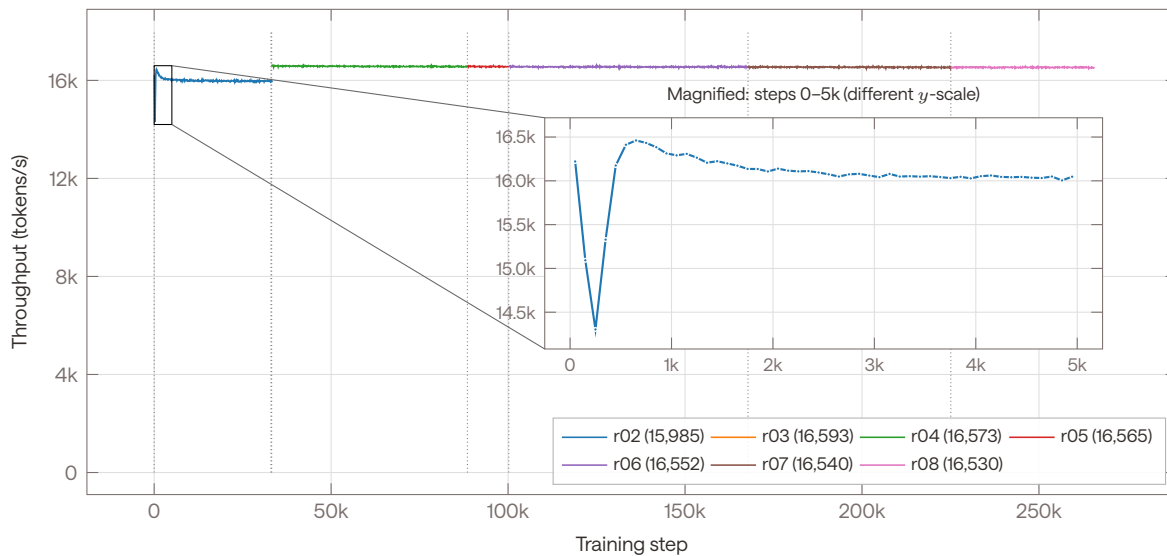


Figure 56. Training throughput of the Kolibri pre-training runs broken down into its constituent runs r02–r08. The plot shows the median throughput in tokens per second and GPU over 100-step wide bins. We do this to smooth out dips caused by our profiling, logging and checkpointing; the plot is to show that there are no expert-parallel related degradations over time. Dotted vertical lines mark restart boundaries. The legend values are per-run medians. The inset magnifies the first 5000 steps. The  $\sim 3.7\%$  increase from r02 to r03 comes from decreasing the logging and profiling frequency.

To balance the workload across all expert parallel ranks, we rely on our MoE load balancing methodology as described in Section 2.1.5. The per-GPU throughput over time in tokens per second is visualised in Figure 56. As we can see, the throughput is very stable after an initial dip. The dip reflects the time until the routing first stabilises, such that the load balancing can be effective.

## B.7.1 Straggler Detection

Our straggler detection approach is three-fold: (1) We do passive health monitoring via NVIDIA Data Center GPU Manager (DCGM). (2) We actively measure the compute time of each transformer block and compare them online. (3) Most intrusively, and thus offline, we take PyTorch profiles on all GPUs and match collective communication kernels across the profiles to figure out when each GPU joined each collective and what throughput it observed. We use three distinct approaches because they can be used online vs. offline and detect different kinds of stragglers or straggler reasons, like compute vs. communication.

To obtain online runtime measurements, we follow the approach of (Ziheng Jiang et al. 2024) and time each transformer block during a forward pass with CUDA events. We inject these events through the `register_forward_pre_hook` and `register_forward_hook` methods of `torch.nn.Module`. To avoid fully draining the GPU work queue when evaluating the CUDA events, we use double-buffering. This means that each training step evaluates the CUDA events of its preceding step. We explicitly gather the runtimes on GPU/process 0 to get more detailed statistics than just the average runtime.

For detecting complicated compute or communication straggler reasons, we use a profiling-based approach. Our main training run writes a PyTorch profile once every 15 min, and we match the collective communication kernels between the profiles of all ranks compare their arrival time and observed through-

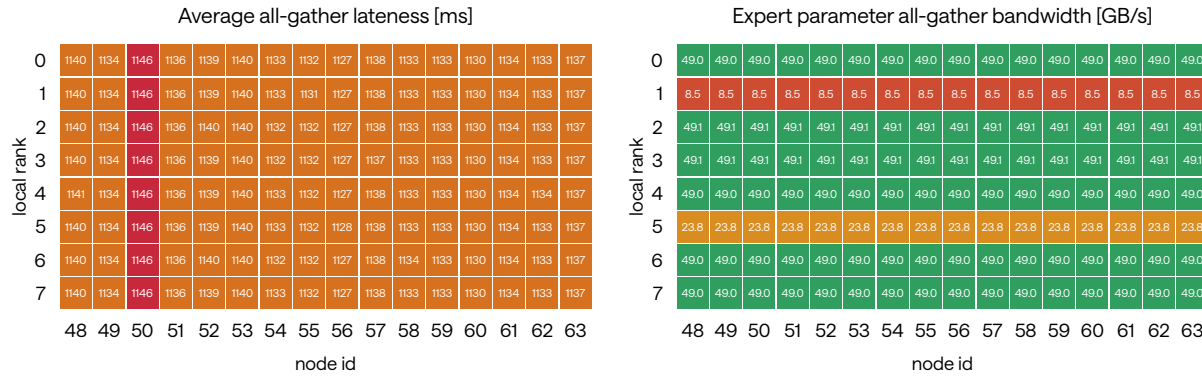


Figure 57. Average all-gather (over all process groups) latency per node and local rank (left) and expert parameter all-gather bandwidth (right) for nodes 48–63 (DP/model replica 3). The intersection is local ranks 1 and 5 on node 50. These visualisations stem from a profile of a timestep, where the PCIe links of NICs 1 and 5 on node 50 were downtrained, i.e., the GPU-to-NIC communication speed was degraded.

put. Take, for example, one FSDP weight all-gather. If the wall clock duration of such an all-gather is significantly faster on one rank than on others, it likely arrived later to the collective than all other ranks. The matching of these collectives across the profiles will tell us if this is the case. One possible reason for a late arrival could be that the process is a compute straggler; the profile will also show us this reason. If all ranks communicate at a rate considerably slower than the maximum throughput, a communication straggler might be present.

We identify communication stragglers by comparing the collective communication arrival times and throughput across all parallelism dimensions separately. Our main training run ran with EP 8 FSDP 16 DP 6. This gives us *four* different communication groups for each rank. Let’s take rank 0 as an example. It communicates with the following partners: (1) with ranks 1-7 expert parallel (all-to-all), (2) with ranks 8, 16, 24, ..., 120 during expert parameter all-gathers/reduce-scatters, (3) with ranks 1-127 during dense parameter all-gathers/reduce-scatters, and (4) with all ranks during data parallel all-reduces. If the InfiniBand speed of rank 0 was reduced, we would notice this in groups (2)–(4). An example of such an analysis is visualised in Figure 57. With this methodology, we were able to pinpoint every communication straggler that occurred during the main pre-training run.

## B.7.2 Fault Tolerance

We observed 38 unplanned interruptions during the net 377 000 GPU hours pre-training run. This yields a rate of 0.1 failures per 1000 GPU hours. Table 34 lists the reasons of the observed failures. The single most frequent hardware failure reported by DCGM was “GPU has fallen off the bus” (Xid 79). Xids are the identifiers of the errors that the NVIDIA driver reports to the kernel, and Xid 79 “GPU has fallen off the bus” indicates that the driver attempts to but cannot access the GPU over PCIe.<sup>8</sup>

Our fault-tolerance setup combines frequent checkpointing with automatic recovery. This setup is the most compatible with our current cluster orchestration, which requires reboots after hardware failures.

<sup>8</sup>C.f. Xid Error specification document: [https://docs.nvidia.com/deploy/pdf/XID\\_Errors.pdf](https://docs.nvidia.com/deploy/pdf/XID_Errors.pdf)

| Reason                          | Number of occurrences | Details                                                    |
|---------------------------------|-----------------------|------------------------------------------------------------|
| GPU / node hardware fault       | 10                    | Xid 79, 94, 145, 154                                       |
| Object-storage read stall (S3)  | 9                     | Timeout (cURL error code 28) during <code>GetObject</code> |
| Bad link                        | 2                     |                                                            |
| Checkpoint-save timeout         | 1                     | DCP checkpoint write to CephFS timed out                   |
| Not investigated / inconclusive | 16                    |                                                            |
| Total                           | 38                    |                                                            |

Table 34. All failure that occurred during the main pre-training run classified by root cause. The most observed one was a GPU or node hardware fault (for example NVLink dropped or NIC problems). Our system classified these automatically, with verification by humans. Some failures could not be attributed, and we thus list them as “inconclusive”.

| Step      | Explanation                                                                     | Average Time | Total time |
|-----------|---------------------------------------------------------------------------------|--------------|------------|
| Downtime  | Hot-swap of faulty node, automatic job restart, Kubernetes scheduling           | 29.4 min     | 18.6 h     |
| Startup   | Distributed launch, rendezvous, <code>torch.compile</code> , checkpoint loading | 7.6 min      | 4.8 h      |
| Recompute | Recomputation of training steps between last checkpoint and failure             | 8.2 min      | 5.2 h      |
| Total     |                                                                                 | 45.3 min     | 28.7 h     |

Table 35. Time for automatic recovery of the 38 failures. After the first couple of failures, we increased checkpoint frequency to lower the “downtime” reported in the table. The average nonetheless includes all samples and is therefore higher than the time we observed during most of the run.

We checkpoint every 250 training steps. For failures that are uncorrelated to the checkpointing this means that we need to recompute on average 125 steps. With a step time of about 6 s that makes 12.5 min. As shown in Table 35, the observed recovery time happens to be lower than expected.

Our cluster orchestration automatically detects faulty nodes and swaps them out of the main training run for automatic reboot and/or manual inspection. The Model Factory (c.f. Part 4) classifies failures as application or system level errors and retries them within their separate budgets. The majority of the time is spent in a grace period for the remaining nodes to checkpoint their state. Together, this took on average 29.4 min as shown in Table 35. This average includes every failure. Initially we swapped out the nodes by hand resulting in downtimes of 30 min to 60 min. After that, we put the automatic cluster recovery in place and the average downtime decreased drastically. We still include these numbers in the calculation of the average of 29.4 min.

The third component in our fault handling is the application startup. This is relatively fixed at 7.6 min and includes launching the application, rendezvous of all GPUs, `torch.compile` as well as checkpoint loading.

## C Optimisation

### C.1 Training Batch Construction

Training batches should represent the dataset distribution. Stochastic Gradient Descent relies on each microbatch gradient being an unbiased, low-variance estimate of the gradient over the whole training

distribution. That estimate is unbiased only if the examples in a batch are drawn independently from that distribution (Bottou, Curtis, and Nocedal 2018). Insufficient shuffling violates this (Bottou 2012), and so do long documents that spread over consecutive batches. Both have two consequences: (1) within a batch, correlated examples shrink the effective batch size, and (2) across batches, the mixture the model sees drifts over time, so the model is chasing a moving target and fits disproportionately what it saw most recently. A third problem is cutting documents: a document longer than the sequence length has to be cut into pieces, and every cut separates tokens from context they could otherwise attend to. The following subsections address these problems.

### C.1.1 Data Loading Pipeline

The training mix is assembled offline according to the mix of each stage, which Olmix (M. F. Chen et al. 2026) selects for pre-training (Section 2.3) and MergeMix (J. Wang et al. 2026) for mid-training (Section 2.3.3). Our materialisation job, described below in more detail, writes the resulting mix into parquet files with row groups of 4096 rows and emits a row index alongside the data. An additional index file fixes the order in which documents are visited, at the granularity of row groups. Dataloader workers load entire row groups and iterate over all documents in each loaded row group sequentially. This approach keeps the amount of S3 GET requests low and avoids overloading the storage system. Since the order is materialised once, every rank reads the same permutation, and a run can be reproduced or resumed from the index alone. At training time we build on the standard PyTorch dataloader with several worker processes per rank. The document stream is split so that each rank sees a disjoint slice of the index and each worker of a rank a disjoint slice of that, and every worker then runs the same per-worker pipeline:

1. It reads its row groups of raw text.
2. unrolls them into individual documents.
3. tokenises each document online and appends the end-of-sequence token.
4. splits documents longer than the sequence length into chunks with the RSDB described below.
5. bin packs chunks into sequences of exactly the sequence length.
6. converts each packed sequence into input, label and position tensors.

Positions restart at zero at every document boundary within a packed sequence, which is how the attention backends recover the document mask. The dataloader collates the sequences of one worker into a microbatch, and its full state, from the position in the index down to the buffer contents of every worker, is checkpointed so that a resumed run continues the exact same data order.

### C.1.2 Chunking and Packing

The common way to fill sequences of a fixed length  $L$  concatenates the tokenised documents, separated by end-of-sequence tokens, and splits the stream at every multiple of  $L$  (H. Ding et al. 2024). This concat-and-chunk packing has two problems. First, it cuts documents that would fit into one sequence: almost every sequence boundary falls inside a document, so a document shorter than  $L$  is split whenever it

straddles a boundary, and the tokens after the cut lose the context before it. H. Ding et al. (2024) report better downstream scores when these cuts are avoided. Second, a document many times longer than  $L$  fills several consecutive sequences on its own. A document of 200 000 tokens at a sequence length of 16k occupies 13 consecutive sequences. With sequence-order packing, those land in a handful of adjacent microbatches, so for a few optimiser steps the gradient is dominated by one document, one domain and one writing style. Across the whole run every token is still seen once, but the per-step batches no longer follow the dataset distribution. Shuffling documents before chunking does not address this: the pieces of one document still come out back to back. The RSDB addresses the second problem and the bin packer the first.

**Random Sequential Document Buffer** We break this locality during data loading with an RSDB, following V. Singh et al. (2026). Each dataloader worker keeps a buffer of 4096 tokenised documents together with a read offset per document. To produce the next piece, the worker draws a buffered document uniformly at random, emits the next contiguous chunk of at most one sequence length starting at that document's offset, and advances the offset. A document leaves the buffer when its last chunk has been emitted, and the slot is refilled from the input stream. A document of  $k$  chunks is therefore drawn  $k$  times over the lifetime of the buffer. Its chunks are separated in expectation by roughly one buffer's worth of chunks from other documents. Within a document the chunks keep their original order; only their spacing in the training stream changes. V. Singh et al. (2026) report a smoother training loss with the RSDB than with sequential chunking, and we observe the same.

**Bin Packing** The chunks then go into the bin packer, which fills each sequence of exactly the sequence length from a pool of 1024 pending chunks. The packer always anchors a sequence on the oldest chunk in its pool, which bounds how long any chunk can wait, and fills the remaining capacity with a subset-sum search over the rest. If no subset fills the sequence exactly, the packer takes the smallest subset that overfills it and drops the tokens beyond the sequence length. At sequence length  $L$ , a document of  $n$  tokens needs  $\lceil n/L \rceil - 1$  cuts. The RSDB makes exactly these cuts, and the packer adds a cut only when no subset fills a sequence exactly. Since the packer can only reorder chunks within its pool of 1024, while the RSDB separates the chunks of one document by several thousand chunks from other documents, pieces that the RSDB emitted far apart stay far apart, and a long document's chunks end up spread over many microbatches rather than concentrated in a few. Positions restart at zero at every chunk, so the model sees the pieces of a long document as independent contexts, but the ordering no longer concentrates them in time.

| Step                    | Drop when                                                                                  |
|-------------------------|--------------------------------------------------------------------------------------------|
| Non-null check          | the extracted text is missing or empty                                                     |
| Character count         | fewer than 150 characters                                                                  |
| Word count              | fewer than 50 or more than 100 000 words                                                   |
| Symbol ratio            | more than 0.1 symbols per word, where a symbol is a hash sign or an ellipsis               |
| Bullet lines            | more than 90 % of lines start with a bullet character, an asterisk or a hyphen             |
| Ellipsis lines          | more than 30 % of non-empty lines end with an ellipsis or “read more”                      |
| Uppercase lines         | more than 50 % of non-empty lines have more than half of their characters in uppercase     |
| Block list              | the host is a blocked domain, the URL is a blocked page, or the URL contains a banned word |
| Language identification | the fastText top label over the first 5000 bytes is neither English nor German             |

Table 36. Language-agnostic heuristic filter steps, in pipeline order. The word-count, symbol-ratio, bullet-line and ellipsis-line steps are Gopher document filters (Rae, Borgeaud, Cai, et al. 2021) at their default thresholds.

## D Data and Training Stages

### D.1 Common Crawl Internal Pipeline

This appendix describes the stages of the CC pipeline of Section 2.3.2.1 in more detail, and Figure 12 shows the documents each stage received and removed.

#### D.1.1 Heuristic Filters

The heuristic filters are based on and extend those of Dolma 3 (Ettinger et al. 2025), which derive from DCLM (J. Li, Fang, Smyrnis, et al. 2024) and RefinedWeb (Penedo, Malartic, Hesslow, et al. 2023). Every extracted document passes the language-agnostic steps of Table 36, and the language identification routes it to an English or a German branch, which applies the steps of Table 37. Figure 58 shows the yield of each step.

A cleanup pass runs on every dataset of the pre-training, mid-training, and long-context pools. It drops empty and degenerate documents, collapses runs of newlines in a random share of the documents to at most three in code and math sources and two elsewhere, and caps horizontal whitespace at 16 characters within a line and 256 characters of indentation.

#### D.1.2 Exact Deduplication

Exact deduplication removes every document whose UTF-8 text, without normalisation, has the same 128-bit xxHash<sup>9</sup> as another document, first within each snapshot and then across all snapshots in chronological order, so the earliest copy remains (Lee et al. 2022). Each kept document records three counts, its copies within its snapshot, its copies in the corpus, and the number of snapshots that contain a copy. The copy count correlates with document quality (Fang et al. 2025), and TxT360 (LLM360 Team 2024) and FineWeb2 (Penedo, Kydlíček, Sabolčec, et al. 2025) upsample by it. The counts thus leave the decision how often to

<sup>9</sup><https://github.com/Cyan4973/xxHash>

| Step                      | Filters / Modifiers                                                                | English                                    | German                                                                             |
|---------------------------|------------------------------------------------------------------------------------|--------------------------------------------|------------------------------------------------------------------------------------|
| Language score            | fastText confidence score below                                                    | 0.65                                       | 0.821                                                                              |
| Mean word length          | mean word length in bytes outside                                                  | 3 to 10                                    | 0 to 15                                                                            |
| Non-alphabetic word ratio | share of words without an alphabetic character above                               | 0.20                                       | 0.226                                                                              |
| Stop words                | fewer than two occurrences of a word from                                          | 8 Gopher words                             | 15 FineWeb2 words                                                                  |
| Boilerplate               | more than 40 % of paragraphs contain a phrase from                                 | 13 English phrases                         | 16 German and the 13 English                                                       |
| Repetition                | duplicate line fraction above                                                      | 0.30                                       | 0.263                                                                              |
|                           | duplicate paragraph fraction above                                                 | 0.30                                       | 0.30                                                                               |
|                           | characters in duplicate lines or paragraphs above                                  | 0.20                                       | 0.20                                                                               |
|                           | most frequent 2-, 3-, 4-gram fraction above                                        | 0.20, 0.18, 0.16                           | 0.169, 0.153, 0.137                                                                |
|                           | repeated 5- to 10-gram fraction above                                              | 0.15 to 0.10, step 0.01                    | 0.141, 0.134, 0.126, 0.117, 0.108, 0.098                                           |
| Line modifiers            | digit-only lines, counter lines such as “3 likes”, lines with fewer than two words | shared                                     |                                                                                    |
| Substring line modifier   | lines of at most ten words that contain one of these substrings                    | “items in cart”, “Read more...”, “Sign-in” | “Artikel im Warenkorb”, “Mehr lesen...”, “Weiterlesen...”, “Anmelden”, “Einloggen” |
| Word removal ratio        | share of words removed by the line modifiers above                                 | 0.05                                       | 0.05                                                                               |
| MADLAD-400 rules 2 and 5  | fewer than five sentences, or at least 20 % questionable sentences                 | applied                                    | not applied                                                                        |

Table 37. Heuristic filter steps of the English and German branches, in pipeline order, with the threshold of each branch. English thresholds follow Gopher, German thresholds and the German stop-word list follow the FineWeb2 deu\_Latn configuration. The paragraph and character thresholds are the Gopher defaults in both branches. The English branch restricts the MADLAD-400 filter (Kudugunta, Caswell, B. Zhang, et al. 2023) to rules 2 and 5, following the ablations of Dolma 3.

repeat a document to the mixing stage, without rerunning deduplication. The global pass stores only the hash table with the counts and the location of each document’s text. A new snapshot therefore merges into the table at a cost proportional to the table and the snapshot, independent of the number of earlier snapshots. We apply no URL deduplication, since one URL can hold distinct documents across crawls.

### D.1.3 Fuzzy Deduplication

Fuzzy deduplication uses MinHash (Broder 1997) with locality-sensitive hashing. Documents are normalised as in RefinedWeb and CCNet (Wenzek et al. 2020), with every digit replaced by a placeholder, and shingled into token 5-grams. The signatures have 26 bands of 11 rows, which pair two documents with probability one half at a Jaccard similarity of 0.72 and 99 % at 0.85. This budget is smaller than those of RefinedWeb and DCLM, since every cluster of candidate pairs is verified on the Jaccard similarity of token 3-grams at a threshold of 0.80. The verification is exhaustive for clusters below 500 documents and uses a second MinHash pass with 200 bands of 31 rows for larger ones. Each kept document records the size of its cluster. The stage runs per language in 30 English and 6 German shards, to which the snapshots are dealt round-robin in chronological order, so near-duplicates in different shards remain. It removes 24.9 % of the English and 33.3 % of the German documents.

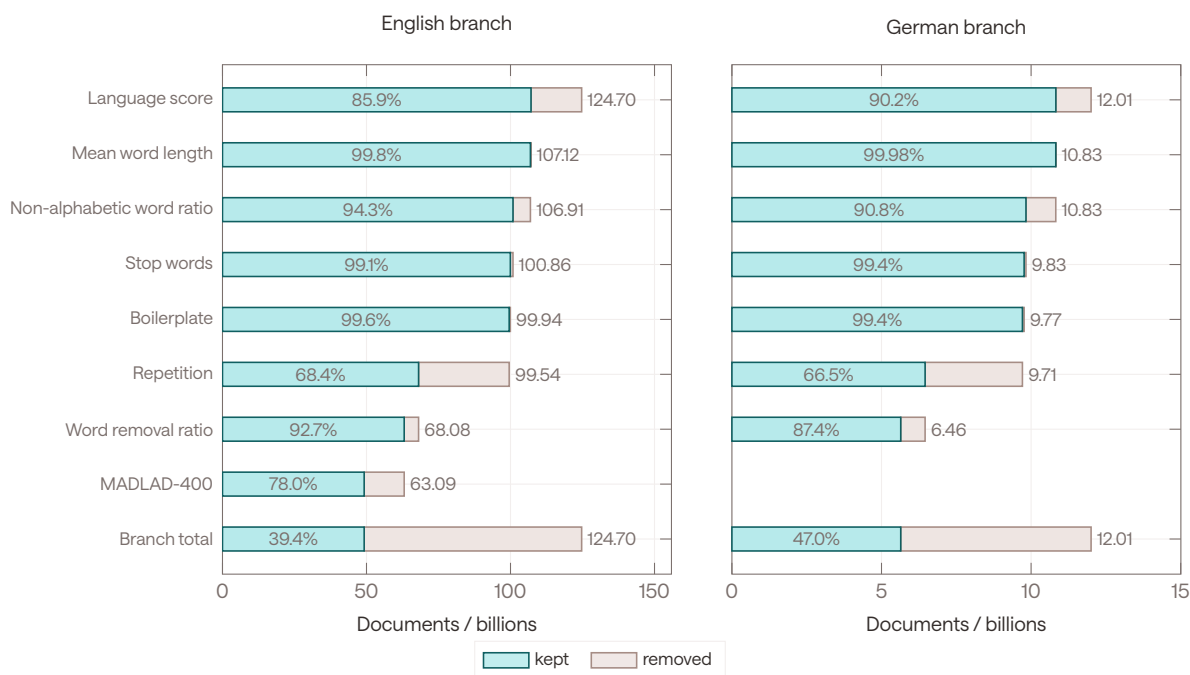


Figure 58. Documents entering each heuristic filter step of the English and German branches, split into the documents the step kept and removed, summed over all processed CC snapshots. The label in the kept segment is the step's yield, the label at the end of the bar the documents it received in billions. Line modifiers drop no documents and are omitted, and the MADLAD-400 filter runs in the English branch only.

## D.1.4 Substring Deduplication

Substring deduplication follows the suffix-array method of Lee et al. (2022), which keeps one occurrence of every repeated byte sequence of at least 500 bytes and cuts the others out of their documents. We keep the occurrence earliest in a shard's byte order, fuse adjacent removal ranges that cover at least 95 % of the fused span, and drop a document with fewer than 100 bytes left. The stage runs per language in 41 English shards of three snapshots and 7 German shards of 16 to 19 snapshots. It removes 20.4 % of the English and 29.6 % of the German bytes. The cuts join the text on either side without a check for coherence, and cut documents skip the heuristic filters.

## D.2 Mid-training Details

This section gives the parts of Section 2.3.3 that the main text summarises or omits: the mixing method, the evals and ablations, the language sweep, decontamination, the transition from the pre-training mix, and the production mix and run.

### D.2.1 Mixing Method

Besides the curated pool of Section 2.3.3.1 and the two sampling rules of Section 2.3.3.3, the language sweep (Appendix D.2.4) uses a second dataset pool and a third sampling rule for the two language domains.

**Repetition Caps** A dataset’s mid-training cap is the four passes of Section 2.3.3.3 minus its epochs in the 20T tokens of pre-training. At the 400B and 90B budgets no repetition cap binds. At the 5.2T production budget every dataset of German instruction and reasoning is pinned (Section 2.3.3.3), so German instruction and reasoning receives fewer tokens than its weight assigns, and every other domain receives its full weight.

**Optimisation of the Domain Weights** Each expert trains on the datasets of its domain under the pre-training-delivered rule. All experts start from the same checkpoint as every other proxy ablation (Appendix D.2.2), so their parameters can be averaged (Wortsman, Ilharco, et al. 2022). We keep the in-domain composition fixed, since optimising it with the same method requires one expert per dataset, 87 trainings in total.

**Sampled Weightings** The sampled weightings centre on the domain weights of C1 and C2, with 30 at concentration 1, 30 at concentration 2, and 40 spread over concentrations from 0.1 to 5, the range RegMix samples from (Q. Liu et al. 2024). The hand-placed weightings step each of the four smallest domains, German instruction and reasoning, Specialized domains, Agentic code and tool use, and STEM (documents), from zero to the largest weight the repetition caps of its datasets allow, with the other domains scaled along the centre.

**Solved Weights** The KL penalty is the regulariser of Olmix’s exact solver, for which M. F. Chen et al. (2026) recommend a weight of 0.05, and the production mix uses 0.01 (Appendix D.2.7). The cap constraint of the solve (Section 2.3.3.3) limits each domain’s weight to the share of the budget its datasets can fill under their caps. The constraint bounds domains only, so a dataset can still be pinned inside its domain at the production budget. In all four projected solutions (Table 38), the projection changes the in-domain composition of German instruction and reasoning and STEM (QA-style), in all but S4 that of English instruction and reasoning, and in all but S6 that of Code. The projections use the caps of a 30T pre-training, and the production mix uses the caps of the 20T pre-training run, under which fewer datasets are pinned. Appendix D.2.2 compares the six solutions to the hand-set candidates.

| Mix | Prior | KL penalty | Projection |
|-----|-------|------------|------------|
| S1  | C2    | 0          | from 5T    |
| S2  | C2    | 0.01       | –          |
| S3  | C2    | 0.01       | from 5T    |
| S4  | C2    | 0.05       | from 5T    |
| S5  | C3    | 0.01       | –          |
| S6  | C3    | 0.01       | from 5T    |

Table 38. The reference mix and KL penalty of each materialised optimised mix, and whether its in-domain composition is projected from a 5T budget (Section 2.3.3.3). Table 43 lists their domain weights.

## D.2.2 Ablation Setup

**Proxy Model and Budgets** The proxy model has 384 experts and a hidden size of 1024. Its pre-training horizon of 3.05T tokens matches the 256 Tokens per Parameter of production, 20T tokens over 78.1B total parameters. Every ablation loads the proxy pre-training checkpoint at step 166 000, 91% of the matched horizon, and mid-trains for 5000 steps at 16.8M tokens per step (84B) or 23 842 steps (400B). The mixes of the 84B runs were materialised for a 90B budget, so their repetition caps are those of Table 43.

**Merged Checkpoints** At 84B the M10 merge averages the ten checkpoints saved every 500 steps, which is the whole run. At 400B, M10 averages ten checkpoints 2100 steps (35B tokens) apart from step 5000 to the endpoint, and M4 averages four checkpoints 4200 steps (70B tokens) apart ending at the endpoint.

**Aggregate Score** The benchmarks of the aggregate score come from the pre-training suite (Section 2.4). The general-knowledge mean averages MMLU (Hendrycks, C. Burns, Basart, Zou, et al. 2021), MMLU-Pro (Yubo Wang et al. 2024), ARC (P. Clark et al. 2018), HellaSwag (Zellers et al. 2019), PIQA (Bisk et al. 2020), and TriviaQA (Joshi et al. 2017) in English, and Global MMLU (S. Singh et al. 2025), ARC, HellaSwag, and PIQA in German. The math mean averages GSM8K (Cobbe et al. 2021) and MATH (Hendrycks, C. Burns, Kadavath, et al. 2021), and the code mean HumanEval (M. Chen et al. 2021) and MBPP (Austin et al. 2021), each in both languages. HumanEval is affected by contamination (Appendix D.2.5), so the tables also report the aggregate score without HumanEval, which drops the English and German HumanEval from the code means, leaves MBPP as the code benchmark, and keeps the rest of the formula unchanged.

**SFT Recipe and Eval Noise** The SFT recipe of this section and of Appendix D.2.8 is 1000 steps at a context of 256k tokens with a global batch of 64 sequences. Its data is one mix of 26 datasets, separate from the Kolibri post-training mix, with 54.5 % chat, 20.7 % math, 10.8 % science, and the rest agentic, instruction following, German, and the model’s identity data. The SFTs models are scored on the following post-training benchmarks. The public benchmarks included: AIME 2026 (Mathematical Association of America 2026) and its German translation, MATH-500 (Lightman et al. 2023), GPQA Diamond (Rein et al. 2023), IFEval (J. Zhou et al. 2023), IFBench (Pyatkin et al. 2025), LongBench Pro (Z. Chen et al. 2026) at two context lengths, HumanEval+ (Jiawei Liu et al. 2023), and LiveCodeBench (Jain et al. 2024). Two are internal German benchmarks, one for science and mathematics and KulturBank for cultural knowledge (Appendix H.5). A majority of the benchmarks have an eval-noise model estimated from 20 replicate SFTs that differ in seed

only. The model gives each score a 95 % interval from the binomial standard error of Miller (2024), with an effective sample size fitted to the replicates, so the interval depends on the score and on the benchmark. It also gives a pooled score, the mean over the 11 calibrated benchmarks, with the variances summed and the degrees of freedom pooled by Welch-Satterthwaite (Welch 1947; Satterthwaite 1946). The interval describes how precisely one score is known. Whether two runs differ is a test of their difference against the noise of both runs, which resolves a difference of about 1.4 times one half-width. Two disjoint intervals need about twice the half-width and are the more conservative test, so two runs with overlapping intervals can still differ.

### D.2.3 Ablation Results

**Budget** The pair tests of the adjacent 400B runs (Section 2.3.3.5) resolve a difference of 2.4 points between continued pre-training and C2, 2.5 between C2 and C3, and 2.6 between C3 and the production mix, and each of the three gaps exceeds its threshold at  $p < 0.01$  (Table 39 and Figure 59).

| Mix                    | 84B  | 400B |      |
|------------------------|------|------|------|
|                        | M10  | M10  | M4   |
| C2                     | 60.9 | 62.2 | 61.1 |
| C3                     | 60.9 | 63.6 | 62.4 |
| production mix         | 62.2 | –    | 65.5 |
| continued pre-training | –    | –    | 60.6 |

Table 39. Aggregate score of the mixes trained at both proxy budgets, at the merged checkpoints. The continued pre-training baseline is an M4 of the proxy pre-training run at 3.2T tokens, whose four checkpoints lie 80B tokens apart, against 70B for the mixes.

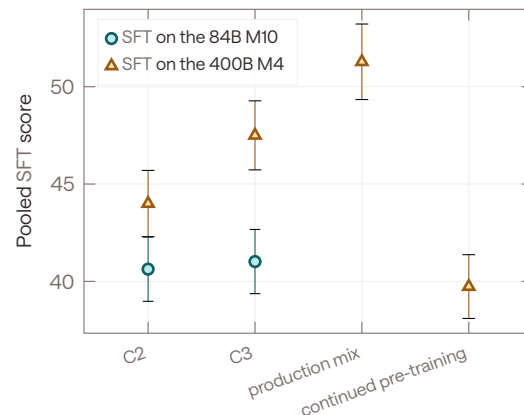


Figure 59. Pooled score of the SFT trained on the merges of Table 39, on the M10 of the smaller budget and on the M4 of the larger, with the 95 % eval-noise interval as error bar.

**Ranking** The ranking of Table 40 covers every 84B run, the budget at which every mix was trained. These runs are the three hand-set candidates, the domain-weight sweep, the language sweep, the production mix, and the Kolibri Origin mid-training mix, an 80/20 blend of high-quality English web data and its translation into German. Runs whose mixes differ only in the projection (S2 and S3, S5 and S6), in the transition (C1 and C1 transition), or in the sampling rule of the language domains (L1 and L2, L3 and L4) score within 0.6 points of each other on the merge, the spread against which we read the ranking. S3 leads the hand-set mixes by 1.2 to 2.0 points, beyond that spread. The production mix ranks fifth of 18, within 0.7 points of the four runs above it.

**Transfer from Merge to SFT** For the four 400B runs of Section 2.3.3.5 and the 16 runs at 84B with an SFT, Figure 60 plots the pooled SFT score against the aggregate score of the merge the SFT starts from. At 84B the eval noise alone has a standard deviation of 0.84 points, against 1.04 for the SFT scores, so

two thirds of their variance is noise. Of the 120 pair tests among these SFTs, 14 clear the noise, against 6 expected by chance, and each of them sets one of the seven highest SFTs against one of the three lowest.

| Mix                             | M10   |      | M10 w/o HumanEval |      | SFT   |       |
|---------------------------------|-------|------|-------------------|------|-------|-------|
|                                 | score | rank | score             | rank | score | $\pm$ |
| S3                              | 62.9  | 1    | 61.9              | 1    | 42.0  | 1.6   |
| L6                              | 62.5  | 2    | 61.7              | 2    | 41.8  | 1.7   |
| S2                              | 62.4  | 3    | 61.0              | 7    | 40.8  | 1.6   |
| S6                              | 62.3  | 4    | 61.0              | 6    | 42.4  | 1.8   |
| production mix                  | 62.2  | 5    | 61.2              | 3    | –     | –     |
| L3                              | 62.1  | 6    | 60.8              | 11   | 42.0  | 1.7   |
| S1                              | 61.8  | 7    | 60.6              | 12   | 40.9  | 1.7   |
| L4                              | 61.7  | 8    | 61.2              | 4    | 42.4  | 1.7   |
| S5                              | 61.7  | 9    | 61.1              | 5    | 42.8  | 1.6   |
| C1                              | 61.6  | 10   | 61.0              | 8    | 40.1  | 1.6   |
| L2                              | 61.6  | 11   | 60.8              | 10   | 42.9  | 1.7   |
| L1                              | 61.5  | 12   | 61.0              | 9    | 42.0  | 1.7   |
| L5                              | 61.4  | 13   | 60.1              | 15   | 41.2  | 1.7   |
| S4                              | 61.1  | 14   | 60.3              | 13   | 39.9  | 1.6   |
| C1 transition                   | 61.1  | 15   | 59.7              | 18   | –     | –     |
| C3                              | 60.9  | 16   | 59.8              | 17   | 41.0  | 1.6   |
| C2                              | 60.9  | 17   | 60.2              | 14   | 40.6  | 1.7   |
| Kolibri Origin mid-training mix | 60.2  | 18   | 59.9              | 16   | 39.5  | 1.6   |

Table 40. Every proxy run at the smaller budget, ordered by the aggregate score of its M10 merge, with the score and rank of the merge and of the merge without HumanEval, and the pooled score of the SFT on the merge with the half-width of its 95 % eval-noise interval. C1 transition is C1 entered through the transition of Appendix D.2.6, and every other run switches directly from the pre-training mix. The merges of C2, C3, and the production mix average the first ten checkpoints of their 400B runs. Dashes mark the two runs without an SFT.

**Domain Weights** The domain weights covary within each sweep, so a correlation of a domain weight with a score (Table 41) describes this set of mixes and includes the effect of the domains that covary with it. The aggregate score and the math means rise with English instruction and reasoning and fall with English web and documents and Specialized domains. TriviaQA rises with English web and documents, at  $r = 0.81$ , and HellaSwag (DE) and the German general-knowledge mean rise with German web and documents, at 0.84 and 0.82. The English general-knowledge mean barely moves with English web and documents (Section 2.3.3.5), so the other English knowledge benchmarks fall on average as TriviaQA rises with it.

**Trade-offs** Across the 84B runs the general-knowledge mean spans 1.2 points, against 5.8 for math and 5.2 for MBPP, so the spread of the aggregate score comes from math and code (Figure 61). English math and MBPP correlate at  $r = -0.18$ , and German knowledge and math at  $-0.73$ , through HellaSwag (DE), which correlates with German math at  $-0.72$ . A knowledge mean averages gains and losses on single benchmarks (Appendix D.2.8), so the means of Figure 61 can hide the loss of one benchmark.

## D.2.4 Language Sweep

The language sweep tests whether the in-domain composition of English web and documents and of German web and documents, the two language domains, causes the losses of the production run on TriviaQA and HellaSwag (DE) (Appendix D.2.8). L1 to L4 share one set of domain weights and differ only

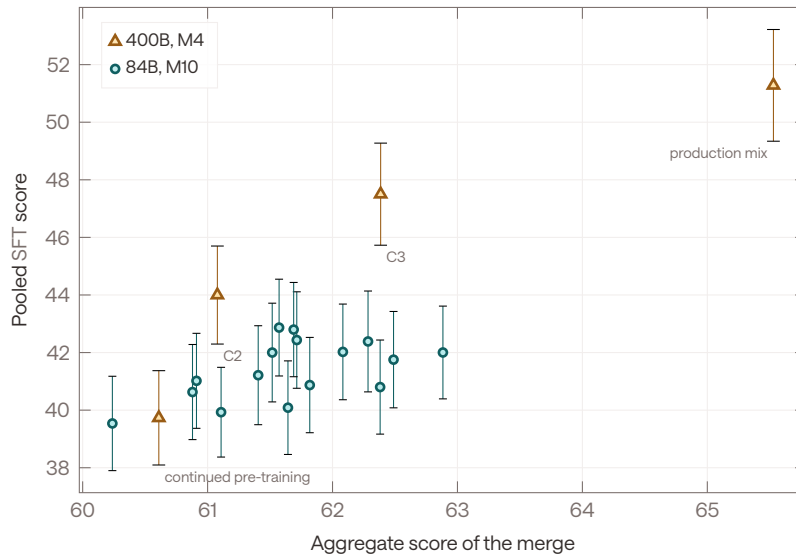


Figure 60. Pooled score of the SFT on each merge against the aggregate score of the merge, with the 95 % eval-noise interval as error bar, for the M4 merges of the four 400B runs and the M10 merges of the 16 runs at 84B with an SFT.

| Domain                            | final | w/o HumanEval | knowledge EN | knowledge DE | math EN | math DE | MBPP EN | MBPP DE | TriviaQA | HellaSwag DE |
|-----------------------------------|-------|---------------|--------------|--------------|---------|---------|---------|---------|----------|--------------|
| English web and documents         | -0.66 | -0.62         | -0.11        | 0.55         | -0.67   | -0.72   | -0.32   | -0.33   | 0.81     | 0.61         |
| German web and documents          | -0.57 | -0.45         | 0.09         | 0.82         | -0.63   | -0.65   | -0.17   | -0.32   | 0.75     | 0.84         |
| English instruction and reasoning | 0.73  | 0.67          | 0.22         | -0.59        | 0.76    | 0.73    | 0.29    | 0.44    | -0.78    | -0.68        |
| German instruction and reasoning  | 0.47  | 0.55          | 0.03         | -0.14        | 0.38    | 0.36    | 0.52    | 0.03    | -0.30    | -0.07        |
| Code                              | 0.41  | 0.26          | 0.01         | -0.85        | 0.53    | 0.57    | 0.02    | -0.02   | -0.48    | -0.92        |
| Agentic code and tool use         | 0.06  | 0.20          | -0.20        | 0.36         | -0.09   | -0.01   | 0.29    | 0.28    | -0.28    | 0.47         |
| STEM (documents)                  | 0.05  | 0.03          | -0.33        | -0.08        | -0.14   | 0.06    | 0.16    | 0.18    | -0.05    | 0.05         |
| STEM (QA-style)                   | 0.48  | 0.48          | 0.13         | -0.38        | 0.58    | 0.52    | 0.19    | 0.28    | -0.71    | -0.47        |
| Specialized domains               | -0.71 | -0.73         | -0.17        | 0.50         | -0.77   | -0.73   | -0.39   | -0.31   | 0.70     | 0.62         |

Table 41. Pearson correlation of each domain weight with the aggregate score and its constituents across the M10 merges of the 17 runs whose mix is in Table 43. The Kolibri Origin mid-training mix is not composed from the nine domains and has no weights. The code columns are MBPP alone.

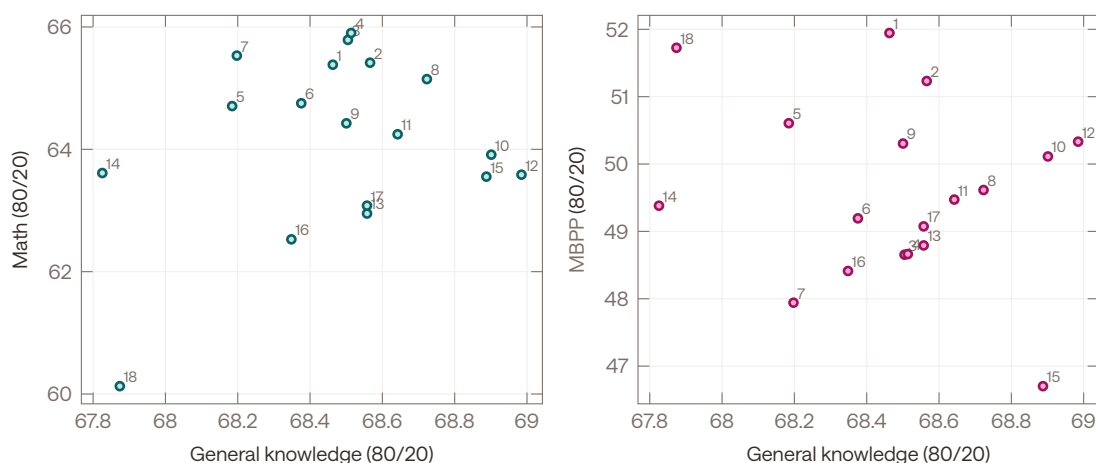


Figure 61. General knowledge against math (left) and against MBPP (right) for the M10 merge of every proxy run in Table 40. Each run trained 84B tokens. Each mean is weighted 80/20 over English and German. The label is the run's rank on the aggregate score in Table 40.

in the dataset pool and sampling rule of the two language domains (Table 42). These weights raise the two language domains of the production mix before its waterfill to 20 % of the budget and scale the other seven domains down by a common factor. Within the 20 %, English web and documents receives 5.8 % and German web and documents 14.2 %, so that the English language and instruction domains together weigh twice as much as the German ones. L5 has the domain weights of C1, and L6 its own (Table 43). The extended pool adds the 26 language datasets of pre-training that the curated pool excludes, the low-quality tiers of the web crawls and the rephrased and translated German web text. Under the optimised sampling rule, the two language domains split their budget at the per-dataset weights that the pre-training mix optimiser assigns over the dataset pool of the run.

After SFT, none of the pair tests among L1 to L6, S5, and S6 clears the noise (Appendix D.2.2). At equal domain weights, runs on the extended pool score 2.4 to 3.1 points lower on HellaSwag (DE) than runs on the curated pool (L4 against L2, L3 against L1, and L5 against C1), against at most 0.9 between curated-pool runs at equal German-web share (Figure 63). In the same pairs, the German general-knowledge mean differs by 0.9 to 1.3 points. On TriviaQA, the dataset pool and the sampling rule of the language domains change the score by at most 0.8 points, the spread between near-identical runs (Appendix D.2.8).

| Mix | Dataset pool  | Sampling               | Other domains as in |
|-----|---------------|------------------------|---------------------|
| L1  | curated pool  | natural                | S5                  |
| L2  | curated pool  | optimised              | S5                  |
| L3  | extended pool | pre-training-delivered | S5                  |
| L4  | extended pool | optimised              | S5                  |
| L5  | extended pool | pre-training-delivered | C1                  |
| L6  | extended pool | optimised              | production mix      |

Table 42. Dataset pool and sampling rule of the two language domains in each run of the language sweep, and the mix whose in-domain composition the other seven domains keep. Table 43 lists the domain weights.

Table 43 lists the domain weights and in-domain composition of every mix trained at proxy scale and of the production mix.

| Mix            | B     | Domain weights (%) |        |           |           |      |         |           |         |       | In-domain composition |                        |
|----------------|-------|--------------------|--------|-----------|-----------|------|---------|-----------|---------|-------|-----------------------|------------------------|
|                |       | EN web             | DE web | EN instr. | DE instr. | Code | Agentic | STEM docs | STEM QA | Spec. | Dataset pool          | Sampling               |
| C1             | 400   | 33.0               | 19.0   | 10.0      | 1.0       | 10.0 | 8.5     | 6.5       | 9.0     | 3.0   | curated pool          | natural                |
| C2             | 400   | 33.0               | 19.0   | 10.0      | 1.0       | 10.0 | 8.5     | 6.5       | 9.0     | 3.0   | curated pool          | pre-training-delivered |
| C3             | 400   | 14.0               | 18.0   | 16.2      | 1.4       | 8.0  | 16.2    | 6.0       | 16.2    | 4.0   | curated pool          | pre-training-delivered |
| S1             | 90    | 0.0                | 0.0    | 31.0      | 1.4       | 42.1 | 0.0     | 0.3       | 25.1    | 0.0   | curated pool          | pre-training-delivered |
| S2             | 90    | 3.1                | 0.4    | 31.5      | 1.4       | 29.4 | 7.8     | 10.8      | 15.5    | 0.1   | curated pool          | pre-training-delivered |
| S3             | 90    | 3.1                | 0.4    | 30.7      | 1.4       | 29.8 | 7.9     | 10.9      | 15.7    | 0.1   | curated pool          | pre-training-delivered |
| S4             | 90    | 20.3               | 2.3    | 14.9      | 1.4       | 27.1 | 9.7     | 13.7      | 9.6     | 1.0   | curated pool          | pre-training-delivered |
| S5             | 90    | 1.3                | 2.1    | 31.5      | 1.4       | 16.4 | 15.5    | 6.0       | 25.5    | 0.2   | curated pool          | pre-training-delivered |
| S6             | 90    | 1.3                | 2.2    | 31.0      | 1.4       | 16.8 | 15.9    | 6.1       | 25.1    | 0.2   | curated pool          | pre-training-delivered |
| L1             | 90    | 5.8                | 14.2   | 25.9      | 1.7       | 13.5 | 12.8    | 4.9       | 21.0    | 0.2   | curated pool          | natural                |
| L2             | 90    | 5.8                | 14.2   | 25.9      | 1.7       | 13.5 | 12.8    | 4.9       | 21.0    | 0.2   | curated pool          | optimised              |
| L3             | 90    | 5.8                | 14.2   | 25.9      | 1.7       | 13.5 | 12.8    | 4.9       | 21.0    | 0.2   | extended pool         | pre-training-delivered |
| L4             | 90    | 5.8                | 14.2   | 25.9      | 1.7       | 13.5 | 12.8    | 4.9       | 21.0    | 0.2   | extended pool         | optimised              |
| L5             | 90    | 33.0               | 19.0   | 10.0      | 1.0       | 10.0 | 8.5     | 6.5       | 9.0     | 3.0   | extended pool         | pre-training-delivered |
| L6             | 90    | 5.8                | 14.2   | 24.0      | 4.0       | 12.5 | 15.4    | 4.5       | 19.4    | 0.2   | extended pool         | optimised              |
| production mix | 5,196 | 1.3                | 2.1    | 31.4      | 1.9       | 16.3 | 15.4    | 5.9       | 25.4    | 0.2   | curated pool          | pre-training-delivered |

Table 43. Every mid-training mix with its domain weights and its in-domain composition. C1 to C3 have hand-set domain weights and were materialised at 400B. S1 to S6 are the optimised mixes (Appendix D.2.1) and L1 to L6 the language sweep (Appendix D.2.4), both at 90B. The production mix is S5 with German instruction and reasoning raised to 1.9% and the other domains scaled down by a common factor (Appendix D.2.7), materialised at 5.2T. Its weights are what its waterfill delivers. Short domain names follow Table 12. In the language sweep, the dataset pool and sampling rule describe the two language domains (Table 42).

## D.2.5 Decontamination

To reduce the risk that a benchmark score reflects memorisation of the benchmark itself, we remove from the training corpus every document that contains text from our evaluation suite. The reference eval set contains all 226 benchmark tasks of our evaluation framework with all available splits (train, validation, and test), because training and validation items share the format, the answer style and potentially near-duplicates of the test items, so a model trained on them scores higher without any test instance leaking.

We detect contamination with the algorithm of the decon tool (Allen Institute for AI 2025).

Multiple-choice tasks are unpacked into one entry per answer option. Questions and answers are indexed separately as 5- and 3-grams over c1100k tokens. Each n-gram is weighted by its IDF across the reference set: a rare, distinctive n-gram is strong evidence of contamination, but one that appears in many evaluation instances, such as a stopword sequence or shared prompt boilerplate, is weak evidence and counts for little.

Inspection is deterministic and involves no model inference. The inspection probes a sample of a training document’s n-grams against the index. On a hit, the inspection expands in both directions and keeps the evaluation instances that continue to match. It scores the span as an IDF-weighted overlap ratio of the question and of the answer, weighted towards the question, and flags a document whose combined score exceeds a threshold. We drop a flagged document as a whole, whichever task it matched. Short matches

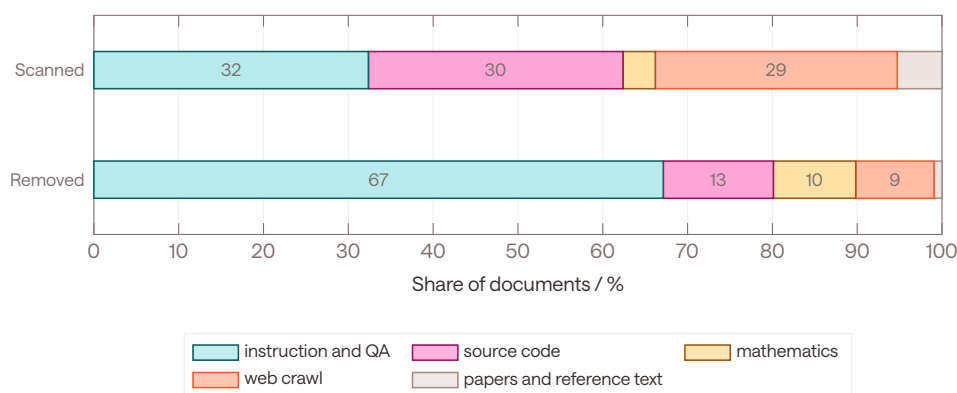


Figure 62. Composition of the inspected curated pool and of the documents decontamination removed from it, by class of data, for our mid-training mix.

must be perfect, which keeps common phrases from being flagged. We keep AI2’s default parameters throughout.

Memorisation concentrates near the end of training (Magar and Schwartz 2022; Bordt et al. 2025). Contamination early in training is forgotten once enough training follows it, whereas contamination in the final stages is followed by too little training to be forgotten. We therefore decontaminate the mid- and post-training pool being most sensitive to task specific data in SFTs among other shares and memorisation, which also feeds the long-context extension stage.

Across all datasets from the curated pool, the decontamination run inspected 8.3B documents and removed 1.5M of them, 0.018 % of documents and 0.06 % of tokens. Removed documents are about three times longer than average, since we drop a document completely once any of its spans matches, and long documents such as committed benchmark files or multi-item synthetic records offer many spans. The aggregates depend on the composition of the curated pool and are indicative only. Contamination is concentrated in a few classes of data, as Figure 62 shows. Synthetic instruction and question-answering data holds 32 % of the inspected documents but 67 % of the removals, and mathematics datasets have the highest rate at 0.05 % of their documents. Source code contributes 13 % of the removals, almost all of them structured data files in which benchmark datasets were committed to repositories verbatim. Web crawl text contributes 9 % at a rate of 0.006 %, and scientific papers, PDF scrapes, and curated reference and legal text together contribute under 1%.

On the benchmark side, MMLU (Hendrycks, C. Burns, Basart, Zou, et al. 2021), MATH (Hendrycks, C. Burns, Kadavath, et al. 2021), and GSM8K (Cobbe et al. 2021) account for 70 % of all matched (document, evaluation instance) pairs, MMLU-Pro (Yubo Wang et al. 2024) for a further 10 %, and ARC (P. Clark et al. 2018), and a small collection of other benchmarks (including HellaSwag (Zellers et al. 2019) and SQuAD (Rajpurkar et al. 2016)) for most of the remainder.

Counted per question rather than per pair, Table 44 gives the share of each benchmark’s held-out questions that at least one document of the pool reproduced. The math and code benchmarks are present almost completely, the knowledge and reasoning benchmarks in part, and TriviaQA hardly at all.

| Benchmark | Split      | Questions | Found | Share / % |
|-----------|------------|-----------|-------|-----------|
| GSM8K     | test       | 1319      | 1319  | 100.0     |
| HumanEval | test       | 164       | 163   | 99.4      |
| MATH      | test       | 5000      | 4654  | 93.1      |
| SQuAD     | validation | 10 205    | 7666  | 75.1      |
| ARC       | test       | 3548      | 2209  | 62.3      |
| MMLU      | test       | 14 042    | 6411  | 45.7      |
| MMLU-Pro  | test       | 12 032    | 4701  | 39.1      |
| HellaSwag | test       | 10 003    | 2981  | 29.8      |
| MBPP      | test       | 500       | 49    | 9.8       |
| TriviaQA  | test       | 7701      | 371   | 4.8       |

Table 44. Share of each benchmark’s questions that at least one document of the curated pool matched before removal, for the held-out split of the suite, or its validation split where the suite has no test split. For the math and code benchmarks the suite whose answer is the reference solution is taken. A question counts once however many documents reproduce it.

Much of this contamination is templated: a benchmark question followed by rewritten options, a stated answer, and a worked explanation, which is what synthetic instruction data generated from benchmark exemplars looks like.

To measure the effect of decontamination, we compared two mid-training runs at proxy-model scale: one on a mix drawn from the curated pool before decontamination, at its natural contamination rate, and one on a mix drawn from the decontaminated pool. The two runs track each other within noise on every benchmark, and the naturally contaminated run shows no drift over a schedule several times longer. At natural dilution and this scale, the contamination is not measurable. This does not make decontamination optional. The contaminated share of a natural mix is roughly the same at any scale, but the expected number of times a contaminated document is seen is not. As the mix grows, each domain’s sampling factor rises towards its repetition cap, and the domains upsampled first, math, code and synthetic instruction data, are the ones that hold the contaminated documents. An upper-bound run shows what such exposure does: adding every document removed from the pool back into the decontaminated proxy mix, 8.2 % of its tokens, inflates HumanEval by 42 points, MMLU by 29 and TriviaQA by 28 at the same step, halves the bits per byte on the benchmark text, and leaves the long-context suites unchanged. This is memorisation and not capability. A benchmark score is evidence of capability only if leakage is ruled out by construction rather than argued away after the fact. We will report the ablations in detail in a forthcoming blog post.

The benchmark scores over the mid-training run show signs of contamination that the method misses, in particular reference solutions that appear without their prompt.

## D.2.6 Transition into the Mid-Training Mix

The step from the pre-training mix to the mid-training mix changes the data distribution at a constant learning rate. Prior work studies such changes of distribution in continual pre-training and mid-training (Gupta et al. 2023; Parmar et al. 2024; Ettinger et al. 2025), and retaining a share of the previous data limits the forgetting they cause (Ibrahim et al. 2024; Xingjin Wang et al. 2025; Abbes et al. 2026; E. Liu, Neubig, and Xiong 2025).

We therefore interpolate between the two mixes over the window of  $H$  tokens of Section 2.3.3.4, in which the token share of the pre-training mix decays linearly as

$$w(t) = 1 - \frac{t}{H}, \quad 0 \leq t \leq H,$$

so that the pre-training mix contributes  $H/2$ , 50B tokens, to the window. The loader index realises the schedule by interleaving the bulk-shuffled indices of the two mixes. It places the  $i$ -th pre-training bulk, with token midpoint  $m_i$ , where the cumulative pre-training tokens  $t - t^2/(2H)$  reach  $m_i$ , that is at

$$t_i = H \left( 1 - \sqrt{1 - \frac{2m_i}{H}} \right).$$

These bulks come from the part of the pre-training index beyond the 20T tokens of the pre-training run, so their documents lie outside the model's training data. The schedule holds up to whole bulks of 4096 rows, and the pre-training documents in the window remain potentially contaminated, since we decontaminate only the mid-training pool.

## D.2.7 Production Mix

The raise of German instruction and reasoning (Section 2.3.3.4) takes the domain to the largest weight its three datasets can fill under their repetition caps at 5T and scales the other eight domains down by a common factor. The solve used the caps of a 30T pre-training, under which German instruction and reasoning caps at 1.4 % and every solution sits at that cap (Table 43), and under the caps of the 20T pre-training run the domain reaches 1.9 %. The run trained 3.44T of the 5.2T build, so it draws the seven pinned datasets at two thirds of the repetition their caps allow. The C3 prior comes from the 400B runs, in which C3 leads C2 by 3.5 points after SFT (Table 39). At 84B, the SFT scores of S1, S2, and S4, with penalties 0, 0.01, and 0.05, lie within 0.9 points of each other, below the resolution of Appendix D.2.2. S1 sets four domains to zero, and S4 lies closest to its prior, so we take 0.01, the smallest tested penalty at which every domain keeps a positive weight (Table 13).

## D.2.8 Production Run

**Knowledge Benchmarks** From the pre-training M4 to the mid-training M4, scores rise by 2.9 points on MMLU-Pro and 1.2 on ARC, fall by 6.8 on TriviaQA, 5.6 on HellaSwag (DE), and 2.6 on HellaSwag, and change by at most 0.7 on the other five knowledge benchmarks (Table 45). At 400B, against continued pre-training, scores rise by 3.6 points on MMLU-Pro, 2.9 on PIQA, 2.4 on MMLU, 1.7 on Global MMLU, and 1.5 on ARC, and fall by 8.4 on TriviaQA, 7.2 on HellaSwag (DE), and 3.9 on HellaSwag. On HellaSwag (DE), C2 and C3, with 18 % to 19 % German web against 2.1 % in the production mix, score within 1.6 points of continued pre-training (Table 14). Across the 84B runs HellaSwag (DE) rises from 65.7 at 0 % to 79.4 at 19 % German web (Figure 63), and HellaSwag rises from 72.6 at 0 % to 75.2 at 33 % English web. Curated-pool runs at equal German-web share differ by at most 0.9 points on HellaSwag (DE), and extended-pool runs score lower at the same share (Appendix D.2.4). TriviaQA falls by 6.2 to 8.4 points at 400B in all three mixes, including C2 with 33 % English web. At 84B and equal domain weights, C1 under the natural sampling rule scores 2.8 points above C2 under the pre-training-delivered rule on TriviaQA, against at most 0.8 between

the near-identical runs of Appendix D.2.2. All three mixes at 400B use the pre-training-delivered rule, and continued pre-training also differs from them in sequence length, which leaves the cause of the TriviaQA loss open. The M20 merge at the end of mid-training lifts the English and German knowledge means by 1.6 and 1.0 points over the M4 merge, and the final base model scores within 0.1 points of it on both.

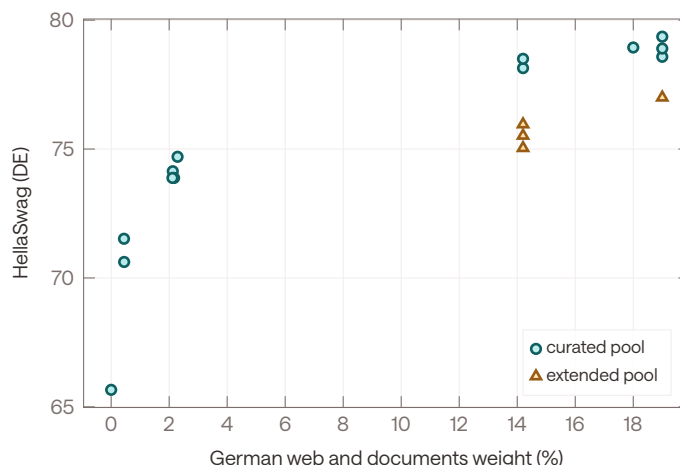


Figure 63. HellaSwag (DE) of the M10 merge of every proxy run at 84B against the weight of German web and documents in its mix, by the dataset pool of the language domains. The figure omits the Kolibri Origin mid-training mix, which has no domain weights.

| Stage        | Step   | M   | Tokens (T) |       | Score     |        | Knowledge |      | Math |      | Code |      | Benchmarks (EN unless marked) |      |      |      |      |       |
|--------------|--------|-----|------------|-------|-----------|--------|-----------|------|------|------|------|------|-------------------------------|------|------|------|------|-------|
|              |        |     | stage      | total | aggregate | w/o HE | EN        | DE   | EN   | DE   | EN   | DE   | HE                            | Pro  | ARC  | TQA  | HS   | HS DE |
| pre-training | 262000 | M4  | 19.78      | 19.78 | 77.7      | 74.8   | 80.7      | 89.6 | 79.4 | 71.4 | 70.3 | 82.3 | 80.0                          | 55.1 | 94.0 | 82.7 | 84.5 | 91.4  |
| mid-training | 33600  | M4  | 3.38       | 23.37 | 80.4      | 77.6   | 79.8      | 88.2 | 84.0 | 75.2 | 75.9 | 84.4 | 85.5                          | 58.0 | 95.2 | 75.9 | 81.9 | 85.8  |
| mid-training | 34000  | M20 | 3.42       | 23.41 | 81.0      | 79.1   | 81.4      | 89.2 | 85.1 | 76.5 | 75.7 | 80.3 | 83.0                          | 61.3 | 95.8 | 77.3 | 83.2 | 86.4  |
| long-context | 1000   | M20 | 0.20       | 23.63 | 81.6      | 78.9   | 81.4      | 89.3 | 85.0 | 77.2 | 76.8 | 85.0 | 86.2                          | 61.5 | 95.7 | 77.5 | 83.1 | 86.9  |

Table 45. The production chain on the pre-training suite at the last evaluated merge of pre-training, mid-training, and long-context extension, whose M20 merge is the final base model. Every intermediate evaluation was on an M4 merge, so the M4 rows are the comparable ones. Mid-training and long-context training start from the raw checkpoints at pre-training step 264750 and mid-training step 34200. The columns are the aggregate score, the aggregate score without HumanEval (HE), the general-knowledge, math, and code means per language, HumanEval (HE), and the knowledge benchmarks with the largest changes (Pro is MMLU-Pro, TQA is TriviaQA, HS is HellaSwag, and HS DE is HellaSwag (DE)). M is the merge window. Tokens are those of the stage at that step and of the whole chain.

**HumanEval Scores and Recitation** During pre-training, English HumanEval moved by more than 25 points from one evaluated merge to the next (Figure 64). For every HumanEval evaluation of the production chain, we compare each completion with the reference solution shipped with the benchmark, after dropping comments and collapsing whitespace, and count it as recited at a similarity of 0.95 or more. During pre-training, 22 % to 95 % of all completions are recited, and pass@1 follows the recited share linearly, with a Pearson correlation of 0.90 over the 96 evaluations, 48 per language. Mid-training on the decontaminated mix lowers the recited share to about a tenth within the first 0.3T tokens, and from 0.3T to 0.8T pass@1 stays at 70 to 72 in English and 72 to 73 in German. Throughout mid-training, pass@1 on the problems a merge does not recite stays at 61 to 76 in English and 61 to 79 in German, against 50 to 96 and 53 to 84 on a shifting set of problems during pre-training. The fall of pass@1 at the start of mid-training thus coincides with the fall of the recited share, and the pre-training HumanEval scores are not comparable

with those of a decontaminated model. The recited solutions come from the pre-training corpus, which remains potentially contaminated (Appendix D.2.5). The recitation that returns in later mid-training merges is a checkpoint-level state, independent of the exposure to HumanEval text in the mid-training data.

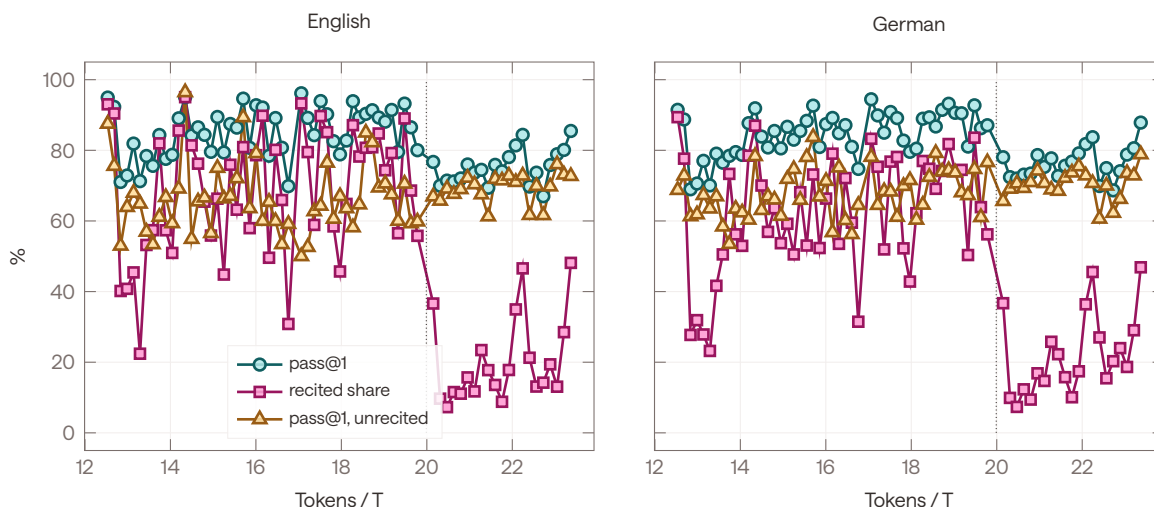


Figure 64. HumanEval pass@1, the share of recited completions, and pass@1 on the problems without a recited completion for every HumanEval evaluation of pre-training and mid-training, against the tokens of the chain. The dotted line marks the start of mid-training.

**Post-Training Benchmarks after SFT** The SFTs on the mid-training endpoint and on the final base model agree within their eval-noise intervals on every benchmark (Table 46). Long-context extension thus changes no post-training benchmark beyond the eval noise. Against the SFT on the 400B proxy run of the same mix (Figure 59), the final base model gains 10 to 44 points per benchmark, a difference in model size and training length at the same mix.

| Benchmark         | Mid-training M20 |      |      | Final base model |      |      |
|-------------------|------------------|------|------|------------------|------|------|
|                   | score            | low  | high | score            | low  | high |
| AIME 2026         | 71.3             | 63.3 | 78.6 | 72.7             | 64.9 | 79.9 |
| AIME 2026 DE      | 63.3             | 52.0 | 74.0 | 73.5             | 62.9 | 83.0 |
| GPQA Diamond CoT  | 66.2             | 58.4 | 73.5 | 66.2             | 58.4 | 73.5 |
| IFBench           | 50.3             | 45.8 | 54.9 | 50.0             | 45.5 | 54.5 |
| IFEval DE         | 74.0             | 68.1 | 79.5 | 73.1             | 67.2 | 78.6 |
| LongBenchPro      | 47.7             | –    | –    | 49.3             | –    | –    |
| LongBenchPro 128k | 37.4             | –    | –    | 38.0             | –    | –    |
| HumanEval+        | 92.7             | 89.4 | 95.4 | 92.1             | 88.7 | 94.9 |
| LiveCodeBench v6  | 66.7             | 62.4 | 70.9 | 67.4             | 63.1 | 71.5 |
| KulturBank        | 48.0             | –    | –    | 48.5             | –    | –    |

Table 46. Post-training benchmark scores of the SFT on the mid-training M20 merge at step 34 000 and on the final base model, with the 95 % eval-noise interval where the benchmark has a noise model. Both SFTs use the recipe of Appendix D.2.2 but ran a smaller benchmark suite than the proxy SFTs, without MATH500 and IFEval, so they have no pooled score comparable with the proxy SFTs. The intervals are those of the proxy-scale noise model, which we assume transfers to this model size.

## D.3 Full Long-Context Results of the Mix Ablation

Table 47 lists the long-context legs of the mix ablation at every evaluated context length, for the merged checkpoint of every branch, together with the aggregate over them.

| Benchmark                     | Mix     | 4k           | 8k           | 16k          | 32k          | 64k          | 128k         | 256k         | Mean         |
|-------------------------------|---------|--------------|--------------|--------------|--------------|--------------|--------------|--------------|--------------|
| RULER NIAH                    | s = 0   | 0.983        | 0.981        | 0.964        | 0.967        | 0.926        | 0.947        | 0.878        | 0.949        |
|                               | s = 1/4 | 0.989        | 0.984        | 0.969        | 0.974        | 0.940        | 0.954        | 0.897        | 0.958        |
|                               | s = 1/3 | <b>0.994</b> | <b>0.988</b> | <b>0.975</b> | <b>0.979</b> | <b>0.942</b> | <b>0.968</b> | <b>0.921</b> | <b>0.967</b> |
|                               | control | 0.992        | 0.976        | 0.964        | 0.968        | 0.917        | 0.936        | 0.860        | 0.945        |
| RULER VT                      | s = 0   | 0.978        | 0.956        | 0.926        | 0.960        | <b>0.960</b> | 0.916        | 0.892        | 0.941        |
|                               | s = 1/4 | <b>0.992</b> | 0.974        | 0.956        | 0.954        | 0.940        | <b>0.940</b> | <b>0.952</b> | 0.958        |
|                               | s = 1/3 | 0.976        | <b>0.978</b> | 0.956        | 0.964        | 0.954        | 0.936        | 0.900        | 0.952        |
|                               | control | 0.984        | <b>0.978</b> | <b>0.974</b> | <b>0.974</b> | 0.952        | 0.934        | 0.940        | <b>0.962</b> |
| RULER QA                      | s = 0   | 0.685        | 0.660        | 0.655        | 0.665        | 0.630        | 0.590        | 0.515        | 0.629        |
|                               | s = 1/4 | <b>0.700</b> | 0.660        | 0.655        | 0.650        | <b>0.645</b> | 0.570        | 0.575        | 0.636        |
|                               | s = 1/3 | 0.695        | <b>0.685</b> | 0.645        | <b>0.680</b> | 0.625        | <b>0.605</b> | <b>0.605</b> | <b>0.649</b> |
|                               | control | 0.685        | 0.675        | <b>0.665</b> | 0.655        | 0.630        | 0.585        | 0.545        | 0.634        |
| RULER WE                      | s = 0   | 0.956        | 0.564        | 0.522        | <b>0.474</b> | 0.462        | 0.458        |              | 0.573        |
|                               | s = 1/4 | <b>0.970</b> | <b>0.667</b> | 0.533        | 0.436        | 0.462        | 0.445        |              | 0.586        |
|                               | s = 1/3 | 0.954        | 0.570        | 0.527        | 0.465        | 0.466        | <b>0.466</b> |              | 0.575        |
|                               | control | 0.963        | 0.654        | <b>0.555</b> | 0.460        | <b>0.476</b> | 0.445        |              | <b>0.592</b> |
| HELMET JsonKV                 | s = 0   |              | 0.990        | <b>1.000</b> | 0.990        | <b>1.000</b> | 0.950        |              | 0.986        |
|                               | s = 1/4 |              | <b>1.000</b> | <b>1.000</b> | <b>1.000</b> | 0.990        | 0.970        |              | 0.992        |
|                               | s = 1/3 |              | 0.990        | <b>1.000</b> | <b>1.000</b> | <b>1.000</b> | <b>0.980</b> |              | <b>0.994</b> |
|                               | control |              | <b>1.000</b> | 0.990        | <b>1.000</b> | <b>1.000</b> | <b>0.980</b> |              | <b>0.994</b> |
| HELMET RagQA                  | s = 0   |              | 0.613        | 0.603        | 0.576        | <b>0.558</b> |              |              | 0.587        |
|                               | s = 1/4 |              | 0.608        | 0.585        | 0.585        | 0.550        |              |              | 0.582        |
|                               | s = 1/3 |              | <b>0.633</b> | <b>0.635</b> | <b>0.615</b> | 0.520        |              |              | <b>0.601</b> |
|                               | control |              | 0.605        | 0.581        | 0.586        | 0.556        |              |              | 0.582        |
| HELMET Rerank                 | s = 0   |              | 0.466        | 0.331        | 0.285        | 0.155        | <b>0.158</b> |              | 0.279        |
|                               | s = 1/4 |              | 0.480        | 0.348        | 0.260        | 0.167        | 0.118        |              | 0.275        |
|                               | s = 1/3 |              | <b>0.489</b> | <b>0.363</b> | <b>0.297</b> | <b>0.194</b> | 0.113        |              | <b>0.291</b> |
|                               | control |              | 0.471        | 0.356        | 0.251        | 0.154        | 0.119        |              | 0.270        |
| HELMET LongQA InfBench QA     | s = 0   |              | 0.157        | 0.185        | 0.265        | 0.300        | 0.329        |              | 0.247        |
|                               | s = 1/4 |              | 0.145        | 0.208        | 0.289        | 0.301        | 0.358        |              | 0.260        |
|                               | s = 1/3 |              | 0.163        | 0.233        | 0.293        | <b>0.321</b> | <b>0.371</b> |              | <b>0.276</b> |
|                               | control |              | <b>0.176</b> | <b>0.236</b> | <b>0.299</b> | 0.303        | 0.331        |              | 0.269        |
| HELMET LongQA InfBench Choice | s = 0   |              | <b>0.460</b> | 0.420        | 0.500        | <b>0.590</b> | 0.590        |              | 0.512        |
|                               | s = 1/4 |              | 0.440        | 0.440        | <b>0.510</b> | 0.560        | 0.570        |              | 0.504        |
|                               | s = 1/3 |              | 0.440        | 0.440        | 0.500        | 0.570        | 0.590        |              | 0.508        |
|                               | control |              | 0.440        | <b>0.460</b> | 0.490        | 0.570        | <b>0.620</b> |              | <b>0.516</b> |

Table 47. Long-context results of the mix ablation for the merged checkpoint of every branch, by benchmark and context length. RULER NIAH, VT and WE report recall and RULER QA substring exact match. HELMET JsonKV and LongQA InfBench Choice report exact match, RagQA substring exact match, Rerank NDCG at 10, and LongQA InfBench QA ROUGE-L F1. An empty cell is a length the leg was not evaluated at, the mean runs over the evaluated lengths, and the best value of every column within a benchmark is bold.

## E Base Model Evaluation

### E.1 Baseline Models

Every model column of Tables 17 and 28 is served with vLLM (Kwon et al. 2023) on one node of eight NVIDIA B200 GPUs, and Table 48 lists each model's checkpoint and deployment. Parallelism is given as tensor parallel (TP) and data parallel (DP) ranks, with EP marking expert-parallel MoE layers; the window is the served context length. Sampling is given as temperature over top-p; “default” takes the sampling parameters of the checkpoint’s generation config. SWE-Bench Verified additionally runs on a dedicated single-replica engine per model.

| Model                           | Checkpoint                                                         | Reference                     | Window | Parallelism | Sampling   | Notes                                          |
|---------------------------------|--------------------------------------------------------------------|-------------------------------|--------|-------------|------------|------------------------------------------------|
| <b>Pre-training table</b>       |                                                                    |                               |        |             |            |                                                |
| Kolibri Base                    | –                                                                  | this report                   | 260k   | TP2 DP4     | 0.6 / 0.6  |                                                |
| Kolibri Origin Base             | –                                                                  | this report                   | 64k    | TP1 DP8     | 0.6 / 0.6  |                                                |
| Apertus 70B Base                | <a href="#">swiss-ai/Apertus-70B-2509</a>                          | (Project Apertus et al. 2025) | 64k    | TP2 DP4     | 0.6 / 0.6  |                                                |
| OLMo 3 7B Base                  | <a href="#">allenai/Olmo-3-1025-7B</a>                             | (Ettinger et al. 2025)        | 64k    | TP1 DP8     | 0.6 / 0.6  |                                                |
| OLMo 3 32B Base                 | <a href="#">allenai/Olmo-3-1125-32B</a>                            | (Ettinger et al. 2025)        | 64k    | TP1 DP8     | 0.6 / 0.6  |                                                |
| Nemotron 3 Nano 30B-A3B Base    | <a href="#">nvidia/NVIDIA-Nemotron-3-Nano-30B-A3B-Base-BF16</a>    | (NVIDIA 2025a)                | 64k    | TP1 DP8 EP  | 0.6 / 0.6  |                                                |
| Nemotron 3 Super 120B-A12B Base | <a href="#">nvidia/NVIDIA-Nemotron-3-Super-120B-A12B-Base-BF16</a> | (NVIDIA 2026a)                | 64k    | TP2 DP4     | 0.6 / 0.6  |                                                |
| Qwen3.5 35B-A3B Base            | <a href="#">Qwen/Qwen3.5-35B-A3B-Base</a>                          | (Qwen Team 2026a)             | 64k    | TP1 DP8 EP  | 0.6 / 0.6  | static YaRN factor 4                           |
| GLM-4.5 Air 106B-A12B Base      | <a href="#">zai-org/GLM-4.5-Air-Base</a>                           | (GLM-4.5 Team 2025)           | 64k    | TP2 DP4 EP  | 0.6 / 0.6  |                                                |
| Gemma 4 26B-A4B Base            | <a href="#">google/gemma-4-26B-A4B</a>                             | (Gemma Team 2026)             | 64k    | TP1 DP8     | 1.0 / 0.95 |                                                |
| <b>Post-training table</b>      |                                                                    |                               |        |             |            |                                                |
| Kolibri                         | –                                                                  | this report                   | 256k   | TP1 DP8     | default    | reasoning effort high                          |
| Kolibri Origin                  | –                                                                  | this report                   | 64k    | TP1 DP8     | 0.6 / 0.95 |                                                |
| Qwen3.6 35B-A3B                 | <a href="#">Qwen/Qwen3.6-35B-A3B</a>                               | (Qwen Team 2026b)             | 256k   | TP1 DP8 EP  | default    |                                                |
| Qwen3-Next 80B-A3B Thinking     | <a href="#">Qwen/Qwen3-Next-80B-A3B-Thinking</a>                   | (Qwen Team 2025b)             | 256k   | TP1 DP8 EP  | default    |                                                |
| Nemotron 3 Super 120B-A12B      | <a href="#">nvidia/NVIDIA-Nemotron-3-Super-120B-A12B-BF16</a>      | (NVIDIA 2026a)                | 256k   | TP2 DP4     | default    |                                                |
| Nemotron 3 Nano 30B-A3B         | <a href="#">nvidia/NVIDIA-Nemotron-3-Nano-30B-A3B-BF16</a>         | (NVIDIA 2025a)                | 256k   | TP1 DP8 EP  | default    |                                                |
| Qwen3.8 27B                     | <a href="#">Qwen/Qwen3.8-27B</a>                                   | (Qwen Team 2026c)             | 256k   | TP1 DP8     | default    |                                                |
| Mistral Small 4 119B-A6B        | <a href="#">mistralai/Mistral-Small-4-119B-2603</a>                | (Mistral AI 2026)             | 256k   | TP2 DP4     | default    | FP8 checkpoint; reasoning effort high          |
| Gemma 4 26B-A4B IT              | <a href="#">google/gemma-4-26B-A4B-it</a>                          | (Gemma Team 2026)             | 256k   | TP1 DP8 EP  | 1.0 / 0.95 | thinking enabled; text only                    |
| GPT-OSS 120B                    | <a href="#">openai/gpt-oss-120b</a>                                | (OpenAI 2025)                 | 128k   | TP8 DP1     | default    | native MXFP4 checkpoint; reasoning effort high |
| Apertus 70B Instruct            | <a href="#">swiss-ai/Apertus-70B-Instruct-2509</a>                 | (Project Apertus et al. 2025) | 64k    | TP2 DP4     | 0.8 / 0.9  | completions capped at 16 384 tokens            |
| Qwen3.5 35B-A3B                 | <a href="#">Qwen/Qwen3.5-35B-A3B</a>                               | (Qwen Team 2026a)             | 256k   | TP1 DP8 EP  | 1.0 / 0.95 |                                                |
| GLM-4.5 Air 106B-A12B           | <a href="#">zai-org/GLM-4.5-Air</a>                                | (GLM-4.5 Team 2025)           | 128k   | TP4 DP2 EP  | 0.6 / 1.0  |                                                |
| GLM-4.7 Flash 30B-A3B           | <a href="#">zai-org/GLM-4.7-Flash</a>                              | (Z.ai 2026a)                  | 198k   | TP1 DP8     | 1.0 / 0.95 |                                                |

Table 48. Checkpoint and deployment of every model of the pre- and post-training benchmark tables. Hugging Face checkpoints are linked. The configurations are read from the release definitions at the commit that rendered the model card. A model without a public reference is internal, and “this report” marks our own checkpoints.

## F “I Don’t Know” Evaluations

This appendix complements Section 2.4 with evaluations of how models distribute probability when they may abstain.

**Motivation.** Kalai et al. (2026) argue that language models hallucinate partly because most benchmarks use binary grading, under which guessing always scores at least as well as abstaining. They propose “open rubric” evaluations that state the scoring in the prompt, with wrong answers penalised by  $L = t/(1 - t)$  points for a correctness threshold  $t$ . Our ternary score (+1 correct, 0 abstain, −1 wrong) is the  $t = 0.5$ ,  $L = 1$  case of that scheme, although we do not state the rubric to the model. The distributional correctness score (DCS) (T. F. Burns 2026) goes further and scores the model’s whole probability distribution over the answer options plus an “I don’t know” (IDK) option:  $DCS := (p_c - p_w)(1 - p_{IDK})$ , where  $p_c$ ,  $p_w$ , and  $p_{IDK}$  are the probability masses placed on correct, wrong, and IDK options, respectively. DCS lies in  $[-1, 1]$  and abstaining scores 0. T. F. Burns (2026) reports that on half of 12 tested benchmarks, every one of the six models tested had negative DCS, so the negative averages below are expected. Confidence-weighted accuracy (C-Accuracy) is  $p_c$  when the argmax option is correct and 0 otherwise. It assumes that base-model probabilities are informative, and Kadavath et al. (2022) found that this largely holds for well-formatted multiple-choice questions. Kumaran et al. (2026) give causal evidence that language models use internal confidence to decide whether to abstain and steered internal confidence signals up or down to change how often models abstained.

**Setup.** We evaluate Kolibri Base, the checkpoints after its pre-training and mid-training stages, and the following baseline models of Section 2.4: Kolibri Origin Base, OLMo 3 7B, OLMo 3 32B, Apertus 70B, Nemotron 3 Nano 30B-A3B, and Qwen3.5 35B-A3B, together with GLM-4.5 Air 106B-A12B and Nemotron 3 Super 120B-A12B. We use 7 English IDK variants. Each task appends an IDK option to an existing multiple-choice task and is scored by log-likelihood. As in the main evaluations, all runs and metrics use eval-framework (Aleph Alpha Research 2026). The source tasks are:

- **Grade-school science.** ARC.
- **Commonsense.** GoldenSwag (a filtered HellaSwag subset), PIQA, and Winogrande.
- **Knowledge and expert questions.** MMLU, MMLU-Pro, and GPQA.

**Results.** Table 49 gives the mean over the 7 tasks. For readability, all scores are the raw metric times 100. Higher scores are better. The largest model, Nemotron 3 Super 120B-A12B, is first on all metrics. Kolibri Base is second on C-Accuracy and DCS. A paired Bayesian bootstrap of the per-question scores gives a posterior probability above 0.9999 that Kolibri Base has the higher average DCS than every model besides Nemotron 3 Super 120B-A12B.

MMLU-Pro gives the clearest single-task result. Kolibri Base scores −20.23, ahead of every comparison model except Nemotron 3 Super 120B-A12B (−11.02). The other models score between −36.33 (GLM-4.5 Air 106B-A12B) and −58.31 (OLMo 3 7B), with Apertus 70B and Kolibri Origin Base also at the bottom (−57.56

| Model                      | Accuracy                           | C-Accuracy                         | Ternary                            | DCS                                 |
|----------------------------|------------------------------------|------------------------------------|------------------------------------|-------------------------------------|
| Kolibri Origin Base        | 37.07 $\pm$ 0.65                   | 22.53 $\pm$ 0.36                   | 27.59 $\pm$ 1.33                   | -24.01 $\pm$ 0.49                   |
| Kolibri (pre-training)     | 41.08 $\pm$ 0.63                   | 26.25 $\pm$ 0.38                   | 36.42 $\pm$ 1.29                   | -19.57 $\pm$ 0.54                   |
| Kolibri (mid-training)     | 42.19 $\pm$ 0.63                   | 27.21 $\pm$ 0.38                   | 38.78 $\pm$ 1.29                   | -18.37 $\pm$ 0.56                   |
| Kolibri Base               | 45.33 $\pm$ 0.61                   | <u>29.84 <math>\pm</math> 0.40</u> | 44.45 $\pm$ 1.24                   | <u>-13.67 <math>\pm</math> 0.59</u> |
| OLMo 3 7B                  | 35.76 $\pm$ 0.65                   | 21.93 $\pm$ 0.35                   | 24.22 $\pm$ 1.35                   | -24.84 $\pm$ 0.48                   |
| OLMo 3 32B                 | 40.03 $\pm$ 0.62                   | 24.96 $\pm$ 0.35                   | 36.87 $\pm$ 1.27                   | -19.84 $\pm$ 0.49                   |
| Apertus 70B                | 37.80 $\pm$ 0.64                   | 24.80 $\pm$ 0.35                   | 33.12 $\pm$ 1.30                   | -23.32 $\pm$ 0.49                   |
| Nemotron 3 Nano 30B-A3B    | 43.37 $\pm$ 0.61                   | 28.56 $\pm$ 0.36                   | 43.31 $\pm$ 1.25                   | -17.71 $\pm$ 0.51                   |
| Qwen3.5 35B-A3B            | <u>45.74 <math>\pm</math> 0.59</u> | 29.58 $\pm$ 0.36                   | <u>47.20 <math>\pm</math> 1.20</u> | -14.53 $\pm$ 0.52                   |
| GLM-4.5 Air 106B-A12B      | 43.93 $\pm$ 0.60                   | 28.92 $\pm$ 0.37                   | 45.72 $\pm$ 1.21                   | -15.20 $\pm$ 0.53                   |
| Nemotron 3 Super 120B-A12B | <b>51.20 <math>\pm</math> 0.57</b> | <b>34.99 <math>\pm</math> 0.39</b> | <b>53.44 <math>\pm</math> 1.14</b> | <b>-7.44 <math>\pm</math> 0.59</b>  |

Table 49. Mean  $\pm$  standard error over the 7 IDK tasks, times 100. In each column, **bold** is best and underline is second.

and -57.99). MMLU-Pro has 10 options instead of MMLU's 4, so it is harder to score well by guessing. DCS adds a penalty for probability left on the wrong options, and Kolibri Base moves more of that probability onto IDK or the correct option than any other model tested except Nemotron 3 Super.

## G Reasoning Mode at Long Context

We evaluate the post-trained Kolibri checkpoint on the RULER tasks of Section 2.3.4.3 with reasoning on and off (Figure 65). Reasoning off raises needle-in-a-haystack recall, while reasoning on scores higher on question answering at the longest context lengths. The Qwen3 technical report also finds lower RULER scores in thinking mode and suggests that extra reasoning can interfere with retrieval (Qwen Team 2025a). The reasoning-on run allows up to 16k generated tokens and the reasoning-off run up to 1k, so this comparison does not isolate the effect of reasoning alone. Nemotron 3 Nano is evaluated with thinking off, as recommended for this task. At 1M tokens, Kolibri with reasoning off scores above Nemotron 3 Nano on variable tracking and question answering, but below it on needle-in-a-haystack and word extraction.

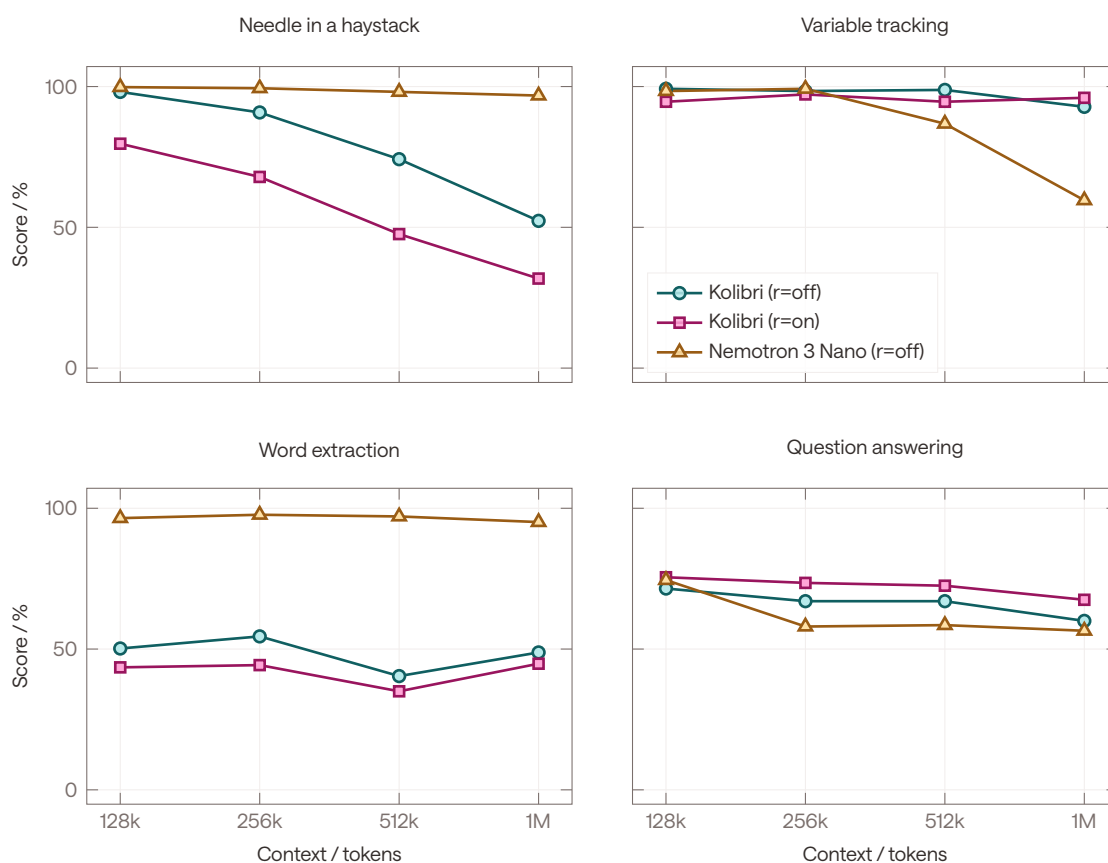


Figure 65. RULER scores of the post-trained Kolibri checkpoint with reasoning off and on, against Nemotron 3 Nano evaluated with thinking off. Kolibri is prompted through its chat template. Reasoning off allows up to 1k generated tokens and reasoning on up to 16k.

## H Supervised Fine-Tuning (SFT)

### H.1 Long-Context SFT Data Ablation

To test whether our long-context data improve reasoning over long documents, we fine-tune the proxy model with the ablation recipe (Appendix H.4) with and without them (Table 50). Variable tracking gains significantly. The gain of AA-LCR and LongBench Pro fall just short of significance, while LongBench Pro gains at every length and more at longer contexts. Single-passage and single-key retrieval change little.

| Benchmark                         | Base        | $\pm$ | Long context mix | $\pm$ | $\Delta$ |
|-----------------------------------|-------------|-------|------------------|-------|----------|
| <i>Long context, %</i>            |             |       |                  |       |          |
| RULER QA 128k (substring EM)      | 51.5        | 6.9   | <b>53.0</b>      | 6.9   | +1.5     |
| RULER VT 128k (recall)            | 17.6        | 6.8   | <b>32.8</b>      | 9.0   | +15.2*   |
| HELMET JsonKV 128k (substring EM) | <b>22.0</b> | 8.2   | 21.0             | 8.0   | -1.0     |
| AA-LCR (accuracy)                 | 25.0        | 8.5   | <b>33.0</b>      | 9.3   | +8.0     |
| <i>LongBench Pro EN, %</i>        |             |       |                  |       |          |
| Overall (8k-128k)                 | 30.2        | 3.2   | <b>33.5</b>      | 3.3   | +3.2     |
| 8k                                | 38.6        | 7.8   | <b>39.4</b>      | 7.6   | +0.7     |
| 16k                               | 34.0        | 7.2   | <b>34.8</b>      | 7.4   | +0.9     |
| 32k                               | 30.0        | 7.2   | <b>33.6</b>      | 7.5   | +3.7     |
| 64k                               | 23.2        | 6.8   | <b>28.8</b>      | 7.2   | +5.6     |
| 128k                              | 25.3        | 6.9   | <b>30.7</b>      | 7.4   | +5.4     |

Table 50. Effect of the long-context SFT data on the proxy model with long-context extension, trained for 1000 steps (16.8B tokens). The long-context mix adds a 5 % share of the synthesised long-context conversations. We evaluate on the long-context benchmarks of pre-training and post-training (Tables 18 and 27), on a subset of the RULER and HELMET tasks. Single runs;  $\pm$  is the half-width of the 95 % confidence interval, and \* marks  $p < 0.05$  in a paired test.

### H.2 Impact of German Reasoning Data

To measure how much German SFT data the model needs to reason in German, we fine-tune the Kolibri base model with the ablation recipe of Table 54 on mixes that upsample all German data by the same factor (Table 51).

| Run                                 | German share (%) |      | AIME'26 DE (%) |              |                  |              |               |              | AIME'26 EN (%) |              |
|-------------------------------------|------------------|------|----------------|--------------|------------------|--------------|---------------|--------------|----------------|--------------|
|                                     | all              | math | score          |              | correct & German |              | loops         |              | score          |              |
|                                     |                  |      | mean           | 95 % CI      | mean             | 95 % CI      | mean          | 95 % CI      | mean           | 95 % CI      |
| de-0                                | 0.0              | 0.0  | <b>70.2**</b>  | [60.5, 79.9] | 42.3*            | [34.2, 50.3] | <b>2.7***</b> | [1.0, 4.4]   | 64.0           | [53.5, 74.4] |
| de-1                                | 2.2              | 0.4  | 57.1           | [44.8, 69.4] | 57.1             | [44.8, 69.4] | 22.5          | [13.3, 31.7] | 71.3           | [61.6, 80.9] |
| de-4                                | 8.2              | 1.6  | 59.0           | [45.3, 72.6] | 59.0             | [45.3, 72.6] | 18.3          | [10.3, 26.4] | <b>78.1</b>    | [67.9, 88.3] |
| de-16                               | 26.5             | 5.0  | 65.8**         | [54.6, 77.0] | <b>65.8***</b>   | [54.6, 77.0] | 17.9*         | [9.4, 26.4]  | 76.0           | [66.2, 85.9] |
| <i>Upsampled vs. no German data</i> |                  |      |                |              |                  |              |               |              |                |              |
| de-16 — de-0                        |                  |      | -4.4           | [-10.6, 1.8] | +23.5***         | [14.8, 32.3] | +15.2**       | [6.8, 23.6]  | +12.1**        | [5.6, 18.5]  |

Table 51. Effect of the German share in the SFT mix on AIME'26 (avg@16), with 95 % Confidence Intervals over the 30 problems. Runs sample every German group  $\times 0$ ,  $\times 1$ ,  $\times 4$  or  $\times 16$ . Correct & German counts rollouts that are correct and in German, and Loops rollouts that end in a repetition loop. The last row is the paired difference of de-16 and de-0. Stars mark Holm-corrected paired  $t$ -tests against de-1, or against zero in the last row: \*  $p < 0.05$ , \*\*  $p < 0.01$ , \*\*\*  $p < 0.001$ .

### H.3 SFT MergeMix Sweep

Following J. Wang et al. (2026, Section 4.2), we compare merged and trained mixes on math and code before the full sweep. We train a math, a code and a chat specialist from the ablation base model of Appendix D.2.2, each for 20.0B packed tokens. They train on all math, all code and all general chat datasets of the SFT mix, each dataset sampled in proportion to its unique tokens (natural sampling, Section 2.3.3.3). A weighting gives each specialist a share, and the shares sum to one. For every weighting, we evaluate its merge, the specialists averaged with these shares, and a model trained for the same budget on data sampled with the same shares. We use two series of weightings. In the first, the math share  $\alpha$  runs from 0 to 1 in steps of 0.2, and code takes the rest. In the second, chat has a share of  $\frac{1}{3}$ , and math and code split the remaining  $\frac{2}{3}$  in the ratio  $\alpha$  to  $1 - \alpha$ . The single specialists at the ends of the first series are the same model either way, so the first series compares 4 weightings and the second 6. For each benchmark<sup>10</sup> we compute the Spearman correlation between merged and trained scores in each series (Table 52 and Figure 66). Merging and training order the weightings alike on AIME 2026 and LiveCodeBench. They disagree on MATH-500 (0.32 and 0.26), where every weighting scores between 84 and 91%. On this benchmark, the scores barely move with the weighting, so neither ordering carries information.

| Benchmark     | $\rho$ , no chat | $\rho$ , $\frac{1}{3}$ chat | Spread, no chat | Spread, $\frac{1}{3}$ chat | Min  | Max  |
|---------------|------------------|-----------------------------|-----------------|----------------------------|------|------|
| AIME 2026     | 0.80             | 0.77                        | 7.9             | 14.8                       | 32.1 | 54.4 |
| LiveCodeBench | 0.80             | 0.94                        | 8.1             | 20.0                       | 23.1 | 46.5 |
| MATH-500      | 0.32             | 0.26                        | 4.4             | 5.6                        | 84.0 | 91.2 |

Table 52. Agreement between merging and data mixing on math and code.  $\rho$  is the Spearman correlation between merged and trained scores, over the 4 weightings of the series without the chat specialist and the 6 of the series with it. Spread is the range of the trained scores in a series, in points; Min and Max bound all merged and trained scores, in percent.

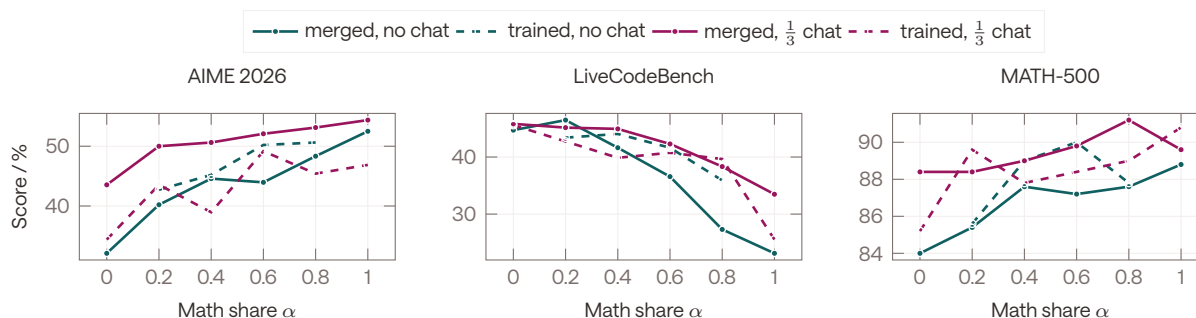


Figure 66. Scores of the merged (solid) and trained (dashed) weightings in the two math and code series, over the math share  $\alpha$  of the math and code part.

Table 53 gives the cluster shares of every weighting trained as a data mix. Within a cluster, the datasets keep their proportions in the reference mix.

<sup>10</sup>The benchmarks are AIME 2026 (Mathematical Association of America 2026), LiveCodeBench (Jain et al. 2024) and MATH-500 (Lightman et al. 2023).

| Cluster                             | Baseline weighting | $\kappa = 50$<br>draw 15 | Refinement<br>draw 13 | Refinement<br>draw 7 | $\kappa = 15$<br>draw 6 | Refinement<br>draw 5 | Gradient-boosted trees | GP   |
|-------------------------------------|--------------------|--------------------------|-----------------------|----------------------|-------------------------|----------------------|------------------------|------|
| Math I                              | 9.8                | 8.2                      | 0.7                   | 3.2                  | 15.6                    | 1.6                  | –                      | –    |
| Math II                             | 6.9                | 4.7                      | 3.4                   | –                    | 1.1                     | 1.6                  | –                      | –    |
| Math III                            | 2.3                | 2.4                      | –                     | 2.7                  | –                       | –                    | –                      | –    |
| Math proofs                         | 1.7                | 1.9                      | –                     | –                    | –                       | –                    | –                      | –    |
| Code I                              | 6.0                | 7.2                      | 4.2                   | 2.1                  | –                       | –                    | –                      | –    |
| Code II                             | 3.4                | –                        | 1.1                   | –                    | 1.9                     | 2.9                  | –                      | –    |
| Software engineering I              | 4.6                | 7.9                      | 11.3                  | 2.2                  | 2.5                     | 3.7                  | –                      | 2.8  |
| Software engineering II             | 2.1                | 3.4                      | –                     | 1.3                  | –                       | –                    | 1.0                    | 2.6  |
| Science I                           | 2.8                | 6.1                      | 13.1                  | 11.7                 | 13.3                    | 15.6                 | 15.4                   | 11.1 |
| Science II                          | 3.6                | 1.1                      | –                     | 7.2                  | 1.3                     | –                    | 9.4                    | 11.6 |
| General chat                        | 1.9                | 1.0                      | –                     | –                    | –                       | 1.1                  | 2.9                    | 5.8  |
| Instruction following I             | 1.6                | 1.3                      | 3.6                   | –                    | –                       | –                    | 5.1                    | 5.3  |
| Instruction following II and safety | 2.1                | 3.2                      | –                     | 0.7                  | 1.1                     | –                    | 6.5                    | 6.8  |
| Tool calling                        | 1.7                | 2.9                      | 7.3                   | 4.7                  | 4.2                     | 8.3                  | 10.8                   | 4.3  |
| Interactive agents                  | 2.3                | 2.0                      | 13.3                  | 15.2                 | –                       | 10.6                 | 2.6                    | 2.0  |
| Terminal I                          | 1.8                | 7.8                      | 1.6                   | 3.2                  | 0.9                     | 4.8                  | 7.1                    | 9.0  |
| Terminal II                         | 2.0                | 2.5                      | 0.7                   | –                    | 3.5                     | –                    | –                      | 4.2  |
| Long context                        | 20.4               | 9.4                      | 9.5                   | 24.7                 | 23.8                    | 37.8                 | 16.1                   | 26.1 |
| German I                            | 12.2               | 8.2                      | –                     | –                    | 11.4                    | –                    | –                      | 5.7  |
| German II                           | 10.9               | 18.7                     | 30.2                  | 20.9                 | 19.2                    | 12.0                 | 22.9                   | 2.6  |

Table 53. Cluster shares of the weightings trained as data mixes (Table 22), in percent of the packed tokens, with the weightings in the order of that table. A dash marks a cluster the sampler pruned below 0.5%.

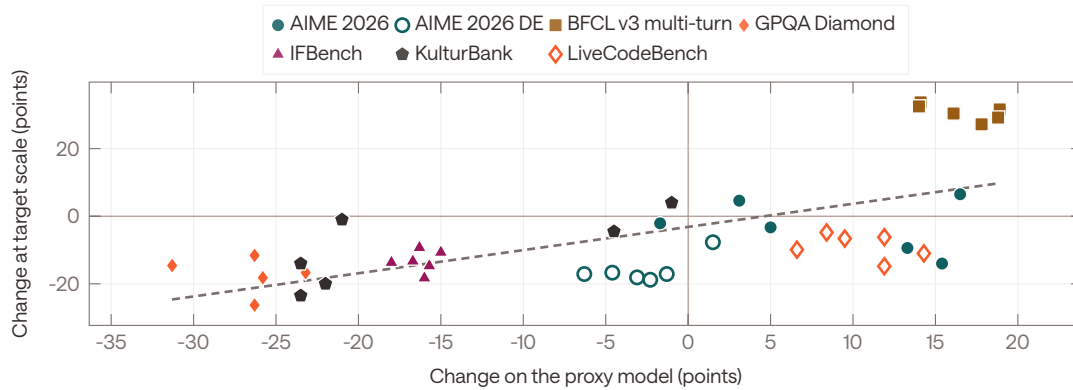
## H.4 SFT Ablation Recipes

Ablations train with a shorter recipe than the Kolibri SFT runs, from either the Kolibri base model at target scale or the mid-trained proxy model (Table 54).

| Setting                | Value                                                                                                  |
|------------------------|--------------------------------------------------------------------------------------------------------|
| Base checkpoint        | Kolibri base model or mid-trained proxy model (Appendix D.2.2)                                         |
| Peak learning rate     | 0.3 (base model) or 0.5 (proxy) of pre-training in every group                                         |
| Context length         | 256k                                                                                                   |
| Batch size             | 64 sequences, 16.8M tokens per step                                                                    |
| Steps                  | 1000, 16.8B training tokens                                                                            |
| Learning-rate schedule | Warmup-Stable-Decay, warmup over 50 steps, linear decay over the last 200 steps to a tenth of the peak |
| Parallelism            | FSDP over 64 (base model) or 32 (proxy) GPUs                                                           |
| Load balancing         | Disabled (Section 2.1.5)                                                                               |

Table 54. Training settings of the SFT ablation recipe, for the two base checkpoints it starts from. A step covers a quarter of the tokens of a step of the Kolibri runs (Table 23), and the recipe trains for a quarter of their steps.

To test whether SFT ablations on the proxy model carry over to target scale, we train 7 SFT configurations on both models, each with the identical dataset mix at both scales. The configurations differ in the data mix, the assignment of reasoning effort and the loss masking. The proxy predicts the direction in which a change of the configuration moves a benchmark at target scale in 33 of 45 cases, but not the ranking of configurations by mean score (Figure 67).



| Environment                                          | Train rows | Steps | Tokens  | Weight  |
|------------------------------------------------------|------------|-------|---------|---------|
| <i>Math and Reasoning</i>                            |            |       |         |         |
| math                                                 | 107 064    | 1     | 131 072 | 0.10497 |
| math (de)                                            | 105 514    | 1     | 131 072 | 0.00995 |
| reasoning                                            | 29 191     | 1     | 65 536  | 0.04020 |
| math proofs                                          | 8 638      | 1     | –       | 0.00677 |
| <i>Code Generation</i>                               |            |       |         |         |
| code                                                 | 10 601     | 1     | 131 072 | 0.10268 |
| code (de)                                            | 11 815     | 1     | 131 072 | 0.01493 |
| <i>Software Engineering and Terminal Use</i>         |            |       |         |         |
| SWE                                                  | 184 886    | 100   | 131 072 | 0.22805 |
| terminal use                                         | 247 193    | 100   | 131 072 | 0.09950 |
| <i>Tool Use</i>                                      |            |       |         |         |
| customer service                                     | 7 353      | 200   | 10 000  | 0.02607 |
| enterprise assistant                                 | 81 815     | 20    | 32 768  | 0.04339 |
| enterprise assistant (de)                            | 5 215      | 20    | 32 768  | 0.01990 |
| <i>Retrieval and Long-Context Question Answering</i> |            |       |         |         |
| retrieval                                            | 16 594     | 100   | 32 768  | 0.03771 |
| long context                                         | 1 221      | 1     | 10 000  | 0.00677 |
| <i>Instruction Following and Structured Output</i>   |            |       |         |         |
| instruction following                                | 88 761     | 1     | 10 000  | 0.14398 |
| <i>Identity and Safety</i>                           |            |       |         |         |
| model identity                                       | 3 012      | 1     | 2 000   | 0.00667 |
| copyright                                            | 1 011      | 1     | 32 768  | 0.00298 |
| copyright (de)                                       | 962        | 1     | 32 768  | 0.00298 |
| safety refusal                                       | 4 520      | 1     | 32 768  | 0.00100 |
| political balance                                    | 6 266      | 1     | –       | 0.00498 |
| <i>Hallucination Reduction</i>                       |            |       |         |         |
| hallucination reduction                              | 288 333    | 1     | –       | 0.00500 |

Table 55. Environments of the RL run with the number of rows in the train split, the maximum number of steps per sequence, the completion-token budget per sequence, and their sampling weight, grouped as in Figure 32. Budgets come in five tiers, 2 000, 10 000, 32 768, 65 536 and 131 072 tokens, set per environment for B300 nodes; on A100 and H100 nodes every budget above 32 768 is capped at 32 768 to fit their shorter context. Environments with a dash have no completion-token budget and are only bound by the context limit of 256k tokens. The weights are the sampling weights of the Kolibri RL run.

# I Reinforcement Learning

## I.1 Environments

### I.1.1 Reward Structure by Environment

**Code Generation** The extracted program runs in a sandbox against every test case of the task with a 20 s timeout per test. Its output is compared with the expected output, with a tolerance for floating-point numbers and case-insensitive yes/no answers, so a solution is not failed for whitespace or formatting alone. The reward is the fraction of test cases passed.

**Software-Engineering Agents** The policy works in a per-task container through a coding harness with a shell tool, and its changes are extracted as a candidate patch. The patch is applied in a fresh container and the task's tests are run: the reward is the fraction of passing fail-to-pass tests floored by pass-to-pass tests regression. Sequences are capped at 100 steps.

**Software-Engineering Repair** The search-and-replace edits are compared with the golden patch by the judge model, without executing tests; the reward is the judge's score, and a non-empty solution block is the *Solution block* signal.

**Terminal Tasks** Terminal tasks are graded by the corpus's own test script, which runs 'pytest' over the final machine state and writes a test report; the reward is the fraction of tests that pass.

**Customer-Service Environment** The reward compares the final database state with the expected one, checks the required actions and judges natural-language assertions about the conversation.

**Retrieval-Augmented Question Answering** The policy searches a document corpus with the tools of Table 56 and gives its final answer in the format that the prompt sets out. The reward is the best token F1 of that answer against the gold answers, and a sequence that never calls a search tool scores zero. The *Boxed answer*, *Answer format* and *Citations* signals of Table 24 score the form of the answer. Appendix I.1.4 gives the step caps and further details of the environment.

All environments have a per-environment token and step-budgets with token exhaustion scoring zero for tool use environments and triggering grading for SWE environments.

## I.1.2 Long Context Environment

For every document, GLM-5.2 (Z.ai 2026b) writes two questions with six options, one to four of them correct. A seeded plan per document fixes which part of the document each question draws on (beginning, middle or several distant passages) and how many options are correct, so the distribution of question shapes is not left to the generator. A question has to combine facts from several passages, may not be answerable from the conversation turns alone, and uses true statements from elsewhere in the document as wrong options.

Each question is answered eight times by a Kolibri checkpoint at low reasoning effort and eight times by GLM-5.2, and enters the training set only if GLM-5.2 solved it at least once and Kolibri at most six times; where Kolibri's attempts agree on a different answer or solve it at most once, Kimi K2.6 (Kimi Team 2026c) rules on the disputed options without seeing the key. The reward is 1 only when the letters on the final `Answer :` line are exactly the correct set, and an empty response scores 0 instead of being dropped, so reasoning past the token budget on a long document is penalised.

### I.1.3 Hallucination Reduction Environment

The hallucination reduction environments implement a Merlin–Arthur protocol (Deiseroth et al. 2026) on various RAG training datasets. Arthur is the model that we train, i.e., the policy  $\pi_\theta$ , and Merlin and Morgana are masking algorithms. For a seed with context  $x$ , question  $q$ , and gold answer  $a$ , they choose masks  $m \in \{0, 1\}^C$  over the  $C$  maskable units (sentences) of  $x$ . The masks are ensured to hide similar amounts of context with some tolerance. The masks optimise the following objective, where  $x \odot m$  denotes a context  $x$  in which parts have been masked out by  $m$ :

$$\hat{m}_{\text{Mer}}, \hat{m}_{\text{Mor}} = \arg \max_{m_{\text{Mer}}, m_{\text{Mor}}} \log \pi_\theta(a \mid x \odot m_{\text{Mer}}, q) + \log(1 - \pi_\theta(a \mid x \odot m_{\text{Mor}}, q)), \quad (25)$$

such that  $|k_{\text{Mer}} - k_{\text{Mor}}| \leq \delta$ , where  $k = |m|$  is the number of units a mask hides. Moreover, each mask hides at least one unit and leaves at least one visible, and the two masked contexts must differ. We approximate both masks by first ranking the sentences by how much masking each one individually changes the log-probability of  $a$ , teacher-forced on the same inference engine that produces the rollouts, and then removing the top  $k$ . During training, we recompute the masks from the policy at every step, so Morgana follows the features that make Arthur prone to hallucinations; no second model or human annotation is involved in generating these abstention examples. Since Merlin and Morgana hide the same number of sentences within an adjustable tolerance, Arthur cannot tell them apart by how much is missing.

Arthur’s training relies on rewards obtained from completions on the question  $q$  paired with Merlin, Morgana, and full (unmasked) contexts. A completion  $y$  receives a reward  $R(y)$  that rewards Arthur for doing what its role asks and penalises a wrong answer on the full context and Merlin’s:

$$R(y) \approx \begin{cases} 1 & \text{if } y \text{ answers } a \text{ on the full context or Merlin's,} \\ -1 & \text{if } y \text{ gives another answer on the full context or Merlin's,} \\ 1 & \text{if } y \text{ abstains on Morgana's context,} \\ 0 & \text{otherwise.} \end{cases}$$

We additionally influence these values by small terms for token overlap with  $a$  and for abstaining on the full context or Merlin’s. An answer counts as correct on an exact match with the gold answer or one of its aliases; otherwise, an LLM judge model decides whether it matches and whether it abstains.

Each of the three versions of a seed is rolled out 8 times, and each role is centred on its own mean; the advantage of rollout  $j$  is computed as

$$A_j \propto R(y_j) - \bar{R}_{\text{role}(j)},$$

where  $\bar{R}_{\text{role}(j)}$  is the mean training reward of the rollouts of its role scaled by a fixed weight per role and by the quality of the masks (for Merlin and Morgana only). Arthur cannot gain advantage over correct answers by abstaining, since answers (and not abstentions) are expected on the full context or on Merlin’s. A separate environment poses unanswerable questions with their full context and rewards only abstention.

The Merlin–Arthur protocol also gives us a hallucination measurement. It measures how often the model answers correctly on Merlin’s context is its *completeness*, and how often it refuses on Morgana’s is its *soundness*. Together, they yield our **M/A grounding score**, a lower bound on how much of the answer

| Tool                            | Description                                                                                            |
|---------------------------------|--------------------------------------------------------------------------------------------------------|
| <i>Search tools</i>             |                                                                                                        |
| <code>vector_search</code>      | Dense embedding search; returns the top- $k$ chunks with score, document id, title and text.           |
| <code>lexical_search</code>     | BM25 keyword search over the same chunks.                                                              |
| <code>hybrid_search</code>      | Merges the dense and lexical results by reciprocal-rank fusion.                                        |
| <code>grep</code>               | Regular-expression search like <code>grep -n</code> over the documents BM25 retrieves for the pattern. |
| <i>Filter tools</i>             |                                                                                                        |
| <code>list_filter_values</code> | Lists the values of a filter column with their document counts, most common first.                     |
| <code>find_documents</code>     | Finds documents by exact filter values without a query; returns their ids, titles and filter values.   |
| <i>Document access tools</i>    |                                                                                                        |
| <code>open_document</code>      | Reads a document by id, in pieces with an offset or whole up to 8000 or 16 000 characters.             |
| <code>open_pages</code>         | Reads up to four fixed-size pages (2000 or 4000 characters) of a document by filename.                 |
| <code>open_window</code>        | Reads 2000 or 4000 characters centred on a character position from a search hit.                       |
| <code>read_lines</code>         | Reads numbered lines from a line given by a search hit, 50 by default and 200 at most.                 |

Table 56. Tools of the retrieval environments, grouped by role. The filter tools are served only when the corpus declares filter columns, and `hybrid_search` and `grep` need a BM25 index over the document chunks.

provably came from the document. The lower bound makes the grounding score conservative: zero means that it cannot be certified that the document was used, not that it went unused. More details on the grounding score and the Merlin–Arthur protocol can be found in our paper (Deiseroth et al. 2026).

## I.1.4 Retrieval-Augmented Question Answering

Many customer applications answer questions from documents within the domain of their organisation, so the model has to search a corpus over several steps with the specific tools the application offers and answer from the retrieved text. Two environments train this behaviour, and both are part of the RL mix (Section 3.2.2.3) and provide the rollouts that cold-start SFT (Section 3.1.1.4). `retrieval` poses multi-hop questions over English data, `retrieval-de` German questions over German data and `retrieval-honeypot`, referred to as Honeypot, poses synthetic multi-hop questions over specialised corpora. This section describes the task, the tools they use, the question sets, the randomisation applied to each episode, the infrastructure that runs the environments during training, and the evaluation and results.

**Task and Tools** In each episode, the policy receives a question and a set of tools, searches the corpus, reads the documents it finds and gives a final answer in the format that the prompt specifies. The answer is graded by F1 score based on word-overlap with the gold answer and its aliases, together with the format signals of Appendix I.1.1. Table 56 lists the tools, which fall into three groups: search tools that return passages, filter tools that restrict a search to documents with given metadata, and document access tools that read a document once a search has found it. The retrieval tools take a query, a result count  $k$  (unless the session fixes it) and an optional `filters` argument that restricts them to documents matching given filter-column values; `grep` takes a regular expression and a match limit. Sequences are capped at a maximum of 100 steps in `retrieval` and `retrieval-de`, and 50 steps in `retrieval-honeypot`.

**Factual Questions** The questions of `retrieval` are taken from the training split of an multi-hop question-answering dataset. They are answered over an English pool, served through one of several index bundles that differ in their Extract, Transform, Load (ETL) parameters: the embedding model and chunk size for vector search, and the BM25 parameters for lexical search. Before training, we apply Kimi-K2.7 (Kimi Team 2026d) to roll out every question in the environment, and these rollouts drive a filtering and repair pass. A question is dropped when one of its supporting articles is missing from the corpus, when Gemma-4-31B (Gemma Team 2026) answers it correctly without retrieval, or when an LLM judge labels it malformed, ambiguous or unsupported on the basis of its rollout. Where the judge considers a question repairable, we apply GLM-5.2 (Z.ai 2026b) to rewrite an outdated gold answer or an underspecified question, and again use GLM-5.2 to check the rewritten question and answer. The training split is restricted to an allow-list of questions that are not in the SFT pipeline, and the test split is excluded from training in both SFT and RL; Section 3.1.1.4 describes the rollouts used in SFT. `retrieval-de` questions are translated and filtered versions of the English questions, processed by the same pipeline and answered over a German pool.

**Honeypot Questions** Honeypot covers nine specialised corpora (industrial product documentation, aerospace certification, financial regulation, EU and German law, scientific reports and economic development publications) that broadly cover our customer’s related domains of interest, and every question is served over its own corpus. Its questions are generated following the Context-1 recipe (Bashir et al. 2026). For each corpus, seed phrases taken mostly from document titles, supplemented by a hand-written list, are issued to the search tools, facts are extracted from the retrieved passages, and a question is generated for each fact, with that fact as its answer. Each fact is then located at further positions in the corpus, and new facts from the surrounding text give follow-up questions in the same way. Chaining an initial question with its follow-ups gives a multi-hop task in which answering hop  $k$  requires the answer to hop  $k - 1$ . A hop is kept only if a search for its clue together with the previous answer finds the target document and a search for the clue alone does not, so that each hop depends on the one before. Each corpus also declares document-level filter columns, and the search tools accept a filter on them, so that every retrieved document satisfies it. The collection holds training and test questions over all nine corpora, and none of the training questions entered the SFT pipeline.

**Randomisation per Episode** A deployed retrieval application fixes its tools, its prompt and its index, and these choices differ between applications. To keep the model independent of any one setup, each episode draws its own configuration. It serves one or more of the search tools and a single document access tool over one of the available index bundles of the corpus. Its prompt preamble comes from a pre-configured set and fixes the answer format, the answer contents, whether tool use is required and the step budget. Each session also samples a name, parameter names and a result format for every search tool, and the name of the document-id parameter (`document_id`, `doc_id` or `ref`). The document access tool is always `read_lines` when `grep` is served, and search results carry whichever address the access tool takes: a document id, a filename and page, a character position or a line number.

**Infrastructure** The retrieval tools in production applications rely on a set of services: a tool-calling server, an embedding-model server, a relational database, a vector-search engine and a lexical search index.

| Name                        | Corpus                                                                                                        | Tasks                                                                                                                                                                                                                    |
|-----------------------------|---------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Semiconductors              | Manufacturer manuals and datasheets for microcontrollers, power-management chips and transceivers.            | Engineering questions on the documented chips, with reference answers. The set mixes questions on parameter values, feature availability, errata, procedures and design advice, and each question names its chip family. |
| German Public Sector        | German public-administration documents: laws, legal documents, IT-governance decisions and reports.           | German fact questions on the documents, with reference answers. Each answer is one fact from one document group, such as an amount, a count or a legal obligation.                                                       |
| Aerospace                   | Public aviation regulations and accident reports, mixed with energy, economy and news documents.              | German questions on aviation regulation, certification, flight-simulator standards and accident investigations, with reference answers.                                                                                  |
| Automotive Supplier         | Synthetic product-development problem reports with tracker metadata, such as status, severity and discipline. | Questions on the reports and their metadata, with reference answers. Most questions filter reports by fields such as status, severity, author or date, and others search report content or count reports.                |
| Industrial Drive Technology | Manufacturer product documentation for motors, brakes, gear units and drive electronics.                      | German service questions on the documented products, with reference answers. The set mixes product-information and troubleshooting questions, and some questions give a serial number instead of the product name.       |

Table 57. Corpus and task sources of the five Industry RAG benchmarks, in-house proxies for deployed customer assistants.

The environment mimics the production scenario by offering the policy the same tools, but simplifies their implementation to reduce the burden on the training infrastructure. A tool handler inside the rollout process serves the tools of Table 56, with the filter tools only where the corpus declares filter columns, and dispatches each tool call as a direct function call on a retrieval engine embedded within the process. Therefore, the only traffic leaving the process is generation requests to the policy's inference endpoint.

For vector search, we use a sentence-`transformers` model that runs on the CPU inside the environment process as the query encoder. An asynchronous batcher coalesces the queries that parallel rollouts send at the same time into batched encodes on a small pool of search-worker threads. We use small embedding models (fewer than 1B parameters) which make CPU-based deployments feasible at sufficient throughput. Lexical search uses a memory-mapped `bm25s` index over the same chunks as vector search.

**Evaluation on Industry RAG Benchmarks** To check that agentic RAG performance generalises beyond MuSiQue and HoneyPot, we built the five Industry RAG benchmarks of Table 57. They are in-house benchmarks, not publicly available, and each is a proxy for a deployed customer assistant in a public or industrial use case; their results are the Industry RAG rows of Table 29. The benchmarks share no documents or questions with the SFT and RL data, although some HoneyPot corpora come from the same domains. Gains on these benchmarks therefore reflect how well the training and the randomisation of tools, prompts and index bundles transfer to out-of-distribution real-world corpora and questions. Their scores over the training runs of September 2026 are shown in Figure 44.

## I.2 Harness Tools

Some of the coding harnesses that are used have dependencies that might need to be installed at runtime. The Terminus 2 harness of Terminal-Bench (Merrill et al. 2026) drives the agent through a `tmux` session inside the sandbox, and most task images do not ship `tmux`. Installing it at rollout time would need network access in every sandbox and would repeat the installation thousands of times per optimiser step. The executor therefore adds a statically linked `tmux` binary as a read-only bottom layer of every sandbox’s OverlayFS union (Section 3.2.3.5), so every sandbox sees `tmux` without any change to its image. The layer places its files at the paths where images keep their own, so an image that ships `tmux` shadows the layer’s copy and the layer only fills gaps. Static linking keeps the binary independent of the task image’s C library.

## I.3 Per-Token Logit-Gradient Norms

By the chain rule, the derivative of the policy-gradient loss term of Equation (16) splits as

$$\nabla_{\theta} \tilde{\ell}_t^{\text{PG}} = \frac{\partial \tilde{\ell}_t^{\text{PG}}}{\partial \mathbf{z}} \frac{\partial \mathbf{z}}{\partial \theta}$$

i.e., into the loss gradient with respect to the training policy logits  $\frac{\partial \tilde{\ell}_t^{\text{PG}}}{\partial \mathbf{z}}$  and the model’s Jacobian  $\partial \mathbf{z} / \partial \theta$ . All other quantities do not depend on  $\theta$  or are non-differentiable.

### I.3.1 Infinity-Norm of the Logit Gradient

Since applying the B-TV mask truncates the gradient, assume that the sampled token is not masked, i.e., it has  $k_t = 1$ . Then, Equation (16) simplifies to  $\tilde{\ell}_t^{\text{PG}} = -A_t r_t$ . This policy-gradient term is linear in the training policy probability of the sampled token through the importance ratio, allowing us to determine the logit gradient norm in closed form.

**Log-Softmax Jacobian** We define  $\mathbf{p} = \text{softmax}(\mathbf{z}) = \pi_{\theta}(\cdot \mid s_t)$  as the training probabilities with entries  $p_v$  over indices  $v$  of the token vocabulary. Let  $w$  be a different vocabulary index, and  $y_t$  denote the sampled token, i.e.,  $p_{\theta,t} = p_{y_t}$ . Splitting the logit gradient further around the normalised log-probabilities  $\log \mathbf{p}$ , yields a partial derivative of the loss with respect to the log-probabilities, and a Jacobian of the log-softmax that maps the unnormalised logits to the log-probabilities. The derivative of the log-softmax can be expressed as

$$\frac{\partial \log p_v}{\partial z_w} = \delta_{vw} - p_w,$$

where  $\delta_{vw}$  is the Kronecker delta. Since the loss depends on the logits only through the log-probability  $\log p_{\theta,t}$  of the sampled token, we only need the corresponding row  $\nabla_{\mathbf{z}} \log p_{\theta,t}$  of the Jacobian. This row is given as

$$\mathbf{g}_t := \nabla_{\mathbf{z}} \log p_{\theta,t} = \mathbf{u}_t - \mathbf{p}, \quad g_{t,v} = \begin{cases} 1 - p_{\theta,t}, & v = y_t, \\ -p_v, & v \neq y_t. \end{cases} \quad (26)$$

**Infinity-Norm of the Likelihood-Ratio Gradient** Since  $\partial r_t / \partial \log p_{\theta,t} = r_t$ , the log-derivative trick yields the gradient of the loss with respect to the sampled normalised log-probability as

$$\frac{\partial \tilde{\ell}_t^{\text{PG}}}{\partial \log p_{\theta,t}} = -A_t r_t,$$

which is a scalar that multiplies the log-softmax Jacobian row Equation (26) in the logit gradient. Using the homogeneity of the  $\infty$ -norm, the norm of the logit gradient is therefore

$$\|\nabla_{\mathbf{z}} \tilde{\ell}_t^{\text{PG}}\|_{\infty} = \left\| \frac{\partial \tilde{\ell}_t^{\text{PG}}}{\partial \log p_{\theta,t}} \mathbf{g}_t \right\|_{\infty} = |A_t| r_t \|\mathbf{g}_t\|_{\infty}.$$

Since the probabilities  $p_v$  are normalised, the  $\infty$ -norm of  $\mathbf{g}_t$  is

$$\max_{v \neq y_t} |-p_v| \leq \sum_{v \neq y_t} p_v = 1 - p_{\theta,t},$$

meaning that the maximum is always attained at the coordinate of the sampled token. Thus the  $\infty$ -norm of the log-softmax Jacobian row simplifies to  $\|\mathbf{g}_t\|_{\infty} = 1 - p_{\theta,t}$ , and the norm of the logit gradient is

$$\|\nabla_{\mathbf{z}} \tilde{\ell}_t^{\text{PG}}\|_{\infty} = |A_t| r_t (1 - p_{\theta,t}) = |A_t| \frac{p_{\theta,t} (1 - p_{\theta,t})}{p_{\mu,t}} =: |A_t| G_t$$

which is Equation (17).

### I.3.2 1- and 2-Norm Bounds

Similarly, the 1-norm of the logit gradient is

$$\|\nabla_{\mathbf{z}} \tilde{\ell}_t^{\text{PG}}\|_1 = |A_t| r_t (|1 - p_{\theta,t}| + \sum_{v \neq y_t} p_v) = 2 |A_t| G_t,$$

and the 2-norm is

$$\|\nabla_{\mathbf{z}} \tilde{\ell}_t^{\text{PG}}\|_2 = |A_t| r_t \sqrt{(1 - p_{\theta,t})^2 + \sum_{v \neq y_t} p_v^2},$$

which is bounded by  $(1 - p_{\theta,t})\sqrt{2} |A_t| r_t$ . Since  $\|\mathbf{x}\|_{\infty} \leq \|\mathbf{x}\|_2 \leq \|\mathbf{x}\|_1$  in general, the  $\infty$ -norm is also a lower bound on the 2-norm.

### I.3.3 Pushforward Analysis

The scaling  $c_t$  of Section 3.2.1.2 bounds the gradient on the logits as a means to limit the change in training policy probability. We next show this relationship by analysing a direct gradient step on the logits, as if they were free parameters, and the resulting effect on the probabilities. In training, the step reaches the logits through the model's Jacobian  $\partial \mathbf{z} / \partial \theta$  and sums over all tokens of the batch, so this analysis describes the contribution of a single token.

To first order, a gradient step  $\delta \mathbf{z} = -\eta \nabla_{\mathbf{z}} \tilde{\ell}_t^{\text{PG}}$  with step size  $\eta$  on the logits changes the probabilities through the Jacobian of the softmax,

$$\delta \mathbf{p} = \mathbf{J} \delta \mathbf{z}, \quad \mathbf{J} := \frac{\partial \mathbf{p}}{\partial \mathbf{z}} = \text{diag}(\mathbf{p}) - \mathbf{p} \mathbf{p}^{\top}.$$

Inserting  $\nabla_{\mathbf{z}} \tilde{\ell}_t^{\text{PG}} = -A_t r_t \mathbf{g}_t$  and  $\mathbf{p}^\top \mathbf{g}_t = p_{\theta,t} - \|\mathbf{p}\|_2^2$ , the probability of the sampled token changes by

$$\delta p_{\theta,t} = \eta A_t r_t p_{\theta,t} (1 - 2p_{\theta,t} + \|\mathbf{p}\|_2^2) = \eta A_t r_t p_{\theta,t} \|\mathbf{g}_t\|_2^2.$$

With the 2-norm bounds  $(1 - p_{\theta,t})^2 \leq \|\mathbf{g}_t\|_2^2 \leq 2(1 - p_{\theta,t})^2$ , this change satisfies

$$\eta |A_t| G_t p_{\theta,t} (1 - p_{\theta,t}) \leq |\delta p_{\theta,t}| \leq 2 \eta |A_t| G_t p_{\theta,t} (1 - p_{\theta,t})$$

and is thus proportional to  $G_t$  up to a factor of at most 2. The scaling  $c_t$  clips  $G_t$  at  $G_{\max}$ , and since  $p_{\theta,t}(1 - p_{\theta,t}) \leq \frac{1}{4}$ , a single token changes its own probability to first order by at most

$$|c_t \delta p_{\theta,t}| \leq 2 \eta |A_t| G_{\max} p_{\theta,t} (1 - p_{\theta,t}) \leq \eta |A_t| G_{\max} / 2.$$

## I.4 Training Recipe Details

### I.4.1 FP8 Quantisation-Aware Training

InfiXAI Team et al. (2025) find that the training–inference mismatch comes from the quantisation itself rather than from the FP8 kernels. Following this finding, the trainer of Section 3.2.1.3 applies the same FP8 rounding as the inference engine but computes in BF16, without FP8 kernels. Even so, the median training–inference mismatch  $|\log r_t|$  is roughly three times that of BF16 serving.

The inference engine stores weights in FP8 (E4M3) in blocks of  $128 \times 128$ , each with an FP32 scale, quantises activations dynamically in groups of  $1 \times 128$ , and keeps the KV cache in FP8. The embeddings, the normalisation layers and the router gates remain in BF16, and both policies keep the LM head’s logits in FP32, which MiniMax (2025) find reduces the training–inference mismatch. In the forward pass, the trainer rounds the weights and input activations of all other linear layers and of the routed experts to FP8 with the same block geometry and scales, and rounds the queries, keys and values to FP8 with unit scales, as the inference engine does. RMS normalisation (B. Zhang and Sennrich 2019) bounds the norm of its output by its largest gain times the square root of its dimension, for any input. The QK normalisation (Henry et al. 2020) therefore bounds every query and key element, and RoPE (Jianlin Su et al. 2024), where applied, preserves this bound as a rotation. The normalisation before the attention block bounds the values together with the value weights. For Kolibri, these bounds lie below the E4M3 maximum of 448 (Micikevicius et al. 2022), so no input can overflow. The backward pass uses the straight-through estimator (Bengio, Léonard, and Courville 2013).

### I.4.2 Staleness at Scale

The loop of rollout generation and optimiser steps (Section 3.2.1.5) runs at the speed of its slowest part, the trainer, inference or saturated environment resources such as sandboxes, and is balanced when inference produces groups as fast as the trainer consumes them. Staleness is counted in optimiser steps, but rollouts take wall-clock time. The staleness of a group is therefore roughly its rollout duration divided by the step time of the trainer, so any speedup of the trainer at a fixed generation latency makes groups staler. Suppose a balanced loop takes 2 min per optimiser step and a code rollout takes 60 min, so code groups arrive with

a staleness of 30. Doubling both the trainer and the inference GPUs at the same batch size halves the step time to about 1 min and doubles inference throughput, so the loop stays balanced. The decode speed per request is unchanged, however, so a code rollout still takes 60 min and now arrives with a staleness of 60. In some of our runs, code groups exceeded the staleness limit in this way after we scaled up the run. Scaling the batch size with the GPU count also keeps the step time, and hence the staleness, constant.

## I.5 RL Hyperparameters

This section lists the hyperparameters of the main RL run. Table 58 covers hardware, precision and optimisation, and Table 59 covers the algorithmic parameters of Sections 3.2.1 and 3.2.2. Table 60 lists the failure classes of the targeted OPSD, and Appendix I.6 gives its hyperparameters.

| Parameter                                       | Value                                                                     |
|-------------------------------------------------|---------------------------------------------------------------------------|
| <i>Hardware and parallelism</i>                 |                                                                           |
| GPUs                                            | NVIDIA B300, 256 on 32 nodes                                              |
| Trainer                                         | 96 GPUs, FSDP without further parallelism                                 |
| Policy inference                                | 144 single-GPU vLLM replicas                                              |
| Judge inference                                 | 16 single-GPU vLLM replicas, frozen                                       |
| Rollout threads per inference replica           | 45                                                                        |
| Share of inference capacity used for validation | 5 %                                                                       |
| <i>Precision and memory</i>                     |                                                                           |
| Inference weights and KV cache                  | FP8                                                                       |
| Trainer                                         | FP8 quantisation-aware training                                           |
| LM head                                         | FP32 for both policies                                                    |
| Exported checkpoints                            | BF16 and FP8                                                              |
| Activation checkpointing                        | full                                                                      |
| <i>Optimisation</i>                             |                                                                           |
| Optimiser steps                                 | 1000                                                                      |
| Attention and expert weights                    | Muon, spectral, Nesterov                                                  |
| Embedding, norm scales, LM head and router      | Adam, $\beta_1 = 0.9$ , $\beta_2 = 0.95$ , $\epsilon = 1 \times 10^{-15}$ |
| Weight decay                                    | none                                                                      |
| Learning rate: attention and expert weights     | $1 \times 10^{-5}$                                                        |
| Learning rate: embedding and norm scales        | $2 \times 10^{-5}$                                                        |
| Learning rate: LM head and router               | $2.5 \times 10^{-7}$                                                      |
| Learning-rate warm-up                           | 50 steps, linear                                                          |
| Learning-rate decay ratio                       | 0                                                                         |
| Global gradient-norm clipping                   | 1.0                                                                       |
| MoE load-balancing loss                         | none                                                                      |
| <i>Sequences and asynchrony</i>                 |                                                                           |
| Context length                                  | 256k tokens                                                               |
| Packed tokens per trainer rank                  | 393 216                                                                   |
| Weight push                                     | after every optimiser step                                                |
| Replay buffer capacity                          | 512 groups in 8 shards                                                    |
| <i>Sandboxes</i>                                |                                                                           |
| Executors                                       | one per node on 31 nodes, 112 CPUs each, shared at nice 10                |
| Writable layer                                  | tmpfs, 8 GiB per sandbox                                                  |
| Address space per command                       | 16 GiB                                                                    |
| Processes per sandbox                           | 4096, then the sandbox is killed                                          |
| Command timeout                                 | 600 s                                                                     |
| Output per command                              | 1 MiB per stream                                                          |
| Network                                         | loopback; egress proxy with an allowlist where dependencies are installed |

Table 58. Technical and optimisation hyperparameters of the main RL run. The optimiser groups follow Table 6.

| Parameter                                                    | Value                                  |
|--------------------------------------------------------------|----------------------------------------|
| <i>Rollouts</i>                                              |                                        |
| Group size $ \mathcal{G} $                                   | 8                                      |
| Groups per batch $B$                                         | 256                                    |
| Sampling                                                     | temperature 1, top- $p$ 1, no top- $k$ |
| Maximum rollout steps                                        | 100                                    |
| Reasoning effort weights (low, medium, high)                 | 0.1, 0.1, 0.8                          |
| Probability of preserving reasoning traces                   | 0.5                                    |
| <i>Curriculum</i>                                            |                                        |
| Target bucket weights $c_b$ (hard, trainable, solved)        | 0.08, 0.9, 0.02                        |
| Pass rate of hard and solved prompts                         | $< 1 \times 10^{-5}$ and 1             |
| Default pass rate                                            | 0.5                                    |
| Policy updates until $\gamma_b$ saturates                    | 50                                     |
| <i>Group filtering</i>                                       |                                        |
| Reward standard deviation below which rewards count as equal | $1 \times 10^{-3}$                     |
| Replacement attempts after a dropped group                   | 2                                      |
| Maximum staleness                                            | 42 policy updates                      |
| <i>Policy loss</i>                                           |                                        |
| B-TV thresholds $\epsilon_+$ , $\epsilon_-$                  | 0.2, 0.2                               |
| Squared log-ratio coefficient $\tau_{\text{KL}}$             | $1 \times 10^{-3}$                     |
| Gradient-norm bound $G_{\text{max}}$                         | 2                                      |
| <i>Format signals</i>                                        |                                        |
| Aggregation norm $p$                                         | 2                                      |
| Global multiplier $m_{\text{min}}$                           | 0.5                                    |
| Minimum factor $f_k$                                         | 0.5, 0 for non_empty_output            |
| Completion-budget threshold (low, medium, high)              | 0.2, 0.4, 0.8 of the budget            |
| Completion-budget sigmoid steepness                          | 6                                      |

Table 59. Algorithmic hyperparameters of the main RL run. Symbols follow Sections 3.2.1 and 3.2.2.

| Family   | Category                   | Failure class              | Gist of the remedial hint                                                                                    |
|----------|----------------------------|----------------------------|--------------------------------------------------------------------------------------------------------------|
| Tool use | invalid tool call          | unparseable_tool_call      | A tool call is a single valid JSON object inside the tool-call tags that matches the tool's schema.          |
|          |                            | wrong_argument_type        | Arguments use the exact JSON types the tool declares.                                                        |
|          |                            | tool_contract_error        | Call tools with exactly the declared argument names and types.                                               |
|          |                            | entity_already_exists      | Re-read the task for the exact identifier and for whether the item is to be created or updated.              |
|          | invented tool or reference | precondition_failed        | Check the entity's current state and complete only the prerequisite steps the task requires.                 |
|          |                            | unknown_tool_name          | Only the tools listed in the system prompt can be called, and the hint lists their signatures.               |
|          |                            | entity_not_found           | Take identifiers from the task or earlier tool results, or look them up with a search tool.                  |
|          |                            | unknown_url                | Open only URLs that an earlier search result shows verbatim, or that result's number, and never guess a URL. |
| Terminal | syntax errors              | result_out_of_range        | A result number is the bracketed index of a search result, counted over all searches so far.                 |
|          |                            | unparseable_tool_call      | A tool call is one JSON object with escaped string arguments, one call per block.                            |
|          |                            | key_name_with_newline      | A key name such as C-c acts as that key only without a trailing newline.                                     |
|          |                            | missing_tool_call          | Every response must contain a tool call through the tool-call channel.                                       |
|          |                            | shell_quoting              | Shell commands must be complete and correctly quoted, and code with its own quotes belongs in a file.        |
|          |                            | inline_script_syntax_error | Check code passed inline to the interpreter for typos and altered quotes, and put multi-line code in a file. |
|          |                            | edited_file_syntax_error   | A file the model wrote earlier that does not parse breaks whatever runs or imports it, so fix it first.      |
|          | wrong tool or target       | unknown_tool_name          | Only the offered tools can be called, and the hint lists their signatures.                                   |
|          |                            | tool_contract_error        | Call tools with exactly the declared argument names and types.                                               |
|          |                            | edit_mismatch              | The text to replace must match the current file exactly and once, so re-read the file first.                 |
|          |                            | noop_edit                  | An edit that reproduces the original text changes nothing.                                                   |
|          |                            | invalid_line_range         | Line numbers must exist in the current version of the file.                                                  |
|          |                            | create_on_existing_file    | Edit an existing file instead of creating it again.                                                          |
|          |                            | no_changes_submit          | Grading uses the changed repository files, and nothing has been changed yet.                                 |
|          |                            | git_unavailable            | Work inside a git repository and keep copies of files before editing them.                                   |
|          | output or time limits      | typed_into_busy_terminal   | Keystrokes reach the shell only once its prompt is the last line on screen, so wait or interrupt first.      |
|          |                            | typed_ahead_in_batch       | Keystrokes after a command that outlives its duration feed that command, so give long commands enough time.  |
|          |                            | command_timeout            | Bound long commands with a timeout, or shrink the work instead of rerunning the command unchanged.           |
|          |                            | output_truncated           | Long tool outputs are cut, so read files in slices and filter command output.                                |

Table 60. Failure classes of the targeted OPSD, grouped by the categories of Figures 42 and 69 and, within a category, in order of priority. The hints are shortened for readability.

## I.6 Further OPSD Analysis

The main run weights the OPSD loss of Equation (14) with a coefficient  $\lambda_{\text{OPSD}} = 1$ , uses the forward KL divergence over the full vocabulary and flags at most 5 steps per sequence. It selects the all-wrong groups, which the group-relative objective drops for low reward variance and whose highest environment reward is

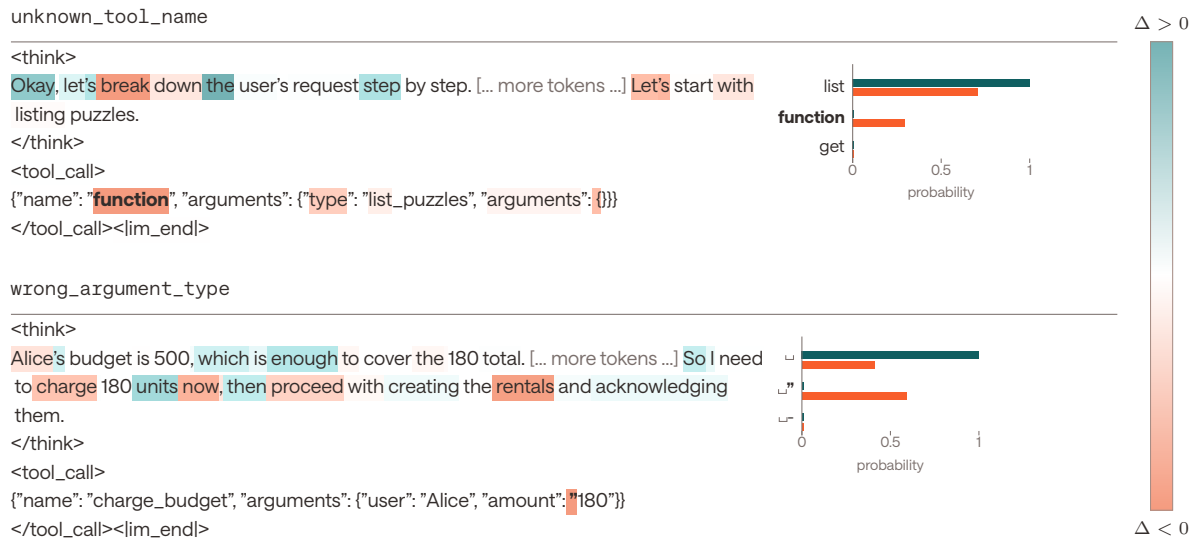


Figure 68. Teacher versus student on a flagged turn for two different failure classes. Token colour is given by  $\Delta = \log p_{\text{teacher}} - \log p_{\text{student}}$  of the turn the student sampled, rescored under the hinted context and the bare context. The hint down-weights a token with  $\Delta < 0$  and up-weights one with  $\Delta > 0$ . The reasoning is shortened.

below 0.2, and Table 60 lists its failure classes. Counts come from the rollout traces, where all 24 detectors score every assistant turn of every training sequence of the 8 OPSD environments. The trainer logs only the first matching detector on at most five turns per OPSD group, so its logs serve only for the shares of trained groups and tokens, the teacher agreement and the entropy. Early and late values span the first and the last 50 steps on the traces, and, to compensate for staleness, steps 25 to 75 and the last 50 steps in the logs.

**Detected Failures** Table 61 shows how detections per training sequence fall in 6 of the 8 environments, with the average decreasing from 4.21 to 2.56. The left of Figure 69 shows that detections per rollout step also fall, by 31 %, though not monotonically. This can be further improved by better hinting and distillation strategies that target the remaining hard-to-solve errors. The right of Figure 69 shows how, over the course of training, errors related to invented tools and references rapidly decrease and eventually are trained away. In contrast, output or time limits rise, making up 59 % of the late detections. Nearly all of them are `output_truncated`, which fires when the harness cuts a long command output, and they rise in the software-engineering environments, where the curriculum keeps drawing long tasks on large repositories. Table 61 shows how OPSD-annotated sequences consistently receive lower environment rewards on average, suggesting that the error detection successfully finds sequences where the model struggles. Figure 68 visualises the differences between teacher and policy token probabilities for two exemplary flagged turns, finding that the teacher focuses on the turn’s error in both cases.

**Share of the Training Signal and Entropy** The left of Figure 70 shows how the share of training groups in the OPSD environments that become OPSD groups falls from 13.0 % to 7.2 % during training, and how only 0.10 % to 0.034 % of tokens are trained with the distillation signal. The right of Figure 70 overlays the

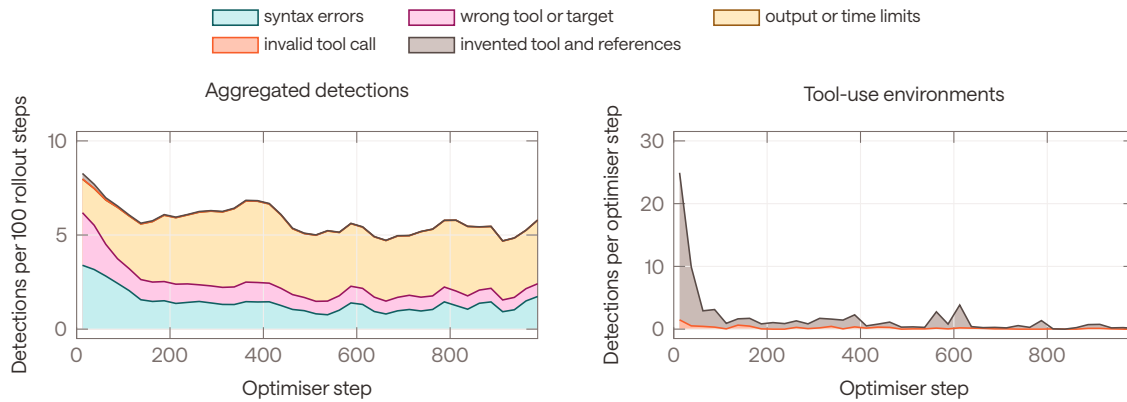


Figure 69. Failures that the targeted OPSD detects over the Kolibri RL run, counted on its rollout traces in bins of 25 steps. Left: Detections per 100 rollout steps in all 8 OPSD environments. When controlling for rollout steps, error detections decrease during training, though not monotonically. Right: Detections over the course of training for the 3 tool-use environments. In contrast to the terminal environments (Figure 42), the tool-use environments quickly converge towards almost no detections.

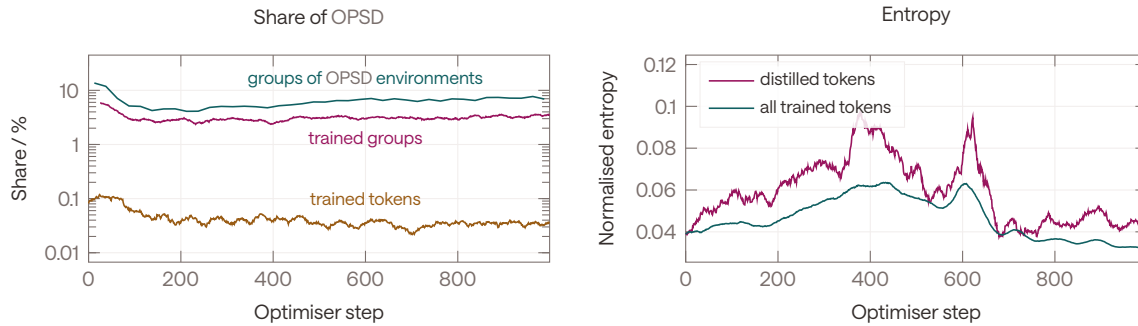


Figure 70. Left: Total share of OPSD groups in eligible environments, total share of OPSD groups among all groups added to the replay buffer, and total share of training tokens using the distillation loss. All three shares drop at the start of training and then stay relatively consistent. Right: Normalised entropy compared between all trained tokens and distilled tokens. Distilled tokens consistently have a higher entropy than average tokens, likely because the policy is less certain about their prediction. Values are averaged over 25 steps.

entropy of the OPSD tokens with the entropy of all trained tokens, as shown in the left panel of Figure 40. Both metrics follow the same rough shape, but the normalised policy entropy stays higher on the distilled tokens than on all trained tokens.

| Environment          | Family   | Detections per sequence |                |            | Reward by detection |         |
|----------------------|----------|-------------------------|----------------|------------|---------------------|---------|
|                      |          | Steps 0–49              | Steps 951–1000 | Change / % | With                | Without |
| General agent        | Tool use | 0.16                    | 0.17           | 3          | 0.57                | 0.68    |
| Customer-service     | Tool use | 1.45                    | 0.02           | –99        | 0.64                | 0.73    |
| Workplace assistant  | Tool use | 0.04                    | 0.03           | –38        | 0.59                | 0.78    |
| Nemotron terminal    | Terminal | 5.09                    | 0.66           | –87        | 0.68                | 0.74    |
| SWE-Gym              | Terminal | 5.21                    | 3.05           | –41        | 0.42                | 0.44    |
| SWE-smith            | Terminal | 4.41                    | 4.74           | 7          | 0.73                | 0.87    |
| SWE-smith non-Python | Terminal | 5.11                    | 4.47           | –13        | 0.78                | 0.92    |
| TMAX                 | Terminal | 5.72                    | 1.04           | –82        | 0.72                | 0.88    |

Table 61. Targeted OPSD per environment of the Kolibri RL run. Detections are the failures the detectors flag per training sequence over the first and the last 50 optimiser steps, and the change is the late value relative to the early one. The reward is the mean environment reward of the training sequences with and without a detected failure over all steps. In most environments, detections decrease over the course of training, while sequences with detected errors receive a lower average reward.

# Acronyms

|                          |                                               |
|--------------------------|-----------------------------------------------|
| <b><math>\mu</math>P</b> | Maximal Update Parametrisation                |
| <b>AIME</b>              | American Invitational Mathematics Examination |
| <b>ARC</b>               | AI2 Reasoning Challenge                       |
| <b>B-TV</b>              | Binary Total Variation                        |
| <b>BFCL</b>              | Berkeley Function Calling Leaderboard         |
| <b>BPB</b>               | Bits Per Byte                                 |
| <b>BPE</b>               | Byte Pair Encoding                            |
| <b>CC</b>                | Common Crawl                                  |
| <b>CCP</b>               | Chinese Communist Party                       |
| <b>CISPO</b>             | Clipped IS-Weight Policy Optimization         |
| <b>CJK</b>               | Chinese, Japanese And Korean                  |
| <b>DCGM</b>              | NVIDIA Data Center GPU Manager                |
| <b>DCLM</b>              | DataComp For Language Models                  |
| <b>DCP</b>               | Distributed Checkpoint                        |
| <b>DP</b>                | Data Parallel                                 |
| <b>DPPO</b>              | Divergence Proximal Policy Optimization       |
| <b>DSM-Directive</b>     | Directive EU 2019/790                         |
| <b>EP</b>                | Expert Parallel                               |
| <b>EQB</b>               | Exact Quantile Balancing                      |
| <b>FFN</b>               | Feed-Forward Network                          |
| <b>FLOP</b>              | Floating-Point Operation                      |
| <b>FSDP</b>              | Fully Sharded Data Parallel                   |
| <b>GDPR</b>              | General Data Protection Regulation            |
| <b>GPAI</b>              | General-Purpose AI                            |
| <b>GQA</b>               | Grouped-Query Attention                       |
| <b>GRPO</b>              | Group Relative Policy Optimization            |
| <b>GSM8K</b>             | Grade School Math 8K                          |
| <b>IDF</b>               | Inverse Document Frequency                    |
| <b>IETF</b>              | Internet Engineering Task Force               |
| <b>KL</b>                | Kullback-Leibler                              |
| <b>KV</b>                | Key-Value                                     |
| <b>LEI</b>               | Load-Error Injection                          |
| <b>LLM</b>               | Large Language Model                          |

**LM** Language Model  
**MBPP** Mostly Basic Python Problems  
**MMLU** Massive Multitask Language Understanding  
**MoE** Mixture Of Experts  
**MTaC** Model Training As Code  
**NIAH** Needle In A Haystack  
**NIC** Network Interface Controller  
**OPSD** On-Policy Self-Distillation  
**PII** Personally Identifiable Information  
**PIQA** Physical Interaction Question Answering  
**QA** Question Answering  
**QB** Quantile Balancing  
**RAG** Retrieval-Augmented Generation  
**RDMA** Remote Direct Memory Access  
**RFC** Request For Comments  
**RL** Reinforcement Learning  
**RMS** Root Mean Square  
**RoPE** Rotary Position Embeddings  
**RSDB** Random Sequential Document Buffer  
**SFT** Supervised Fine-Tuning  
**SiLU** Sigmoid Linear Unit  
**SMEBU** Soft-Clamped Momentum Expert Bias Updates  
**SWA** Sliding-Window Attention  
**VT** Variable Tracking  
**WE** Word Extraction  
**WSM** Warmup-Stable-Merge  
**YaRN** Yet Another RoPE Extension